

SIMATIC

S7-PLCSIM Advanced API

Programming Manual

Introduction

1

Industrial cybersecurity

2

Product overview

3

Initialize API

4

Shut down API

5

Global functions (Native C++)

6

IBaseRuntimeManager

7

ISimulationRuntimeManager

8

IInstance

9

IRemoteRuntimeManager

10

Data types

11

Examples/applications

12

Legal information

Warning notice system

This manual contains notices you have to observe in order to ensure your personal safety, as well as to prevent damage to property. The notices referring to your personal safety are highlighted in the manual by a safety alert symbol, notices referring only to property damage have no safety alert symbol. These notices shown below are graded according to the degree of danger.

DANGER

indicates that death or severe personal injury **will** result if proper precautions are not taken.

WARNING

indicates that death or severe personal injury **may** result if proper precautions are not taken.

CAUTION

indicates that minor personal injury can result if proper precautions are not taken.

NOTICE

indicates that property damage can result if proper precautions are not taken.

If more than one degree of danger is present, the warning notice representing the highest degree of danger will be used. A notice warning of injury to persons with a safety alert symbol may also include a warning relating to property damage.

Qualified Personnel

The product/system described in this documentation may be operated only by **personnel qualified** for the specific task in accordance with the relevant documentation, in particular its warning notices and safety instructions. Qualified persons are those who, because of their training and experience, are familiar with the installation, assembly, commissioning, operation, decommissioning and disassembly of the product and can recognize risks and avoid possible hazards.

Proper use of Siemens products

Note the following:

WARNING

Siemens products may only be used for the application described in the catalog and the associated usage information. If products and components from other manufacturers are used, these must be recommended or approved by Siemens. Proper transport, storage, installation, assembly, commissioning, operation and maintenance are required to ensure that the products operate safely and without any problems. The permissible ambient conditions must be complied with. The information in the relevant documentation must be observed.

Trademarks

All names identified by ® are registered trademarks of Siemens Aktiengesellschaft. The remaining trademarks in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owner.

Disclaimer of Liability

We have reviewed the contents of this publication to ensure consistency with the hardware and software described. Since variance cannot be precluded entirely, we cannot guarantee full consistency. However, the information in this publication is reviewed regularly and any necessary corrections are included in subsequent editions.

Table of contents

1	Introduction.....	8
1.1	General information.....	8
1.2	S7-PLCSIM Advanced documentation guide.....	9
1.3	SIMATIC Technical Documentation.....	13
1.4	Tool support.....	14
1.5	Administrator rights.....	15
2	Industrial cybersecurity.....	16
2.1	Introduction.....	16
2.1.1	Cybersecurity information.....	16
2.2	Basic information on industrial cybersecurity.....	17
2.2.1	Definition of industrial cybersecurity.....	17
2.2.2	Objectives of industrial cybersecurity.....	17
2.3	Operational application environment and security assumptions.....	17
2.3.1	Intended use.....	17
2.3.2	Requirements for the operational application environment and security assumptions.....	18
2.4	Secure operation.....	19
2.4.1	Security update notification.....	19
2.4.2	Hardening measures.....	20
2.4.3	Access control.....	20
2.4.4	Notifications about security vulnerabilities (Siemens Security Advisories).....	20
2.4.5	Data backup.....	21
2.4.6	Secure uninstallation.....	21
3	Product overview.....	22
3.1	What is S7-PLCSIM Advanced?	22
3.2	Functions for licenses.....	23
3.3	System requirements.....	25
3.4	Restrictions due to antivirus programs	25
3.5	Access to instances.....	27
3.6	Interfaces of the Simulation Runtime.....	27
3.7	Overview of interfaces for native C++.....	29
3.8	Overview of interfaces for managed code	33
3.9	Restrictions for file paths.....	36
3.10	Overview of data types for native C++.....	36

3.11	Overview of data types for managed code.....	38
4	Initialize API	39
4.1	Load API library.....	39
4.2	Native C++	40
4.2.1	InitializeApi()	40
4.2.2	RuntimeApiEntry_Initialize.....	41
4.3	.NET (C#).....	43
4.3.1	Initialize	43
5	Shut down API.....	44
5.1	Native C++.....	44
5.1.1	DestroyInterface().....	44
5.1.2	RuntimeApiEntry_DestroyInterface.....	45
5.1.3	FreeApi().....	46
5.1.4	ShutdownAndFreeApi().....	47
5.2	.NET (C#).....	48
5.2.1	Shut down API	48
6	Global functions (Native C++).....	49
7	IBaseRuntimeManager.....	54
7.1	Simulation runtime instances.....	54
7.2	Interfaces for IBaseRuntimeManager.....	60
8	ISimulationRuntimeManager.....	68
8.1	Remote connections.....	68
8.1.1	RunAutodiscover().....	73
8.2	Events for ISimulationRuntimeManager.....	75
8.2.1	OnConfigurationChanged events.....	75
8.2.2	OnRuntimeManagerLost events.....	77
8.2.3	OnAutodiscoverData events.....	80
9	IInstance.....	82
9.1	Interfaces for IInstances.....	82
9.2	Controller - Information and settings.....	90
9.3	Operating state	100
9.4	Stop simulation.....	110
9.5	Tag list.....	112
9.6	I/O access.....	117
9.6.1	Synchronizing inputs and outputs.....	117
9.6.2	Simulate peripheral I/O.....	118
9.6.3	I/O access via address - Reading.....	118
9.6.4	I/O access via address - Writing.....	123
9.6.5	I/O access via tag name - Reading.....	128
9.6.6	I/O access via tag name - Writing.....	150

9.7	Cycle control.....	174
9.8	Virtual and real time.....	179
9.9	Monitoring overflow	183
9.10	Display functions and events.....	183
9.10.1	Display functions.....	183
9.10.2	OnDisplayDataChanged events.....	187
9.11	Acyclic services.....	190
9.11.1	Overview.....	190
9.11.2	ReadRecordDone/WriteRecordDone.....	192
9.11.3	AlarmNotification.....	194
9.11.4	ProcessEvent.....	196
9.11.5	PullOrPlugEvent.....	197
9.11.6	StatusEvent.....	199
9.11.7	ProfileEvent.....	200
9.11.8	UpdateEvent.....	201
9.11.9	GetConfiguredProcessEvent.....	202
9.11.10	RackOrStationFaultEvent.....	202
9.12	Events for Instances.....	203
9.12.1	Events for operating state and cycle control.....	203
9.12.1.1	Synchronize simulation partner.....	204
9.12.1.1.1	Synchronize simulation partner cycle-controlled.....	204
9.12.1.1.2	Synchronize simulation partner time-controlled.....	208
9.12.1.2	OnOperatingStateChanged events.....	210
9.12.1.3	OnLedChanged events.....	213
9.12.1.4	OnSoftwareConfigurationChanged events.....	216
9.12.1.5	OnIPAddressChanged events.....	218
9.12.1.6	OnSyncPointReached events.....	220
9.12.1.7	OnHardwareConfigChanged events	222
9.12.2	Events for acyclic services.....	225
9.12.2.1	OnAlarmNotification events.....	227
9.12.2.2	OnProcessEvent events.....	228
9.12.2.3	OnPullOrPlugEvent events.....	229
9.12.2.4	OnStatusEvent events.....	230
9.12.2.5	OnProfileEvent events.....	231
9.12.2.6	OnUpdateEvent events.....	232
9.12.2.7	RackOrStationFault events.....	233
10	IRemoteRuntimeManager.....	234
10.1	Interfaces - Information and settings.....	234
10.2	Events for IRemoteRuntimeManager.....	236
10.2.1	OnConnectionLost events.....	236
11	Data types.....	239
11.1	DLL import functions (Native C++).....	239
11.1.1	ApiEntry_Initialize.....	239
11.1.2	ApiEntry_DestroyInterface.....	240
11.2	Event callback functions (Native C++).....	240
11.2.1	EventCallback_VOID.....	240

11.2.2	EventCallback_SRCC_UINT32_UINT32_INT32.....	241
11.2.3	EventCallback_SRRSI_AD.....	241
11.2.4	EventCallback_IRRTM.....	242
11.2.5	EventCallback_II_SREC_ST_SROS_SROS	242
11.2.6	EventCallback_II_SREC_ST_UINT32_INT64_INT64_UINT32.....	243
11.2.7	EventCallback_II_SREC_ST.....	243
11.2.8	EventCallback_II_SREC_ST_SRLT_SRLM.....	244
11.2.9	EventCallback_II_SREC_ST_SDRI.....	244
11.2.10	EventCallback_II_SREC_ST_SDRI_BYTE.....	245
11.2.11	EventCallback_II_SREC_ST_UINT32_UINT32.....	245
11.2.12	EventCallback_II_SREC_ST_UINT32_UINT32_EPET_UINT32	246
11.2.13	EventCallback_II_SREC_ST_UINT32_EPPET_UINT32	246
11.2.14	EventCallback_II_SREC_ST_UINT32_ERSFET.....	247
11.2.15	EventCallback_II_SREC_ST_UINT32.....	247
11.3	Delegate definitions (managed code).....	248
11.3.1	Delegate_Void.....	248
11.3.2	Delegate_DisplayDataChanged.....	248
11.3.3	Delegate_SRCC_UINT32_UINT32_INT32.....	248
11.3.4	Delegate_SRRSI_AD.....	249
11.3.5	Delegate_II_EREC_DT.....	249
11.3.6	Delegate_II_EREC_DT_EOS_EOS.....	250
11.3.7	Delegate_II_EREC_DT_ELT_ELM.....	250
11.3.8	Delegate_II_EREC_DT_UINT32_INT64_INT64_UINT32.....	251
11.3.9	Delegate_IRRTM	251
11.3.10	Delegate_II_EREC_DT_SDRI.....	252
11.3.11	Delegate_II_EREC_DT_SDR.....	252
11.3.12	Delegate_SREC_ST_UINT32_EPPET_UINT32.....	253
11.3.13	Delegate_SREC_ST_UINT32_UINT32_EPET_UINT32.....	253
11.3.14	Delegate_SREC_ST_UINT32.....	254
11.3.15	Delegate_SREC_ST_UINT32_UINT32.....	254
11.3.16	Delegate_SREC_ST_UINT32_ERSFET.....	255
11.4	Definitions and constants.....	255
11.5	Unions (Native C++).....	256
11.5.1	UIP.....	256
11.5.2	UDataValue.....	257
11.6	Structures.....	258
11.6.1	SDataValue.....	258
11.6.2	SDVBNI.....	259
11.6.3	SDataValueByAddress.....	260
11.6.4	SDataValueByAddressWithCheck.....	261
11.6.5	SDataValueByName.....	262
11.6.6	SDataValueByNameWithCheck.....	263
11.6.7	SConnectionInfo.....	263
11.6.8	SDisplayData.....	264
11.6.9	SInstanceInfo.....	264
11.6.10	SDimension.....	265
11.6.11	STagInfo.....	265
11.6.12	SIP	267
11.6.13	SIPSuite4.....	267
11.6.14	SONSyncPointReachedResult.....	268

11.6.15	SDataRecordInfo.....	270
11.6.16	SDataRecord.....	270
11.6.17	SConfiguredProcessEvents	271
11.6.18	SDiagExtChannelDescription.....	272
11.6.19	SAutodiscoverData.....	273
11.6.20	SMAC address.....	274
11.6.21	SNetInterfaceInfo.....	274
11.7	Enumerations.....	274
11.7.1	ERuntimeErrorCode.....	275
11.7.2	EArea.....	278
11.7.3	EOperatingState.....	278
11.7.4	EOperatingMode.....	279
11.7.5	ECPUType.....	280
11.7.6	ECommunicationInterface.....	282
11.7.7	EDisplayKey.....	282
11.7.8	ENetworkMode.....	283
11.7.9	EPLCInterface.....	283
11.7.10	ELEDType.....	283
11.7.11	ELEDMode.....	284
11.7.12	EPrimitiveDataType.....	285
11.7.13	EDataType.....	287
11.7.14	ETagListDetails.....	289
11.7.15	ERuntimeConfigChanged.....	290
11.7.16	EPullOrPlugEventType.....	291
11.7.17	EProcessEventTypes.....	291
11.7.18	ESoftwareConfigChanged.....	292
11.7.19	EDirection.....	292
11.7.20	EDiagProperty.....	292
11.7.21	EDiagSeverity.....	293
11.7.22	ERackOrStationFaultType.....	293
11.7.23	ECycleTimeMonitoringMode.....	293
11.7.24	EAutodiscoverType.....	294
11.7.25	ESecurityMode.....	294
12	Examples/applications.....	295
12.1	Example: Communication via TCP/IP in Single Adapter Network Mode (promiscuous mode)	295
12.2	Example: Communication via TCP/IP in Multi-Adapter Network Mode (non-promiscuous mode)	296

Introduction

1.1 General information

Purpose of the documentation

This manual describes the API for the simulation software, SIMATIC S7-PLCSIM Advanced. You can use the API to simulate and test your SIMATIC STEP 7 programs on a simulated PLC instance. You can also use the API to connect the simulated PLC instance to a system or machine simulation (co-simulation).

Basic knowledge required

The API must only be used by qualified staff.
The following knowledge is required:

- Industrial Automation and Automation Technology
- Programming with STEP 7 (TIA Portal)
- SIMATIC CPUs and CPU programming
- PC-based automation using S7-1500 and WinCC Runtime Advanced
- Knowledge of programming with C++ or C#
- PC technology
- Windows operating system

Conventions

STEP 7: In this documentation, "STEP 7" is used as a synonym for all versions of the configuration and programming software "SIMATIC STEP 7 (TIA Portal)".

Also observe notes marked as follows:

NOTE

A note contains important information on the product described in the documentation, on the handling of the product or on the section of the documentation to which particular attention should be paid.

Scope

This manual is valid for the following product variants:

Article number	Product variants
6ES7823-1FA06-0YA5	SIMATIC S7-PLCSIM Advanced V8.0
6ES7823-1FE06-0YA5	SIMATIC S7-PLCSIM Advanced V8.0 Download
6ES7823-1FA06-0YE5	SIMATIC S7-PLCSIM Advanced V8.0 Upgrade V1.0 -> V8.0
6ES7823-1FE06-0YE5	SIMATIC S7-PLCSIM Advanced V8.0 Upgrade V1.0 -> V8.0 Download
6ES7823-1FE06-0YA7	SIMATIC S7-PLCSIM Advanced V8.0 POC
6ES7823-1FE00-0YN5	SIMATIC S7-PLCSIM Advanced Subscription Download

The articles each contain one license, which is valid for two instances.

NOTE

You can obtain online readme updates to the S7-PLCSIM manuals on the Internet (<https://support.industry.siemens.com/cs/us/en/view/109824282>).

You can find the following application examples for S7-PLCSIM Advanced on the Internet:

- SIMATIC S7-PLCSIM Advanced: Co-Simulation via API
(<https://support.industry.siemens.com/cs/ww/de/view/109739660/en>)
- Digitalization with TIA Portal: Virtual commissioning with SIMATIC and Simulink
(<https://support.industry.siemens.com/cs/ww/en/document/109749187>)

Recycling and disposal

For environmentally sustainable recycling and disposal of your old equipment, contact a certified electronic waste disposal service and dispose of the equipment according to the applicable regulations in your country.

Industry Mall

The Industry Mall (<https://mall.industry.siemens.com>) is the catalog and order system of Siemens AG for automation and drive solutions on the basis of Totally Integrated Automation (TIA) and Totally Integrated Power (TIP).

1.2 S7-PLCSIM Advanced documentation guide

The control panel buttons display the English S7-PLCSIM Advanced Function Manual and API Manual in PDF format. Download the free Adobe Reader if necessary to view PDFs. The installation folder of S7-PLCSIM Advanced includes German PDFs and English PDFs.

Other display formats are available at Siemens Industry Online Support (<https://support.industry.siemens.com>) (SIOS).

Accessing documentation on SIOS

Documentation for this product is available at Siemens Industry Online Support (<https://support.industry.siemens.com/>). Formats include the following for display or download:

- PDF document
- Multimedia document (HTML5 Web site)
- Website with chapter structure (useful for providing a specific topic link to other people)

From the Web site, search for your product name. Filter the search results by manuals if you are only looking for documentation. For your product, select the edition that corresponds to the version of your product. You can switch the display language for the page and the documents from the menu.

The various document formats provide navigation and search features based on their documentation format.

Navigating an HTML5 site

When you open an HTML5 Web site, click Begin from the Start page.

The table of contents displays on the left. Each chapter collapses as you navigate from one chapter to the next to optimize screen space on devices such as phones and tablets. You can also click the  button to hide the table of contents. Click  to restore it.

Left and right arrow buttons allow to navigate back and forward through topics that you have previously viewed:  

Previous and next page buttons allow you to navigate sequentially through topics and chapters in the order of the table of contents:  

Searching in an HTML5 site

To search an HTML5 Web site, click the  button on the right. When you enter your search text, you see a list of topics that contain the search text. Click any topic to view it. The page highlights occurrences of the matched search text.

To see where the topic exists in the table of contents, click the  button to switch from the search view to the table of contents view. To switch back to your search, click  again. The search functions much the same as any search from a Web browser. Ctrl+F searches within the displayed topic.

Changing the language of the HTML5 site

The S7-PLCSIM Advanced HTML5 documentation includes German and English. Click the  button to switch to another language.



The documentation for the SIMATIC S7-1500 automation system, for the 1513/1516pro-2 PN, SIMATIC Drive Controller CPUs based on SIMATIC S7-1500 and the SIMATIC ET 200MP, ET 200SP, ET 200AL and ET 200eco PN distributed I/O systems is arranged into three areas.

This arrangement enables you to access the specific content you require.

You can download the documentation free of charge from the Internet

(<https://support.industry.siemens.com/cs/ww/en/view/109742705>).

Basic information



The system manuals and Getting Started describe in detail the configuration, installation, wiring and commissioning of the SIMATIC S7-1500, SIMATIC Drive Controller, ET 200MP, ET 200SP, ET 200AL and ET 200eco PN systems. Use the corresponding operating instructions for 1513/1516pro-2 PN CPUs.

The STEP 7 online help supports you in the configuration and programming.

Examples:

- Getting Started S7-1500
- System manuals
- Operating instructions ET 200pro and 1516pro-2 PN CPU
- Online help TIA Portal

Device information



Equipment manuals contain a compact description of the module-specific information, such as properties, wiring diagrams, characteristics and technical specifications.

Examples:

- Equipment manuals for CPUs
- Equipment manuals for interface modules
- Equipment manuals for digital modules
- Equipment manuals for analog modules
- Equipment manuals for communication modules
- Equipment manuals for technology modules
- Equipment manuals for power supply modules
- Equipment manuals for BaseUnits

General information



The function manuals contain detailed descriptions on general topics relating to the SIMATIC Drive Controller and the S7-1500 automation system.

Examples:

- Function Manual Diagnostics
- Function Manual Communication
- Function Manuals Motion Control
- Function Manual Web Server
- Function Manual Cycle and Response Times
- PROFINET Function Manual
- PROFIBUS Function Manual

Product Information

Changes and supplements to the manuals are documented in a Product Information. The Product Information takes precedence over the device and system manuals.

You will find the latest Product Information on the Internet:

- S7-1500/ET 200MP (<https://support.industry.siemens.com/cs/de/en/view/68052815>)
- SIMATIC Drive Controller (<https://support.industry.siemens.com/cs/de/en/view/109772684/en>)
- Motion Control (<https://support.industry.siemens.com/cs/de/en/view/109794046/en>)
- ET 200SP (<https://support.industry.siemens.com/cs/de/en/view/73021864>)
- ET 200eco PN (<https://support.industry.siemens.com/cs/ww/en/view/109765611>)

Manual Collections

The Manual Collections contain the complete documentation of the systems put together in one file.

You will find the Manual Collections on the Internet:

- S7-1500/ET 200MP/SIMATIC Drive Controller (<https://support.industry.siemens.com/cs/ww/en/view/86140384>)
- ET 200SP (<https://support.industry.siemens.com/cs/ww/en/view/84133942>)
- ET 200AL (<https://support.industry.siemens.com/cs/ww/en/view/95242965>)
- ET 200eco PN (<https://support.industry.siemens.com/cs/ww/en/view/109781058>)

1.3 SIMATIC Technical Documentation

Additional SIMATIC documents will complete your information. You can find these documents and their use at the following links and QR codes.

The Industry Online Support gives you the option to get information on all topics. Application examples support you in solving your automation tasks.

Overview of the SIMATIC Technical Documentation

Here you will find an overview of the SIMATIC documentation available in Siemens Industry Online Support:



Industry Online Support International
<https://support.industry.siemens.com/cs/ww/en/view/109742705>

Watch this short video to find out where you can find the overview directly in Siemens Industry Online Support and how to use Siemens Industry Online Support on your mobile device:



Quick introduction to the technical documentation of automation products per video
<https://support.industry.siemens.com/cs/us/en/view/109780491>



YouTube video: Siemens Automation Products - Technical Documentation at a Glance
<https://youtu.be/TwLSxxRQQA>

mySupport

With "mySupport" you can get the most out of your Industry Online Support.

Registration	You must register once to use the full functionality of "mySupport". After registration, you can create filters, favorites and tabs in your personal workspace.
Support requests	Your data is already filled out in support requests, and you can get an overview of your current requests at any time.
Documentation	In the Documentation area you can build your personal library.
Favorites	You can use the "Add to mySupport favorites" to flag especially interesting or frequently needed content. Under "Favorites", you will find a list of your flagged entries.
Recently viewed articles	The most recently viewed pages in mySupport are available under "Recently viewed articles".
CAX data	The CAX data area gives you access to the latest product data for your CAX or CAE system. You configure your own download package with a few clicks: • Product images, 2D dimension drawings, 3D models, internal circuit diagrams, EPLAN macro files • Manuals, characteristics, operating manuals, certificates • Product master data

You can find "mySupport" on the Internet. (<https://support.industry.siemens.com/My/ww/en>)

Application examples

The application examples support you with various tools and examples for solving your automation tasks. Solutions are shown in interplay with multiple components in the system - separated from the focus on individual products.

You can find the application examples on the Internet.
(<https://support.industry.siemens.com/cs/ww/en/ps/ae>)

1.4 Tool support

Tools

The tools described below support you in all steps: from planning, over commissioning, all the way to analysis of your system.

TIA Selection Tool

The TIA Selection Tool tool supports you in the selection, configuration, and ordering of devices for Totally Integrated Automation (TIA).

As successor of the SIMATIC Selection Tools, the TIA Selection Tool assembles the already known configurators for automation technology into a single tool.

With the TIA Selection Tool, you can generate a complete order list from your product selection or product configuration.

You can find the TIA Selection Tool on the Internet.
(<https://support.industry.siemens.com/cs/us/en/view/109767888>)

SINETPLAN

SINETPLAN, the Siemens Network Planner, supports you in planning automation systems and networks based on PROFINET. The tool facilitates professional and predictive dimensioning of your PROFINET installation as early as in the planning stage. In addition, SINETPLAN supports you during network optimization and helps you to exploit network resources optimally and to plan reserves. This helps to prevent problems in commissioning or failures during productive operation even in advance of a planned operation. This increases the availability of the production plant and helps improve operational safety.

The advantages at a glance

- Network optimization thanks to port-specific calculation of the network load
- Increased production availability thanks to online scan and verification of existing systems
- Transparency before commissioning through importing and simulation of existing STEP 7 projects
- Efficiency through securing existing investments in the long term and the optimal use of resources

You can find SINETPLAN on the Internet

(<https://new.siemens.com/global/en/products/automation/industrial-communication/profinet/sinetplan.html>).

1.5 Administrator rights

NOTE

Administrator rights

For normal operation, the standard user rights are sufficient. You need administrator rights only when you install or repair S7-PLCSIM Advanced or change the network configuration.

Industrial cybersecurity

2.1 Introduction

Due to the digitalization and increasing networking of machines and industrial plants, the risk of cyber attacks is also growing. Appropriate protective measures are therefore mandatory, particularly in the case of critical infrastructure facilities.

Refer to the relevant system manual of the devices used for general information and measures regarding industrial cybersecurity.

2.1.1 Cybersecurity information

Siemens provides products and solutions with industrial cybersecurity functions that support the secure operation of plants, systems, machines and networks.

In order to protect plants, systems, machines and networks against cyber threats, it is necessary to implement – and continuously maintain – a holistic, state-of-the-art industrial cybersecurity concept. Siemens' products and solutions constitute one element of such a concept.

Customers are responsible for preventing unauthorized access to their plants, systems, machines and networks. Such systems, machines and components should only be connected to an enterprise network or the internet if and to the extent such a connection is necessary and only when appropriate security measures (e.g. firewalls and/or network segmentation) are in place.

For additional information on industrial cybersecurity measures that may be implemented, please visit

<https://www.siemens.com/cybersecurity-industry>.

Siemens' products and solutions undergo continuous development to make them more secure. Siemens strongly recommends that product updates are applied as soon as they are available and that the latest product versions are used. Use of product versions that are no longer supported, and failure to apply the latest updates may increase customer's exposure to cyber threats.

To stay informed about product updates, subscribe to the Siemens Industrial Cybersecurity RSS Feed under

<https://new.siemens.com/cert>.

2.2 Basic information on industrial cybersecurity

2.2.1 Definition of industrial cybersecurity

Industrial cybersecurity is generally understood to mean all measures to protect against the following threats:

- Loss of confidentiality due to unauthorized access to data
- Loss of integrity due to manipulation of data
- Loss of availability (for example, due to the destruction of data or denial of service (DoS))

2.2.2 Objectives of industrial cybersecurity

The objectives of industrial cybersecurity are:

- Ensuring trouble-free operation of industrial plants and production processes
- Preventing threats to people and production from cybersecurity attacks
- Protection of industrial plants against espionage and manipulation
- Protection of industrial automation systems and components against unauthorized access and data loss
- Provision of a practical and cost-effective concept for securing existing plants and devices that do not have their own security functions
- Use of existing, open, and proven industrial cybersecurity standards
- Compliance with legal requirements

An optimized and adapted security concept applies to automation and drive technology. The security measures must not impede or endanger production.

2.3 Operational application environment and security assumptions

2.3.1 Intended use

SIMATIC products are intended for use in industry. If you plan to use the product in a different environment, check the conditions required for such use.

The product may be operated only by personnel qualified for the specific task in accordance with the relevant documentation, in particular its warning notices and safety information. Qualified personnel are those who, based on their training and experience, are capable of identifying risks and avoiding potential hazards when working with these products.

2.3.2 Requirements for the operational application environment and security assumptions

Siemens recommends the following security measures:

- Threat and Risk Assessments (as part of security management)
- Network security concepts
 - Network segmentation
 - Asset and network management
 - Network protection
 - Remote access
- Access control concepts (utilizing access control systems)
 - Physical protection
 - Physical corporate security
 - Physical product security

Threat and Risk Assessment

Vulnerabilities and risks are identified, and countermeasures are proposed to ensure the security of the system, networks, and data.

Network security concepts

You can find information on network security in the whitepaper "Industrial Network Security Architecture", available at the Download Center (<https://www.siemens.com/us/en/company/topic-areas/cybersecurity/industrial-security/downloads.html>) on the Industrial Cybersecurity (<https://www.siemens.com/us/en/company/topic-areas/cybersecurity/industrial-security.html>) website.

Access control concepts

Physical protection

In addition to closing off and/or monitoring entire production facilities, it may be necessary to physically secure cabinets or even individual components such as circuit breakers.

Physical corporate security

You can ensure physical corporate security by the following measures, among others:

- Closed off and monitored company premises
- Access control, locks/card readers, and/or security personnel
- Accompaniment of non-employees by company personnel
- Employees are trained on and embrace security processes within the company

Physical production security

You can ensure the physical production security by, among others, the following measures:

- Separate access control for critical areas, such as production zones.
- Installation of critical components in lockable cabinets/control rooms with monitoring and alarm capabilities. The cabinets/control rooms must be secured with a cylinder lock. Do not use simple locks, such as universal, triangular/square, or double-bit locks.
- Radio field planning to limit WLAN coverage areas, preventing them from extending beyond defined zones (e.g. factory floor).
- Guidelines that prohibit the use of external data storage media (such as USB flash drives) and IT devices (such as laptops) classified as unsafe on systems.

2.4 Secure operation

This section describes measures that Siemens recommends to protect against manipulation and unauthorized access.

Guidelines

Observe the following guidelines for secure operation:

- Use only the original version as provided on the Internet and follow the Installation notes.
- Check regularly whether there are new versions available for download and use the latest version.

Dependencies on other software components

Verify that your computer meets the System requirements.

2.4.1 Security update notification

Read the special information on maintaining the operational safety of your plant in the Introduction (Page 16) section.

Set up notification of security updates

To receive notifications about security updates, proceed as follows:

1. Register with mySiePortal (<https://sieportal.siemens.com/en-ww/mysieportal>).
2. Enter the keyword "Security" in the search engine.
3. Choose the "Search in Knowledge base" option.
4. Enter the name or article number of the product in the product search.
5. Select the "Other types" option from the filter menu for "Type," and then choose "Download" and "Product note".

6. For the document you want to be informed about, select "Add to my favorites" using the 3 dots on the right.

For example, if you want to receive notifications about all firmware versions of the S7-1500 CPUs, select the entry "Firmware update S7-1500 CPUs incl. Displays and ET 200 CPUs (ET 200SP, ET 200pro)".

7. In the following dialog, enter a name for the favorite, select the storage location and select the "Notify me" option. Then click the "+ Add to my favorites" button.

Result: You will be notified by email each time the document is changed.

Under "mySiePortal" > "Lists & notifications" > "My notifications", you can display your notifications and delete them if necessary.

2.4.2 Hardening measures

System hardening, also simply referred to as hardening, is the secure configuration of products or systems. The aim is to close security gaps and take various measures to reduce the attack surfaces for cyberattacks.

Measures for system hardening include, for example:

- Secure configuration in which only necessary software components and services are installed or activated for proper operation.
- Access control, by which a restrictive user and rights management system is implemented.

2.4.3 Access control

In addition to physical protection, also establish logical safeguards to control access to your system:

- Use a restrictive user and rights management system, for example, for accessing the CPUs and TIA Portal.
- Refer to the information about password management in the section Users and roles (https://docs.tia.siemens.cloud/access?ft:originId=./100365717643/100365717643_enUS.htm&ft:clusterId./100365717643/100365717643.htm) and in the STEP 7 online help (TIA Portal).

2.4.4 Notifications about security vulnerabilities (Siemens Security Advisories)

A vulnerability is a security vulnerability in information security. A vulnerability can pose a threat as it provides intruders with the opportunity to access system resources and manipulate or steal data. Many vulnerabilities allow availability to be impaired.

Siemens ProductCERT

If Siemens identifies or fixes Security Vulnerabilities in the products, this will be published in the Security Advisories.

You can find the documents for SIMATIC on the following website of the Siemens Aktiengesellschaft: Siemens ProductCERT and Siemens CERT (<https://new.siemens.com/global/en/products/services/cert.html?s=SIMATIC#SecurityPublications>)

"Search Security Advisories" is the default entry in the "SIMATIC" search field. You can also enter other product names or other terms in the search field and search for them.

On this page, you will also find all the required information on how to deal with vulnerabilities:

- Contact persons for matters related to vulnerabilities
- Options for automated notifications regarding vulnerabilities
- Notifications are also possible in CSAF format
- Option to subscribe to RSS feeds and newsletters
- List of all current vulnerabilities and detailed information such as:
 - Description
 - Classification according to Common Vulnerability Scoring System (CVSS)
 - Measures
 - Availability
 - Etc.

Set up Security feeds (<https://www.siemens.com/cert>) to receive notifications about security-related topics.

If you suspect or have discovered a vulnerability in a Siemens product, please let us know immediately. To do this, press the "Contact" button on the CERT Services page (<https://www.siemens.com/cert>) and follow the instructions.

2.4.5 Data backup

Secure your configuration and parameter settings so that you can quickly restore this data if needed.

2.4.6 Secure uninstallation

Use the Windows control panel to uninstall the product and follow the dialog prompts.

Product overview

3.1 What is S7-PLCSIM Advanced?

You use S7-PLCSIM Advanced to simulate your CPU programs on a simulated PLC instance. You do not need any real controllers for simulation. You configure your CPU with STEP 7 in TIA Portal, program your application logic and then load the hardware configuration and the program into the simulated CPU. From STEP 7, you run your program logic, monitor the effects of simulated inputs and outputs, and adapt your programs.

In addition to communication via Softbus, which limits communication to a local PC or virtual machine, S7-PLCSIM Advanced offers a full Ethernet connection and thus also supports distributed I/O systems.

S7-PLCSIM Advanced also includes an application programming interface (API), which enables interaction with native C++/C# programs or simulation software. The API extends simulation capabilities and provides features such as the following:

- Simulation of hardware interrupts
- Stopping a simulation
- Synchronization with simulation partners

For these and other API functions, refer to the S7-PLCSIM Advanced API manual included with your release.

Application areas

Typical application areas of S7-PLCSIM Advanced are:

- Verification of the user program (TIA Portal)
- Testing of the STEP 7 program
- Software in the loop simulation for the virtual commissioning of machine tools/production machines, production cells, and production lines in a plant.
- Operator training through the connection of a real HMI

Advantages

S7-PLCSIM Advanced offers numerous advantages:

- Improve the quality of automation projects by early error detection
- Avoid costs for hardware in simulation environments
- Shorten commissioning time
- Reduce risk for commissioning
- Train operators earlier
- Increase production efficiency by optimizing program components

- Increase efficiency during replacement of machine components
- Increase efficiency during expansion of existing plants

3.2 Functions for licenses

API functions for licenses

S7-PLCSIM Advanced regularly checks whether a license is available. The following return values provide information about the status (for example, for C++):

- Return values for API function `PowerOn()` and callback function `OnOperatingStateChanged`
 - `SREC_OK` when a floating license is available.
 - `SREC_WARNING_TRIAL_MODE_ACTIVE` when an instance is started with the Trial License.
 - `SREC_WARNING_RUNNING_ON_TIA_PORTAL_TEST_SUITE` when no valid license for S7-PLCSIM Advanced is available, but a "TIA Portal Test Suite" license is available.

S7-PLCSIM Advanced starts with this license. A download from the TIA Portal is possible, but the instance terminates without feedback if the download was not made from the TIA Portal Test Suite.

- `SREC_NOT_EMPTY` when no valid license for S7-PLCSIM Advanced is available, but a "TIA Portal Test Suite" license is available.

If this is the case, power-up from the Virtual SIMATIC Memory Card is not supported.

- Return value for callback function `OnOperatingStateChanged`
 - `SREC_LICENSE_NOT_FOUND` when the instance is automatically shut down after 21 days.

Components of the Simulation Runtime

The following components are relevant for handling the Simulation Runtime of S7-PLCSIM Advanced:

Table 3-1 Components of the Simulation Runtime

Components	Description
• "Siemens.Simatic.Simulation.Runtime.Manager.exe"	A Windows process that runs in the background. Main component of Runtime that manages all other Runtime components. The process is started automatically as soon as an application attempts to initialize the Runtime API. It is ended automatically as soon as there is no longer any application running that initialized the Runtime API.
• "Siemens.Simatic.Simulation.Runtime.Instance.exe"	The process of the instance that loads a DLL of a simulated PLC instance. Each simulated PLC instance generates its own process.
• "Siemens.Simatic.Simulation.Runtime.Api.x86.dll" • "Siemens.Simatic.Simulation.Runtime.Api.x64.dll"	API libraries that must load an application to use the Simulation Runtime. The libraries contain interfaces for native code and managed code. The "Runtime.Api.x86.dll" is loaded exclusively by 32-bit applications, and the "Runtime.Api.x64.dll" by 64-bit applications.

Components	Description
• "SimulationRuntimeApi.h"	Header file that describes all data types that require a native C++ application to use the API library.
• "Siemens.Simatic.PlcSim.Vplc1500.ODKClient.so.exe"	ODK client process for a CPU function library (original Shared Object)
• "Siemens.Simatic.PlcSim.Vplc1500.ODKClient.x86.exe"	ODK client process for a 32-bit application
• "Siemens.Simatic.PlcSim.Vplc1500.ODKClient.x64.exe"	ODK client process for a 64-bit application

External applications and Simulation Runtime

The following figure schematically presents the access of external applications to Simulation Runtime via the Runtime API. The Simulation Runtime Manager manages the Runtime instances. These load the libraries of the simulated PLC instances.

An external application can be, for example, another simulation software or a graphical user interface (GUI).

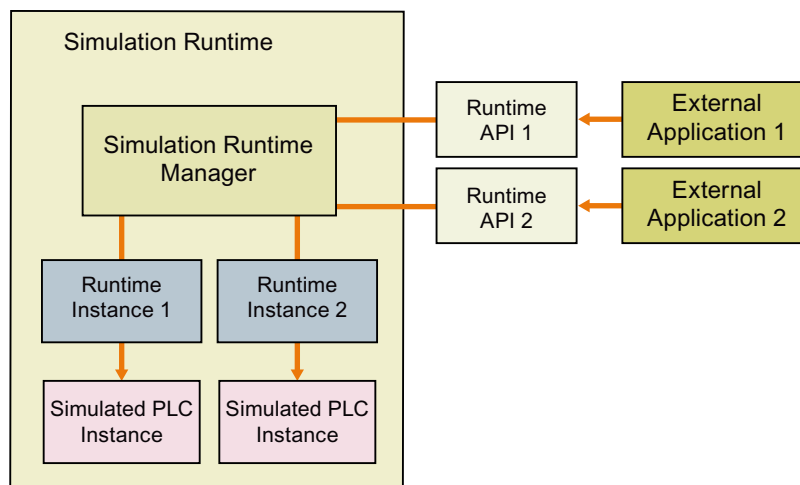


Figure 3-1 External applications and Simulation Runtime

3.3 System requirements

For S7-PLCSIM Advanced to operate efficiently, the following minimum requirements for computer hardware or for a virtual machine must be met.

	Hardware
Processor	- A modern multi-core processor with sufficient performance - One core per simulation instance started - At least one additional core for the operating system - At least one additional core for extra active applications For virtual machines: A corresponding number of processors must be physically available on the host.
RAM	16 GB minimum Memory requirements increase with the number of instances and the complexity of the simulation
Free hard disk space	20 GB
Screen resolution	Minimum 1920 x 1080 (Full HD)

Operating systems and virtualization platforms

S7-PLCSIM Advanced supports the same operating systems, virtualization, and hard disk encryption technology as the corresponding STEP 7 Basic version, with the exception of the Windows Home operating systems. S7-PLCSIM does not support the Windows Home operating systems. See the TIA Portal Information System for a full list of supported technologies.

Refer to online information (<https://support.industry.siemens.com/cs/ww/en/view/78788417>) for installing STEP 7 (TIA Portal) and S7-PLCSIM Advanced on a virtual machine.

3.4 Restrictions due to antivirus programs

NOTICE
Restrictions due to virus scanners and Advanced Threat Protection software (ATP software) Virus scanners and ATP software that monitor the behavior of processes and communication can have a significant influence on the performance of the runtime and communication of S7-PLCSIM Advanced and even prevent S7-PLCSIM Advanced instances from starting.

Solution

You can decrease restrictions during installation and runtime of S7-PLCSIM Advanced. To do so, define exceptions for the virus scanner for secure files and folders. Add the following folders to the exceptions:

- "C:\Program Files\Common Files\Siemens\PLCSIMADV\Drivers"
- "C:\Program Files (x86)\Common Files\Siemens\PLCSIMADV"
- "C:\Program Files (x86)\SIEMENS\Automation\PLCSIMADV\bin"

If the virus scanner only allows files as exceptions, add all files from the listed folders to the exceptions. The procedure is described in the instructions of the respective manufacturer.

NOTE

Refer to the latest readme (<https://support.industry.siemens.com/cs/us/en/view/109824282>) on the Siemens Industry Online Support site for updates. Select the Edition that corresponds to your release version.

Virus scanner and security software support

Refer to "General software and hardware requirements" in the TIA Portal information system for the supported virus scanners and security software.

Siemens recommends Windows Defender that is part of the Windows operating system.

Symantec Endpoint Protection 14.3

Note the following restriction for Symantec Endpoint Protection 14.3 when using TCP/IP Single Adapter or TCP/IP Multiple Adapter:

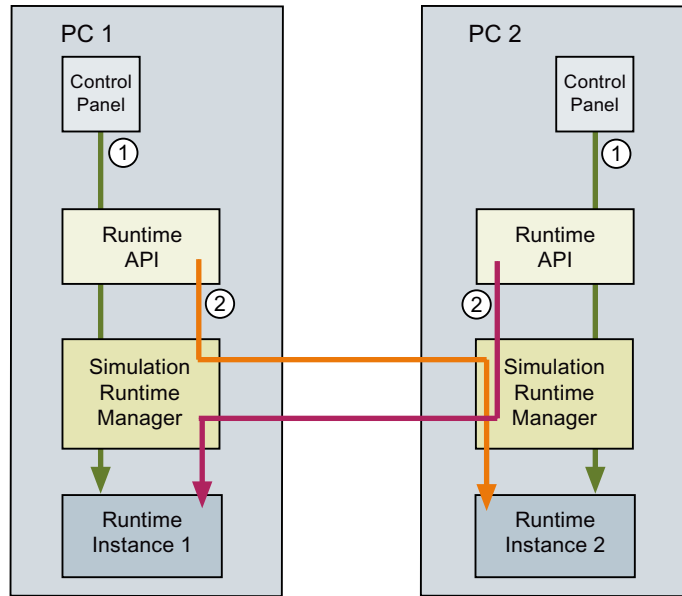
If you use Npcap with Symantec Endpoint Protection, select one of the following options:

- Uninstall Symantec Endpoint Protection.
- Use Symantec Endpoint Protection with Npcap V1.60 or earlier.

3.5 Access to instances

Access via the Control Panel and the API

You can access a local instance on the PC via the Control Panel. It does not matter on which PC an instance was created and started. With distributed communication, the Runtime API accesses the instance of the other PCs via the Simulation Runtime Manager.



- ① Access to a local instance via the Control Panel
- ② Access to a remote instance on the Runtime API

Figure 3-2 Access to instances with distributed communication

API functions

The following interfaces are available for native C++ [\(Page 29\)](#) and managed code [\(Page 33\)](#):

- Overview of interfaces for native C++ [\(Page 29\)](#)
- Overview of interfaces for managed code [\(Page 33\)](#)

3.6 Interfaces of the Simulation Runtime

The user interfaces of the Simulation Runtime include functions you use for tasks such as the following:

- Creating instances
- Changing the operating mode of a simulated PLC instance
- Exchanging I/O data

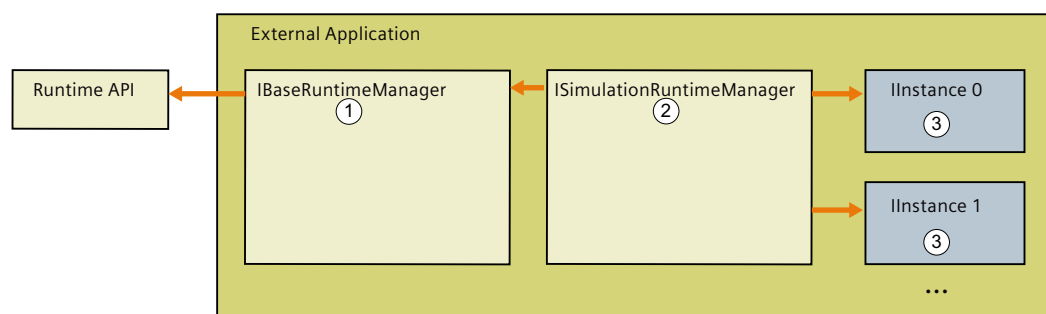
Simulation Runtime has the following user interfaces:

- IBaseRuntimeManager
- ISimulationRuntimeManager
- IInstances
- IRemoteRuntimeManager

ISimulationRuntimeManager and IRemoteRuntimeManager are derived from IBaseRuntimeManager.

API and external applications

The Runtime API makes the interfaces available to an external application.



- ① **IBaseRuntimeManager**
Base interface of the Runtime Manager. It is used to register new Runtime instances, to search through existing Runtime instances, and to receive an interface of a registered instance. Up to 16 instances can be registered in one Runtime Manager.
- ② **ISimulationRuntimeManager**
Derived from IBaseRuntimeManager. It is used to handle events, manage connections with other Runtime Managers, and check API and Runtime Manager status.
- ③ **IInstances**
Interface of a Runtime instance. It is used to change the operating state of a simulated PLC instance and to exchange I/O data. Each instance has a unique name and an ID.

Figure 3-3 API and external applications

Access to API functions and data types

Required functions and data types are available for native C++ and .NET (C#).

- Overview of interfaces for native C++ [\(Page 29\)](#)
- Overview of data types for native C++ [\(Page 36\)](#)
- Overview of interfaces for managed code [\(Page 33\)](#)
- Overview of data types for managed code [\(Page 38\)](#)

NOTE

The list of tables in this manual gives you direct access to the description of the individual functions and data types.

Transfer parameters for API functions

All API functions that return a value using the function parameters expect a user-allocated memory area as a transfer parameter. Null pointers are not permitted. Exceptions to this are the functions that return an interface of a simulated PLC instance:

- An IBaseRuntimeManager interface
- An ISimulationRuntimeManager interface
- An IRemoteRuntimeManager interface
- An IInstance interface

3.7 Overview of interfaces for native C++

Initializing and shutting down API

Table 3-2 Overview of initializing and shutting down API - Native C++

Actions	Functions
Initialize API (Page 40)	InitializeApi RuntimeApiEntry_Initialize
Shut down API (Page 44)	DestroyInterface RuntimeApiEntry_DestroyInterface
Logging off API library (Page 46)	FreeApi ShutdownAndFreeApi

Global functions

Table 3-3 Overview of global functions - Native C++

Actions	Functions
Global functions (Page 49)	GetNameOfAreaSection() GetNameOfCPUType() GetNameOfCommunicationInterface() GetNameOfDataType() GetNameOfLEDMode() GetNameOfLEDType() GetNameOfOperatingMode() GetNameOfOperatingState() GetNameOfPrimitiveDataType() GetNameOfTagListDetails() GetNameOfErrorCode() GetNameOfRuntimeConfigChanged() GetNameOfInstanceConfigChanged() GetNameOfDiagSeverity() GetNameOfDirection() GetNameOfRackOrStationFaultType() GetNameOfProcessEvent() GetNameOfPullOrPlugEventType() GetNameOfCycleTimeMonitoringMode() GetNameOfDiagProperty() GetNameOfAutodiscoverType() GetNameOfSoftwareConfigChanged() GetNameOfNetworkMode()

API IBaseRuntimeManager

Table 3-4 Overview of IBaseRuntimeManager functions - Native C++

Settings	Functions
Interface (Page 60)	GetVersion() GetPort() GetStrictMotionTiming() SetStrictMotionTiming() GetNetworkMode() SetNetworkMode() GetNetInterfaces() SetNetInterfaceBindings() ResetNetInterfaceBindings() GetDefaultStoragePath() SetDefaultStoragePath()
Simulation Runtime instances (Page 54)	GetRegisteredInstancesCount() GetRegisteredInstanceInfoAt() RegisterInstance() RegisterCustomInstance() CreateInterface()

API ISimulationRuntimeManager

Table 3-5 Overview of API ISimulationRuntimeManager functions - Native C++

Settings	Functions
Interface (Page 80)	IsInitialized() IsRuntimeManagerAvailable() Shutdown()
Remote connections (Page 68)	OpenPort() ClosePort() GetRemoteConnectionsCount() GetRemoteConnectionInfoAt() RemoteConnect() RunAutodiscover()

Table 3-6 Overview of API ISimulationRuntimeManager events - Native C++

Events	Functions
OnConfigurationChanged (Page 75)	RegisterOnConfigurationChangedCallback() UnregisterOnConfigurationChangedCallback() RegisterOnConfigurationChangedEvent() UnregisterOnConfigurationChangedEvent() WaitForOnConfigurationChangedEvent()
OnRuntimeManagerLost (Page 77)	RegisterOnRuntimeManagerLostCallback() UnregisterOnRuntimeManagerLostCallback() RegisterOnRuntimeManagerLostEvent() UnregisterOnRuntimeManagerLostEvent() WaitForOnRuntimeManagerLostEvent()
OnAutodiscover (Page 80)	RegisterOnAutodiscoverCallback() UnregisterOnAutodiscoverCallback()

API Instance

Table 3-7 Overview of Instance functions - Native C++

Settings	Functions
Interface (Page 82)	GetID() GetName() GetCPUType() SetCPUType() GetCommunicationInterface() SetCommunicationInterface() GetInfo() UnregisterInstance() GetStrictMotionTiming() SetStrictMotionTiming()
Controller (Page 90)	GetControllerName() GetControllerShortDesignation() GetControllerIPCount() GetControllerIP() GetControllerIPSuite4() SetIPSuite() GetStoragePath() SetStoragePath() ArchiveStorage() RetrieveStorage() CleanupStoragePath()
Operating state (Page 100)	PowerOn() PowerOff() Run() Stop() GetOperatingState() MemoryReset()
Tag list (Page 112)	UpdateTagList() GetTagListStatus() GetTagInfoCount() GetTagInfos() CreateConfigurationFile()
I/O access via address - Reading (Page 118)	GetAreaSize() ReadBit() ReadByte() ReadBytes() ReadSignals()
I/O access via address - Writing (Page 123)	WriteBit() WriteByte() WriteBytes() WriteSignals()
I/O access via tag name - Reading (Page 128)	Read() ReadBool() ReadChar(), ReadWChar() ReadDouble() ReadFloat() ReadInt8(), ReadInt16(), ReadInt32(), ReadInt64() ReadUInt8(), ReadUInt16(), ReadUInt32(), ReadUInt64() ReadSignals() ReadString() ReadWString()
I/O access via tag name - Writing (Page 150)	Write() WriteBool() WriteChar(), WriteWChar() WriteDouble() WriteFloat() WriteInt8(), WriteInt16(), WriteInt32(), WriteInt64(), WriteUInt8(), WriteUInt16(), WriteUInt32(), WriteUInt64() WriteSignals() WriteString() WriteWString()

3.7 Overview of interfaces for native C++

Settings	Functions
Virtual time (Page 172)	GetSystemTime() SetSystemTime() GetScaleFactor() SetScaleFactor()
Cycle control (Page 174)	GetOperatingMode() SetOperatingMode() SetSendSyncEventInDefaultModeEnabled() IsSendSyncEventInDefaultModeEnabled() GetOverwrittenMinimalCycleTime_ns() SetOverwrittenMinimalCycleTime_ns() RunToNextSyncPoint() StartProcessing() SetCycleTimeMonitoringMode() GetCycleTimeMonitoringMode()
Acyclic services (Page 190)	

Table 3-8 Overview of Instance events - Native C++

Events	Functions
OnOperatingStateChanged (Page 210)	RegisterOnOperatingStateChangedCallback() UnregisterOnOperatingStateChangedCallback() RegisterOnOperatingStateChangedEvent() UnregisterOnOperatingStateChangedEvent() WaitForOnOperatingStateChangedEvent()
OnLedChanged (Page 213)	RegisterOnLedChangedCallback() UnregisterOnLedChangedCallback() RegisterOnLedChangedEvent() UnregisterOnLedChangedEvent() WaitForOnLedChangedEvent()
OnSoftwareConfigurationChanged (Page 216)	RegisterOnSoftwareConfigurationChangedCallback() UnregisterOnSoftwareConfigurationChangedCallback() RegisterOnSoftwareConfigurationChangedEvent() UnregisterOnSoftwareConfigurationChangedEvent() WaitForOnSoftwareConfigurationChangedEvent()
OnIPAddressChanged (Page 218)	RegisterOnIPAddressChangedCallback() UnregisterOnIPAddressChangedCallback() RegisterOnIPAddressChangedEvent() UnregisterOnIPAddressChangedEvent() WaitForOnIPAddressChangedEvent()
OnSyncPointReached (Page 220)	RegisterOnSyncPointReachedCallback() UnregisterOnSyncPointReachedCallback() RegisterOnSyncPointReachedEvent() UnregisterOnSyncPointReachedEvent() WaitForOnSyncPointReachedEvent()
OnHardwareConfigChanged (Page 222)	RegisterOnHardwareConfigChangedCallback() UnregisterOnHardwareConfigChangedCallback() RegisterOnHardwareConfigChangedEvent() UnregisterOnHardwareConfigChangedEvent() WaitForOnHardwareConfigChangedEvent()

API IRemoteRuntimeManager

Table 3-9 Overview of IRemoteRuntimeManager functions - Native C++

Settings	Functions
Interface (Page 234)	GetIP() GetRemoteComputerName() Disconnect()

Table 3-10 Overview of IRemoteRuntimeManager events - Native C++

Events	Functions
OnConnectionLost (Page 236)	RegisterOnConnectionLostCallback() UnregisterOnConnectionLostCallback() RegisterOnConnectionLostEvent() UnregisterOnConnectionLostEvent() WaitForOnConnectionLostEvent()

3.8 Overview of interfaces for managed code

Initializing and shutting down API

Table 3-11 Overview of initializing and shutting down API - .NET (C#)

Actions	Functions
Initialize API (Page 43)	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeManager
Shut down API (Page 48)	

API IBaseRuntimeManager

Table 3-12 Overview of IBaseRuntimeManager functions - .NET (C#)

Settings	Functions
Interface (Page 60)	Version { get; } Port { get; } StrictMotionTiming { get; set; } NetworkMode { get; set; } NetInterfaces { get; } SetNetInterfaceBindings(); ResetNetInterfaceBindings(); DefaultStoragePath { get; set; }
Simulation Runtime instances (Page 54)	RegisteredInstanceInfo { get; } RegisterInstance(); RegisterCustomInstance(); CreateInterface();

API ISimulationRuntimeManager

Table 3-13 Overview of ISimulationRuntimeManager functions - .NET (C#)

Settings	Functions
Interface (Page 80)	IsInitialized { get; } IsRuntimeManagerAvailable { get; } Shutdown() StrictMotionTiming { get; }
Remote connections (Page 68)	OpenPort() ClosePort() Port { get; } RemoteConnectionInfo { get; } RemoteConnect() RunAutodiscover()

3.8 Overview of interfaces for managed code

Table 3-14 Overview of ISimulationRuntimeManager events - .NET (C#)

Events	Functions
OnConfigurationChanged (Page 75)	OnConfigurationChanged() RegisterOnConfigurationChangedEvent() UnregisterOnConfigurationChangedEvent() WaitForOnConfigurationChangedEvent()
OnRuntimeManagerLost (Page 77)	OnRuntimeManagerLost() RegisterOnRuntimeManagerLostEvent() UnregisterOnRuntimeManagerLostEvent() WaitForOnRuntimeManagerLostEvent()
OnAutodiscover (Page 80)	OnAutodiscoverData()

API Instance

Table 3-15 Overview of IInstance functions - .NET (C#)

Settings	Functions
Interface (Page 82)	Dispose() ID { get; } Name { get; } CommunicationInterface { get; } CPUType { get; set; } Info { get; } UnregisterInstance() StrictMotionTiming { get; set; }
Controller - Information and settings (Page 90)	ControllerName { get; } ControllerShortDesignation { get; } ControllerIPSuite4 { get; } SetIPSuite() StoragePath { get; set; } ArchiveStorage() RetrieveStorage() CleanupStoragePath()
Operating state (Page 100)	PowerOn() PowerOff() Run() Stop() OperatingState { get; } MemoryReset()
Tag list (Page 112)	UpdateTagList() GetTagListStatus() TagInfos { get; } CreateConfigurationFile()
I/O access via address - Reading (Page 118)	InputArea MarkerArea OutputArea { get; } AreaSize { get; } ReadBit() ReadByte() ReadBytes() ReadSignals()
I/O access via address - Writing (Page 123)	WriteBit() WriteByte() WriteBytes() WriteSignals()

3.8 Overview of interfaces for managed code

Settings	Functions
I/O access via tag name - Reading (Page 128)	Read() ReadBool() ReadChar(), ReadWChar() ReadDouble() ReadFloat() ReadInt8(), ReadInt16(), ReadInt32(), ReadInt64() ReadUInt8(), ReadUInt16(), ReadUInt32(), ReadUInt64() ReadSignals() ReadString() ReadWString()
I/O access via tag name - Writing (Page 150)	Write() WriteBool() WriteChar(), WriteWChar() WriteDouble() WriteFloat() WriteInt8(), WriteInt16(), WriteInt32(), WriteInt64(), WriteUInt8(), WriteUInt16(), WriteUInt32(), WriteUInt64() WriteSignals() WriteString() WriteWString()
Virtual time (Page 172)	SystemTime { get; set; } ScaleFactor { get; set; }
Cycle control (Page 174)	OperatingMode { get; set; } IsSendSyncEventInDefaultModeEnabled { get; set; } OverwrittenMinimalCycleTime_ns { get; set; } RunToNextSyncPoint() StartProcessing() SetCycleTimeMonitoringMode() GetCycleTimeMonitoringMode()
Acyclic services (Page 190)	

Table 3-16 Overview of Instance events - .NET (C#)

Events	Functions
OnOperatingStateChanged (Page 210)	OnOperatingStateChanged() RegisterOnOperatingStateChangedEvent() UnregisterOnOperatingStateChangedEvent() WaitForOnOperatingStateChangedEvent()
OnLedChanged (Page 213)	OnLedChanged() RegisterOnLedChangedEvent() UnregisterOnLedChangedEvent() WaitForOnLedChangedEvent()
OnSoftwareConfigurationChanged (Page 216)	OnSoftwareConfigurationChanged() RegisterOnSoftwareConfigurationChangedEvent() UnregisterOnSoftwareConfigurationChangedEvent() WaitForOnSoftwareConfigurationChangedEvent()
OnIPAddressChanged (Page 218)	OnIPAddressChanged() RegisterOnIPAddressChangedEvent() UnregisterOnIPAddressChangedEvent() WaitForOnIPAddressChangedEvent()
OnSyncPointReached (Page 220)	OnSyncPointReached() RegisterOnSyncPointReachedEvent() UnregisterOnSyncPointReachedEvent() WaitForOnSyncPointReachedEvent()
OnHardwareConfigChanged (Page 222)	OnHardwareConfigChanged() RegisterOnHardwareConfigChangedEvent() UnregisterOnHardwareConfigChangedEvent() WaitForOnHardwareConfigChangedEvent()

3.10 Overview of data types for native C++

API IRemoteRuntimeManager

Table 3-17 Overview of IRemoteRuntimeManager functions - .NET (C#)

Settings	Functions
Interface (Page 234)	Dispose() IP { get; } RemoteComputerName { get; } Disconnect() StrictMotionTiming { get; }

Table 3-18 Overview IRemoteRuntimeManager events - .NET (C#)

Events	Functions
OnConnectionLost() (Page 236)	OnConnectionLost() RegisterOnConnectionLostEvent() UnregisterOnConnectionLostEvent() WaitForOnConnectionLostEvent()

3.9 Restrictions for file paths

The following restrictions apply for user interfaces which expect a path or a complete file name as the transfer parameter:

Restrictions for local paths	Write permissions to system-critical folders such as the Windows folder (%Windows%) or the program folders (%Program Files%, %Program Files (x86)%) are not allowed. In this case the C++ user interface returns the error code SREC_WRONG_ARGUMENT. In this case the managed user interface returns an exception with the error code <code>RuntimeErrorCode.WrongArgument</code> .
Restrictions for network paths	In order to be able to use network paths, you must incorporate them as a network drive. Otherwise, the C++ user interface returns the error code SREC_WRONG_ARGUMENT. The managed user interface returns an exception with the error code <code>RuntimeErrorCode.WrongArgument</code> .

3.10 Overview of data types for native C++

The following table shows which data types are available for the simulation in Runtime Manager.

Table 3-19 Overview of data types - Native C++

Data type	
DLL import functions (Page 239)	ApiEntry_Initialize ApiEntry_DestroyInterface
Event callback functions (Page 240)	EventCallback_VOID EventCallback_SRCC_UINT32_UINT32_INT32 EventCallback_SRSI_AD EventCallback_IRRTM EventCallback_II_SREC_ST_SROS_SROS EventCallback_II_SREC_ST_UINT32_INT64_INT64_UINT32 EventCallback_II_SREC_ST EventCallback_II_SREC_ST_SRLT_SRLM EventCallback_II_SREC_ST_SDRI EventCallback_II_SREC_ST_SDRI_BYTE EventCallback_II_SREC_ST_UINT32_UINT32 EventCallback_II_SREC_ST_UINT32_UINT32_EPET_UINT32 EventCallback_II_SREC_ST_UINT32_EPPET_UINT32 EventCallback_II_SREC_ST_UINT32_ERSFET

Data type	
	EventCallback_II_SREC_ST_UINT32 EventCallback_II_SS CCP
Definitions and constants (Page 255)	
Unions (Page 256)	UIP UDataValue
Structures (Page 258)	SDataValue SDVBNI SDataValueByAddress SDataValueByName SConnectionInfo SInstanceInfo SDimension STagInfo SIP SIPSuite4 SOnSyncPointReachedResult SDataValueByAddressWithCheck SDataValueByNameWithCheck SDataRecordInfo SDataRecord SConfiguredProcessEvents SdiagExtChannelDescription SAutodiscoverData SOnSoftwareConfigChangedParameter
Enumerations (Page 274)	ERuntimeErrorCode EArea ECommunicationInterface EOperatingState EOperatingMode ECPUType ENetworkMode EPLCInterface ELEDType ELEDMode EPrimitiveDataTypes EDataType ETagListDetails ERuntimeConfigChanged EPullOrPlugEventType EProcessEventTypes EDirection EDiagProperty EDiagSeverity ERackOrStationFaultType ECycleTimeMonitoringMode EAutodiscoverType

3.11 Overview of data types for managed code

The following table shows which data types are available for the simulation in Runtime Manager.

Table 3-20 Overview of data types - .NET (C#)

Data type	
Delegate definitions (Page 248) - Event handler methods	Delegate_Void Delegate_SRCC_UINT32_UINT32_INT32 Delegate_SRRSI_AD Delegate_IRRTM Delegate_II_EREC_DT_EOS_EOS Delegate_II_EREC_DT_UINT32_INT64_INT64_UINT32 Delegate_II_EREC_DT_ Delegate_II_EREC_DT_ELT_ELM Delegate_II_EREC_DT_SDR_I Delegate_II_EREC_DT_SDR Delegate_SREC_ST_UINT32_UINT32 Delegate_SREC_ST_UINT32_UINT32_EPET_UINT32 Delegate_SREC_ST_UINT32_EPPET_UINT32 Delegate_SREC_ST_UINT32_ERSFET Delegate_SREC_ST_UINT32_
Definitions and constants (Page 255)	
Structures (Page 258)	SDataValue SDVBNI SDataValueByAddress SDataValueByName SConnectionInfo SInstanceInfo SDimension STagInfo SIP SIPSuite4 SOnSyncPointReachedResult SDataValueByAddressWithCheck SDataValueByNameWithCheck SDataRecordInfo SDataRecord SConfiguredProcessEvents SDiagExtChannelDescription SAutodiscoverData
Enumerations (Page 274)	ERuntimeErrorCode EArea ECommunicationInterface EOperatingState EOperatingMode ECPUType ELEDType ELEDMode ENetworkMode EPrimitiveDataTypes EDataType ETagListDetails ERuntimeConfigChanged EPullOrPlugEventType EProcessEventTypes EDirection EDiagProperty EDiagSeverity ERackOrStationFaultType ECycleTimeMonitoringMode EAutodiscoverType

Initialize API

4.1 Load API library

Description

For S7-PLCSIM Advanced, the interfaces of the API V8.0 are not compatible with the interfaces of previous API versions. However, the Runtime Manager of S7-PLCSIM Advanced V8.0 is compatible with the API of previous S7-PLCSIM Advanced versions.

Earlier versions of the API are also installed during the installation of S7-PLCSIM Advanced V8.0.

The default path is C:\Program Files (x86)\Common Files\Siemens\PLCSIMADV\API

NOTE

API versions

To access and use the latest features and bug fixes, use the latest API version.

The installation path of S7-PLCSIM Advanced is contained in the registry:

- Key: "HKEY_LOCAL_MACHINE\SOFTWARE\Wow6432Node\Siemens\Shared Tools\PLCSIMADV_SimRT"
- Value: "Path"

To maintain the path to the API, add the character string of the following subdirectory at the end: "API\<API version>".

When you use this path the API library (DLL) is loaded directly from the installation directory.

Reference

Additional information can be found in:

- For Native C++ in section InitializeApi() ([Page 40](#)).
- For .NET via the call of the function "System.Reflection.Assembly.LoadFile(string)" in the online documentation for MSDN.

4.2 Native C++

4.2.1 InitializeApi()

Description

The `InitializeApi` function loads the API library (DLL) and initializes the API. The function loads the version of the DLL that is compatible with the architecture of your application and with the header file of the API ("SimulationRuntimeApi.h").

To load the DLL, the function `InitializeApi` searches in the following folders one after the other:

- Folder to which the parameter of the function leads
(`in_SimulationRuntimeApiDllPath`)
- Folder, which also contains your application that calls this function
- Installation directory of S7-PLCSIM Advanced

If no DLL is available, the function accesses the next folder.

The function returns an interface to the Simulation Runtime Manager. Use this interface to create a new simulated PLC instance or to obtain access to an existing instance.

Table 4-1 InitializeApi() - Native C++

Syntax	<pre>ERuntimeErrorCode InitializeApi(ISimulationRuntimeManager** out_SimulationRuntimeManagerInterface); ERuntimeErrorCode InitializeApi(WCHAR* in_SimulationRuntimeApiDllPath, ISimulationRuntimeManager** inOut_SimulationRuntimeManagerInterface);</pre>	
Parameters	• <code>ISimulationRuntimeManager** out_SimulationRuntimeManagerInterface</code>: Pointer to a Runtime Manager interface pointer. The pointer must be initialized with <code>NULL</code>. The interface is created within the function. See Data types (Page 239). • <code>WCHAR* in_SimulationRuntimeApiDllPath</code>: The path to the Runtime API library.	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_WRONG_ARGUMENT</code>	The pointer to the Runtime Manager interface does not equal <code>NULL</code> .
	<code>SREC_WRONG_VERSION</code>	• The required version of the interface is incompatible with the version used to compile the API. • The version of the API is not compatible with Runtime. See the topic on compatibility in the S7-PLCSIM Advanced Function Manual.
	<code>SREC_CONNECTION_ERROR</code>	Unable to establish a connection to the Runtime Manager.
	<code>SREC_ERROR_LOADING_DLL</code>	The API library cannot be loaded.

Return values	SREC_RUNTIME_NOT_AVAILABLE	No Runtime Manager runs in this Windows user session.
	SREC_CONFIG_FILE_ERROR	Operation regarding the configuration file "UserInterfaceConfiguration.xml" has failed, for example, create, read, write.
Example C++	<pre>// Include The Headerfile Of The API #include "SimulationRuntimeApi.h" // Prepare The Variables ERuntimeErrorCode result = SREC_INVALID_ERROR_CODE; ISimulationRuntimeManager* api = NULL; // Initialize The API And Get The RuntimeManager Interface result = InitializeApi(&api);</pre>	

NOTE

If you no longer require the interface, delete the interface.
See DestroyInterface() [\(Page 44\)](#).

4.2.2 RuntimeApiEntry_Initialize

Description

Use the function `RuntimeApiEntry_Initialize` only if the API library (DLL) is to be loaded from a different folder than the folder of your application calling this function.

When the API is initialized, the API library is first loaded and the `Initialize` function is then imported and called.

The function returns an interface to the Simulation Runtime Manager. Use this interface to create a new simulated PLC instance to obtain access to an existing instance.

Table 4-2 RuntimeApiEntry_Initialize - Native C++

Syntax	<pre>_declspec(dllexport) ERuntimeErrorCode RuntimeApiEntry_Initialize(ISimulationRuntimeManager** out_SimulationRuntimeManagerInterface,UINT32 in_InterfaceVersion);</pre>	
Parameters	ISimulationRuntimeManager** out_SimulationRuntimeManagerInterface: Pointer to a Runtime Manager interface pointer. The pointer must be initialized with NULL. The interface is created within the function. See Data types (Page 239). UINT32 in_InterfaceVersion: Version of the API interface to be downloaded: <code>DAPI_DLL_INTERFACE_VERSION</code>.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_WRONG_ARGUMENT	The pointer to the Runtime Manager interface does not equal NULL.
	SREC_WRONG_VERSION	- The required version of the interface is incompatible with the version used to compile the API. - The version of the API is not compatible with Runtime. See information about compatibility in the S7-PLCSIM Advanced Function Manual.
	SREC_CONNECTION_ERROR	Unable to establish a connection to the Runtime Manager.

Return values	SREC_RUNTIME_NOT_AVAILABLE	No Runtime Manager runs in this Windows user session.
	SREC_CONFIG_FILE_ERROR	Operation regarding the configuration file "UserInterfaceConfiguration.xml" has failed, for example, create, read, write.
Example C++	<pre>// Include The Headerfile Of The API #include "SimulationRuntimeApi.h" // Prepare The Variables ERuntimeErrorCode result = SREC_INVALID_ERROR_CODE; HMODULE dllHandle = NULL; ApiEntry_Initialize Initialize = NULL; ISimulationRuntimeManager* api = NULL; // Load The DLL And Import The "Initialize" Function (using the Windows API) dllHandle = LoadLibrary(DAPI_DLL_NAME_X86); if (dllHandle != NULL) { Initialize = (ApiEntry_Initialize)GetProcAddress(dllHandle, DAPI_ENTRY_INITIALIZE); } // Initialize The API And Get The RuntimeManager Interface if (Initialize != NULL) { result = Initialize(&api, DAPI_DLL_INTERFACE_VERSION); } </pre>	

NOTE

If you no longer require the interface, delete the interface.

See [DestroyInterface\(\)](#) (Page 44).

4.3 .NET (C#)

4.3.1 Initialize

Description

The entry point to the API is the static class
`Siemens.Simatic.Simulation.Runtime.SimulationRuntimeManager`.
 The API is initialized when a function of this class is used for the first time.

Table 4-3 Initialize - .NET (C#)

Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationInitializationException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.ConnectionError</code>	Unable to establish a connection to the Runtime Manager.
	<code>ERuntimeErrorCode.WrongVersion</code>	The version of the API is not compatible with Runtime. See information about compatibility in the S7-PLCSIM Advanced Function Manual.
	<code>ERuntimeErrorCode.RuntimeNotAvailable</code>	No Runtime Manager runs in this Windows user session.
	<code>ERuntimeErrorCode.ConfigFileError</code>	Operation regarding the configuration file "UserInterfaceConfiguration.xml" has failed, for example, create, read, write.

Shut down API

5.1 Native C++

Basic procedure for deleting the user interfaces

To delete all user interfaces, generally follow these steps:

1. Delete the interfaces `IIInstances` and `IRemoteRuntimeManager`.
2. Call the `Shutdown()` function of the `ISimulationRuntimeManager` interface.
3. Delete the `ISimulationRuntimeManager` interface.
4. Unload the API library (DLL) with the Windows API function `FreeLibrary()`.

Deleting the user interfaces via functions

Deleting the user interfaces is also possible via functions.

If the API was initialized using the `InitializeApi()` function, you delete the user interfaces using the following functions:

- `FreeApi()` [\(Page 46\)](#)
- `ShutdownAndFreeApi()` [\(Page 47\)](#)

5.1.1 DestroyInterface()

Description

`DestroyInterface()` is a function pointer that points to the `RuntimeApiEntry_DestroyInterface` function. It is only valid if you have successfully called the `InitializeApi` function.

The function unloads the memory of an `ISimulationRuntimeManager`, `IRemoteRuntimeManager` or `IIInstance` interface.

If you call `DestroyInterface()` on an instance and the number of interfaces reaches zero, the instance becomes unregistered.

Table 5-1 `DestroyInterface()` - Native C++

Syntax	<code>ERuntimeErrorCode DestroyInterface(IBaseInterface* in_Interface);</code>	
Parameters	<code>IBaseInterface* in_Interface</code>: The interface to be deleted.	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_WRONG_ARGUMENT</code>	The pointer to the interface is <code>NULL</code> .
Example C++	<pre>// Include The Headerfile Of The API #include "SimulationRuntimeApi.h" // The Interfaces ERuntimeErrorCode result; ISimulationRuntimeManager* api = NULL; IInstance* instance = NULL; // Init the DLL and create an instance result = InitializeApi(&api); result = api->RegisterInstance(&instance); // Destroy Instance Interfaces result = DestroyInterface(instance); instance = NULL;</pre>	

5.1.2 RuntimeApiEntry_DestroyInterface

Description

Use the `RuntimeApiEntry_DestroyInterface` function only if the API library (DLL) is to be loaded from a different directory than the Startup folder of the application calling this function.

If the API was initialized using the `InitializeApi` function, you select the `DestroyInterface()` [\(Page 44\)](#) function.

5.1 Native C++

The function unloads the memory of an ISimulationRuntimeManager, IRemoteRuntimeManager or IInstance interface.

Table 5-2 RuntimeApiEntry_DestroyInterface() - Native C++

Syntax	<code>__declspec(dllexport) ERuntimeErrorCode RuntimeApiEntry_DestroyInterface(IBaseInterface* in_Interface);</code>	
Parameters	IBaseInterface* in_Interface: The interface to be deleted.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_WRONG_ARGUMENT	The pointer to the interface is NULL.
Example C++	<pre>// Include The Headerfile Of The API #include "SimulationRuntimeApi.h" // Prepare The Variables ERuntimeErrorCode result = SREC_INVALID_ERROR_CODE; HMODULE dllHandle = NULL; ApiEntry_DestroyInterface Destroy = NULL; IInstance* instance = NULL; // Load The DLL And Import The "DestroyInterface" Function (using the Windows API) dllHandle = LoadLibraryA(DAPI_DLL_NAME_X86); if (dllHandle != NULL) { Destroy = (ApiEntry_DestroyInterface)GetProcAddress (dllHandle, DAPI_ENTRY_DESTROY_INTERFACE); } ... // Frees the memory of an IInstance interface result = Destroy(instance);</pre>	

5.1.3 FreeApi()

Description

The FreeApi () function unloads the library of the Runtime API.

This function can only be called after the successful call of the `InitializeApi`. If the `InitializeApi` function was not called, the library must be unloaded using the Windows API function `FreeLibrary()`.

Table 5-3 FreeApi() - Native C++

Syntax	<code>ERuntimeErrorCode FreeApi();</code>	
Parameters	None	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_API_NOT_INITIALIZED</code>	The <code>InitializeApi</code> function was not called successfully.
Example C++	<pre>// Include The Headerfile Of The API #include "SimulationRuntimeApi.h" // The Interfaces ERuntimeErrorCode result; ISimulationRuntimeManager* api = NULL; IInstance* instance = NULL; // Init the API result = InitializeApi(&api); ... // Shutdown The API api->Shutdown(); result = DestroyInterface(api); api = NULL; result = FreeApi();</pre>	

5.1.4 ShutdownAndFreeApi()

Description

The `ShutdownAndFreeApi()` function shuts down the Runtime API, deletes the `IRuntimeManager` interface and unloads the library of the Runtime API.

5.2 .NET (C#)

This function can only be called after the successful call of the `InitializeApi`. If the `InitializeApi` function was not called, the library must be unloaded using the Windows API function `FreeLibrary()`.

Table 5-4 ShutdownAndFreeApi() - Native C++

Syntax	<code>ERuntimeErrorCode ShutdownAndFreeApi(ISimulationRuntimeManager* in_SimulationRuntimeManagerInterface);</code>	
Parameters	<code>ISimulationRuntimeManager* in_SimulationRuntimeManagerInterface:</code> The interface of the Runtime Manager to be deleted.	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_API_NOT_INITIALIZED</code>	The <code>InitializeApi</code> function was not called successfully.
	<code>SREC_WRONG_ARGUMENT</code>	The pointer to the interface is NULL.
Example C++	<pre>// Include The Headerfile Of The API #include "SimulationRuntimeApi.h" // The Interfaces ERuntimeErrorCode result; ISimulationRuntimeManager* api = NULL; IInstance* instance = NULL; // Init the API result = InitializeApi(&api); ... // Shutdown The API result = ShutdownAndFreeApi(api); api = NULL;</pre>	

5.2 .NET (C#)

5.2.1 Shut down API

You can terminate the .NET components of the API for the `IInstance` and `IRemoteRuntimeManager` interfaces by calling the `Dispose` (Page 82) function.

These interfaces can also be cleared automatically with the .NET Garbage Collector.

Manually clearing the API

To manually clear the API, follow these steps:

1. Delete all interfaces.

You can find more information at Interfaces for `IInstances` (Page 82).

2. Call the `Shutdown()` (Page 80) function of the `ISimulationRuntimeManager` interface.

Global functions (Native C++)

The global functions `GetNameOf...` return the name of the enumeration entry (`const WCHAR*`).

GetNameOfAreaSection()

Table 6-1 `GetNameOfAreaSection()` - Native C++

Syntax	<code>const WCHAR* GetNameOfAreaSection(EArea in_AreaSection);</code>
Parameters	<code>EArea in_AreaSection</code> : Enumeration entry
Return values	<code>const WCHAR*</code> : Name of the enumeration entry

GetNameOfCPUType()

Table 6-2 `GetNameOfCPUType()` - Native C++

Syntax	<code>const WCHAR* GetNameOfCPUType(ECPUType in_CPUType);</code>
Parameters	<code>ECPUType in_CPUType</code> : Enumeration entry
Return values	<code>const WCHAR*</code> : Name of the enumeration entry

GetNameOfCommunicationInterface()

Table 6-3 `GetNameOfCommunicationInterface()` - Native C++

Syntax	<code>const WCHAR* GetNameOfCommunicationInterface(ECommunicationInterface in_CommunicationInterface);</code>
Parameters	<code>ECommunicationInterface in_CommunicationInterface</code> : Enumeration entry
Return values	<code>const WCHAR*</code> : Name of the enumeration entry

GetNameOfDataType()

Table 6-4 `GetNameOfDataType()` - Native C++

Syntax	<code>const WCHAR* GetNameOfDataType(EDataType in_DataType);</code>
Parameters	<code>EDataType in_DataType</code> : Enumeration entry
Return values	<code>const WCHAR*</code> : Name of the enumeration entry

GetNameOfErrorCode()

Table 6-5 GetNameOfErrorCode() - Native C++

Syntax	<code>const WCHAR* GetNameOfErrorCode(ERuntimeErrorCode in_ErrorCode);</code>
Parameters	<code>ERuntimeErrorCode in_ErrorCode</code> : Enumeration entry
Return values	<code>const WCHAR*</code> : Name of the enumeration entry

GetNameOfLEDMode()

Table 6-6 GetNameOfLEDMode() - Native C++

Syntax	<code>const WCHAR* GetNameOfLEDMode(ELEDMode in_LEDMode);</code>
Parameters	<code>ELEDMode in_LEDMode</code> : Enumeration entry
Return values	<code>const WCHAR*</code> : Name of the enumeration entry

GetNameOfLEDType()

Table 6-7 GetNameOfLEDType() - Native C++

Syntax	<code>const WCHAR* GetNameOfLEDType(ELEDType in_LEDType);</code>
Parameters	<code>ELEDType in_LEDType</code> : Enumeration entry.
Return values	<code>const WCHAR*</code> : Name of the enumeration entry

GetNameOfOperatingMode()

Table 6-8 GetNameOfOperatingMode() - Native C++

Syntax	<code>const WCHAR* GetNameOfOperatingMode(EOperatingMode in_OperatingMode);</code>
Parameters	<code>EOperatingMode in_OperatingMode</code> : Enumeration entry
Return values	<code>const WCHAR*</code> : Name of the enumeration entry

GetNameOfOperatingState

Table 6-9 GetNameOfOperatingState() - Native C++

Syntax	<code>const WCHAR* GetNameOfOperatingState(EOperatingState in_OperatingState);</code>
Parameters	<code>EOperatingState in_OperatingState</code> : Enumeration entry
Return values	<code>const WCHAR*</code> : Name of the enumeration entry

GetNameOfPrimitiveDataType

Table 6-10 GetNameOfPrimitiveDataType() - Native C++

Syntax	<code>const WCHAR* GetNameOfPrimitiveDataType(EPrimitiveDataType in_DataType);</code>
Parameters	<code>EPrimitiveDataType in_DataType</code> : Enumeration entry
Return values	<code>const WCHAR*</code> : Name of the enumeration entry

GetNameOfTagListDetails

Table 6-11 GetNameOfTagListDetails() - Native C++

Syntax	<code>const WCHAR* GetNameOfTagListDetails(ETagListDetails in_TagListDetails);</code>
Parameters	<code>ETagListDetails in_TagListDetails</code> : Enumeration entry
Return values	<code>const WCHAR*</code> : Name of the enumeration entry

GetNameOfRuntimeConfigChanged()

Table 6-12 GetNameOfRuntimeConfigChanged() - Native C++

Syntax	<code>const WCHAR* GetNameOfRuntimeConfigChanged(ERuntimeConfigChanged in_RuntimeConfigChanged);</code>
Parameters	<code>ERuntimeConfigChanged in_RuntimeConfigChanged</code> : Enumeration entry
Return values	<code>const WCHAR*</code> : Name of the enumeration entry

GetNameOfDirection()

Table 6-13 GetNameOfDirection() - Native C++

Syntax	<code>const WCHAR* GetNameOfDirection(EDirection in_Direction);</code>
Parameter	<code>EDirection in_Direction</code> : Enumeration entry
Return values	<code>const WCHAR*</code> : Name of the enumeration entry

GetNameOfDiagSeverity()

Table 6-14 GetNameOfDiagSeverity() - Native C++

Syntax	<code>const WCHAR* GetNameOfDiagSeverity(EDiagSeverity in_DiagSeverity);</code>
Parameter	<code>EDiagSeverity in_DiagSeverity</code> : Enumeration entry
Return values	<code>const WCHAR*</code> : Name of the enumeration entry

GetNameOfRackOrStationFaultType()

Table 6-15 GetNameOfRackOrStationFaultType() - Native C++

Syntax	<code>const WCHAR* GetNameOfRackOrStationFaultType (ERackOrStationFaultType in_RackOrStationFaultType);</code>
Parameter	<code>ERackOrStationFaultType in_RackOrStationFaultType:</code> Enumeration entry
Return values	<code>const WCHAR*:</code> Name of the enumeration entry

GetNameOfProcessEventTypes()

Table 6-16 GetNameOfProcessEventTypes() - Native C++

Syntax	<code>const WCHAR* (GetNameOfProcessEventTypes (EProcessEventTypes in_ProcessEventTypes);</code>
Parameters	<code>EProcessEventTypes in_ProcessEventTypes:</code> Enumeration entry
Return values	<code>const WCHAR*:</code> Name of the enumeration entry

GetNameOfPullOrPlugEventTypes()

Table 6-17 GetNameOfPullOrPlugEventTypes() - Native C++

Syntax	<code>const WCHAR* GetNameOfPullOrPlugEventTypes (EPullOrPlugEventTypes in_PullOrPlugEventTypes);</code>
Parameters	<code>EPullOrPlugEventTypes in_PullOrPlugEventTypes:</code> Enumeration entry
Return values	<code>const WCHAR*:</code> Name of the enumeration entry

GetNameOfCycleTimeMonitoringMode()

Table 6-18 GetNameOfCycleTimeMonitoringMode() - Native C++

Syntax	<code>const WCHAR* GetNameOfCycleTimeMonitoringMode (ECycleTimeMonitoringMode in_CycleTimeMonitoringMode);</code>
Parameters	<code>ECycleTimeMonitoringMode in_CycleTimeMonitoringMode:</code> Enumeration entry
Return values	<code>const WCHAR*:</code> Name of the enumeration entry

GetNameOfDiagProperty()

Table 6-19 GetNameOfDiagProperty() - Native C++

Syntax	<code>const WCHAR* GetNameOfDiagProperty (EDiagProperty in_DiagProperty);</code>
Parameters	<code>EDiagProperty in_DiagProperty:</code> Enumeration entry
Return values	<code>const WCHAR*:</code> Name of the enumeration entry

GetNameOfAutodiscoverType()

Table 6-20 GetNameOfAutodiscoverType() - Native C++

Syntax	<code>const WCHAR* GetNameOfAutodiscoverType(EAutodiscoverType in_AutodiscoverType);</code>
Parameters	<code>EAutodiscoverType in_AutodiscoverType</code> : Enumeration entry
Return values	<code>const WCHAR*</code> : Name of the enumeration entry

GetNameOfSoftwareConfigChanged()

Table 6-21 GetNameOfSoftwareConfigChanged() - Native C++

Syntax	<code>const WCHAR* GetNameOfSoftwareConfigChanged(ESoftwareConfigChanged in_SoftwareConfigChanged);</code>
Parameters	<code>ESoftwareConfigChanged in_SoftwareConfigChanged</code> : Enumeration entry
Return values	<code>const WCHAR*</code> : Name of the enumeration entry

GetNameOfNetworkMode()

Table 6-22 GetNameOfNetworkMode() - Native C++

Syntax	<code>const WCHAR* GetNameOfNetworkMode(ENetworkMode in_NetworkMode);</code>
Parameters	<code>ENetworkMode in_NetworkMode</code> : Enumeration entry
Return values	<code>const WCHAR*</code> : Name of the enumeration entry

IBaseRuntimeManager

7.1 Simulation runtime instances

CreateInterface()

Creates and returns an interface of an already registered instance of a simulated PLC.

The instance could have been registered via the application or another application that uses the Simulation Runtime API.

Table 7-1 CreateInterface() - Native C++

Syntax	<pre>ERuntimeErrorCode CreateInterface(WCHAR* in_InstanceName, IInstance** out_InstanceInterface); ERuntimeErrorCode CreateInterface(INT32 in_InstanceID, IInstance** out_InstanceInterface);</pre>	
Parameters	- INT32 in_InstanceID: The ID of the registered instance from which you want to receive the interface. - WCHAR* in_InstanceName: The name of the registered instance from which you want to receive the interface. - IInstance** out_InstanceInterface: Pointer to a Simulation Runtime interface pointer. The pointer must be initialized with NULL. The interface is created within the function.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The interface is disconnected from the remote Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_WRONG_ARGUMENT	The name, the ID or the IInstance- pointer is invalid.
	SREC_DOES_NOT_EXIST	The instance is not registered in Runtime Manager.
Example C++	<pre>ISimulationRuntimeManager * api = NULL; ERuntimeErrorCode result = InitializeApi(&api); IInstance* psa1 = NULL; IInstance* psa2 = NULL; if (result == SREC_OK) { result = api->CreateInterface(0, &psa1); result = api->CreateInterface(0, &psa2); // psa2 will be the same as psa1 }</pre>	
Example C++	<pre>ISimulationRuntimeManager * api = NULL; ERuntimeErrorCode result = InitializeApi(&api); IInstance* psa = NULL; if (result == SREC_OK) { result = api->CreateInterface(L"My SimulationRuntime Instance", &psa); }</pre>	

NOTE**Native C++**

If you no longer require the interface, delete the interface.

See DestroyInterface() (Page 44)

Table 7-2 CreateInterface() - .NET (C#)

Syntax	<pre>IInstance CreateInterface(string in_InstanceName); IInstance CreateInterface(INT32 in_InstanceID);</pre>	
Parameters	- INT32 in_InstanceID: The ID of the registered instance from which you want to receive the interface. - string in_InstanceName: The name of the registered instance from which you want to receive the interface.	
Return values	If the function is successful, an interface of a simulated PLC instance; otherwise, a Null pointer.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The interface is disconnected from the remote Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.WrongArgument	The name or the ID is invalid.
	ERuntimeErrorCode.DoesNotExist	The instance is not registered in Runtime Manager.

GetRegisteredInstanceInfoAt()

Returns information about an already registered instance.

You can use the ID or name to create an interface of this instance (see CreateInterface()).

Table 7-3 GetRegisteredInstanceInfoAt() - Native C++

Syntax	<pre>ERuntimeErrorCode GetRegisteredInstanceInfoAt(UINT32 in_Index, SInstanceInfo* out_InstanceInfo);</pre>	
Parameters	- UINT32 in_Index: Index of the created instance from which you want to receive the information. The index must be less than the value you receive when you call GetRegisteredInstanceCount(). - SInstanceInfo* out_InstanceInfo: The information with name and ID of the instance.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The interface is disconnected from the remote Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INDEX_OUT_OF_RANGE	There is no instance information for this index.

RegisteredInstanceInfo { get; }

Returns information about an already registered instance. You can use the ID or name of this instance to create an interface of this instance, see `CreateInterface()`.

Table 7-4 RegisterInstanceInfo { get; } - .NET (C#)

Syntax	<code>SInstanceInfo[] RegisteredInstanceInfo { get; }</code>	
Parameters	None	
Return values	None	
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The interface is disconnected from the remote Runtime Manager.
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.

GetRegisteredInstancesCount()

Returns the number of instances that are registered in Runtime Manager. If the function fails, the return value is 0.

Table 7-5 GetRegisteredInstancesCount() - Native C++

Syntax	<code>UINT32 GetRegisteredInstancesCount();</code>
Parameters	None
Return values	<code>UINT32</code> : Number of available instances.

RegisterInstance()

Registers a new instance of a simulated PLC instance in Runtime Manager. Creates and returns an interface of this instance.

Table 7-6 RegisterInstance() - Native C++

Syntax	<pre>ERuntimeErrorCode RegisterInstance(IInstance** out_InstanceInterface); ERuntimeErrorCode RegisterInstance(WCHAR* in_InstanceName, IInstance** out_InstanceInterface); ERuntimeErrorCode RegisterInstance(ECPUType in_CPUType, IInstance** out_InstanceInterface); ERuntimeErrorCode RegisterInstance(ECPUType in_CPUType, WCHAR* in_InstanceName, IInstance** out_InstanceInterface); ERuntimeErrorCode RegisterInstance(WCHAR* in_ArticleNumber, WCHAR* in_InstanceName, IInstance** out_InstanceInterface);</pre>
Parameters	<code>ECPUType in_CPUType</code>: Defines which CPU type is simulated at the start of the instance. The default setting is "SRCT_1500_Unspecified". When a different CPU type is loaded via STEP 7 or from the Virtual SIMATIC Memory Card, this CPU type applies. <code>WCHAR* in_InstanceName</code>: Name to be assigned to the instance. Every instance must have a unique name. If no name is assigned when registering a new instance, the instance is given the name "Instance_#" (# is the ID of the instance). If this name already exists, the name "Instance_#. #" is used, in which the second # is a counter that is incremented until the name is unique. The length of the name must be less than <code>DINSTANCE_NAME_LENGTH</code>. You can use basic Latin alphanumeric characters

	(A-Z, a-z, 0-9), "!", "=", and "-" to name the instance, but most other special characters, such as currency, Chinese characters, or mathematical symbols are unsupported. See Data types (Page 239). IInstance** out_InstanceInterface: Pointer to a Simulation Runtime interface pointer. The pointer must be initialized with NULL. The interface is created within the function. WCHAR* in_ArticleNumber: Article number for a specific PLC device. The article number must be 19 characters long.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_CONFIG_FILE_ERROR	Error found in configuration file.
	SREC_INTERFACE_REMOVED	The interface is disconnected from the remote Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_WRONG_ARGUMENT	One or more of the following parameters are invalid: - The instance name - The IInstance pointer - The length of in_ArticleNumber
	SREC_LIMIT_REACHED	There are already 16 instances registered in Runtime Manager.
	SREC_ALREADY_EXISTS	An instance with this name already exists.
Example C++	<pre>ISimulationRuntimeManager * api = NULL; ERuntimeErrorCode result = InitializeApi(&api); // Example: How To Create And Register An Instance // And To Get An Interface Of The Instance The Same Time IInstance* psa = NULL; if (result == SREC_OK) { result = api->RegisterInstance(&psa); }</pre>	

NOTE**Native C++**

If you no longer require the interface, delete the interface.

See DestroyInterface() [\(Page 44\)](#).

Table 7-7 RegisterInstance() - .NET (C#)

Syntax	<pre>IInstance RegisterInstance(); IInstance RegisterInstance(string in_InstanceName); IInstance RegisterInstance(ECPUType in_CPUType); IInstance RegisterInstance(ECPUType in_CPUType string in_InstanceName); IInstance RegisterInstance(string in_ArticleNumber, string in_InstanceName);</pre>
Parameters	ECPUType in_CPUType: Defines which CPU type is simulated at the start of the instance. The default setting is "ECPUType.Unspecified". When a different CPU type is loaded via STEP 7 or from the Virtual SIMATIC Memory Card, this CPU type applies. string in_InstanceName: Name to be assigned to the instance. Every instance must have a unique name. If no name is assigned when registering a new instance, the instance is given the name "Instance_#" (# is the ID of the instance). If this name already exists, the name "Instance_#. #" is used, in which

7.1 Simulation runtime instances

	the second # is a counter that is incremented until the name is unique. The length of the name must be less than <code>DINSTANCE_NAME_LENGTH</code>. You can use basic Latin alphanumeric characters (A-Z, a-z, 0-9), "!", "=", and "-" to name the instance, but most other special characters, such as currency, Chinese characters, or mathematical symbols are unsupported. See Data types (Page 239). • <code>string in_ArticleNumber</code>: Article number for a specific PLC device. The article number must be 19 characters long.	
Return values	If the function is successful, an interface of a simulated PLC. Otherwise, a Null pointer.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	<code>ERuntimeErrorCode.ConfigFileError</code>	Error found in configuration file.
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The interface is disconnected from the remote Runtime Manager.
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.
	<code>ERuntimeErrorCode.WrongArgument</code>	One or more of the following parameters are invalid: • The instance name • The <code>IInstance</code> pointer • The length of <code>in_ArticleNumber</code>
	<code>ERuntimeErrorCode.LimitReached</code>	There are already 16 instances registered in Runtime Manager.
	<code>ERuntimeErrorCode.AlreadyExists</code>	An instance with this name already exists.

RegisterCustomInstance()

Registers a new simulated PLC instance in the Runtime Manager. Creates and returns an interface of this instance.

Table 7-8 RegisterCustomInstance() - Native C++

Syntax	<pre>ERuntimeErrorCode RegisterCustomInstance(WCHAR* in_VplcDll, IInstance** out_InstanceInterface); ERuntimeErrorCode RegisterCustomInstance(WCHAR* in_VplcDll, WCHAR* in_InstanceName, IInstance** out_InstanceInterface);</pre>	
Parameters	• <code>WCHAR* in_VplcDll</code>: The complete path to the DLL of the simulated PLC instance that Siemens.Simatic.Simulation.Runtime.Instance.exe loads at PowerOn. • <code>WCHAR* in_InstanceName</code>: Name to be assigned to the instance. Every instance must have a unique name. If no name is assigned when registering a new instance, the instance is given the name "Instance_#" (# is the ID of the instance). If this name already exists, the name "Instance_#. #" is used, in which the second # is a counter that is incremented until the name is unique. The length of the name must be less than <code>DINSTANCE_NAME_LENGTH</code>. See Data types (Page 239). • <code>IInstance** out_InstanceInterface</code>: Pointer to a Simulation Runtime interface pointer. The pointer must be initialized with <code>NULL</code>. The interface is created within the function.	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_INTERFACE_REMOVED</code>	The interface is disconnected from the remote Runtime Manager.
	<code>SREC_TIMEOUT</code>	The function does not return on time.

Return values	SREC_WRONG_ARGUMENT	The DLL name, the instance name or the IInstance pointer is invalid.
	SREC_LIMIT_REACHED	There are already 16 instances registered in Runtime Manager.
	SREC_ALREADY_EXISTS	An instance with this name already exists.
Example C++	<pre>ISimulationRuntimeManager * api = NULL; ERuntimeErrorCode result = InitializeApi(&api); // Example: How To Create And Register An Instance // And To Get An Interface Of The Instance The Same Time IInstance* psa = NULL; if (result == SREC_OK) { result = api->RegisterCustomInstance(L"C:\\Temp\\vplc.dll", &psa); }</pre>	

NOTE

Native C++

If you no longer require the interface, delete the interface.

See DestroyInterface() (Page 44).

Table 7-9 RegisterCustomInstance() - .NET (C#)

Syntax	<pre>IInstance RegisterCustomInstance(string in_VplcDll); IInstance RegisterCustomInstance(string in_VplcDll, string in_InstanceName);</pre>	
Parameters	- string in_VplcDll: The complete path to the DLL of the simulated PLC instance that Siemens.Simatic.Simulation.Runtime.Instance.exe loads at PowerOn. - string in_InstanceName: Name to be assigned to the instance. Every instance must have a unique name. If no name is assigned when registering a new instance, the instance is given the name "Instance_#" (# is the ID of the instance). If this name already exists, the name "Instance_#. #" is used, in which the second # is a counter that is incremented until the name is unique. The length of the name must be less than DINSTANCE_NAME_LENGTH. See Data types (Page 239).	
Return values	If the function is successful, an interface of a simulated PLC instance; otherwise, a Null pointer.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The interface is disconnected from the remote Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.WrongArgument	The name or the ID is invalid.
	ERuntimeErrorCode.LimitReached	There are already 16 instances registered in Runtime Manager.
	ERuntimeErrorCode.AlreadyExists	An instance with this name already exists.

7.2 Interfaces for IBaseRuntimeManager

GetPort() / Port { get; }

Returns the open port of the PC on which the Runtime Manager is running. If the function fails, the return value is 0.

Table 7-10 GetPort() - Native C++

Syntax	UINT16 GetPort();
Parameters	None
Return values	UINT16: Open port of the PC on which the remote Runtime Manager is running.

Table 7-11 Port { get; } - .NET (C#)

Syntax	UInt16 Port { get; }
Parameters	None
Return values	UInt16: Open port of the PC on which the remote Runtime Manager is running.

GetVersion() / Version { get; }

Returns the version of the Runtime Manager. If the function fails, version 0.0 is returned.

Table 7-12 GetVersion() - Native C++

Syntax	UINT32 GetVersion();
Parameters	None
Return values	UINT32: Remote Runtime Manager Version (HIWORD = Major, LOWORD = Minor)

Table 7-13 Version { get; } - .NET (C#)

Syntax	UInt32 Version { get; }
Parameters	None
Return values	UInt32: Remote Runtime Manager Version (HIWORD = Major, LOWORD = Minor)

GetStrictMotionTiming() / StrictMotionTiming { get; }

Returns the current global setting for the "Strict Motion Timing" feature that has an effect on newly created instances. The "Strict Motion Timing" feature is located on the S7-PLCSIM Advanced Control Panel.

Table 7-14 GetStrictMotionTiming() - Native C++

Syntax	ERuntimeErrorCode GetStrictMotionTiming(bool* enabled);	
Parameters	bool* enabled: Receives the current setting. true: Active false: Inactive	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_TIMEOUT	The function does not return on time.
	SREC_CONFIG_FILE_ERROR	The setting could not be read from the configuration file UserInterfaceConfiguration.xml.

Table 7-15 StrictMotionTiming { get; } - .NET (C#)

Syntax	bool StrictMotionTiming { get; }	
Parameters	None	
Return values	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.ConfigFileError	The setting could not be read from the configuration file UserInterfaceConfiguration.xml.

SetStrictMotionTiming() / StrictMotionTiming { set; }

Sets the global setting for the "Strict Motion Timing" feature that has an effect on newly created instances. The "Strict Motion Timing" feature is located on the S7-PLCSIM Advanced Control Panel.

Table 7-16 SetStrictMotionTiming() - Native C++

Syntax	ERuntimeErrorCode SetStrictMotionTiming(bool enable);	
Parameters	bool enable: The value to be set. true: Active false: Inactive	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_ALREADY_EXISTS	An instance is registered. No instance must be registered to change the setting.
	SREC_TIMEOUT	The function does not return on time.
	SREC_ACCESS_DENIED	No write rights for the configuration file.
	SREC_CONFIG_FILE_ERROR	The setting could not be written to the configuration file UserInterfaceConfiguration.xml.

7.2 Interfaces for IBaseRuntimeManager

Table 7-17 StrictMotionTiming { set; } - .NET (C#)

Syntax	bool StrictMotionTiming { set; }	
Parameters	None	
Return values	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.AlreadyExists	An instance is registered. No instance must be registered to change the setting.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.AccessDenied	No write rights for the configuration file.
	ERuntimeErrorCode.ConfigFileError	The setting could not be written to the configuration file UserInterfaceConfiguration.xml.

GetNetworkMode

Returns the current network mode settings that are used in the network adapter for distributed communication.

Table 7-18 GetNetworkMode() - Native C++

Syntax	ERuntimeErrorCode IBaseRuntimeManager::GetNetworkMode(ENetworkMode* out_Mode);	
Parameters	ENetworkMode* out_Mode Receives the network mode as #ENetworkMode that is currently set in the driver of the S7-PLCSIM Advanced Virtual Switch	
Return values	Runtime error code	Description
	SREC_OK	The function call is successful.

SetNetworkMode()

Sets the network mode as used in the network adapter for distributed communication.

Check that the binding on the network interfaces is correct.

Table 7-19 SetNetworkMode() - Native C++

Syntax	ERuntimeErrorCode SetNetworkMode(const ENetworkMode in_mode);	
Parameters	ENetworkMode in_Mode Type of network communication mode to be set	
Return values	Runtime error code	Description
	SREC_OK	The function call is successful.
	SREC_VIRTUAL_SWITCH_MISCONFIGURED	The driver of the S7-PLCSIM Advanced Virtual Switch is not running.
	SREC_INSTANCE_ALREADY_RUNNING	Network mode cannot be changed because there is already a running instance.
	SREC_PCAP_DRIVER_NOT_RUNNING	The Nmap Packet Capture Driver (Npcap) is not active on the system. Remedy: 1. Start the command line in administrator mode. 2. Execute this command: net start npcap

NetworkMode { get; set; };

Returns or sets the network mode used on the network adapter for distributed communication.

Check that the binding on the network interfaces is correct.

Table 7-20 NetworkMode { get; } - .NET (C#)

Syntax	ENetworkMode NetworkMode { get; };
Return value	ENetworkMode Reveives the network mode as #ENetworkMode that is currently set in the driver of the S7-PLCSIM Advanced Virtual Switch
Exceptions	None

Table 7-21 NetworkMode { set; } - .NET (C#)

Syntax	ENetworkMode NetworkMode { set; };	
Parameters	None	
Exceptions	Runtime error code	Description
	ERuntimeErrorCode.VirtualSwitchMisconfigured	The S7-PLCSIM Advanced Virtual Switch driver is not running.
	ERuntimeErrorCode.InstanceAlreadyRunning	Network mode cannot be changed because there is already a running instance.
	ERuntimeErrorCode.PCAPDriverNotRunning	The Nmap Packet Capture Driver (Npcap) is not active on the system. Remedy: 1. Start the command line in administrator mode. 2. Execute this command: net start npcap

GetNetInterfaces() / NetInterfaces { get; }

Provides information on the PC interfaces, which you can use for SetNetInterfaceMapping.

Table 7-22 GetNetInterfaces() - Native C++

Syntax	ERuntimeErrorCode GetNetInterfaces(SNetInterfaceInfo* inout_InterfaceInfo, UINT* inout_InterfaceCount);	
Parameters	SNetInterfaceInfo* inout_InterfaceInfo: Address of the first item of the SNetInterfaceInfo array, which contains information about PC network adapters. If the value is NULL, only the number of network adapters has been requested. UINT* inout_InterfaceCount: For the input: Number of SNetInterfaceInfo items assigned by the user. For the output: Number of network adapters (array items)	
Return values	Runtime error code	Description
	SREC_OK	The function call is successful.

7.2 Interfaces for IBaseRuntimeManager

Return values	SREC_INDEX_OUT_OF_RANGE	The number of array units is lower than the number of network adapters.
	SREC_INTERNAL_ERROR	The system call has caused an internal error.
Example C++	<pre>#include <iostream> #include "SimulationRuntimeApi.h" ISimulationRuntimeManager* rtm = nullptr; ERuntimeErrorCode result = SREC_INVALID_ERROR_CODE; //init connection InitializeApi(&rtm); //get count of network interfaces UINT count; result = rtm->GetNetInterfaces(nullptr, &count); //get actual network interfaces std::unique_ptr<SNetInterfaceInfo[]> netInterfaces = std::make_unique<SNetInterfaceInfo[]>(count); result = rtm->GetNetInterfaces(netInterfaces.get(), &count); ShutdownAndFreeApi(rtm);</pre>	

Table 7-23 NetInterfaces { get; } - .NET (C#)

Syntax	SNetInterfaceInfo[] NetInterfaces { get; }	
Return values	Runtime error code	Description
	SNetInterfaceInfo[]	Information about the PC network interfaces
Exceptions	ERuntimeErrorCode.InternalError	The system call has caused an internal error.

SetNetInterfaceBindings()

Sets the binding of an S7-PLCSIM Advanced Virtual Switch to PC interfaces based on the network mode and the mapping configuration of the network interfaces for the instance.

You must call the function from a parent session (administrator mode), otherwise you cannot access it, and the message SREC_ACCESS_DENIED appears. This function is not valid for the Softbus NetworkMode.

Table 7-24 SetNetInterfaceBindings() - Native C++

Syntax	ERuntimeErrorCode SetNetInterfaceBindings(DWORD in_PCInterfaceIndex = 0);	
Parameters	DWORD in_PCInterfaceIndex = 0: - When Single Adapter Network Mode (promiscuous mode) is set, you can specify the index of the PC interfaces for network communication. To enable only local PC communication, use the value (0). - When Multi-Adapter Network Mode (non-promiscuous mode) is set, this parameter is not used and the binding is set according to the configured mapping of the interfaces.	
Return values	Runtime error code	Description
	SREC_OK	The function call is successful.

Return values	SREC_INVALID_CONFIGURATION	The configuration is invalid because an invalid adapter is mapped on an instance. For example, a sim_hwdb.ini file of a registered instance uses the MAC address of an invalid adapter such as a PLCSIM vAdapter, a disabled adapter, or an adapter that does not support PLCSIM vSwitch.
	SREC_ACCESS_DENIED	You do not have authorization to connect to the PC network interfaces.
	SREC_WRONG_COMMUNICATION_INTERFACE	The PC interface does not exist or is disabled.
	SREC_INTERNAL_ERROR	The system call has caused an internal error. Enable the traces to obtain detailed information.
	SREC_INSTANCE_ALREADY_RUNNING	In Single Adapter Network Mode (promiscuous mode), bindings cannot change while an instance is running.
	SREC_VIRTUAL_SWITCH_MISCONFIGURED	The configuration of an S7-PLCSIM Advanced Virtual Switch is not correct or the driver is not functioning properly.
	SREC_NOT_SUPPORTED	You cannot call the function in the Softbus NetworkMode.

Table 7-25 SetNetInterfaceBindings() - .NET (C#)

Syntax	<pre>void SetNetInterfaceBindings(); void SetNetInterfaceBindings(DWORD in_PCInterfaceIndex);</pre>	
Exceptions	Runtime error code	Description
	ERuntimeErrorCode.InvalidConfiguration	The configuration is invalid because, for example, a non-existent adapter was used.
	ERuntimeErrorCode.AccessDenied	You do not have authorization to connect to the PC network interfaces.
	ERuntimeErrorCode.WrongCommunicationInterface	The PC interface does not exist or is disabled.
	ERuntimeErrorCode.InternalError	The system call has caused an internal error. Enable the traces to obtain detailed information.
	ERuntimeErrorCode.InstanceAlreadyRunning	In Single Adapter Network Mode (promiscuous mode), bindings cannot change while an instance is running.
	ERuntimeErrorCode.VirtualSwitchMisconfigured	The configuration of an S7-PLCSIM Advanced Virtual Switch is not correct or the driver is not functioning properly.
	ERuntimeErrorCode.NotSupported	You cannot call the function in the Softbus NetworkMode.

ResetNetInterfaceBindings()

Deactivates the binding of an S7-PLCSIM Advanced Virtual Switch to PC interfaces. The binding on the Siemens PLCSIM Virtual Ethernet Adapter remains.

You must call the function from a parent session (administrator mode), otherwise you cannot access it, and the message SREC_ACCESS_DENIED appears. For local runtime manager, the application using the API must be in an elevated session; for remote use, the remote runtime

7.2 Interfaces for IBaseRuntimeManager

manager must be in the elevated session. This function is not valid for the Softbus NetworkMode.

Table 7-26 ResetNetInterfaceBindings() - Native C++

Syntax	ERuntimeErrorCode ResetNetInterfaceBindings();	
Parameters	None	
Return values	Runtime error code	Description
	SREC_OK	The function call is successful.
	SREC_ACCESS_DENIED	You do not have authorization to connect to the PC network interfaces.
	SREC_INSTANCE_ALREADY_RUNNING	In Single Adapter Network Mode (promiscuous mode), the bindings cannot change while an instance is running.
	SREC_INTERNAL_ERROR	The system call has caused an internal error. Enable the traces to obtain detailed information.
	SREC_NOT_SUPPORTED	You cannot call the function in the Softbus NetworkMode.

ResetNetInterfaceBindings() - .NET (C#)

Syntax	void ResetNetInterfaceBindings();	
Exceptions	Runtime error code	Description
	ERuntimeErrorCode.AccessDenied	You do not have authorization to connect to the PC network interfaces.
	ERuntimeErrorCode.InternalError	The system call has caused an internal error. Enable the traces to obtain detailed information.
	ERuntimeErrorCode.InstanceAlreadyRunning	In Single Adapter Network Mode (promiscuous mode), the bindings cannot change while an instance is running.
	ERuntimeErrorCode.NotSupported	You cannot call the function in the Softbus NetworkMode.

GetDefaultStoragePath()

Returns the full path of the default folder in which the instance stores its data.

Table 7-27 GetDefaultStoragePath() - Native C++

Syntax	ERuntimeErrorCode GetDefaultStoragePath(WCHAR* out_DefaultStoragePath);	
Parameters	WCHAR* out_DefaultStoragePath Output parameter that is filled with the path of the default folder.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.

SetDefaultStoragePath()

Sets the full path of the default folder in which the instance stores its data.

Table 7-28 SetDefaultStoragePath() - Native C++

Syntax	ERuntimeErrorCode SetDefaultStoragePath(const WCHAR* in_DefaultStoragePath);	
Parameters	const WCHAR* in_DefaultStoragePath Input parameter that specifies the path of the default folder.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_WRONG_ARGUMENT	The path contains invalid characters.
	SREC_CONFIG_FILE_ERROR	The configuration file UserInterfaceConfiguration.xml could not be read.

DefaultStoragePath { get; set; }

Returns or sets the full path of the default folder in which the instance stores its data.

Table 7-29 DefaultStoragePath { get; set; } - .NET (C#)

Syntax	string DefaultStoragePath { get; set; }	
Return values	Runtime error code	Description
	OK	The function is successful.
	WrongArgument	The path contains invalid characters.
	ConfigFileError	The configuration file UserInterfaceConfiguration.xml could not be read.

ISimulationRuntimeManager

8.1 Remote connections

Remote connections use TLS and temporary self-signed certificates to provide increased security.

NOTE

For secure communication, only open ports and connect to remote runtime managers on trusted local networks. Without restricting access, unwanted users could access a running remote runtime manager.

OpenPort()

Opens a port to which another Runtime Manager can connect. The default communication using only the inPort parameter is secured communication. Setting the in_SecurityMode to UNSECURED provides the ability to use legacy communication, but this mode is not recommended.

Table 8-1 OpenPort() - Native C++

Syntax	<pre>ERuntimeErrorCode OpenPort(UINT16 in_Port); ERuntimeErrorCode ISimulationRuntimeManager::OpenPort(UINT16 in_Port, ESecurityMode in_SecurityMode);</pre>	
Parameters	- UINT16 in_Port: The port. The value must be greater than 1024. - ESecurityMode in_SecurityMode Security mode	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_TIMEOUT	The function does not return on time.
	SREC_ALREADY_EXISTS	A port is already open.
	SREC_WRONG_ARGUMENT	The port is invalid.
	SREC_CONNECTION_ERROR	The port cannot be opened.
	SREC_WARNING_UNSECURED_COMMUNICATION	The port was opened without secured protection level. ESecurityMode::UNSECURED.

NOTE

Check for the security warning in the return value of the function. If communication is unsecured, the port communication is not encrypted.

Table 8-2 OpenPort() - .NET (C#)

Syntax	<pre>void OpenPort(UInt16 in_Port); void SimulationRuntimeManager::OpenPort (UINT16 in_Port, ESecurityMode in_SecurityMode);</pre>	
Parameters	• UInt16 in_Port: The port. The value must be greater than 1024. • ESecurityMode in_SecurityMode Security mode	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.AlreadyExists	A port is already open.
	ERuntimeErrorCode.WrongArgument	The port is invalid.
	ERuntimeErrorCode.ConnectionError	The port cannot be opened.
	ERuntimeErrorCode.WarningUnsecured-Communication	The port was opened without secured protection level. ESecurityMode::UNSECURED.

ClosePort()

Closes an open port and all open connections that another Runtime Manager has created to this open port.

Table 8-3 ClosePort() - Native C++

Syntax	ERuntimeErrorCode ClosePort();	
Parameters	None	
Return values	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_TIMEOUT	The function does not return on time.
	SREC_WARNING_INVALID_CALL	No port is open.

Table 8-4 ClosePort() - .NET (C#)

Syntax	void ClosePort();	
Parameters	None	
Return values	None	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.Timeout	The function does not return on time.

GetRemoteConnectionsCount()

Supplies the number of open remote connections to which you have connected.

Table 8-5 GetRemoteConnectionsCount() - Native C++

Syntax	UINT32 GetRemoteConnectionsCount();
Parameters	None
Return values	UINT32: Number of open remote connections.

GetRemoteConnectionInfoAt()

Returns information about an open connection to which you are connected.

Table 8-6 GetRemoteConnectionInfoAt()- Native C++

Syntax	ERuntimeErrorCode GetRemoteConnectionInfoAt(UINT32 in_Index, SConnectionInfo* out_ConnectionInfo);	
Parameters	- UINT32 in_Index: Index of the connection information that is expected. - SConnectionInfo* out_ConnectionInfo: The connection information for this index.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INDEX_OUT_OF_RANGE	Connection information for this index does not exist.

RemoteConnectionInfo { get; }

Returns an array of information about all open connections to which you are connected.

Table 8-7 RemoteConnectionInfo { get; } - .NET (C#)

Syntax	SConnectionInfo[] RemoteConnectionInfo { get; }	
Parameters	None	
Return values	SConnectionInfo[] : An array of information about all open connections.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.Timeout	The function does not return on time.

RemoteConnect()

Creates a new connection to a remote Runtime Manager or uses an existing connection to create an IRemoteRuntimeManager interface.

Table 8-8 RemoteConnect() - Native C++

Syntax	<pre>ERuntimeErrorCode RemoteConnect(UINT8 in_IP3, UINT8 in_IP2, UINT8 in_IP1, UINT8 in_IP0, UINT16 in_Port, IRemoteRuntimeManager**out_RemoteRuntimeManagerInterface); ERuntimeErrorCode RemoteConnect(UIP in_IP, UINT16 in_Port, IRemoteRuntimeManager** out_RunTimeManagerInterface);</pre>	
Parameters	• UINT8 in_IP3: First part of the IP address of the remote PC. • UINT8 in_IP2: Second part of the IP address of the remote PC. • UINT8 in_IP1: Third part of the IP address of the remote PC. • UINT8 in_IP0: Last part of the IP address of the remote PC. • UIP in_IP: IP address of the remote PC. • UINT16 in_Port: The port that is open on the remote PC. • IRemoteRuntimeManager** out_RemoteRuntimeManagerInterface: Pointer to a remote Runtime Manager interface pointer. The pointer must be initialized with NULL. The interface is created in the function.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_TIMEOUT	The function does not return on time.
	SREC_CONNECTION_ERROR	The connection to the remote Runtime Manager cannot be established because secured connection requirements are not met.
	SREC_WRONG_ARGUMENT	IP, port or Instance pointer is invalid.
	SREC_WRONG_VERSION	The version of the API is not compatible with Runtime. See information on compatibility in the S7-PLCSIM Advanced Function Manual.
	SREC_WARNING_UNSECURED_COMMUNICATION	The remote session is not protected.
Example C++	<pre>ISimulationRuntimeManager* api = NULL; ERuntimeErrorCode result = InitializeApi(&api); IRemoteRuntimeManager * client = NULL; if (result == SREC_OK) { result = api->RemoteConnect(192,203,145,144, 4444, &client); }</pre>	

NOTE

After calling this method, check the return value for unsecure communication warnings. If the connection is unsecured, the communication is not encrypted.

NOTE**Native C++**

If you no longer require the interface, delete the interface.

See DestroyInterface() (Page 44).

Table 8-9 RemoteConnect() - .NET (C#)

Syntax	<pre>IRemoteRuntimeManager RemoteConnect(string in_ConnectionString); IRemoteRuntimeManager RemoteConnect(SIP in_IP, UInt16 in_Port); IRemoteRuntimeManager RemoteConnect(Byte in_IP3, Byte in_IP2, Byte in_IP1, Byte in_IP0, UInt16 in_Port);</pre>	
Parameters	- Byte in_IP3: First part of the IP address of the remote PC. - Byte in_IP2: Second part of the IP address of the remote PC. - Byte in_IP1: Third part of the IP address of the remote PC. - Byte in_IP0: Last part of the IP address of the remote PC. - string in_ConnectionString: A string in the form of "<IP3>.<IP2>.<IP1>.<IP0>:<Port>" Example: "182.203.145.144:4444". - SIP in_IP: IP address of the remote PC. - UInt16 in_Port: The port that is open on the remote PC.	
Return values	IRemoteRuntimeManager: Interface to the remote Runtime Manager.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.ConnectionError	Connection to the remote Runtime Manager cannot be established.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.WrongArgument	IP or port is invalid.
	ERuntimeErrorCode.WrongVersion	The version of the API is not compatible with Runtime. See information on compatibility in the S7-PLCSIM Advanced Function Manual.

NOTE

After calling this function, call GetSecurityMode() (Page 234) to check the security mode (Page 294) of the connected remote runtime manager. If the connection is unsecured, the communication is not encrypted.

GetSecurityMode()

GetSecurityMode returns the current ESecurityMode of the opened port.

Table 8-10 GetSecurityMode - Native C++

Syntax	ESecurityMode ISimulationRuntimeManager::GetSecurityMode();	
Parameters	None	
Return values	Runtime error code	Description
	ESecurityMode::UNSECURED	No security is used.
	ESecurityMode::ENCRYPTED_COMMUNICATION	Secure communication
	ESecurityMode::UNKNOWN	No port is opened.

Table 8-11 GetSecurityMode - .NET (C#)

Syntax	ESecurityMode SimulationRuntimeManager::SecurityMode { get; };	
Parameters	None	
Return values	Runtime error code	Description
	ESecurityMode::Unsecured	No security is used.
	ESecurityMode::EncryptedCommunication	Secure communication
	ESecurityMode::Unknown	No port is opened.

8.1.1 RunAutodiscover()**Description**

This function identifies all Runtime Managers that are on the network and that are ready to establish a remote connection.

NOTE

The function identifies Runtime Managers as of S7-PLCSIM Advanced V5.0.

Requirements

- The Runtime Manager must be running and allowing remote connections.
- The firewall of the remote PC must not block traffic on the selected UDP port.
- Devices in the local network (such as routers, switches, firewalls) must not block multicast packets of the selected class.

RunAutodiscover()

The function starts the identification of the Runtime Manager in the network.

Table 8-12 RunAutodiscover() - Native C++

Syntax	<code>ERuntimeErrorCode RunAutodiscover(UINT32 in_Timeout = 2000);</code>	
Parameters	<code>UINT32 in_Timeout</code> A timeout value in milliseconds that defines how long the local Runtime Manager waits for responses from the Remote Manager. A value between 500 ms and 30000 ms is valid. Default: 2000 ms.	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_WRONG_ARGUMENT</code>	The timeout value is outside the permissible range.
	<code>SREC_AUTODISCOVER_ALREADY_RUNNING</code>	A <code>RunAutodiscover()</code> call is already running in the background. Wait for the message <code>SRRSI_DISCOVER_FINISHED</code> in the callback function. See <code>EAutodiscoverType</code> (Page 294).
	<code>SREC_TIMEOUT</code>	Communication errors in the local Runtime Manager.

Table 8-13 RunAutodiscover() - .NET (C#)

Syntax	<code>void RunAutodiscover(UInt32 in_Timeout = 2000);</code>	
Parameters	<code>UInt32 in_Timeout</code> A timeout value in milliseconds that defines how long the local Runtime Manager waits for responses from the Remote Manager. A value between 500 ms and 30000 ms is valid. Default: 2000 ms.	
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.WrongArgument</code>	The timeout value is outside the permissible range.
	<code>ERuntimeErrorCode.AutodiscoverAlreadyRunning</code>	A <code>RunAutodiscover()</code> call is already running in the background. Wait for the message <code>AutodiscoverFinished</code> in the callback function. See <code>EAutodiscoverType</code> (Page 294).
	<code>ERuntimeErrorCode.Timeout</code>	Communication errors in the local Runtime Manager.

8.2 Events for ISimulationRuntimeManager

Events for runtime instances and remote connections

The following events are triggered for the ISimulationRuntimeManager interface:

Table 8-14 Events for ISimulationRuntimeManager

Event	Cause
OnConfigurationChanged events (Page 75)	The Runtime Manager configuration has changed. The Control Panel uses this event to update the list of available instances.
OnRuntimeManagerLost (Page 77)	The connection to the Runtime Manager is interrupted.
OnAutodiscoverData events (Page 80)	The network searches for Runtime Managers which are ready to establish a remote connection.

8.2.1 OnConfigurationChanged events

OnConfigurationChanged

The OnConfigurationChanged event reports configuration changes.

Table 8-15 OnConfigurationChanged() - .NET C#

Syntax	<code>event Delegate_SRCC_UINT32_UINT32_INT32 OnConfigurationChanged(); event Delegate_SRCC_UINT32_UINT32_INT32 OnConfigurationChanged(ERuntimeCon- figChanged in_RuntimeConfigChanged, uint in_Param1, uint in_Param2, int in_Param3);</code>
Parameters	<code>in_RuntimeConfigChanged, in_Param1, in_Param2, in_Param3</code>
Return values	None
Exceptions	None
Note	The event handler method runs in a separate thread.

RegisterOnConfigurationChangedCallback()

When receiving the event the registered callback function will be called. The callback function runs in a separate thread. More callback functions can be registered to the event.

Table 8-16 RegisterOnConfigurationChangedCallback() - Native C++

Syntax	<code>void RegisterOnConfigurationChangedCallback(EventCallback_SRCC_UINT32_UINT32- _INT32 in_CallbackFunction);</code>
Parameters	<code>in_CallbackFunction</code>
Return values	None
Note	A callback function to subscribe to the event

RegisterOnConfigurationChangedEvent()

When receiving the event, the registered event object is set to signaled state. There can be only one event object registered to the event. Registering a new one will automatically unregister the previous one

Table 8-17 RegisterOnConfigurationChangedEvent() - Native C++

Syntax	<code>void RegisterOnConfigurationChangedEvent();</code> <code>void RegisterOnConfigurationChangedEvent(HANDLE* in_Event);</code>
Parameters	- None An internal event object is registered. - HANDLE* A handle for a user-specific event object. The event object is registered.
Return values	None

UnregisterOnConfigurationChangedCallback()

Unregisters the callback function. When the event occurs, no callback function is called.

Table 8-18 UnregisterOnConfigurationChangedCallback() - Native C++

Syntax	<code>void UnregisterOnConfigurationChangedCallback();</code>
Parameters	None
Return values	None

UnregisterOnConfigurationChangedEvent()

Unregisters the event object.

Table 8-19 UnregisterOnConfigurationChangedEvent() - Native C++

Syntax	<code>void UnregisterOnConfigurationChangedEvent();</code>
Parameters	None
Return values	None

Table 8-20 UnregisterOnConfigurationChangedEvent() - .NET (C#)

Syntax	<code>void UnregisterOnConfigurationChangedEvent();</code>
Parameters	None
Return values	None

WaitForOnConfigurationChangedEvent()

The function will block until the registered event object is in the signaled state or the timeout interval elapses.

Table 8-21 WaitForOnConfigurationChangedEvent() - Native C++

Syntax	<code>bool WaitForOnConfigurationChangedEvent();</code> <code>bool WaitForOnConfigurationChangedEvent(uint in_Time_ms);</code>
Parameters	• None: The time limit is set to INFINITE. • UINT32 in_Time_ms: Value for the time limit in milliseconds.
Return values	• true: If the event object was set to the signaled state. • false: If no event was received during the defined time limit.
Note	The function blocks the CoSimulation program until a software download is set to the signaled state or the timeout interval is exceeded.

Table 8-22 WaitForOnConfigurationChangedEvent() - .NET (C#)

Syntax	<code>bool WaitForOnConfigurationChangedEvent();</code> <code>bool WaitForOnConfigurationChangedEvent(uint in_Time_ms);</code>
Parameters	• None: The time limit is set to INFINITE. • UINT32 in_Time_ms: Value for the time limit in milliseconds.
Return values	• true: If the event object was set to the signaled state. • false: If no event was received during the defined time limit.
Note	The function blocks the CoSimulation program until a software download is set to the signaled state or the timeout interval is exceeded.

8.2.2 OnRuntimeManagerLost events**OnRuntimeManagerLost**

Registers or unregisters an event handler method.

Table 8-23 OnRuntimeManagerLost - .NET (C#)

Syntax	<code>event Delegate_Void OnRuntimeManagerLost;</code>
Parameters	None. See Delegate_Void (Page 248) .
Return values	None
Exceptions	None
Note	The event handler method runs in a separate thread.

RegisterOnRuntimeManagerLostCallback()

When the event occurs, the registered callback function is called. Multiple callback functions can be registered for the event.

Table 8-24 RegisterOnRuntimeManagerLostCallback() - Native C++

Syntax	<code>void RegisterOnRuntimeManagerLostCallback(EventCallback_VOID in_CallbackFunction);</code>
Parameters	EventCallback_VOID in_CallbackFunction: A callback function that subscribes to the event. See EventCallback_VOID (Page 240).
Return values	None
Note	The event handler method runs in a separate thread.

RegisterOnRuntimeManagerLostEvent()

When the event occurs, the registered event object is set to the signaled state. Only one event object can be registered for the event. Registration of a new event object causes the previous event object to be deleted.

Table 8-25 RegisterOnRuntimeManagerLostEvent() - Native C++

Syntax	<code>void RegisterOnRuntimeManagerLostEvent();</code> <code>void RegisterOnRuntimeManagerLostEvent(HANDLE* in_Event);</code>
Parameters	None: An internal event handle is registered. HANDLE* in_Event: A user-specific event handle is registered.
Return values	None

Table 8-26 RegisterOnRuntimeManagerLostEvent() - .NET (C#)

Syntax	<code>void RegisterOnRuntimeManagerLostEvent();</code>
Parameters	None
Return values	None

UnregisterOnRuntimeManagerLostCallback()

Unregisters the callback function. When the event occurs, no callback function is called.

Table 8-27 UnregisterOnRuntimeManagerLostCallback() - Native C++

Syntax	<code>void UnregisterOnRuntimeManagerLostCallback();</code>
Parameters	Optional parameter of the same type as for the Register*() function If you use the parameter, it uninstalls the specific function; otherwise, the last registered callback function is unregistered.
Return values	None

UnregisterOnRuntimeManagerLostEvent()

Unregisters the event object.

Table 8-28 UnregisterOnRuntimeManagerLostEvent() - Native C++

Syntax	<code>void UnregisterOnRuntimeManagerLostEvent();</code>
Parameters	None
Return values	None

Table 8-29 UnregisterOnRuntimeManagerLostEvent() - .NET (C#)

Syntax	<code>void UnregisterOnRuntimeManagerLostEvent();</code>
Parameters	None
Return values	None

WaitForOnRuntimeManagerLostEvent()

The function will block the program until the registered event object is set to the signaled state or the timeout interval is exceeded.

Table 8-30 WaitForOnRuntimeManagerLostEvent() - Native C++

Syntax	<code>bool WaitForOnRuntimeManagerLostEvent();</code> <code>bool WaitForOnRuntimeManagerLostEvent(UINT32 in_Time_ms);</code>
Parameters	- None: The time limit is set to INFINITE. - UINT32 in_Time_ms: Value for the time limit in milliseconds.
Return values	- true: If the event object was set to the signaled state. - false: If no event was received during the defined timeout interval.

Table 8-31 WaitForOnRuntimeManagerLostEvent() - .NET (C#)

Syntax	<code>bool WaitForOnRuntimeManagerLostEvent();</code> <code>bool WaitForOnRuntimeManagerLostEvent(UInt32 in_Time_ms);</code>
Parameters	- None: The time limit is set to INFINITE. - UInt32 in_Time_ms: Value for the time limit in milliseconds.
Return values	- true: If the event object was set to the signaled state. - false: If no event was received during the defined timeout interval.

8.2.3 OnAutodiscoverData events

OnAutodiscoverData

Registers or unregisters an event handler method.

Table 8-32 OnAutodiscoverData - .NET (C#)

Syntax	event Delegate_SRRSI_AD OnAutodiscoverData
Parameters	None. See Delegate_SRRSI_AD (Page 249) .
Return values	None
Exceptions	None
Note	The event handler method runs in a separate thread.

RegisterOnAutodiscoverCallback()

When the event occurs, the registered callback function is called. Multiple callback functions can be registered for the event.

Table 8-33 RegisterOnAutodiscoverCallback() - Native C++

Syntax	void RegisterOnAutodiscoverCallback(EventCallback_SRRSI_AD in_CallbackFunction);
Parameters	EventCallback_SRRSI_AD in_CallbackFunction: Pointer to a user-defined callback function. See EventCallback_SRRSI_AD (Page 241).
Return values	None

UnregisterOnAutodiscoverCallback()

Unregisters the callback function.

Table 8-34 UnregisterOnAutodiscoverCallback() - Native C++

Syntax	void UnregisterOnAutodiscoverCallback();
Parameters	Optional parameter of the same type as for the Register*() function If you use the parameter, it uninstalls the specific function; otherwise, the last registered callback function is unregistered.
Return values	None

IsInitialized() / IsInitialized { get; }

Returns a value that indicates whether the API was successfully initialized.

Table 8-35 IsInitialized() - Native C++

Syntax	bool IsInitialized();
Parameters	None
Return values	- false: If the API was not initialized. - true: If the API was initialized.

Table 8-36 IsInitialized { get; } - .NET (C#)

Syntax	bool IsInitialized { get; }
Parameters	None
Return values	• false: If the API was not initialized. • true: If the API was initialized.

IsRuntimeManagerAvailable() / IsRuntimeManagerAvailable { get; }

The function returns `false` when the connection to Runtime Manager is interrupted. This happens only when the Runtime Manager process is closed.

Subscribe to the `OnRuntimeManagerLost()` event to find out whether the connection is interrupted. See Events [\(Page 75\)](#).

Table 8-37 IsRuntimeManagerAvailable() - Native C++

Syntax	bool IsRuntimeManagerAvailable();
Parameters	None
Return values	• false: If the connection is interrupted. • true: If the connection is active.

Table 8-38 IsRuntimeManagerAvailable { get; } - .NET (C#)

Syntax	bool IsRuntimeManagerAvailable{ get; }
Parameters	None
Return values	• false: If the connection is interrupted. • true: If the connection is active.

Shutdown()

Ends communication with Runtime Manager and clears the interfaces.

Call this function in the following cases:

- Immediately before the API library (DLL) is unregistered (native C++).
- When your application is no longer using Runtime Manager.

Table 8-39 Shutdown() - Native C++

Syntax	ERuntimeErrorCode Shutdown()	
Parameters	None	
Return values	Runtime error code	Description
	SREC_OK	The function call is successful.

Table 8-40 Shutdown() - .NET (C#)

Syntax	void Shutdown()
Parameters	None
Return values	None

Instance

9.1 Interfaces for Instances

Dispose()

Deletes the managed interface and unloads the native components of the user interfaces.

Table 9-1 Dispose() - .NET (C#)

Syntax	<code>void Dispose();</code>
Parameters	None
Return values	None

GetID() / ID { get; }

Returns the instance ID. The ID is assigned by Runtime Manager when the instance is registered.

Table 9-2 GetID() - Native C++

Syntax	<code>INT32 GetID();</code>
Parameters	None
Return values	INT32: Instance ID

Table 9-3 ID { get; } - .NET (C#)

Syntax	<code>UInt32 ID { get; }</code>
Parameters	None
Return values	UInt32: Instance ID
Exceptions	None

GetName() / Name { get; }

Returns the name of the instance.

Table 9-4 GetName() - Native C++

Syntax	<code>ERuntimeErrorCode GetName(WCHAR inout_Name[], UINT32 in_ArrayLength);</code>	
Parameters	- WCHAR inout_Name[]: A user-allocated storage for the name of the instance. The field length should be at least as long as DINSTANCE_NAME_MAX_LENGTH. See Definitions and constants (Page 255). - UINT32 in_ArrayLength: Field length (Wide character)	
Return values	Runtime error code	Description

Return values	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_WRONG_ARGUMENT	The name does not fit in the storage.
Example C++	<pre>ISimulationRuntimeManager * api = NULL; ERuntimeErrorCode result = InitializeApi(&api); IInstance* psa = NULL; if (result == SREC_OK) { result = api->RegisterInstance(&psa); } WCHAR name[DINSTANCE_NAME_MAX_LENGTH]; if (result == SREC_OK) { result = psa->GetName(name, DINSTANCE_NAME_MAX_LENGTH); }</pre>	

Table 9-5 Name { get; } - .NET (C#)

Syntax	string Name { get; }	
Parameters	None	
Return values	Name of the instance.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.

GetCPUType()

Returns the CPU type of the simulated PLC instance.

Table 9-6 GetCPUType() - Native C++

Syntax	ECPUType GetCPUType();	
Parameters	None	
Return values	An enumeration element that defines the CPU type. See ECPUType (Page 280) .	

SetCPUType()

Sets the CPU type of the simulated PLC instance. A change of CPU type occurs only when the controller is restarted.

NOTE

When you create a new instance and set the CPU type, S7-PLCSIM Advanced uses precise simulation to simulate the PLC after PowerOn. The API loads the quantity structure based on the selected type. You can only overload the CPU type using a hardware-compatible PLC.

9.1 Interfaces for Instances

Table 9-7 SetCPUType() - Native C++

Syntax	<code>void SetCPUType(ECPUType in_Value);</code>
Parameters	ECPUType in_Value: Defines which CPU type is simulated at the start of the instance. When a different CPU type is loaded via STEP 7 or from the Virtual Memory Card, this CPU type applies.
Return values	None

CPUType { get; set; }

Returns or sets the CPU type of the simulated PLC instance. A change of CPU type occurs only when the controller is restarted.

When a different CPU type is loaded via STEP 7 or from the Virtual Memory Card, this CPU type applies.

NOTE

When you create a new instance and set the CPU type, S7-PLCSIM Advanced uses precise simulation to simulate the PLC after `PowerOn`. The API loads the quantity structure based on the selected type. You can only overload the CPU type using a hardware-compatible PLC.

Table 9-8 CPUType { get; set; } - .NET (C#)

Syntax	<code>ECPUType CPUType { get; set; }</code>
Parameters	None
Return values	An enumeration element that defines the CPU type.
Exceptions	None

GetArticleNumber() / ArticleNumber { get; }

Returns the set ArticleNumber of the simulated instance. If you only set the CPU type, the method returns the article number of the specified CPU type.

To use this method, you must first create a 20-character array for the article number using the constant `HWDB_B1_ORDER_ID_LEN` in `SimulationRuntimeAPI.h`. Pass a pointer to the array into the method. The method fills the array with a string containing the article number plus a zero in position 19.

Table 9-9 GetArticleNumber() - Native C++

Syntax	<code>GetArticleNumber(WCHAR *out_ArticleNumber);</code>
Parameters	<code>WCHAR *out_ArticleNumber</code>
Return values	None

Table 9-10 ArticleNumber{ get; } - .NET (C#)

Syntax	<code>System::String^ ArticleNumber { System::String^ get(); }</code>
Parameters	None
Return values	<code>System::String^</code> containing the article number

GetCommunicationInterface()

This property is now read-only. To set the network mode, refer to the NetworkMode section.

Returns the communication interface of the simulated controller: Local communication (Softbus) or TCPIP.

Table 9-11 GetCommunicationInterface() - Native C++

Syntax	<code>ECommunicationInterface GetCommunicationInterface();</code>
Parameters	None
Return values	<code>SRCI_NONE</code> Cannot be selected. Is returned if the instance interface is no longer valid. <code>SRCI_SOFTBUS</code> Is returned if the simulated controller uses the Softbus. <code>SRCI_TCPIP</code> Is returned if the simulated controller communicates over the virtual adapter.

CommunicationInterface { get; }

This property is now read-only. To set the network mode, refer to the NetworkMode section [\(Page 80\)](#).

Returns the communication interface of the simulated controller: Local communication (Softbus) or TCPIP.

Table 9-12 CommunicationInterface { get; } - .NET (C#)

Syntax	<code>ECommunicationInterface CommunicationInterface { get; }</code>
Parameters	None
Return values	<code>ECommunicationInterface.None</code> Cannot be selected. Is returned if the instance interface is no longer valid. <code>ECommunicationInterface.Softbus</code> Is returned if the simulated controller uses the Softbus. <code>ECommunicationInterface.TCPIP</code> Is returned if the simulated controller communicates over the virtual adapter.
Exceptions	None

GetInfo() / Info { get; }

Returns a structure that provides information about the instance.

Table 9-13 GetInfo() - Native C++

Syntax	<code>SInstanceInfo GetInfo();</code>
Parameters	None
Return values	<code>SInstanceInfo</code> : A structure that provides information about the instance. See <code>SInstanceInfo</code> (Page 264) .

9.1 Interfaces for Instances

Table 9-14 Info { get; } - .NET (C#)

Syntax	SInstanceInfo Info { get; }
Parameters	None
Return values	SInstanceInfo: A structure that provides information about the instance.
Exceptions	None

GetNetInterfaceMapping()

Returns an interface index of the PC interface to which a specific CPU interface is mapped. Use the `GetNetInterfaces()` function to get a list of PC interfaces with interface index. The value 0 means that no mapping has been set.

Note that `GetNetInterfaceMapping()` only returns a mapping table and not the actual address while the instance is running.

NOTE

If two network adapters have the same MAC address, the `GetNetInterfaceMapping()` function might not return the adapter that you previously set using the `SetNetInterfaceMapping()` function. This issue can occur specifically with adapters that use the 'MAC Address Pass-through' feature, leading to potential conflicts in adapter identification and mapping.

Table 9-15 GetNetInterfaceMapping() - Native C++ und .NET (C#)

Syntax C++	INT32 GetNetInterfaceMapping(EPLCInterface in_PLCInterfaceID);	
Syntax .NET (C#)	int GetNetInterfaceMapping(EPLCInterface in_PLCInterfaceID);	
Parameters	EPLCInterface in_PLCInterfaceID: ID of the PLC interface for which you want to get the mapping.	
Return values	Value	Description
	Greater than 0	Index of mapped computer interface
	0	No mapping is defined for the PLC interface or the NetworkMode is Softbus
	-1	Cannot get the mapping

SetNetInterfaceMapping()

Sets interface mappings between CPU interface and PC interface. Ethernet packets that a certain PC interface receives are forwarded to the assigned CPU interface. Use the `GetNetInterfaces()` function to get a list of PC interfaces.

The function is not valid for the Softbus NetworkMode.

Table 9-16 SetNetInterfaceMapping() - Native C++

Syntax	<pre>ERuntimeErrorCode SetNetInterfaceMapping(EPLCInterface in_PLCInterfaceID, DWORD in_PCInterfaceIndex); ERuntimeErrorCode SetNetInterfaceMapping(EPLCInterface in_PLCInterfaceID, const WCHAR* in_PCInterfaceName); ERuntimeErrorCode SetNetInterfaceMapping(EPLCInterface in_PLCInterfaceID, const SMACAddress* in_MACAddressParam);</pre>	
Parameters	EPLCInterface in_PLCInterfaceID: ID of the CPU interface for which a MAC address is to be set. DWORD* in_PCInterfaceIndex: Index of the PC interface; if this value is 0, the mapping is deleted. const WCHAR* in_PCInterfaceName: Name of the PC interface const UMACAddress* in_MACAddressParam: Hardware MAC address of the PC interface in the form FF-FF-FF-FF-FF-FF, where the octet separator is a "-". You get these values with the function GetNetInterfaces().	
Return values	Runtime error code	Description
	SREC_OK	The function call is successful.
	SREC_WRONG_COMMUNICATION_INTERFACE	This error is returned in TCPIP modes in case the specified/mapped computer interface in the sim_hwdb.ini file is disabled or it is the PLCSIM Virtual Adapter.
	SREC_ALREADY_EXISTS	The PC interface has already been assigned to another CPU interface.
	SREC_CONFIG_FILE_ERROR	An error occurred when writing the configuration to the file "sim_hwdb.ini".
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_INSTANCE_ALREADY_RUNNING	Bindings cannot change while the instance is running.
	SREC_NOT_SUPPORTED	You cannot call the function in Softbus NetworkMode.

Table 9-17 SetNetInterfaceMapping() - .NET (C#)

Syntax	<pre>SetNetInterfaceMapping(EPLCInterface in_PLCInterfaceID, uint in_PCInterfaceIndex); SetNetInterfaceMapping(EPLCInterface in_PLCInterfaceID, const String in_PCInterfaceName); SetNetInterfaceMapping(EPLCInterface in_PLCInterfaceID, const SMACAddress in_MACAddressParam);</pre>	
Exceptions	Runtime error code	Description
	ERuntimeErrorCode.WrongCommunicationInterface	This error is returned in TCPIP modes in case the specified/mapped computer interface in the sim_hwdb.ini file is disabled or it is the PLCSIM Virtual Adapter.
	ERuntimeErrorCode.AlreadyExists	The PC interface has already been assigned to another CPU interface.
	ERuntimeErrorCode.ConfigFileError	An error occurred when writing the configuration to the file "sim_hwdb.ini".

Exceptions	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.InstanceAlreadyRunning</code>	Bindings cannot change while the instance is running.
	<code>ERuntimeErrorCode.NotSupported</code>	You cannot call the function in Softbus NetworkMode.

CheckNetInterfaceMapping()

The function checks the validity of the interface mapping in order to start the instance. All PC instances already running are checked for possible duplicate mappings. The function also checks whether the binding to the PC network interfaces is valid. The function is not valid for the Softbus NetworkMode.

Table 9-18 CheckNetInterfaceMapping() - Native C++

Syntax	<code>ERuntimeErrorCode CheckNetInterfaceMapping();</code>	
Parameters	None	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function call is successful.
	<code>SREC_INVALID_CONFIGURATION</code>	The configuration is invalid. Examples: - The current instance has a mapped PC network interface that is already in use by another running instance. - A mapped PLCSIM Virtual Ethernet Adapter exists. - You have set the Multi-Adapter Network Mode (non-promiscuous mode) without interface mapping.
	<code>SREC_VIRTUAL_SWITCH_MISCONFIGURED</code>	The binding of an S7-PLCSIM Advanced Virtual Switch on a PC network interface is not correct, or the driver of an S7-PLCSIM Advanced Virtual Switch is not functioning properly. If the binding is not correct, call the <code>SetNetInterfaceBindings()</code> function or set the binding manually before starting the instance.
	<code>SREC_INTERNAL_ERROR</code>	The system call has caused an internal error.
	<code>SREC_WRONG_COMMUNICATION_INTERFACE</code>	This error is returned in TCPIP modes in case the specified/mapped computer interface in the <code>sim_hwdb.ini</code> file is disabled or it is the PLCSIM Virtual Adapter.
	<code>SREC_INTERFACE_REMOVED</code>	The instance is not registered in Runtime Manager.
	<code>SREC_PCAP_DRIVER_NOT_RUNNING</code>	The PCAP driver is not running.
	<code>SREC_NOT_SUPPORTED</code>	You cannot call the function in the Softbus NetworkMode.

Table 9-19 CheckNetInterfaceMapping() - .NET (C#)

Syntax	ERuntimeErrorCode CheckNetInterfaceMapping();	
Return values	Runtime error code	Description
	ERuntimeErrorCode.OK	The function call is successful.
	ERuntimeErrorCode.InvalidConfiguration	The configuration is invalid. Examples: - The current instance has a mapped PC network interface that is already in use by another running instance. - A mapped PLCSIM Virtual Ethernet Adapter exists. - You have set the Multi-Adapter Network Mode (non-promiscuous mode) without interface mapping.
	ERuntimeErrorCode.VirtualSwitchMisconfigured	The binding of an S7-PLCSIM Advanced Virtual Switch on a PC network interface is not correct, or the driver of an S7-PLCSIM Advanced Virtual Switch is not functioning properly. If the binding is not correct, call the SetNetInterfaceBindings() function or set the binding manually before starting the instance.
	ERuntimeErrorCode.InternalError	The system call has caused an internal error.
	ERuntimeErrorCode.WrongCommunicationInterface	This error is returned in TCPIP modes in case the specified/mapped computer interface in the sim_hwdb.ini file is disabled or it is the PLCSIM Virtual Adapter.
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.PCAPDriverNotRunning	The PCAP driver is not running.
	ERuntimeErrorCode.NotSupported	You cannot call the function in the Softbus NetworkMode.

UnregisterInstance()

Unregisters this instance from Runtime Manager.

NOTE

Loss of the interfaces

Other applications that are connected to this instance will lose their interface to this instance.

NOTE

Do not unregister an instance or free the API while either the control panel or the API is starting an instance. Wait until the PowerOn action for this instance is complete.

9.2 Controller - Information and settings

Table 9-20 UnregisterInstance() - Native C++

Syntax	<code>ERuntimeErrorCode UnregisterInstance();</code>	
Parameters	None	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.

Table 9-21 UnregisterInstance() - .NET (C#)

Syntax	<code>void UnregisterInstance();</code>	
Parameters	None	
Return values	None	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.

9.2 Controller - Information and settings

GetControllerName() / ControllerName { get; }

Returns the downloaded name of the simulated PLC instance.

Table 9-22 GetControllerName() - Native C++

Syntax	<code>ERuntimeErrorCode GetControllerName(WCHAR inout_Name[], UINT32 in_ArrayLength);</code>	
Parameters	<code>WCHAR inout_Name[]</code>: A user-allocated storage for the name. <code>UINT32 in_ArrayLength</code>: The length of the storage.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INDEX_OUT_OF_RANGE	The name does not fit in the storage.

Table 9-23 ControllerName { get; } - .NET (C#)

Syntax	string ControllerName { get; }	
Parameters	None	
Return values	string: The downloaded name of the simulated PLC instance.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.

GetControllerShortDesignation() / ControllerShortDesignation { get; }

Returns the downloaded short designation of the simulated PLC instance.

Table 9-24 GetControllerShortDesignation() - Native C++

Syntax	ERuntimeErrorCode GetControllerShortDesignation(WCHAR inout_ShortDesignation[], UINT32 in_ArrayLength);	
Parameters	- WCHAR inout_ShortDesignation[]: A user-allocated storage for the short designation. - UINT32 in_ArrayLength: The length of the storage.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INDEX_OUT_OF_RANGE	The name does not fit in the storage.

Table 9-25 ControllerShortDesignation { get; } - .NET (C#)

Syntax	string ControllerShortDesignation { get; }	
Parameters	None	
Return values	string: The downloaded short designation of the simulated PLC instance.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.

GetControllerIPCount()

Returns the number of configured IP addresses of the simulated PLC instance. If the function fails, the return value is 0.

Table 9-26 GetControllerIPCount() - Native C++

Syntax	UINT32 GetControllerIPCount();
Parameters	None
Return values	UINT32: Number of configured IP addresses of the simulated PLC instance.

GetControllerIP() / ControllerIP { get; }

Returns a configured IP address of the instance.

Table 9-27 GetControllerIP() - Native C++

Syntax	UIP GetControllerIP(); UIP GetControllerIP(UINT32 in_Index);
Parameters	WCHAR in_Index: The index of the IP address you want to receive. The index must be less than the value you receive from GetControllerIPCount(). The default setting is 0.
Return values	UIP: IP address of the simulated PLC instance. If the function fails, the return value is 0.

Table 9-28 ControllerIP { get; } - .NET (C#)

Syntax	string[] ControllerIP { get; }
Parameters	None
Return values	string: All downloaded IP addresses of the simulated PLC instance. If the function fails, the field is empty.
Exceptions	None

GetControllerIPSuite4() / ControllerIPSuite4 { get; }

Returns the IP suite instance. If the "Softbus" communication interface is used, the subnet mask and default gateway are 0.

Table 9-29 GetControllerIPSuite4() Native C++

Syntax	SIPSuite4 GetControllerIPSuite4(); SIPSuite4 GetControllerIPSuite4(UINT32 in_Index);
Parameters	WCHAR in_Index: The index of the IP address you want to receive. The index must be less than the value you receive from GetControllerIPCount(). The default setting is 0.
Return values	SIPSuite4: The IP suite of the simulated PLC instance. If the function fails, the return values are 0.

Table 9-30 ControllerIPSuite4 { get; } - .NET (#)

Syntax	SIPSuite4[] ControllerIPSuite4 { get; };
Parameters	None
Return values	SIPSuite4[]: All downloaded IP suites of the simulated PLC instance. If the function fails, the field is empty.
Exceptions	None

SetIPSuite()

Sets the IP suite of the network interface of a simulated PLC instance.

To change the subnet mask or gateway, you must also change the IP address. If you do not change the IP address, the API does not apply your other changes.

Table 9-31 SetIPSuite() - Native C++

Syntax	<code>ERuntimeErrorCode SetIPSuite(UINT32 in_InterfaceID, SIPSuite4 in_IPSuite, bool in_IsRemanent);</code>	
Parameters	<code>UINT32 in_InterfaceID</code>: The ID of the network interface. <code>SIPSuite4 in_IPSuite</code>: The IP suite that is to be assigned to the network interface. The IP suite contains the IP address, the subnet mask and the standard gateway. If the communication interface is "Softbus", the subnet mask and standard gateway are ignored. <code>bool in_IsRemanent</code>: If <code>true</code>, the IP suite is saved after restart of the simulated PLC instance. If the communication interface is "Softbus", this flag is ignored.	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_INTERFACE_REMOVED</code>	The instance is not registered in Runtime Manager.
	<code>SREC_TIMEOUT</code>	The function does not return on time.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	The process of the simulated PLC instance is not running.
	<code>SREC_DOES_NOT_EXIST</code>	There is no network interface with this ID.
	<code>SREC_INVALID_OPERATING_STATE</code>	The simulated PLC instance has not yet completed the boot process or is already in the shutdown phase.

Table 9-32 SetIPSuite() - .NET (C#)

Syntax	<code>void SetIPSuite(UInt32 in_InterfaceID, SIPSuite4 in_IPSuite, bool in_IsRemanent);</code>	
Parameters	<code>UInt32 in_InterfaceID</code>: The ID of the network interface. <code>SIPSuite4 in_IPSuite</code>: If the communication interface is "Softbus", the subnet mask and standard gateway are ignored. <code>bool in_IsRemanent</code>: If <code>true</code>, the IP suite is saved after restart of the simulated PLC instance. If the communication interface is "Softbus", this flag is ignored.	
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	The process of the simulated PLC instance is not running.
	<code>ERuntimeErrorCode.DoesNotExist</code>	There is no network interface with this ID.
	<code>ERuntimeErrorCode.InvalidOperatingState</code>	The simulated PLC instance has not yet completed the boot process or is already in the shutdown phase.

GetStoragePath()

Returns the full path of the folder in which the instance stores its data.

Table 9-33 GetStoragePath() - Native C++

Syntax	ERuntimeErrorCode GetStoragePath(WCHAR inout_StoragePath[], UINT32 in_ArrayLength);	
Parameters	- WCHAR inout_StoragePath[]: A user-allocated storage for the storage path. The length of the array should be at least as long as DSTORAGE_PATH_MAX_LENGTH. See Data types (Page 239). - UINT32 in_ArrayLength: Length of the array (Wide character)	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INDEX_OUT_OF_RANGE	The path does not fit in the storage.

SetStoragePath()

Sets the full path of the folder in which the instance stores its data. This can also be a network share.

Set the path before you start the instance. A change to the path takes effect only when the controller is restarted.

If no path is set, the default setting:

<My Documents>\Siemens\Simatic\Simulation\Runtime\Persistence\<Instance Name> is used.

Table 9-34 SetStoragePath() - Native C++

Syntax	ERuntimeErrorCode SetStoragePath(WCHAR* in_StoragePath);	
Parameters	WCHAR* in_StoragePath: Full name of the storage path. The length of the name must be less than DSTORAGE_PATH_MAX_LENGTH. You can use basic Latin alphanumeric characters (A-Z, a-z, 0-9), "!", "=", and "-" to name the instance, but most other special characters, such as currency, Chinese characters, or mathematical symbols are unsupported. See Data types (Page 239).	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INDEX_OUT_OF_RANGE	The length of the path exceeds the limit.
	SREC_WRONG_ARGUMENT	The path contains invalid characters.

StoragePath { get; set; }

Returns or sets the full path of the folder in which the instance stores its retentive data. This can also be a network share.

Set the path before you start the instance. A change to the path takes effect only when the controller is restarted.

If no path is set, the default setting:

<My Documents>\Siemens\Simatic\Simulation\Runtime\Persistence\<Instance Name> is used.

Table 9-35 StoragePath { get; set; } - .NET (C#)

Syntax	string StoragePath { get; set; }	
Parameters	None	
Return values	string: The configured storage path.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.IndexOutOfRange	The length of the path exceeds the limit.
	ERuntimeErrorCode.WrongArgument	The path contains invalid characters.

ArchiveStorage()

The user program, the hardware configuration, and the retentive data are stored in a file to the Virtual SIMATIC Memory Card. `ArchiveStorage()` stores this file as a ZIP file. The simulated PLC instance must be in OFF operating state for this. You must call the `PowerOn()` function at least once to avoid errors.

NOTE

Stored files can contain sensitive information. It is your responsibility to handle this information securely.

Table 9-36 ArchiveStorage() - Native C++

Syntax	ERuntimeErrorCode ArchiveStorage(WCHAR* in_FullFileName);	
Parameters	WCHAR in_FullFileName: The full path to the ZIP file. The path relates to directories of the computer where the API is being called.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INVALID_OPERATING_STATE	The instance is not in OFF operating state.
	SREC_INVALID_ARCHIVE_PATH	The archive path is invalid.
	SREC_CREATE_DIRECTORIES_FAILED	The folder for the ZIP file could not be created.
	SREC_ARCHIVE_STORAGE_FAILED	The ZIP file could not be created.
	SREC_STORAGE_TRANSFER_ERROR	Error during network data transfer. Memory data between client and server computers do not match.
	SREC_NO_STORAGE_PATH_SET	No storage path is set. You must execute the <code>SetStoragePath()</code> function beforehand.
	SREC_ARCHIVE_STORAGE_NOT_CREATED	The storage file could not be created. You must call the <code>PowerOn()</code> function at least once.

9.2 Controller - Information and settings

Table 9-37 ArchiveStorage() - .NET (C#)

Syntax	void ArchiveStorage(string in_FullFileName);	
Parameters	string in_FullFileName: The full path to the ZIP file. The path relates to folders of the computer where the API is being called.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InvalidOperating-State	The instance is not in OFF operating state.
	ERuntimeErrorCode.InvalidArchivePath	The archive path is invalid.
	ERuntimeErrorCode.CreateDirectoryFailed	The folder for the ZIP file could not be created.
	ERuntimeErrorCode.ArchiveStorageNotCreated	The ZIP file could not be created.
	ERuntimeErrorCode.StorageTransferError	Error during network data transfer. Memory data between client and server computers do not match.

RetrieveStorage()

RetrieveStorage() creates a Virtual SIMATIC Memory Card from the archived ZIP file. The simulated PLC instance must be in OFF operating state for this.

Table 9-38 RetrieveStorage() - Native C++

Syntax	ERuntimeErrorCode RetrieveStorage(WCHAR* in_FullFileName);	
Parameters	WCHAR* in_FullFileName: The full path to the ZIP file. The path relates to folders of the computer where the API is being called.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INVALID_OPERATING_STATE	The instance is not in OFF operating state.
	SREC_INVALID_ARCHIVE_PATH	The archive path is invalid.
	SREC_DELETE_EXISTING_STORAGE_FAILED	The old storage cannot be deleted.
	SREC_RETRIEVE_STORAGE_FAILURE	The ZIP file cannot be unzipped.
	SREC_STORAGE_TRANSFER_ERROR	Error during network data transfer. Memory data between client and server computers do not match.
	SREC_NO_STORAGE_PATH_SET	No storage path is set. The user has to execute the SetStoragePath() function beforehand.

Table 9-39 RetrieveStorage() - .NET (C#)

Syntax	<code>void RetrieveStorage(string in_FullFileName);</code>	
Parameters	<code>string in_FullFileName:</code> The full path to the ZIP file. The path relates to folders of the computer where the API is being called.	
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.
	<code>ERuntimeErrorCode.InvalidOperating-State</code>	The instance is not in OFF operating state.
	<code>ERuntimeErrorCode.InvalidArchivePath</code>	The archive path is invalid.
	<code>ERuntimeErrorCode.DeleteExisting-StorageFailed</code>	The old storage cannot be deleted.
	<code>ERuntimeErrorCode.RetrieveStorage-Failure</code>	The ZIP file cannot be unzipped.
	<code>ERuntimeErrorCode.StorageTransferError</code>	Error during network data transfer. Memory data between client and server computers do not match.

NOTE**Editing an instance created with S7-PLCSIM Advanced using S7-PLCSIM Advanced API V3.0 with the functions ArchiveStorage() and RetrieveStorage()**

When you edit an instance created with S7-PLCSIM Advanced V4.0 or higher with the API functions ArchiveStorage() and RetrieveStorage() of V3.0, API V3.0 does not save the entire card content of the virtual memory card. The retain.pms and sim_hwdb.ini files are missing. This means that the password to protect confidential configuration data is not saved and will be lost.

Solution:

You have a variety options:

- Set the password to protect the confidential configuration data before each simulation again using STEP 7, the SIMATIC Automation Tool, or TIA Portal Openness.
- Establish a remote connection of the instance via the Runtime Manager even when the instance communicates locally. In this case, the password is saved to the virtual memory card.
- Do not assign a password to protect confidential configuration data.

CleanupStoragePath()

The function deletes the folder with the Virtual SIMATIC Memory Card of a local instance or a remote instance. To do this, the function checks whether required and invalid files are available. Even if the folder is missing, the function is considered successful.

To make sure that the correct folder is deleted, the function checks whether there are any files that need to be present in the Virtual SIMATIC Memory Card:

- ".\sim_hwdb.ini"
- ".\SIMATIC_MC\SIMATIC.S7S\"
- ".\SIMATIC_MC\RDdata\"

To permanently delete the folder, only the following folders with files are also allowed:

- ".\CrashDump\"
- ".\Traces\"
- ".\retain.pms"

The instance must be in OFF operating state ("PowerOff").

Table 9-40 CleanupStoragePath() - Native C++

Syntax	<code>ERuntimeErrorCode CleanupStoragePath();</code>	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INVALID_OPERATING_STATE	The instance is not in OFF operating state.
	SREC_DELETE_EXISTING_STORAGE_FAILED	The folder with the memory cannot be deleted.
	SREC_INVALID_STORAGE	The memory is invalid. It contains files or folders that are not permitted.

Table 9-41 CleanupStoragePath() - .NET (C#)

Syntax	<code>void CleanupStoragePath();</code>	
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.
	<code>ERuntimeErrorCode.InvalidOperatingState</code>	The instance is not in OFF operating state.
	<code>ERuntimeErrorCode.DeleteExistingStorageFailed</code>	The folder with the memory cannot be deleted.
	<code>SREC_INVALID_STORAGE</code>	The memory is invalid. It contains files or folders that are not permitted.

GetStrictMotionTiming() / StrictMotionTiming { get; }

Returns the current instance setting for the "Strict Motion Timing" feature. The "Strict Motion Timing" feature is located on the S7-PLCSIM Advanced Control Panel.

Table 9-42 GetStrictMotionTiming() - Native C++

Syntax	<code>ERuntimeErrorCode GetStrictMotionTiming (bool* enabled);</code>	
Parameters	bool* enabled: Receives the current setting. true: Active false: Inactive	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_TIMEOUT	The function does not return on time.

Table 9-43 StrictMotionTiming { get; } - .NET (C#)

Syntax	<code>bool StrictMotionTiming { get; }</code>	
Parameters	None	
Return values	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.

SetStrictMotionTiming() / StrictMotionTiming { set; }

Sets the instance setting for the "Strict Motion Timing" feature. The "Strict Motion Timing" feature is located on the S7-PLCSIM Advanced Control Panel.

Table 9-44 SetStrictMotionTiming() - Native C++

Syntax	<code>ERuntimeErrorCode GetStrictMotionTiming (bool* enabled);</code>	
Parameters	bool enabled: The value to be set. true: Active false: Inactive	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INVALID_OPERATING_STATE	The instance is in Power On. The instance must not be in Power On to change the setting.

Table 9-45 StrictMotionTiming { set; } - .NET (C#)

Syntax	<code>bool StrictMotionTiming { set; }</code>	
Parameters	None	
Return values	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.
	<code>ERuntimeErrorCode.InvalidOperating-State</code>	The instance is in Power On. The instance must not be in Power On to change the setting.

9.3 Operating state

PowerOn()

The function creates the process for the Simulation Runtime instance and starts the firmware of the simulated PLC instance.

The PowerOn() state is reached without a timeout value:

1. As soon as an instance was created
2. Before the user program has been started on the CPU

Table 9-46 PowerOn() - Native C++

Syntax	<pre>ERuntimeErrorCode PowerOn(); ERuntimeErrorCode PowerOn(UINT32 in_Timeout_ms);</pre>	
Parameters	• <code>UInt32 in_Timeout_ms</code>: A timeout value in milliseconds. – If no timeout value is set, the function returns as soon as an instance is created and before the user program has been started on the CPU. Subscribe to the <code>OnOperatingStateChanged()</code> event to find out when the operation has been completed. – If the value is greater than 0 (a timeout value of at least 8500 ms is recommended), the function returns when the operation has been completed or after a timeout. – Expected operating states when this function is successful: { <code>SROS_STOP</code> or <code>SROS_RUN</code>, <code>SROS_FREEZE</code>, <code>SROS_HOLD</code> }	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_INTERFACE_REMOVED</code>	The instance is not registered in Runtime Manager.
	<code>SREC_TIMEOUT</code>	The expected operating state does not occur on time.
	<code>SREC_ERROR_LOADING_DLL</code>	The "Siemens.Simatic.Simulation.Runtime.Instance.exe" cannot load the "Siemens.Simatic.PlcSim.Vplc1500.dll" or the "Siemens.Simatic.PlcSim.Vplc1200g2.dll".
	<code>SREC_STORAGE_PATH_ALREADY_IN_USE</code>	The selected path for this instance is already being used by another instance.
	<code>SREC_NO_STORAGE_PATH_SET</code>	No storage path is set. The user has to execute the <code>SetStoragePath()</code> function beforehand.
	<code>SREC_WARNING_ALREADY_EXISTS</code>	Warning: The instance is started.
	<code>SREC_VIRTUAL_SWITCH_MISCONFIGURED</code>	• The S7-PLCSIM Advanced Virtual Switch is configured incorrectly. • The binding of an S7-PLCSIM Advanced Virtual Switch on a PC network interface is not correct, or the driver of an S7-PLCSIM Advanced Virtual Switch is not functioning properly. If the binding is not correct, call the <code>SetNetInterfaceBindings()</code> function or set the binding manually before starting the instance.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	The process of the simulated PLC instance is no longer running.

Return values	SREC_UNSUPPORTED_PCAP_DRIVER	The PCAP driver used is not supported. S7-PLCSIM Advanced supports Npcap as of V0.9995. TCP/IP communication is not possible.
	SREC_WARNING_TRIAL_MODE_ACTIVE	Warning: No license available. You can use the instance without restrictions with the Trial License. Afterwards, the instance is shut down.
	SREC_WARNING_RUNNING_ON_TIA_PORTAL_TEST_SUITE	Warning: No valid license is available, but a "TIA Portal Test Suite" license. S7-PLCSIM Advanced starts with this license. A download from the TIA Portal is possible, but the instance terminates without feedback if the download was not made from the TIA Portal Test Suite.
	SREC_NOT_EMPTY	Warning: No valid license for S7-PLCSIM Advanced is available, but a "TIA Portal Test Suite" license. If this is the case, power-up from the Virtual SIMATIC Memory Card is not supported.
	SREC_COMMUNICATION_INTERFACE_NOT_AVAILABLE	For local communication via Softbus S7-PLCSIM Advanced cannot connect to the Softbus. Solution - Try again to establish the connection. - Close S7-PLCSIM Advanced and the TIA Portal and restart the applications. - Reboot the PC. - Repair the S7-PLCSIM Advanced installation. For TCP/IP communication Another application is connected to the Softbus on your PC. Solution - Close all SIMATIC applications, e.g. TIA Portal, WinCC, PLCSIM. - Reboot the PC. - Repair the S7-PLCSIM Advanced installation.
	SREC_ACCESS_DENIED	No write access in the Storage Directory.
	SREC_PCAP_DRIVER_NOT_RUNNING	The Nmap Packet Capture Driver (Npcap) is not active on the system. Remedy - Start the command line in administrator mode. - Execute the command "net start npcapi".
	SREC_WRONG_COMMUNICATION_INTERFACE	- You have tried to supply the second instance with the communication interface that is different from the first instance that is already switched on. Example: - PowerOn() of the first instance with Softbus is successful. - PowerOn() of the second instance with TCP/IP; the error code is returned. - The specified/mapped computer interface in the sim_hwdb.ini file is disabled or it is the PLCSIM Virtual Adapter.

9.3 Operating state

Return values	SREC_WARNING_PASSWORD_PROTECTION_ERROR	The password protection prevented the memory from being opened. View the restrictions with activated password protection.
	SREC_INVALID_CONFIGURATION	The configuration is invalid. Examples: - The current instance has a mapped PC network interface that is already in use by another running instance. - A mapped PLCSIM Virtual Ethernet Adapter exists. - You have set the Multi-Adapter Network Mode (non-promiscuous mode) without interface mapping.
	SREC_INTERNAL_ERROR	The system call has caused an internal error.
	SREC_LICENSE_NOT_FOUND	This error occurs under one of the following conditions: - There is no valid license available in the Automation License Manager - The Automation License Manager is corrupt
	SREC_CONFIGURED_DEVICE_NOT_FOUND	The article number you provided is invalid.
	SREC_OUT_OF_MEMORY	The computer has insufficient memory. This can occur when you create multiple unspecified instances. Solution If you encounter an out of memory error try one of the following actions: - Add more RAM to your computer. - Register an instance for a specific CPU type. - Increase the Windows virtual memory paging file size.

Table 9-47 PowerOn() - .NET (C#)

Syntax	<pre>ERuntimeErrorCode PowerOn(); ERuntimeErrorCode PowerOn(UInt32 in_Timeout_ms);</pre>	
Parameters	- UInt32 in_Timeout_ms: A timeout value in milliseconds. - If no timeout value is set, the function returns as soon as an instance is created and before the user program has been started on the CPU. Subscribe to the OnOperatingStateChanged() event to find out when the operation has been completed. - If the value is greater than 0 (a timeout value of at least 8500 ms is recommended), the function returns when the operation has been completed or after a timeout. - Expected operating states when this function is successful: <pre>{ EOperatingState.Run or EOperatingState.Stop, EOperatingState.Freeze, E OperatingState.Hold }</pre>	
Return values	Runtime error code	Description
	ERuntimeErrorCode.OK	The function is successful.
	ERuntimeErrorCode.WarningAlreadyExists	Warning: The instance is started.
	ERuntimeErrorCode.UnsupportedPcapDriver	The PCAP driver used is not supported. S7-PLCSIM Advanced supports Npcap as of V0.9995. TCP/IP communication is not possible.

Return values	<code>ERuntimeErrorCode.WarningTrialModeActive</code>	Warning: No license available. You can use the instance without restrictions with the Trial License. Afterwards, the instance is shut down.
	<code>ERuntimeErrorCode.WarningRunningOnTiaPortalTestSuite</code>	Warning: No valid license is available, but a "TIA Portal Test Suite" license. S7-PLCSIM Advanced starts with this license. A download from the TIA Portal is possible, but the instance terminates without feedback if the download was not made from the TIA Portal Test Suite.
	<code>ERuntimeErrorCode.NotEmpty</code>	Warning: No valid license for S7-PLCSIM Advanced is available, but a "TIA Portal Test Suite" license. If this is the case, power-up from the Virtual SIMATIC Memory Card is not supported.
	<code>ERuntimeErrorCode.CommunicationInterfaceNotAvailable</code>	For local communication via Softbus S7-PLCSIM Advanced cannot connect to the Softbus. Solution • Try again to establish the connection. • Close S7-PLCSIM Advanced and the TIA Portal and restart the applications. • Reboot the PC. • Repair the S7-PLCSIM Advanced installation. For TCP/IP communication Another application is connected to the Softbus on your PC. Solution • Close all SIMATIC applications, e.g. TIA Portal, WinCC, PLCSIM. • Reboot the PC. • Repair the S7-PLCSIM Advanced installation.
	<code>ERuntimeErrorCode.AccessDenied</code>	No write access in the Storage Directory.
	<code>ERuntimeErrorCode.PCAPDriverNotRunning</code>	The Nmap Packet Capture Driver (Npcap) is not active on the system. Remedy 1. Start the command line in administrator mode. 2. Execute the command "net start npcapi".
	<code>ERuntimeErrorCode.WrongCommunicationInterface</code>	• You have tried to supply the second instance with the communication interface that is different from the first instance that is already switched on. – Example: 1. PowerOn() of the first instance with Softbus is successful. 2. PowerOn() of the second instance with TCP/IP; the error code is returned. • The specified/mapped computer interface in the <code>sim_hwdb.ini</code> file is disabled or it is the PLCSIM Virtual Adapter.
	<code>ERuntimeErrorCode.WarningPasswordForProtectionError</code>	The password protection prevented the memory from being opened. View the restrictions with activated password protection.

9.3 Operating state

Return values	<code>ERuntimeErrorCode.InvalidConfiguration</code>	The configuration is invalid. Examples: - The current instance has a mapped PC network interface that is already in use by another running instance. - A mapped PLCSIM Virtual Ethernet Adapter exists. - You have set the Multi-Adapter Network Mode (non-promiscuous mode) without interface mapping.
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.Timeout</code>	The expected operating state does not occur on time.
	<code>ERuntimeErrorCode.ErrorLoadingDll</code>	The "Siemens.Simatic.Simulation.Runtime.Instance.exe" cannot load the "Siemens.Simatic.PlcSim.Vplc1500.dll" or the "Siemens.Simatic.PlcSim.Vplc1200g2.dll".
	<code>ERuntimeErrorCode.StoragePathAlreadyInUse</code>	The selected path for this instance is already being used by another instance.
	<code>ERuntimeErrorCode.NoStoragePathSet</code>	The path could not be created. The length of the <code>DSTORAGE_PATH_MAX_LENGTH</code> characters might be exceeded.
	<code>ERuntimeErrorCode.VirtualSwitchMisconfigured</code>	- The S7-PLCSIM Advanced Virtual Switch is configured incorrectly. - The binding of an S7-PLCSIM Advanced Virtual Switch on a PC network interface is not correct, or the driver of an S7-PLCSIM Advanced Virtual Switch is not functioning properly. If the binding is not correct, call the <code>SetNetInterfaceBindings()</code> function or set the binding manually before starting the instance.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	The process of the simulated PLC instance is no longer running.
	<code>ERuntimeErrorCode.InternalError</code>	The system call has caused an internal error.
	<code>ERuntimeErrorCode.LicenseNotFound</code>	This error occurs under one of the following conditions: - There is no valid license available in the Automation License Manager - The Automation License Manager is corrupt
	<code>ERuntimeErrorCode.ConfiguredDeviceNotFound</code>	The article number you provided is invalid.
	<code>ERuntimeErrorCode.OutOfMemory</code>	The computer has insufficient memory. This can occur when you create multiple unspecified instances. Solution

		If you encounter an out of memory error try one of the following actions: • Add more RAM to your computer. • Register an instance for a specific CPU type. • Increase the Windows virtual memory paging file size.
--	--	---

PowerOff()

Shuts down the Simulation Runtime and closes its process.

Table 9-48 PowerOff() - Native C++

Syntax	<pre>ERuntimeErrorCode PowerOff(); ERuntimeErrorCode PowerOff(UINT32 in_Timeout_ms);</pre>	
Parameters	• UINT32 in_Timeout_ms: A timeout value in milliseconds. – If no timeout value is set, the function returns immediately. Subscribe to the <code>OnOperatingStateChanged()</code> event to find out when the operation has been completed. – If the value is greater than 0 (a timeout value of at least 800 ms is recommended), the function returns when the operation has been completed or after a timeout. Expected operating state when this function is successful: { SROS_OFF }	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The expected operating state does not occur on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.

Table 9-49 PowerOff() - .NET (C#)

Syntax	<pre>void PowerOff(); void PowerOff(UInt32 in_Timeout_ms);</pre>	
Parameters	• UInt32 in_Timeout_ms: A timeout value in milliseconds. – If no timeout value is set, the function returns immediately. Subscribe to the <code>OnOperatingStateChanged()</code> event to find out when the operation has been completed. – If the value is greater than 0 (a timeout value of at least 800 ms is recommended), the function returns when the operation has been completed or after a timeout. Expected operating state when this function is successful: { EOperatingState.Stop }	
Return values	None	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The expected operating state does not occur on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.

9.3 Operating state

Run()

Changes the simulated PLC instance to the RUN operating state.

Table 9-50 Run() - Native C++

Syntax	<pre>ERuntimeErrorCode Run(); ERuntimeErrorCode Run(UINT32 in_Timeout_ms);</pre>	
Parameters	- UINT32 in_Timeout_ms: A timeout value in milliseconds. - If no timeout value is set, the function returns immediately. Subscribe to the OnOperatingStateChanged() event to find out when the operation has been completed. - If the value is greater than 0 (a timeout value of at least 20 ms is recommended), the function returns when the operation has been completed or after a timeout. Expected operating states if this function is successful: <pre>{ SROS_STOP , SROS_RUN }</pre>	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The expected operating state does not occur on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_IS_EMPTY	One of the following two conditions applies here: Condition: After creating and starting an instance, you have tried to switch the simulated PLC instance to the RUN operating state without downloading a STEP 7 project. Condition: After creating and enabling an instance with download of a STEP 7 project with enabled password for the protection of confidential configuration data, you have moved the virtual memory card from one system to another. You have tried to start the instance on the new system. The following error code is returned after the start: "SREC_WARNING_PASSWORD_PROTECTION_ERROR" Afterwards, you have tried to switch the simulated PLC instance to the RUN operating state.

Table 9-51 Run() - .NET (C#)

Syntax	<pre>void Run(); void Run(UInt32 in_Timeout_ms);</pre>	
Parameters	- UInt32 in_Timeout_ms: A timeout value in milliseconds. - If no timeout value is set, the function returns immediately. Subscribe to the OnOperatingStateChanged() event to find out when the operation has been completed. - If the value is greater than 0 (a timeout value of at least 20 ms is recommended), the function returns when the operation has been completed or after a timeout. Expected operating states when this function is successful: <pre>{ EOperatingState.Run }</pre>	
Exceptions	Runtime error code	Description

Exceptions	ERuntimeErrorCode.IsEmpty	One of the following two conditions applies here: Condition: After creating and starting an instance, you have tried to switch the simulated PLC instance to the RUN operating state without downloading a STEP 7 project. Condition: After creating and enabling an instance with download of a STEP 7 project with enabled password for the protection of confidential configuration data, you have moved the virtual memory card from one system to another. You have tried to start the instance on the new system. The following error code is returned after the start: "SREC_WARNING_PASSWORD_PROTECTION_ERROR" Afterwards, you have tried to switch the simulated PLC instance to the RUN operating state.
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The expected operating state does not occur on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.

RUN operating state after enabling an instance

When you expect the RUN operating state after enabling an instance, check whether the instance is already in RUN by subscribing to the event `OnOperatingStateChanged()`. As an alternative, call `Run()` immediately after enabling the API function to switch the instance to the RUN operating state.

Stop()

Changes the simulated PLC instance to the STOP operating state.

Table 9-52 Stop() - Native C++

Syntax	<pre>ERuntimeErrorCode Stop(); ERuntimeErrorCode Stop(UINT32 in_Timeout_ms);</pre>	
Parameters	- UINT32 in_Timeout_ms: A timeout value in milliseconds. - If no timeout value is set, the function returns immediately. Subscribe to the <code>OnOperatingStateChanged()</code> event to find out when the operation has been completed. - If the value is greater than 0 (a timeout value of at least 200 ms is recommended), the function returns when the operation has been completed or after a timeout. Expected operating state when this function is successful: { SROS_STOP }	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.

9.3 Operating state

Return values	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The expected operating state does not occur on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.

Table 9-53 Stop() - .NET (C#)

Syntax	<pre>void Stop(); void Stop(UInt32 in_Timeout_ms);</pre>	
Parameters	- UInt32 in_Timeout_ms: A timeout value in milliseconds. - If no timeout value is set, the function returns immediately. Subscribe to the OnOperatingStateChanged() event to find out when the operation has been completed. - If the value is greater than 0 (a timeout value of at least 200 ms is recommended), the function returns when the operation has been completed or after a timeout. Expected operating state if this function is successful: <pre>{ EOperatingState.Stop }</pre>	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The expected operating state does not occur on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.

GetOperatingState() / OperatingState { get; }

Returns the operating state of the simulated PLC instance. When the operating state changes, the OnOperatingStateChanged() (Page 210) event is triggered. For details about the operating state, see Data types (Page 278).

Table 9-54 GetOperatingState() - Native C++

Syntax	EOperatingState GetOperatingState();	
Parameters	None	
Return values	- SROS_INVALID_OPERATING_STATE: If the function fails. - SROS_OFF: If the Simulation Runtime instance is not running. - SROS_BOOTING: If PowerOn() was called while in this state and the simulated PLC instance is not yet ready to start the user program. - SROS_STOP: If the simulated PLC instance is in STOP state. - SROS_STARTUP: If the user program is currently changing from STOP to RUN. - SROS_RUN: If the user program is running. - SROS_FREEZE: If the user program is being stopped (Freeze status).	

	• SROS_HOLD: If the user program is set to HOLD when the breakpoint is reached. • SROS_SHUTTING_DOWN: If <code>PowerOff()</code> was called but the simulated PLC instance is still in the Shutdown phase.
--	---

Table 9-55 OperatingState { get; } - .NET (C#)

Syntax	<code>EOperatingState OperatingState { get; }</code>
Parameters	None
Return values	• EOperatingState.InvalidOperatingState: If the function fails. • EOperatingState.Off: If the Simulation Runtime instance is not running. • EOperatingState.Booting: If <code>PowerOn()</code> was called while in this state and the simulated PLC instance is not yet ready to start the user program. • EOperatingState.Stop: If the simulated PLC instance is in STOP state. • EOperatingState.Startup: If the user program is currently changing from STOP to RUN. • EOperatingState.Run: If the user program is running. • EOperatingState.Freeze: If the user program is being stopped (Freeze status). • EOperatingState.Hold: If the user program is set to HOLD when the breakpoint is reached. • EOperatingState.ShuttingDown: If <code>PowerOff()</code> was called but the simulated PLC instance is still in the Shutdown phase.

MemoryReset()

Shuts down the simulated PLC instance, closes its processes and performs a restart.

Closing the processes automatically performs a general reset.

Table 9-56 MemoryReset() - Native C++

Syntax	<pre>ERuntimeErrorCode MemoryReset(); ERuntimeErrorCode MemoryReset(UINT32 in_Timeout_ms);</pre>	
Parameters	• UINT32 in_Timeout_ms: A timeout value in milliseconds. – If no timeout value is set, the function returns immediately. Subscribe to the <code>OnOperatingStateChanged()</code> event to find out when the operation has been completed. – If the value is greater than 0 (a value of 60000 is recommended), the function returns when the operation has been completed or after a timeout. Expected operating states if this function is successful: <pre>{ SROS_STOP , SROS_RUN }</pre>	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The expected operating state does not occur on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.

9.4 Stop simulation

Table 9-57 MemoryReset() - .NET (C#)

Syntax	<pre>void MemoryReset(); void MemoryReset(UInt32 in_Timeout_ms);</pre>	
Parameters	UInt32 in_Timeout_ms: A timeout value in milliseconds. - If no timeout value is set, the function returns immediately. Subscribe to the <code>OnOperatingStateChanged()</code> event to find out when the operation has been completed. - If the value is greater than 0 (a value of 60000 is recommended), the function returns when the operation has been completed or after a timeout. Expected operating states when this function is successful: { <code>EOperatingState.Run</code>, <code>EOperatingState.Stop</code> }	
Return values	None	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.Timeout</code>	The expected operating state does not occur on time.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	The process of the simulated PLC instance is not running.

9.4 Stop simulation

Freeze state of the simulated PLC instance

To stop a simulation and synchronize simulation partners, you can set a simulated PLC instance to a freeze state via the API. When the instance has reached a synchronization point, the instance sends the event `OnSyncPointReached` to the API clients.

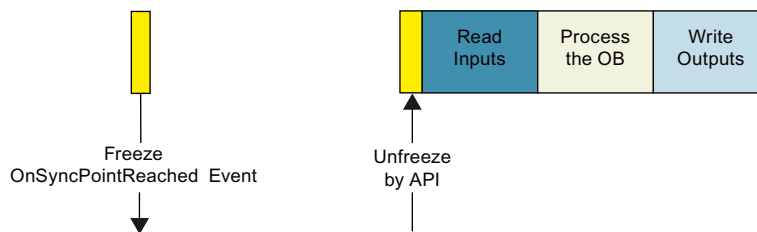


Figure 9-1 Freeze state of the simulated PLC instance

The following occurs in the freeze state:

- The virtual time is stopped.
- The OBs currently being executed are stopped.
The procedure corresponds to the behavior of a program that has reached a breakpoint.
- The user program is no longer executed.
- The simulated PLC instance is still accessible from the TIA Portal.

NOTE**Freeze state during downloading**

To complete a download in freeze state, the simulated PLC instance must pass through a suspended or terminated freeze state.

NOTE**Freeze-state ≠ operating state**

The freeze state is an internal operating state of the simulated PLC instance. It does not correspond to RUN/STOP mode of a CPU. In the freeze state, the instance maintains the last operating state.

- The LED display on the Control Panel and on the Web server accordingly shows RUN or STOP for the instance.
- The instance shows the operating state `SROS_FREEZE / Freeze`, see `EOperatingState` (Page 278).

NOTE**Reaction of an S7-PLCSIM Advanced API to a freeze state**

If you use a SingleStep operating mode, you must register a callback function for the `OnSyncPointReached` event. Calling the mode-specific `RunToNextSyncPoint` method ensures that the freeze state of the simulated PLC instance is exited again.

Synchronization points

A synchronization point always exists **before** inputs are read in, for example at the cycle control point or at the beginning of a cyclic OB.

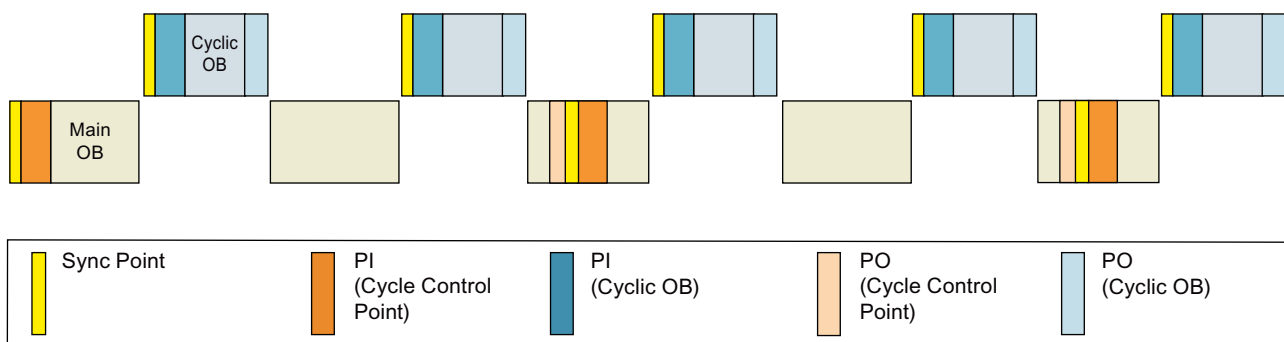


Figure 9-2 Overview of the synchronization points

Trigger freeze state

To trigger the freeze state, following modes are available for the simulated PLC instance:

- SingleStep operating modes
See Synchronize simulation partner cycle-controlled (Page 204).
- TimespanSynchronized operating modes
See Synchronize simulation partner time-controlled (Page 208).

In Default operating mode, the simulated PLC instance does not change into a freeze state.

API functions

- Settings for the cycle control (Page 174)
- GetOperatingMode() (Page 174)
- SetOperatingMode() (Page 174)
- OperatingMode { get; set; } (Page 174)
- EOperatingMode (Page 279)

9.5 Tag list

NOTE

Elements with data types not known to the API (`EDataType.Unknown`) are not included in the tag list.

UpdateTagList()

The function reads the tags from the simulated PLC instance and writes them to the shared storage arranged by name.

If the tag is an array or a structure, there are multiple entries.

In the case of a structure, there is an entry for the structure itself and an additional entry for each structure element.

```
Entry_1: "StructName"
Entry_2: "StructName.ElementName_1"
..
Entry_N: "StructName.ElementName_n"
```

In the case of an array, in this example a two-dimensional array, there is an entry for the array itself and an additional entry for each array element.

```
Entry_1: "ArrayName"
Entry_2: "ArrayName[a,b]", {a} and {b} correspond to the first index of the respective
dimension)
..
```


Entry_N: "ArrayName[x,y]", {x} and {y} correspond to the last index of the respective dimension)

Memory for up to 500000 entries (not PLC tags) is reserved for the list. If the list becomes too large, the function returns the error/exception "NOT_ENOUGH_MEMORY".

If there are problems with the maximum number of entries and not all tags are needed, two filters can be used when refreshing the tag table.

Table 9-58 UpdateTagList() - Native C++

Syntax	<pre>ERuntimeErrorCode UpdateTagList(); ERuntimeErrorCode UpdateTagList(ETagListDetails in_TagListDetails); ERuntimeErrorCode UpdateTagList(ETagListDetails in_TagListDetails, bool in_IsHMIVisibleOnly); ERuntimeErrorCode UpdateTagList(ETagListDetails in_TagListDetails, bool in_IsHMIVisibleOnly, WCHAR* in_DataBlockFilterList);</pre>	
Parameters	- ETagListDetails in_TagListDetails: Every combination of the following four areas: IO: Inputs and Outputs M: Bit memory CT: Counters and Timers DB: Data Blocks The default setting is IOMCTDB. Example: IOM reads only the tags from the area Inputs / Outputs and Bit memory. - bool in_IsHMIVisibleOnly: If true, only tags marked with "HMI Visible" are read. The default setting is true. - WCHAR* in_DataBlockFilterList: A string that includes the name of all data blocks that are supposed to be available in the tag table. The string must be in quotation marks. Example: ""DB_1", "DB_2" "DB_3" "DB_4" "DB_5"" All characters within the quotation marks are interpreted as a DB name. If the data block does not exist in the PLC program, it is not added to the tag table memory. No error is triggered in the process. For this list to be taken into consideration, in_DataBlockFilterList has to be unequal to NULL and in_TagListDetails has to contain "DB".	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_NOT_ENOUGH_MEMORY	More than 500000 entries are requested.
	SREC_WARNING_ALREADY_EXISTS	The tag table is current.
	SREC_WRONG_ARGUMENT	The syntax of in_DataBlockFilterList is invalid. The list has to be 3 characters long; the first and last character have to be a quotation mark.
Example C++	<pre>ISimulationRuntimeManager * api = NULL; ERuntimeErrorCode result = InitializeApi(&api); IInstance* psa = NULL; if (result == SREC_OK) { result = api->RegisterInstance(&psa); }</pre>	

9.5 Tag list

	<pre> if (result == SREC_OK) { // Example: How to filter data blocks // DB Name: TEST_DB, DB Number: 2 result = psa->UpdateTagList(SRTLD_MDB, false, L"\"TEST_DB\""); } </pre>
--	---

Table 9-59 UpdateTagList() - .NET (C#)

Syntax	<pre> void UpdateTagList(); void UpdateTagList(ETagListDetails in_TagListDetails); void UpdateTagList(ETagListDetails in_TagListDetails, bool in_IsHMIVisibleOnly); ERuntimeErrorCode UpdateTagList(ETagListDetails in_TagListDetails, bool in_IsHMIVisibleOnly, string in_DataBlockFilterList); </pre>	
Parameters	- ETagListDetails in_TagListDetails: Every combination of the following four areas: IO: Inputs and Outputs M: Bit memory CT: Counters and Timers DB: Data Blocks The default setting is IOMCTDB. Example: IOM reads only the tags from the area Inputs / Outputs and Bit memory. - bool in_IsHMIVisibleOnly: If true, only tags marked with "HMI Visible" are read. The default setting is true. - string in_DataBlockFilterList: A string that includes the name of all data blocks that are supposed to be available in the tag table. The string must be in quotation marks. Example: "\"DB_1\", \"DB_2\", \"DB_3\", \"DB_4\", \"DB_5\"" All characters within the quotation marks are interpreted as a DB name. If the data block does not exist in the PLC program, it is not added to the tag table memory. No error is triggered in the process. For this list to be taken into consideration, in_DataBlockFilterList has to be unequal to NULL and in_TagListDetails has to contain "DB".	
Return values	None	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.NotEnoughMemory	More than 500000 entries are requested.
	ERuntimeErrorCode.WrongArgument	The syntax of in_DataBlockFilterList is invalid. The list has to be 3 characters long; the first and last character have to be a quotation mark.

GetTagListStatus()

Returns the current update status of the tag list storage.

"inout_TagListDetails" is NONE, if the list needs to be updated.

Table 9-60 GetTagListStatus() - Native C++

Syntax	ERuntimeErrorCode GetTagListStatus(ETagListDetails* out_TagListDetails, bool* out_IsHMIVisibleOnly);	
Parameters	- ETagListDetails out_TagListDetails: Status of the tag list details. SRTLD_NONE when an update of the list is required. - bool out_IsHMIVisibleOnly: If true, only tags marked with "HMI Visible" are available in the list.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.

Table 9-61 GetTagListStatus() - .NET (C#)

Syntax	void GetTagListStatus(out ETagListDetails out_TagListDetails, out bool out_IsHMIVisibleOnly);	
Parameters	- out ETagListDetails out_TagListDetails: Status of the tag list details. ETagListDetails.None when an update of the list is required. - out bool out_IsHMIVisibleOnly: If true, only tags marked with "HMI Visible" are available in the list.	
Return values	None	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.

GetTagInfoCount()

Returns the number of entries in the tag list storage. If the function fails, the return value is 0.

Table 9-62 GetTagInfoCount() - Native C++

Syntax	UINT32 GetTagInfoCount();
Parameters	None
Return values	Number of entries in the tag list storage.

GetTagInfos() / TagInfos { get; }

Returns a list of all tags.

Table 9-63 GetTagInfos() - Native C++

Syntax	ERuntimeErrorCode GetTagInfos(UINT32 in_BufferLength, STagInfo* inout_TagInfos, UINT32* out_TagCount);	
Parameters	- UINT32 in_BufferLength: The number of elements that the storage can accommodate. - STagInfo* inout_TagInfos: The user-allocated storage that accommodates the tags. - UINT32* out_TagCount: Returns the number of tags that were written to the storage.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The elements do not fit in the storage.

Table 9-64 TagInfos { get; } - .NET (C#)

Syntax	STagInfo[] TagInfos { get; }	
Parameters	None	
Return values	An array that contains all available entries of the storage.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.

CreateConfigurationFile()

Writes all entries from the tag list to an XML file.

Table 9-65 CreateConfigurationFile() - Native C++

Syntax	ERuntimeErrorCode CreateConfigurationFile(WCHAR* in_FullFileName);	
Parameters	WCHAR* in_FullFileName: Full file name of the XML file: <Path> + <File name> + <File extension>.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_WRONG_ARGUMENT	The file name is invalid.

Table 9-66 CreateConfigurationFile() - .NET (C#)

Syntax	void CreateConfigurationFile(string in_FullFileName);	
Parameters	None	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.WrongArgument	The file name is invalid.

9.6 I/O access

9.6.1 Synchronizing inputs and outputs

Description

In S7-PLCSIM Advanced the complete scope of the input and output area is used (see GetAreaSize/AreaSize [\(Page 118\)](#)). This is also possible when no I/O module is configured.

Inputs and outputs which are defined via configured I/O modules are synchronized to the defined update of the process image partition (PIP).

Inputs and outputs which are not assigned to an I/O module are synchronized in the cycle control point.

Note the following when synchronizing these inputs and outputs:

- Inputs can only be used as inputs.
You can write values via the API. However, values written via STEP 7 or via communication to the input area will not be visible in the API. For example, if you go online in the TIA Portal and write data to the input area, those data values are not visible in the API. Methods such as ReadBytes() only read values written from the API.
- Outputs can be used as output and as input.
You can write values via the API and via the CPU / STEP 7 user program. If API and user program write to the same area, the values from the API will overwrite the values from the user program.

NOTE

Conditions for synchronizing Fail-safe I/O

Fail-safe inputs and outputs are only synchronized if a Fail-safe module is plugged in and the Fail-safe program uses at least one bit of the module.

9.6.2 Simulate peripheral I/O

S7-PLCSIM Advanced writes to and reads from a memory area. This memory is synchronized with the internal process image of the simulated S7-1500 controller at the cycle control point and when calling cyclic and acyclic OBs (process image partitions, interrupts, events). The direct I/O accesses are made to this memory area. Only one process can access this memory at a given time.

The instance must be in RUN mode to apply changes made by the API.

NOTE

Precedence of write operations

If the STEP 7 user program is writing to a memory range at the same time an API application is writing to the same memory range, the writes from the API application take precedence. Writes from the API application override writes from the STEP 7 program to the simulated PLC instance.

9.6.3 I/O access via address - Reading

InputArea { get; }, MarkerArea { get; }, OutputArea { get; }

Returns an interface that you use to call the .NET functions in this section.

Table 9-67 InputArea { get; } MarkerArea { get; } OutputArea { get; } - .NET (C#)

Syntax	<code>IIOArea InputArea { get; } IIOArea MarkerArea { get; } IIOArea OutputArea { get; }</code>
Parameters	None
Return values	<code>IIOArea</code> : The interface is used to call the "I/O access via address" functions.

GetAreaSize() / AreaSize { get; }

Returns the size of the area in bytes.

Table 9-68 GetAreaSize() - Native C++

Syntax	<code>UINT32 GetAreaSize(EArea in_Area);</code>
Parameters	<code>EArea in_Area</code>: The area whose size you want to receive. Permissible values: {<code>SRA_INPUT</code>, <code>SRA_MARKER</code>, <code>SRA_OUTPUT</code>}. See EArea (Page 278).
Return values	<code>UINT32</code> : Size of the area in bytes. If the function was successful, the value is not equal to 0.

Table 9-69 AreaSize { get; } - .NET (C#)

Syntax	<code>UInt32 InputArea.AreaSize { get; } UInt32 MarkerArea.AreaSize { get; } UInt32 OutputArea.AreaSize { get; }</code>
Parameters	None
Return values	<code>UInt32</code> : Size of the area in bytes. If the function was successful, the value is not equal to 0.

ReadBit()

Reads an individual bit from the area.

See Synchronizing inputs and outputs (Page 117) for information on the API handling of I/O values.

Table 9-70 ReadBit() - Native C++

Syntax	ERuntimeErrorCode ReadBit(EArea in_Area, UINT32 in_Offset, UINT8 in_Bit, bool* out_Value);	
Parameters	- EArea in_Area: The area to be read from. Permissible values: {SRA_INPUT, SRA_MARKER, SRA_OUTPUT}. See EArea (Page 278). - UINT32 in_Offset: The byte offset within the area. The value must be between 0 and the value that GetAreaSize() returns. - UINT8 in_Bit: The bit offset within the byte. The value must be between 0 and 7. - bool* out_Value: Returns the bit value.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	Offset or bits are invalid.
	SREC_WRONG_ARGUMENT	The area is invalid.

Table 9-71 ReadBit() - .NET (C#)

Syntax	bool InputArea.ReadBit(UInt32 in_Offset, Byte in_Bit); bool MarkerArea.ReadBit(UInt32 in_Offset, Byte in_Bit); bool OutputArea.ReadBit(UInt32 in_Offset, Byte in_Bit);	
Parameters	- UInt32 in_Offset: The byte offset within the area. The value must be between 0 and the value that AreaSize returns. - Byte in_Bit: The bit offset within the byte. The value must be between 0 and 7.	
Return values	bool: Bit value	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	Offset or bits are invalid.

ReadByte()

Reads an individual byte from the area.

See Synchronizing inputs and outputs (Page 117) for information on the API handling of I/O values.

Table 9-72 ReadByte() - Native C++

Syntax	<code>ERuntimeErrorCode ReadByte(EArea in_Area, UINT32 in_Offset, BYTE* out_Value);</code>	
Parameters	EArea in_Area: The area from which you want to read. Permissible values: {SRA_INPUT, SRA_MARKER, SRA_OUTPUT}. See EArea (Page 117). UINT32 in_Offset: The byte offset within the area. The value must be between 0 and the value that <code>GetAreaSize()</code> returns. BYTE* out_Value: Returns the byte value.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	Offset is invalid.
	SREC_WRONG_ARGUMENT	The area is invalid.

Table 9-73 ReadByte() - .NET (C#)

Syntax	<code>Byte InputArea.ReadByte(UInt32 in_Offset);</code> <code>Byte MarkerArea.ReadByte(UInt32 in_Offset);</code> <code>Byte OutputArea.ReadByte(UInt32 in_Offset);</code>	
Parameters	UInt32 in_Offset: The byte offset within the area. The value must be between 0 and the value that <code>AreaSize</code> returns.	
Return values	Byte: Byte value.	
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	The process of the simulated PLC instance is not running.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Offset is invalid.

ReadBytes()

Reads a byte array from the area.

See Synchronizing inputs and outputs (Page 117) for information on the API handling of I/O values.

Table 9-74 ReadByte() - Native C++

Syntax	ERuntimeErrorCode ReadBytes(EArea in_Area, UINT32 in_Offset, UINT32 in_BytesToRead, UINT32* out_BytesRead, BYTE inout_Values[]);	
Parameters	- EArea in_Area: The area from which you want to read. Permissible values: {SRA_INPUT, SRA_MARKER, SRA_OUTPUT}. See EArea (Page 278). - UINT32 in_Offset: The byte offset within the area. The value must be between 0 and the value that GetAreaSize() returns. - UINT32 in_BytesToRead: Contains the size of the value storage. - UINT32* out_BytesRead: Returns the number of bytes that were just written to the value storage. - BYTE inout_Values[]: The storage for the bytes that are read from the area.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No byte could be read.
	SREC_WRONG_ARGUMENT	The area is invalid.

Table 9-75 ReadBytes() - .NET (C#)

Syntax	Byte[] InputArea.ReadBytes(UInt32 in_Offset, UInt32 in_BytesToRead); Byte[] MarkerArea.ReadBytes(UInt32 in_Offset, UInt32 in_BytesToRead); Byte[] OutputArea.ReadBytes(UInt32 in_Offset, UInt32 in_BytesToRead);	
Parameters	- UInt32 in_Offset: The byte offset within the area. The value must be between 0 and the value that AreaSize returns. - UInt32 in_BytesToRead: The number of bytes to be read.	
Return values	Byte[]: The read bytes.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No byte could be read.

ReadSignals()

Structures and fields can be emulated through signal lists and be read by using the `ReadSignals()` function.

The function also takes into consideration the byte order (Endianness).

Only primitive data type signals are supported, but the function is not type-safe.

See Synchronizing inputs and outputs [\(Page 117\)](#) for information on the API handling of I/O values.

Table 9-76 ReadSignals() - Native C++

Syntax	<pre>ERuntimeErrorCode ReadSignals(EArea in_Area, SDataValueByAddress* inout_Signals, UINT32 in_SignalCount, bool validateAddressTag = false); ERuntimeErrorCode ReadSignals(EArea in_Area, SDataValueByAddressWithCheck* inout_Signals, UINT32 in_SignalCount, bool* out_SignalsHaveChanged, bool validateAddressTag = false);</pre>	
Parameters	EArea in_Area The area from which you want to read. Permissible values: {SRA_INPUT, SRA_MARKER, SRA_OUTPUT}. See EArea (Page 278). SDataValueByAddress* inout_Signals The signal list that is read. The result is stored in the structure. SDataValueByAddressWithCheck* inout_Signals The signal list that is read. The result is stored in the structure. "ValueHasChanged" is set to true if the value of the signal has changed since the preceding call. UINT32 in_SignalCount Number of signals in the list. bool* out_SignalsHaveChanged Returns true if the value of at least one signal has changed since the preceding call. bool validateAddressTag Boolean value to enable or disable the Memory Address to Tag association check.	
Signal error	Error code	Description
	SREC_OK	The signal operation is successful.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_WRONG_ARGUMENT	The area is invalid.
	SREC_SIGNAL_CONFIGURATION_ERROR	At least one signal error is in the list.

Table 9-77 ReadSignals() - .NET (C#)

Syntax	<pre>void ReadSignals(ref SDataValueByAddress[] inout_Signals, bool validateAddressTag); void ReadSignals(ref SDataValueByAddressWithCheck[] inout_Signals out bool out_SignalsHaveChanged, bool validateAddressTag);</pre>	
Parameters	- ref SDataValueByAddress[] inout_Signals The signal list that is read. - ref SDataValueByAddressWithCheck[] inout_Signals The signal list that is read. The result is stored in the structure. "ValueHasChanged" is set to true if the value of the signal has changed since the preceding call. - out bool out_SignalsHaveChanged Returns true if the value of at least one signal has changed since the preceding call. - bool validateAddressTag Optional boolean value to enable or disable the Memory Address to Tag association check.	
Signal error	Runtime error code	Description
	ERuntimeErrorCode.OK	The signal operation is successful.
	ERuntimeErrorCode.IndexOutOfRange	Offset or bits are invalid.
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.SignalConfigura- tionError	At least one signal error is in the list.

9.6.4 I/O access via address - Writing

WriteBit()

Writes an individual bit to the area.

NOTE

Data can be overwritten

The function allows access to the entire storage area of the simulated PLC instance. Therefore, use access via the tag name [\(Page 150\)](#) and not via the address areas.

Table 9-78 WriteBit() - Native C++

Syntax	<pre>ERuntimeErrorCode WriteBit(EArea in_Area, UINT32 in_Offset, UINT8 in_Bit, bool in_Value);</pre>	
Parameters	- EArea in_Area: The area that is to be written. Permissible values: {SRA_INPUT, SRA_MARKER, SRA_OUTPUT}. See EArea (Page 278). - UINT32 in_Offset: The byte offset within the area. The value must be between 0 and the value that GetAreaSize() returns.	

9.6 I/O access

	• UINT8 in_Bit: The bit offset within the byte. The value must be between 0 and 7. • bool in_Value: Bit value.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	Offset or bits are invalid.
	SREC_WRONG_ARGUMENT	Area is invalid.

Table 9-79 WriteBit() - .NET (C#)

Syntax	<pre>void InputArea WriteBit(UInt32 in_Offset, Byte in_Bit, bool in_Value); void MarkerArea WriteBit(UInt32 in_Offset, Byte in_Bit, bool in_Value); void OutputArea WriteBit(UInt32 in_Offset, Byte in_Bit, bool in_Value);</pre>	
Parameters	• UInt32 in_Offset: The byte offset within the area. The value must be between 0 and the value that <code>AreaSize</code> returns. • Byte in_Bit: The bit offset within the byte. The value must be between 0 and 7. • bool in_Value: Bit value.	
Return values	None	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	Offset or bits are invalid.

WriteByte()

Writes an individual byte to the area.

NOTE

Data can be overwritten

The function allows access to the entire storage area of the simulated PLC instance. Therefore, use access via the tag name [\(Page 150\)](#) and not via the address areas.

Table 9-80 WriteByte() - Native C++

Syntax	<code>ERuntimeErrorCode WriteByte(EArea in_Area, UINT32 in_Offset, BYTE in_Value);</code>	
Parameters	EArea in_Area: The area that is to be written. Permissible values: {SRA_INPUT, SRA_MARKER, SRA_OUTPUT}. See EArea (Page 278). UINT32 in_Offset: The byte offset within the area. The value must be between 0 and the value that <code>GetAreaSize()</code> returns. BYTE in_Value: Byte value.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	Offset is invalid.
	SREC_WRONG_ARGUMENT	Area is invalid.

Table 9-81 WriteByte() - .NET (C#)

Syntax	<code>void InputArea.WriteByte(UInt32 in_Offset, Byte in_Value);</code> <code>void MarkerArea.WriteByte(UInt32 in_Offset, Byte in_Value);</code> <code>void OutputArea.WriteByte(UInt32 in_Offset, Byte in_Value);</code>	
Parameters	UINT32 in_Offset: The byte offset within the area. The value must be between 0 and the value that <code>AreaSize</code> returns. BYTE in_Value: Byte value.	
Return values	None	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	The process of the simulated PLC instance is not running.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	Offset is invalid.

WriteBytes()

Writes a byte array to the area.

NOTE

Data can be overwritten

The function allows access to the entire storage area of the simulated PLC instance.

In particular, do not write to bytes that belong to other applications or contain internal data, for example, qualifier bits for fail-safe I/O modules.

Therefore, use access via the tag name (Page 150) and not via the address areas.

Table 9-82 WriteBytes() - Native C++

Syntax	ERuntimeErrorCode WriteBytes(EArea in_Area, UINT32 in_Offset, UINT32 in_BytesToWrite, UINT32* out_BytesWritten, BYTE in_Values[]);	
Parameters	- EArea in_Area: The area that is to be written. Permissible values: {SRA_INPUT, SRA_MARKER, SRA_OUTPUT}. See EArea (Page 278). - UINT32 in_Offset: The byte offset within the area. The value must be between 0 and the value that GetAreaSize() returns. - UINT32 in_BytesToWrite: Contains the size of the array value to be written. - UINT32* out_BytesWritten: Contains the number of bytes that were just written. - BYTE in_Values[]: Byte array that is to be written to the area.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No byte could be written.
	SREC_WRONG_ARGUMENT	The area is invalid.

Table 9-83 WriteBytes() - .NET (C#)

Syntax	<pre> UInt32 InputArea.WriteBytes(UInt32 in_Offset, Byte[] in_Values); UInt32 InputArea.WriteBytes(UInt32 in_Offset, UInt32 in_BytesToWrite, Byte[] in_Values); UInt32 MarkerArea.WriteBytes(UInt32 in_Offset, Byte[] in_Values); UInt32 MarkerArea.WriteBytes(UInt32 in_Offset, UInt32 in_BytesToWrite, Byte[] in_Values); UInt32 OutputArea.WriteBytes(UInt32 in_Offset, Byte[] in_Values); UInt32 OutputArea.WriteBytes(UInt32 in_Offset, UInt32 in_BytesToWrite, Byte[] in_Values); </pre>	
Parameters	- UINT32 in_Offset: The byte offset within the area. The value must be between 0 and the value that AreaSize returns. - UInt32 in_BytesToWrite: Contains the number of bytes to be written. The value must be between 1 and the size of the array value. - BYTE in_Value: Byte value.	
Return values	UInt32: Contains the number of bytes that were just written.	

Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No byte could be written.

WriteSignals()

Writes multiple signals within an API call.

The function also takes into consideration the byte order (Endianness).

The function supports only primitive data type signals, but it is not typical.

NOTE

Data can be overwritten

The function allows access to the entire storage area of the simulated PLC instance.

Therefore, use access via the tag name ([Page 150](#)) and not via the address areas.

Table 9-84 WriteSignals() - Native C++

Syntax	ERuntimeErrorCode WriteSignals(EArea in_Area, SDataValueByAddress* in_Signals, UINT32 in_SignalCount, bool validateAddressTag = false);	
Parameters	- EArea in_Area The area that is to be written. Permissible values: {SRA_INPUT, SRA_MARKER, SRA_OUTPUT}. See EArea (Page 278). - SDataValueByAddress* inout_Signals The signal list to be written. - UINT32 in_SignalCount Number of signals in the list. - bool validateAddressTag Boolean value to enable or disable the Memory Address to Tag association check.	
Signal error	Error code	Description
	SREC_OK	The signal operation is successful.
	SREC_INDEX_OUT_OF_RANGE	Offset or bits are invalid.
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_SIGNAL_CONFIGURATION_ERROR	At least one signal error is in the list.
	SREC_WRONG_ARGUMENT	The area is invalid.

9.6 I/O access

Table 9-85 WriteSignals() - .NET (C#)

Syntax	void WriteSignals(SDataValueByAddress[] in_Signals, bool validateAddressTag);	
Parameters	- SDataValueByAddress[] in_Signals The signal list to be written. - bool validateAddressTag Optional boolean value to enable or disable the Memory Address to Tag association check.	
Signal error	Error code	Description
	ERuntimeErrorCode.OK	The signal operation is successful.
	ERuntimeError-Code.IndexOutOfRange	Offset or bits are invalid.
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.SignalConfigur- ationError	At least one signal error is in the list.

9.6.5 I/O access via tag name - Reading

Individual access to I/O data is used for displaying and writing values that are not refreshed regularly in a graphical user interface (GUI).

NOTE

To simulate a regular exchange of signals, create a signal list for each set of signals. Use this signal list for all further accesses. Create a new list as soon as the set of signals changes.

For the signal lists use the functions ReadSignals() and WriteSignals().

Read()

Reads the value of a PLC tag.

See Synchronizing inputs and outputs (Page 117) for information on the API handling of I/O values.

Table 9-86 Read() - Native C++

Syntax	ERuntimeErrorCode Read(WCHAR* in_Tag, SDataValue* inout_Value);	
Parameters	- WCHAR* in_Tag: The name of the PLC tag that is to be read. - SDataValue* inout_Value: Contains the value and the expected type of the PLC tag. If the expected type is UNSPECIFIC, it is set to the stored type when the function was successful. The STRUCT type is not supported. Structures and fields can be emulated through signal lists and be read by using the ReadSignals() function.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.

Return values	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be read.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-87 Read() - .NET (C#)

Syntax	SDataValue Read(string in_Tag)	
Parameters	string in_Tag: The name of the PLC tag that is to be read.	
Return values	SDataValue: Contains the value and the type of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be read.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285-286).
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

ReadBool()

Reads the value of a PLC tag.

See Synchronizing inputs and outputs [\(Page 117\)](#) for information on the API handling of I/O values.

Table 9-88 ReadBool() - Native C++

Syntax	ERuntimeErrorCode ReadBool(WCHAR* in_Tag, bool* out_Value);	
Parameters	- WCHAR* in_Tag: The name of the PLC tag that is to be read. - bool* out_Value: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be read.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-89 ReadBool() - .NET (C#)

Syntax	bool ReadBool(string in_Tag)	
Parameters	string in_Tag: The name of the PLC tag that is to be read.	
Return values	bool: Contains the value of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be read.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .

Exceptions	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

ReadInt8()

Reads the value of a PLC tag.

See Synchronizing inputs and outputs ([Page 117](#)) for information on the API handling of I/O values.

Table 9-90 ReadInt8() - Native C++

Syntax	ERuntimeErrorCode ReadInt8(WCHAR* in_Tag, INT8* out_Value);	
Parameters	- WCHAR* in_Tag: The name of the PLC tag that is to be read. - INT8* out_Value: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be read.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-91 ReadInt8() - .NET (C#)

Syntax	Int8 ReadInt8(string in_Tag)	
Parameters	string in_Tag: The name of the PLC tag that is to be read.	
Return values	Int8: Contains the value and the type of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.

Exceptions	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	The process of the simulated PLC instance is not running.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	The offset lies outside the area range. No value could be read.
	<code>ERuntimeErrorCode.DoesNotExist</code>	The entry does not exist in the stored tag list.
	<code>ERuntimeErrorCode.NotSupported</code>	Access to entire structures or arrays is not supported.
	<code>ERuntimeErrorCode.TypeMismatch</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	<code>ERuntimeErrorCode.NotUpToDate</code>	The stored tag list must be updated.
	<code>ERuntimeErrorCode.NotEnoughMemory</code>	Unexpected error occurred; turn on logging and contact support.
	<code>ERuntimeErrorCode.InternalError</code>	Internal error occurred; turn on logging and contact support.

ReadInt16()

Reads the value of a PLC tag.

See Synchronizing inputs and outputs (Page 117) for information on the API handling of I/O values.

Table 9-92 ReadInt16() - Native C++

Syntax	<code>ERuntimeErrorCode ReadInt16(WCHAR* in_Tag, INT16* out_Value);</code>	
Parameters	<code>WCHAR* in_Tag</code>: The name of the PLC tag that is to be read. <code>INT16* out_Value</code>: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_INTERFACE_REMOVED</code>	The instance is not registered in Runtime Manager.
	<code>SREC_TIMEOUT</code>	The function does not return on time.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	The process of the simulated PLC instance is not running.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	The offset lies outside the area range. No value could be read.
	<code>SREC_DOES_NOT_EXIST</code>	The entry does not exist in the stored tag list.
	<code>SREC_NOT_SUPPORTED</code>	Access to entire structures or arrays is not supported.
	<code>SREC_TYPE_MISMATCH</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	<code>SREC_NOT_UP_TO_DATE</code>	The stored tag list must be updated.
	<code>SREC_NOT_ENOUGH_MEMORY</code>	Unexpected error occurred; turn on logging and contact support.
	<code>SREC_INTERNAL_ERROR</code>	Internal error occurred; turn on logging and contact support.

Table 9-93 ReadInt16() - .NET (C#)

Syntax	Int16 ReadInt16(string in_Tag)	
Parameters	string in_Tag: The name of the PLC tag that is to be read.	
Return values	Int16: Contains the value and the type of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be read.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

ReadInt32()

Reads the value of a PLC tag.

See Synchronizing inputs and outputs (Page 117) for information on the API handling of I/O values.

Table 9-94 ReadInt32() - Native C++

Syntax	ERuntimeErrorCode ReadInt32(WCHAR* in_Tag, INT32* out_Value);	
Parameters	- WCHAR* in_Tag: The name of the PLC tag that is to be read. - INT32* out_Value: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be read.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.

Return values	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-95 ReadInt32() - .NET (C#)

Syntax	Int32 ReadInt32(string in_Tag)	
Parameters	string in_Tag: The name of the PLC tag that is to be read.	
Return values	Int32: Contains the value and the type of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be read.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

ReadInt64()

Reads the value of a PLC tag.

See Synchronizing inputs and outputs ([Page 117](#)) for information on the API handling of I/O values.

Table 9-96 ReadInt64() - Native C++

Syntax	<code>ERuntimeErrorCode ReadInt64(WCHAR* in_Tag, INT64* out_Value);</code>	
Parameters	<code>WCHAR* in_Tag</code>: The name of the PLC tag that is to be read. <code>INT64* out_Value</code>: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_INTERFACE_REMOVED</code>	The instance is not registered in Runtime Manager.
	<code>SREC_TIMEOUT</code>	The function does not return on time.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	The process of the simulated PLC instance is not running.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	The offset lies outside the area range. No value could be read.
	<code>SREC_DOES_NOT_EXIST</code>	The entry does not exist in the stored tag list.
	<code>ERuntimeErrorCode.NotSupported</code>	Access to entire structures or arrays is not supported.
	<code>ERuntimeErrorCode.TypeMismatch</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	<code>SREC_NOT_UP_TO_DATE</code>	The stored tag list must be updated.
	<code>SREC_NOT_ENOUGH_MEMORY</code>	Unexpected error occurred; turn on logging and contact support.
	<code>SREC_INTERNAL_ERROR</code>	Internal error occurred; turn on logging and contact support.

Table 9-97 ReadInt64() - .NET (C#)

Syntax	<code>Int64 ReadInt64(string in_Tag)</code>	
Parameters	<code>string in_Tag</code>: The name of the PLC tag that is to be read.	
Return values	<code>Int64</code> : Contains the value and the type of the PLC tag.	
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	The process of the simulated PLC instance is not running.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	The offset lies outside the area range. No value could be read.
	<code>ERuntimeErrorCode.DoesNotExist</code>	The entry does not exist in the stored tag list.
	<code>ERuntimeErrorCode.NotSupported</code>	Access to entire structures or arrays is not supported.
	<code>ERuntimeErrorCode.TypeMismatch</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).

Exceptions	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

ReadUInt8()

Reads the value of a PLC tag.

See Synchronizing inputs and outputs [\(Page 117\)](#) for information on the API handling of I/O values.

Table 9-98 ReadUInt8() - Native C++

Syntax	ERuntimeErrorCode ReadUInt8(WCHAR* in_Tag, UINT8* out_Value);	
Parameters	- WCHAR* in_Tag: The name of the PLC tag that is to be read. - UINT8* out_Value: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be read.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-99 ReadUInt8() - .NET (C#)

Syntax	UInt8 ReadUInt8(string in_Tag)	
Parameters	string in_Tag: The name of the PLC tag that is to be read.	
Return values	UInt8: Contains the value and the type of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.

Exceptions	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	The process of the simulated PLC instance is not running.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	The offset lies outside the area range. No value could be read.
	<code>ERuntimeErrorCode.DoesNotExist</code>	The entry does not exist in the stored tag list.
	<code>ERuntimeErrorCode.NotSupported</code>	Access to entire structures or arrays is not supported.
	<code>ERuntimeErrorCode.TypeMismatch</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	<code>ERuntimeErrorCode.NotUpToDate</code>	The stored tag list must be updated.
	<code>ERuntimeErrorCode.NotEnoughMemory</code>	Unexpected error occurred; turn on logging and contact support.
	<code>ERuntimeErrorCode.InternalError</code>	Internal error occurred; turn on logging and contact support.

ReadUInt16()

Reads the value of a PLC tag.

See Synchronizing inputs and outputs ([Page 117](#)) for information on the API handling of I/O values.

Table 9-100 ReadUInt16() - Native C++

Syntax	<code>ERuntimeErrorCode ReadUInt16(WCHAR* in_Tag, UINT16* out_Value);</code>	
Parameters	<code>WCHAR* in_Tag</code>: The name of the PLC tag that is to be read. <code>UINT16* out_Value</code>: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_INTERFACE_REMOVED</code>	The instance is not registered in Runtime Manager.
	<code>SREC_TIMEOUT</code>	The function does not return on time.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	The process of the simulated PLC instance is not running.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	The offset lies outside the area range. No value could be read.
	<code>SREC_DOES_NOT_EXIST</code>	The entry does not exist in the stored tag list.
	<code>SREC_NOT_SUPPORTED</code>	Access to entire structures or arrays is not supported.
	<code>SREC_TYPE_MISMATCH</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	<code>SREC_NOT_UP_TO_DATE</code>	The stored tag list must be updated.
	<code>SREC_NOT_ENOUGH_MEMORY</code>	Unexpected error occurred; turn on logging and contact support.
	<code>SREC_INTERNAL_ERROR</code>	Internal error occurred; turn on logging and contact support.

Table 9-101 ReadUInt16() - .NET (C#)

Syntax	UInt16 ReadUInt16(string in_Tag)	
Parameters	string in_Tag: The name of the PLC tag that is to be read.	
Return values	UInt16: Contains the value and the type of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be read.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

ReadUInt32()

Reads the value of a PLC tag.

See Synchronizing inputs and outputs (Page 117) for information on the API handling of I/O values.

Table 9-102 ReadUInt32() - Native C++

Syntax	ERuntimeErrorCode ReadUInt32(WCHAR* in_Tag, UINT32* out_Value);	
Parameters	- WCHAR* in_Tag: The name of the PLC tag that is to be read. - UINT32* out_Value: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be read.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.

Return values	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-103 ReadUInt32() - .NET (C#)

Syntax	UInt32 ReadUInt32(string in_Tag)	
Parameters	string in_Tag: The name of the PLC tag that is to be read.	
Return values	UInt32: Contains the value and the type of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be read.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

ReadUInt64()

Reads the value of a PLC tag.

See Synchronizing inputs and outputs [\(Page 117\)](#) for information on the API handling of I/O values.

Table 9-104 ReadInt64() - Native C++

Syntax	ERuntimeErrorCode ReadUInt64(WCHAR* in_Tag, UINT64* out_Value);	
Parameters	- WCHAR* in_Tag: The name of the PLC tag that is to be read. - UINT64* out_Value: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be read.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-105 ReadUInt64() - .NET (C#)

Syntax	UInt64 ReadUInt64(string in_Tag)	
Parameters	string in_Tag: The name of the PLC tag that is to be read.	
Return values	UInt64: Contains the value and the type of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be read.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .

Exceptions	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

ReadFloat()

Reads the value of a PLC tag.

See Synchronizing inputs and outputs ([Page 117](#)) for information on the API handling of I/O values.

Table 9-106 ReadFloat() - Native C++

Syntax	ERuntimeErrorCode ReadFloat(WCHAR* in_Tag, float* out_Value);	
Parameters	- WCHAR* in_Tag: The name of the PLC tag that is to be read. - float* out_Value: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be read.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-107 ReadFloat() - .NET (C#)

Syntax	float ReadFloat(string in_Tag)	
Parameters	string in_Tag: The name of the PLC tag that is to be read.	
Return values	float: Contains the value and the type of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.

Exceptions	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	The process of the simulated PLC instance is not running.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	The offset lies outside the area range. No value could be read.
	<code>ERuntimeErrorCode.DoesNotExist</code>	The entry does not exist in the stored tag list.
	<code>ERuntimeErrorCode.NotSupported</code>	Access to entire structures or arrays is not supported.
	<code>ERuntimeErrorCode.TypeMismatch</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	<code>ERuntimeErrorCode.NotUpToDate</code>	The stored tag list must be updated.
	<code>ERuntimeErrorCode.NotEnoughMemory</code>	Unexpected error occurred; turn on logging and contact support.
	<code>ERuntimeErrorCode.InternalError</code>	Internal error occurred; turn on logging and contact support.

ReadDouble()

Reads the value of a PLC tag.

See Synchronizing inputs and outputs (Page 117) for information on the API handling of I/O values.

Table 9-108 ReadDouble() - Native C++

Syntax	<code>ERuntimeErrorCode ReadDouble(WCHAR* in_Tag, double* out_Value);</code>	
Parameters	<code>WCHAR* in_Tag</code>: The name of the PLC tag that is to be read. <code>double* out_Value</code>: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_INTERFACE_REMOVED</code>	The instance is not registered in Runtime Manager.
	<code>SREC_TIMEOUT</code>	The function does not return on time.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	The process of the simulated PLC instance is not running.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	The offset lies outside the area range. No value could be read.
	<code>SREC_DOES_NOT_EXIST</code>	The entry does not exist in the stored tag list.
	<code>SREC_NOT_SUPPORTED</code>	Access to entire structures or arrays is not supported.
	<code>SREC_TYPE_MISMATCH</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	<code>SREC_NOT_UP_TO_DATE</code>	The stored tag list must be updated.
	<code>SREC_NOT_ENOUGH_MEMORY</code>	Unexpected error occurred; turn on logging and contact support.
	<code>SREC_INTERNAL_ERROR</code>	Internal error occurred; turn on logging and contact support.

Table 9-109 ReadDouble() - .NET (C#)

Syntax	double ReadDouble(string in_Tag)	
Parameters	string in_Tag: The name of the PLC tag that is to be read.	
Return values	double: Contains the value and the type of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be read.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

ReadChar()

Reads the value of a PLC tag.

See Synchronizing inputs and outputs (Page 117) for information on the API handling of I/O values.

Table 9-110 ReadChar() - Native C++

Syntax	ERuntimeErrorCode ReadChar(WCHAR* in_Tag, char* out_Value);	
Parameters	- WCHAR* in_Tag: The name of the PLC tag that is to be read. - char* out_Value: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be read.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.

Return values	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-111 ReadChar() - .NET (C#)

Syntax	sbyte ReadChar(string in_Tag)	
Parameters	string in_Tag: The name of the PLC tag that is to be read.	
Return values	sbyte: Contains the value and the type of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be read.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

ReadWChar()

Reads the value of a PLC tag.

See Synchronizing inputs and outputs ([Page 117](#)) for information on the API handling of I/O values.

Table 9-112 ReadWChar() - Native C++

Syntax	ERuntimeErrorCode ReadWChar(WCHAR* in_Tag, WCHAR* out_Value);	
Parameters	- WCHAR* in_Tag: The name of the PLC tag that is to be read. - WCHAR* out_Value: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be read.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-113 ReadWChar() - .NET (C#)

Syntax	char ReadWChar(string in_Tag)	
Parameters	string in_Tag: The name of the PLC tag that is to be read.	
Return values	char: Contains the value and the type of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be read.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285).

Exceptions	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

ReadSignals()

Reads multiple signals within an API call.

When the function is called for the first time, it stores internal information in the structures `SDataValueByName*` to improve the performance of the subsequent calls.

NOTE

To simulate a regular exchange of signals, create a signal list for each set of signals. Use this signal list for all further accesses. Create a new list as soon as the set of signals changes.

See Synchronizing inputs and outputs ([Page 117](#)) for information on the API handling of I/O values.

Table 9-114 ReadSignals() - Native C++

Syntax	<pre>ERuntimeErrorCode ReadSignals(SDataValueByName* inout_Signals, UINT32 in_SignalCount); ERuntimeErrorCode ReadSignals(SDataValueByNameWithCheck* inout_Signals, UINT32 in_SignalCount bool* out_SignalsHaveChanged);</pre>	
Parameters	<code>SDataValueByName* inout_Signals</code>: Contains the name, the value and the expected type of the PLC tag. If the expected type is <code>UNSPECIFIC</code>, it is set to the stored type when the function was successful. The <code>STRUCT</code> type is not supported. <code>SDataValueByNameWithCheck* inout_Signals</code>: Contains the name, the value and the expected type of the PLC tag. If the expected type is <code>UNSPECIFIC</code>, it is set to the stored type when the function was successful. The <code>STRUCT</code> type is not supported. "ValueHasChanged" is set to <code>true</code> if the value of the signal has changed since the preceding call. <code>UINT32 in_SignalCount</code>: The number of signals to be read. <code>bool* out_SignalsHaveChanged</code>: Returns <code>true</code> if the value of at least one signal has changed since the preceding call.	
Signal error	Runtime error code	Description
	<code>SREC_OK</code>	The signal operation is successful.
	<code>SREC_DOES_NOT_EXIST</code>	The entry does not exist in the stored tag list.
	<code>SREC_NOT_SUPPORTED</code>	Access to entire structures or arrays is not supported.
	<code>SREC_TYPE_MISMATCH</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	<code>SREC_INDEX_OUT_OF_RANGE</code>	Offset or bits are invalid.
Return values	Runtime error code	Condition
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_INTERFACE_REMOVED</code>	The instance is not registered in Runtime Manager.

Return values	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_SIGNAL_CONFIGURATION_ERROR	At least one signal error is in the list.

Table 9-115 ReadSignals() - .NET (C#)

Syntax	<pre>void ReadSignals(ref SDataValueByName[] inout_Signals) void ReadSignals(ref SDataValueByNameWithCheck[] inout_Signals out bool out_SignalsHaveChanged);</pre>	
Parameters	- ref SDataValueByName[] inout_Signals: Contains the name, the value and the expected type of the PLC tag. If the expected type is UNSPECIFIC, it is set to the stored type when the function was successful. The STRUCT type is not supported. - ref SDataValueByNameWithCheck[] inout_Signals: Contains the name, the value and the expected type of the PLC tag. If the expected type is UNSPECIFIC, it is set to the stored type when the function was successful. The STRUCT type is not supported. "ValueHasChanged" is set to true if the value of the signal has changed since the preceding call. - out bool out_SignalsHaveChanged: Returns true if the value of at least one signal has changed since the preceding call.	
Return values	SDataValue: Contains the value and the type of the PLC tag.	
Signal error	Runtime error code	Description
	ERuntimeErrorCode.OK	The signal operation is successful.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	ERuntimeErrorCode.IndexOutOfRange	Offset or bits are invalid.
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.

ReadString()

Reads the value of a PLC tag.

See Synchronizing inputs and outputs [\(Page 117\)](#) for information on the API handling of I/O values.

Table 9-116 ReadString() - Native C++

Syntax	<pre>ERuntimeErrorCode IInstance::ReadString(const WCHAR* in_Tag, CHAR* out_Value, const UINT32 in_BufferSize, UINT32* out_CharsRead);</pre>	
Parameters	- const WCHAR* in_Tag: The name of the PLC tag that is to be read. - CHAR* out_Value: Contains the value of the PLC tag - const UINT32 in_BufferSize: Size of the buffer provided - UINT32* out_CharsRead: Number of characters read	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be read.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_BUFFER_OVERFLOW	The buffer provided is too small.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-117 ReadString() - .NET (C#)

Syntax	<pre>string ReadString(string in_Tag);</pre>	
Parameters	string in_Tag: The name of the PLC tag that is to be read.	
Return values	string: Contains the value of the PLC tag.	
Exceptions	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.

Exceptions	<code>ERuntimeErrorCode.IndexOutOfRange</code>	The offset lies outside the area range. No value could be read.
	<code>ERuntimeErrorCode.DoesNotExist</code>	The entry does not exist in the stored tag list.
	<code>ERuntimeErrorCode.NotSupported</code>	Access to entire structures or arrays is not supported.
	<code>ERuntimeErrorCode.TypeMismatch</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	<code>ERuntimeErrorCode.NotUpToDate</code>	The stored tag list must be updated.
	<code>ERuntimeErrorCode.BufferOverflow</code>	The buffer provided is too small.
	<code>ERuntimeErrorCode.NotEnoughMemory</code>	Unexpected error occurred; turn on logging and contact support.
	<code>ERuntimeErrorCode.InternalError</code>	Internal error occurred; turn on logging and contact support.

ReadWString()

Reads the value of a PLC tag.

See Synchronizing inputs and outputs ([Page 117](#)) for information on the API handling of I/O values.

Table 9-118 ReadWString() - Native C++

Syntax	<pre>ERuntimeErrorCode IInstance::ReadWString(const WCHAR* in_Tag, WCHAR* out_Value, const UINT32 in_BufferSize, UINT32* out_CharsRead);</pre>	
Parameters	<code>const WCHAR* in_Tag</code>: The name of the PLC tag that is to be read. <code>WCHAR* out_Value</code>: Contains the value of the PLC tag <code>const UINT32 in_BufferSize</code>: Size of the buffer provided <code>UINT32* out_CharsRead</code>: Number of characters read	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_INTERFACE_REMOVED</code>	The instance is not registered in Runtime Manager.
	<code>SREC_TIMEOUT</code>	The function does not return on time.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	The process of the simulated PLC instance is not running.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	The offset lies outside the area range. No value could be read.
	<code>SREC_DOES_NOT_EXIST</code>	The entry does not exist in the stored tag list.
	<code>SREC_NOT_SUPPORTED</code>	Access to entire structures or arrays is not supported.
	<code>SREC_TYPE_MISMATCH</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).

9.6 I/O access

Return values	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_BUFFER_OVERFLOW	The buffer provided is too small.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-119 ReadWString() - .NET (C#)

Syntax	wstring ReadWString(wstring in_Tag);	
Parameters	wstring in_Tag: The name of the PLC tag that is to be read.	
Return values	wstring: Contains the value of the PLC tag.	
Exceptions	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be read.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.BufferOverflow	The buffer provided is too small.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

9.6.6 I/O access via tag name - Writing

Individual access to I/O data is used for displaying and writing values that are not refreshed regularly in a graphical user interface (GUI).

NOTE

To simulate a regular exchange of signals, create a signal list for each set of signals. Use this signal list for all further accesses. Create a new list as soon as the set of signals changes. For the signal lists use the functions `ReadSignals()` and `WriteSignals()`.

Write()

Writes the value of a PLC tag.

Table 9-120 Write() - Native C++

Syntax	ERuntimeErrorCode Write(WCHAR* in_Tag, SDataValue* in_Value);	
Parameters	- WCHAR* in_Tag: The name of the PLC tag that is to be written. - SDataValue* in_Value: Contains the value and the expected type of the PLC tag. The UNSPECIFIC and STRUCT types are not supported. Structures and fields can be emulated through signal lists and then be read by using the ReadSignals() function and written by using the WriteSignals() function.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be written.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_WRONG_ARGUMENT	The expected type is UNSPECIFIC.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-121 Write() - .NET (C#)

Syntax	void Write(string in_Tag SDataValue in_Value)	
Parameters	- string in_Tag: The name of the PLC tag that is to be written. - SDataValue in_Value: Contains the value and the expected type of the PLC tag. The UNSPECIFIC and STRUCT types are not supported. Structures and fields can be emulated through signal lists and then be written by using the WriteSignals() function.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.

Exceptions	<code>ERuntimeErrorCode.IndexOutOfRange</code>	The offset lies outside the area range. No value could be written.
	<code>ERuntimeErrorCode.DoesNotExist</code>	The entry does not exist in the stored tag list.
	<code>ERuntimeErrorCode.NotSupported</code>	Access to entire structures or arrays is not supported.
	<code>ERuntimeErrorCode.TypeMismatch</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .
	<code>ERuntimeErrorCode.NotUpToDate</code>	The stored tag list must be updated.
	<code>ERuntimeErrorCode.WrongArgument</code>	The expected type is <code>UNSPECIFIC</code> .
	<code>ERuntimeErrorCode.NotEnoughMemory</code>	Unexpected error occurred; turn on logging and contact support.
	<code>ERuntimeErrorCode.InternalError</code>	Internal error occurred; turn on logging and contact support.

WriteBool()

Writes the value of a PLC tag.

Table 9-122 WriteBool() - Native C++

Syntax	<code>ERuntimeErrorCode WriteBool(WCHAR* in_Tag, bool in_Value);</code>	
Parameters	<code>WCHAR* in_Tag</code>: The name of the PLC tag that is to be written. <code>bool in_Value</code>: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_INTERFACE_REMOVED</code>	The instance is not registered in Runtime Manager.
	<code>SREC_TIMEOUT</code>	The function does not return on time.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	The process of the simulated PLC instance is not running.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	The offset lies outside the area range. No value could be written.
	<code>SREC_DOES_NOT_EXIST</code>	The entry does not exist in the stored tag list.
	<code>SREC_NOT_SUPPORTED</code>	Access to entire structures or arrays is not supported.
	<code>SREC_TYPE_MISMATCH</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .
	<code>SREC_NOT_UP_TO_DATE</code>	The stored tag list must be updated.
	<code>SREC_NOT_ENOUGH_MEMORY</code>	Unexpected error occurred; turn on logging and contact support.
	<code>SREC_INTERNAL_ERROR</code>	Internal error occurred; turn on logging and contact support.

Table 9-123 WriteBool() - .NET (C#)

Syntax	void WriteBool(string in_Tag bool in_Value)	
Parameters	- string in_Tag: The name of the PLC tag that is to be written. - bool in_Value: Contains the value of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be written.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285-286).
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

WriteInt8()

Writes the value of a PLC tag.

Table 9-124 WriteInt8() - Native C++

Syntax	ERuntimeErrorCode WriteInt8(WCHAR* in_Tag, INT8 in_Value);	
Parameters	- WCHAR* in_Tag: The name of the PLC tag that is to be written. - INT8 in_Value: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be written.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.

9.6 I/O access

Return values	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-125 WriteInt8() - .NET (C#)

Syntax	void WriteInt8(string in_Tag Int8 in_Value)	
Parameters	- string in_Tag: The name of the PLC tag that is to be written. - Int8 in_Value: Contains the value of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be written.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

WriteInt16()

Writes the value of a PLC tag.

Table 9-126 WriteInt16() - Native C++

Syntax	<code>ERuntimeErrorCode WriteInt16(WCHAR* in_Tag, INT16 in_Value);</code>	
Parameters	<code>WCHAR* in_Tag</code>: The name of the PLC tag that is to be written. <code>INT16 in_Value</code>: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_INTERFACE_REMOVED</code>	The instance is not registered in Runtime Manager.
	<code>SREC_TIMEOUT</code>	The function does not return on time.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	The process of the simulated PLC instance is not running.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	The offset lies outside the area range. No value could be written.
	<code>SREC_DOES_NOT_EXIST</code>	The entry does not exist in the stored tag list.
	<code>SREC_NOT_SUPPORTED</code>	Access to entire structures or arrays is not supported.
	<code>SREC_TYPE_MISMATCH</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	<code>SREC_NOT_UP_TO_DATE</code>	The stored tag list must be updated.
	<code>SREC_NOT_ENOUGH_MEMORY</code>	Unexpected error occurred; turn on logging and contact support.
	<code>SREC_INTERNAL_ERROR</code>	Internal error occurred; turn on logging and contact support.

Table 9-127 WriteInt16() - .NET (C#)

Syntax	<code>void WriteInt16(string in_Tag Int16 in_Value)</code>	
Parameters	<code>string in_Tag</code>: The name of the PLC tag that is to be written. <code>Int16 in_Value</code>: Contains the value of the PLC tag.	
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	The process of the simulated PLC instance is not running.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	The offset lies outside the area range. No value could be written.
	<code>ERuntimeErrorCode.DoesNotExist</code>	The entry does not exist in the stored tag list.
	<code>ERuntimeErrorCode.NotSupported</code>	Access to entire structures or arrays is not supported.
	<code>ERuntimeErrorCode.TypeMismatch</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).

9.6 I/O access

Exceptions	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

WriteInt32()

Writes the value of a PLC tag.

Table 9-128 WriteInt32() - Native C++

Syntax	ERuntimeErrorCode WriteInt32(WCHAR* in_Tag, INT32 in_Value);	
Parameters	- WCHAR* in_Tag: The name of the PLC tag that is to be written. - INT32 in_Value: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be written.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-129 WriteInt32() - .NET (C#)

Syntax	void WriteInt32(string in_Tag Int32 in_Value)	
Parameters	- string in_Tag: The name of the PLC tag that is to be written. - Int32 in_Value: Contains the value of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.

Exceptions	<code>ERuntimeErrorCode.InstanceNotRunning</code>	The process of the simulated PLC instance is not running.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	The offset lies outside the area range. No value could be written.
	<code>ERuntimeErrorCode.DoesNotExist</code>	The entry does not exist in the stored tag list.
	<code>ERuntimeErrorCode.NotSupported</code>	Access to entire structures or arrays is not supported.
	<code>ERuntimeErrorCode.TypeMismatch</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	<code>ERuntimeErrorCode.NotUpToDate</code>	The stored tag list must be updated.
	<code>ERuntimeErrorCode.NotEnoughMemory</code>	Unexpected error occurred; turn on logging and contact support.
	<code>ERuntimeErrorCode.InternalError</code>	Internal error occurred; turn on logging and contact support.

WriteInt64()

Writes the value of a PLC tag.

Table 9-130 WriteInt64() - Native C++

Syntax	<code>ERuntimeErrorCode WriteInt64(WCHAR* in_Tag, INT64 in_Value);</code>	
Parameters	<code>WCHAR* in_Tag</code>: The name of the PLC tag that is to be written. <code>INT64 in_Value</code>: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_INTERFACE_REMOVED</code>	The instance is not registered in Runtime Manager.
	<code>SREC_TIMEOUT</code>	The function does not return on time.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	The process of the simulated PLC instance is not running.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	The offset lies outside the area range. No value could be written.
	<code>SREC_DOES_NOT_EXIST</code>	The entry does not exist in the stored tag list.
	<code>SREC_NOT_SUPPORTED</code>	Access to entire structures or arrays is not supported.
	<code>SREC_TYPE_MISMATCH</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	<code>SREC_NOT_UP_TO_DATE</code>	The stored tag list must be updated.
	<code>SREC_NOT_ENOUGH_MEMORY</code>	Unexpected error occurred; turn on logging and contact support.
	<code>SREC_INTERNAL_ERROR</code>	Internal error occurred; turn on logging and contact support.

Table 9-131 WriteInt64() - .NET (C#)

Syntax	void WriteInt64(string in_Tag Int64 in_Value)	
Parameters	- string in_Tag: The name of the PLC tag that is to be written. - Int64 in_Value: Contains the value of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be written.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

WriteUInt8()

Writes the value of a PLC tag.

Table 9-132 WriteUInt8() - Native C++

Syntax	ERuntimeErrorCode WriteUInt8(WCHAR* in_Tag, UINT8 in_Value);	
Parameters	- WCHAR* in_Tag: The name of the PLC tag that is to be written. - UINT8 in_Value: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be written.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.

Return values	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-133 WriteUInt8() - .NET (C#)

Syntax	void WriteUInt8(string in_Tag UInt8 in_Value)	
Parameters	- string in_Tag: The name of the PLC tag that is to be written. - UInt8 in_Value: Contains the value of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be written.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

WriteUInt16()

Reads the value of a PLC tag.

Table 9-134 WriteUInt16() - Native C++

Syntax	ERuntimeErrorCode WriteUInt16(WCHAR* in_Tag, UINT16 in_Value);	
Parameters	- WCHAR* in_Tag: The name of the PLC tag that is to be written. - UINT16 in_Value: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.

Return values	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be written.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-135 WriteUInt16() - .NET (C#)

Syntax	void WriteUInt16(string in_Tag UInt16 in_Value)	
Parameters	- string in_Tag: The name of the PLC tag that is to be written. - UInt16 in_Value: Contains the value of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be written.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

WriteUInt32()

Writes the value of a PLC tag.

Table 9-136 WriteUInt32() - Native C++

Syntax	<code>ERuntimeErrorCode WriteUInt32(WCHAR* in_Tag, UINT32 in_Value);</code>	
Parameters	WCHAR* in_Tag: The name of the PLC tag that is to be written. UINT32 in_Value: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be written.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-137 WriteUInt32() - .NET (C#)

Syntax	<code>void WriteUInt32(string in_Tag UInt32 in_Value)</code>	
Parameters	string in_Tag: The name of the PLC tag that is to be written. UInt32 in_Value: Contains the value of the PLC tag.	
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	The process of the simulated PLC instance is not running.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	The offset lies outside the area range. No value could be written.
	<code>ERuntimeErrorCode.DoesNotExist</code>	The entry does not exist in the stored tag list.
	<code>ERuntimeErrorCode.NotSupported</code>	Access to entire structures or arrays is not supported.
	<code>ERuntimeErrorCode.TypeMismatch</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).

9.6 I/O access

Exceptions	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

WriteUInt64()

Writes the value of a PLC tag.

Table 9-138 WriteUInt64() - Native C++

Syntax	ERuntimeErrorCode WriteUInt64(WCHAR* in_Tag, UINT64 in_Value);	
Parameters	- WCHAR* in_Tag: The name of the PLC tag that is to be written. - UINT64 in_Value: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be written.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-139 WriteUInt64() - .NET (C#)

Syntax	void WriteUInt64(string in_Tag UInt64 in_Value)	
Parameters	- string in_Tag: The name of the PLC tag that is to be written. - UInt64 in_Value: Contains the value of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.

Exceptions	<code>ERuntimeErrorCode.InstanceNotRunning</code>	The process of the simulated PLC instance is not running.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	The offset lies outside the area range. No value could be written.
	<code>ERuntimeErrorCode.DoesNotExist</code>	The entry does not exist in the stored tag list.
	<code>ERuntimeErrorCode.NotSupported</code>	Access to entire structures or arrays is not supported.
	<code>ERuntimeErrorCode.TypeMismatch</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	<code>ERuntimeErrorCode.NotUpToDate</code>	The stored tag list must be updated.
	<code>ERuntimeErrorCode.NotEnoughMemory</code>	Unexpected error occurred; turn on logging and contact support.
	<code>ERuntimeErrorCode.InternalError</code>	Internal error occurred; turn on logging and contact support.

WriteFloat()

Writes the value of a PLC tag.

Table 9-140 WriteFloat() - Native C++

Syntax	<code>ERuntimeErrorCode WriteFloat(WCHAR* in_Tag, float in_Value);</code>	
Parameters	<code>WCHAR* in_Tag</code>: The name of the PLC tag that is to be written. <code>float in_Value</code>: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_INTERFACE_REMOVED</code>	The instance is not registered in Runtime Manager.
	<code>SREC_TIMEOUT</code>	The function does not return on time.
	<code>SREC_INSTANCE_NOT_RUNNING</code>	The process of the simulated PLC instance is not running.
	<code>SREC_INDEX_OUT_OF_RANGE</code>	The offset lies outside the area range. No value could be written.
	<code>SREC_DOES_NOT_EXIST</code>	The entry does not exist in the stored tag list.
	<code>SREC_NOT_SUPPORTED</code>	Access to entire structures or arrays is not supported.
	<code>SREC_TYPE_MISMATCH</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	<code>SREC_NOT_UP_TO_DATE</code>	The stored tag list must be updated.
	<code>SREC_NOT_ENOUGH_MEMORY</code>	Unexpected error occurred; turn on logging and contact support.
	<code>SREC_INTERNAL_ERROR</code>	Internal error occurred; turn on logging and contact support.

Table 9-141 WriteFloat() - .NET (C#)

Syntax	void WriteFloat(string in_Tag float in_Value)	
Parameters	- string in_Tag: The name of the PLC tag that is to be written. - float in_Value: Contains the value of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be written.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

WriteDouble()

Writes the value of a PLC tag.

Table 9-142 WriteDouble() - Native C++

Syntax	ERuntimeErrorCode WriteDouble(WCHAR* in_Tag, double in_Value);	
Parameters	- WCHAR* in_Tag: The name of the PLC tag that is to be written. - double in_Value: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be written.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.

Return values	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-143 WriteDouble() - .NET (C#)

Syntax	void WriteDouble(string in_Tag double in_Value)	
Parameters	- string in_Tag: The name of the PLC tag that is to be written. - double in_Value: Contains the value of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be written.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

WriteChar()

Writes the value of a PLC tag.

Table 9-144 WriteChar() - Native C++

Syntax	ERuntimeErrorCode WriteChar(WCHAR* in_Tag, char in_Value);	
Parameters	- WCHAR* in_Tag: The name of the PLC tag that is to be written. - char in_Value: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.

Return values	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be written.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-145 WriteChar() - .NET (C#)

Syntax	void WriteChar(string in_Tag sbyte in_Value)	
Parameters	- string in_Tag: The name of the PLC tag that is to be written. - sbyte in_Value: Contains the value of the PLC tag.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be written.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

WriteWChar()

Writes the value of a PLC tag.

Table 9-146 WriteWChar() - Native C++

Syntax	<code>ERuntimeErrorCode WriteWChar(WCHAR* in_Tag, WCHAR in_Value);</code>	
Parameters	WCHAR* in_Tag: The name of the PLC tag that is to be written. WCHAR in_Value: Contains the value of the PLC tag.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be written.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-147 WriteWChar() - .NET (C#)

Syntax	<code>void WriteWChar(string in_Tag char in_Value)</code>	
Parameters	string in_Tag: The name of the PLC tag that is to be written. char in_Value: Contains the value of the PLC tag.	
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.
	<code>ERuntimeErrorCode.InstanceNotRunning</code>	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be written.
	<code>ERuntimeErrorCode.DoesNotExist</code>	The entry does not exist in the stored tag list.
	<code>ERuntimeErrorCode.NotSupported</code>	Access to entire structures or arrays is not supported.
	<code>ERuntimeErrorCode.TypeMismatch</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285).

Exceptions	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

WriteSignals()

Writes multiple signals within an API call.

When the function is called for the first time, it stores internal information in the structures `SDataValueByName*` to improve the performance of the subsequent calls.

NOTE

To simulate a regular exchange of signals, create a signal list for each set of signals. Use this signal list for all further accesses. Create a new list as soon as the set of signals changes.

Table 9-148 WriteSignals() - Native C++

Syntax	ERuntimeErrorCode WriteSignals(SDataValueByName* inout_Signals, UINT32 in_SignalCount);	
Parameters	<code>SDataValueByName* inout_Signals</code>: Contains the name, the value and the expected type of the PLC tag. The <code>UNSPECIFIC</code> and <code>STRUCT</code> types are not supported. <code>UINT32 in_SignalCount</code>: Number of signals.	
Signal error	Error code	Description
	SREC_INDEX_OUT_OF_RANGE	Offset or bits are invalid.
Return values	Runtime error code	Condition
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_WRONG_ARGUMENT	The expected type is <code>UNSPECIFIC</code> .

Table 9-149 WriteSignals() - .NET (C#)

Syntax	void WriteSignals(SDataValueByName[] in_Signals)	
Parameters	SDataValueByName Contains the name, the value and the expected type of the PLC tag. The UNSPECIFIC and STRUCT types are not supported.	
Signal error	Error code	Description
	ERuntimeErrorCode.IndexOutOfRange	Offset or bits are invalid.
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285-286).
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.WrongArgument	The expected type is UNSPECIFIC.

WriteString()

Writes the value of a PLC tag.

Table 9-150 WriteString() - Native C++

Syntax	<pre>ERuntimeErrorCode IInstance::WriteString(const WCHAR* in_Tag, const CHAR* in_Value, const UINT32 in_CharsToWrite);</pre>	
Parameters	- const WCHAR* in_Tag: The name of the PLC tag that is to be written. - CHAR* in_Value: Contains the value of the PLC tag - const UINT32 in_CharsToWrite: Number of characters to write	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be written.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.

9.6 I/O access

Return values	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_BUFFER_OVERFLOW	The characters to be written are greater than the maximum value of the tag.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-151 WriteString() - .NET (C#)

Syntax	<pre>void WriteString(string in_Tag, string in_Value);</pre>	
Parameters	- string in_Tag: The name of the PLC tag that is to be written. - string in_Value: The value to be written to the PLC tag.	
Exceptions	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.IndexOutOfRange	The offset lies outside the area range. No value could be written.
	ERuntimeErrorCode.DoesNotExist	The entry does not exist in the stored tag list.
	ERuntimeErrorCode.NotSupported	Access to entire structures or arrays is not supported.
	ERuntimeErrorCode.TypeMismatch	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .
	ERuntimeErrorCode.NotUpToDate	The stored tag list must be updated.
	ERuntimeErrorCode.BufferOverflow	The characters to be written are greater than the maximum value of the tag.
	ERuntimeErrorCode.NotEnoughMemory	Unexpected error occurred; turn on logging and contact support.
	ERuntimeErrorCode.InternalError	Internal error occurred; turn on logging and contact support.

WriteWString()

Writes the value of a PLC tag.

Table 9-152 WriteWString() - Native C++

Syntax	<pre>ERuntimeErrorCode IInstance::WriteWString(const WCHAR* in_Tag, const WCHAR* in_Value, const UINT32 in_CharsToWrite,);</pre>	
Parameters	- const WCHAR* in_Tag: The name of the PLC tag that is to be written. - const WCHAR* in_Value: Contains the value of the PLC tag - const UINT32 in_CharsToWrite: Number of characters that are to be written.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_INDEX_OUT_OF_RANGE	The offset lies outside the area range. No value could be written.
	SREC_DOES_NOT_EXIST	The entry does not exist in the stored tag list.
	SREC_NOT_SUPPORTED	Access to entire structures or arrays is not supported.
	SREC_TYPE_MISMATCH	The expected type does not match the stored type. See Compatible primitive data types (Page 285).
	SREC_NOT_UP_TO_DATE	The stored tag list must be updated.
	SREC_BUFFER_OVERFLOW	The characters to be written are greater than the maximum value of the tag.
	SREC_NOT_ENOUGH_MEMORY	Unexpected error occurred; turn on logging and contact support.
	SREC_INTERNAL_ERROR	Internal error occurred; turn on logging and contact support.

Table 9-153 WriteWString() - .NET (C#)

Syntax	<pre>void WriteWString(wstring in_Tag, wstring in_Value);</pre>	
Parameters	- wstring in_Tag: The name of the PLC tag that is to be written. - wstring in_Value: The value to be written to the PLC tag.	
Exceptions	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.

9.6 I/O access

Exceptions	<code>ERuntimeErrorCode.IndexOutOfRange</code>	The offset lies outside the area range. No value could be written.
	<code>ERuntimeErrorCode.DoesNotExist</code>	The entry does not exist in the stored tag list.
	<code>ERuntimeErrorCode.NotSupported</code>	Access to entire structures or arrays is not supported.
	<code>ERuntimeErrorCode.TypeMismatch</code>	The expected type does not match the stored type. See Compatible primitive data types (Page 285) .
	<code>ERuntimeErrorCode.NotUpToDate</code>	The stored tag list must be updated.
	<code>ERuntimeErrorCode.BufferOverflow</code>	The characters to be written are greater than the maximum value of the tag.
	<code>ERuntimeErrorCode.NotEnoughMemory</code>	Unexpected error occurred; turn on logging and contact support.
	<code>ERuntimeErrorCode.InternalError</code>	Internal error occurred; turn on logging and contact support.

GetSystemTime()

Returns the virtual system time of the simulated PLC instance. Returns an empty structure when the function fails.

Table 9-154 GetSystemTime() - Native C++

Syntax	<code>SYSTEMTIME GetSystemTime();</code>
Parameters	None
Return values	<code>SYSTEMTIME</code> : System time of the simulated PLC instance.

SetSystemTime()

Sets the virtual system time of the simulated PLC instance. A system time between "Jan 1 1970 00:00:00:000" and "Dec 31 2200 23:59:59:999" is valid.

Table 9-155 SetSystemTime() - Native C++

Syntax	<code>ERuntimeErrorCode SetSystemTime(SYSTEMTIME in_SystemTime);</code>	
Parameters	<code>SYSTEMTIME in_SystemTime</code>: System time that is to be set for the simulated PLC instance.	
Return values	Runtime error code	Condition
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_INTERFACE_REMOVED</code>	The instance is not registered in Runtime Manager.
	<code>SREC_TIMEOUT</code>	The function does not return on time.
	<code>SREC_WRONG_ARGUMENT</code>	The value is outside the limits.

SystemTime { get; set; }

Sets or returns the virtual system time of the simulated PLC instance. A system time between "Jan 1 1970 00:00:00:000" and "Dec 31 2200 23:59:59:999" is valid.

Table 9-156 SystemTime { get; set; } - .NET (C#)

Syntax	DateTime SystemTime { get; set; }	
Parameters	None	
Return values	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Condition
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.WrongArgument	The value is outside the limits.

GetScaleFactor()

Returns the scaling factor with which the virtual time advances.

Table 9-157 GetScaleFactor() - Native C++

Syntax	double GetScaleFactor();
Parameters	None
Return values	double: Scaling factor of the virtual time.

SetScaleFactor()

Sets the scaling factor with which the virtual time advances.

Siemens recommends that you start with a small scaling factor and gradually work your way up to a scaling factor where the simulated PLC instance remains in RUN.

A value between 0.01 and 100 is valid. The default setting is 1.

- If the value is less than 1, the virtual time of the simulated PLC instance runs X-times slower than the real time.
- If the value is greater than 1, the virtual time of the simulated PLC instance runs X-times faster than the real time.

A change in the value during runtime only takes effect at the cycle control point.

Table 9-158 SetScaleFactor() - Native C++

Syntax	ERuntimeErrorCode SetScaleFactor (double in_Value);	
Parameters	double in_Value: Scaling factor of the virtual time.	
Return values	Runtime error code	Condition
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_WRONG_ARGUMENT	The value is outside the limits.

ScaleFactor { get; set; }

Sets or returns the scaling factor with which the virtual time advances.

Start with a small scaling factor and incrementally approach a scaling factor at which the simulated PLC instance remains in RUN.

A value between 0.01 and 100 is valid. The default setting is 1.

- If the value is less than 1, the virtual time of the simulated PLC instance runs X-times slower than the real time.
- If the value is greater than 1, the virtual time of the simulated PLC instance runs X-times faster than the real time.

A change in the value during runtime only takes effect at the cycle control point.

Table 9-159 ScaleFactor { get; set; } - .NET (C#)

Syntax	double ScaleFactor { get; set; }	
Parameters	None	
Return values	double: Scaling factor of the virtual time.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Condition
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.WrongArgument	The value is outside the limits.

9.7 Cycle control

GetOperatingMode()

Returns the operating mode [\(Page 279\)](#) of the simulated PLC instance.

Table 9-160 GetOperatingMode() - Native C++

Syntax	EOperatingMode GetOperatingMode();
Parameters	None
Return values	EOperatingMode: Operating mode of the simulated PLC instance

SetOperatingMode()

Sets the operating mode of the simulated PLC instance.

A change in the value during runtime only takes effect at the synchronization point.

Table 9-161 SetOperatingMode() - Native C++

Syntax	void SetOperatingMode(EOperatingMode in_OperatingMode);
Parameters	EOperatingMode in_OperatingMode: Operating mode of the simulated PLC instance

OperatingMode { get; set; }

Returns or sets the operating mode of the simulated PLC instance.

A change in the value during runtime only takes effect at the synchronization point.

Table 9-162 OperatingMode { get; set; } - .NET (C#)

Syntax	EOperatingMode OperatingMode { get; set; }	
Parameters	None	
Return values	EOperatingMode: Operating mode of the simulated PLC instance	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.

SetSendSyncEventInDefaultModeEnabled()

Sets the SendSyncEventInDefault mode. In this mode the OnSyncPointReached event is triggered after each cycle end in the Default operating mode. See OnSyncPointReached ([Page 220](#)).

Table 9-163 SetSendSyncEventInDefaultModeEnabled() - Native C++

Syntax	ERuntimeErrorCode SetSendSyncEventInDefaultModeEnabled(bool in_Enable);	
Parameters	bool in_Enable: If true, the OnSyncPointReached event is triggered after each cycle in the Default operating mode.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.

IsSendSyncEventInDefaultModeEnabled()

Returns the SendSyncEventInDefaultMode mode. When the function fails, the return value is false.

Table 9-164 IsSendSyncEventInDefaultModeEnabled() - Native C++

Syntax	bool IsSendSyncEventInDefaultModeEnabled();	
Parameters	None	
Return values	- false: The event is not triggered (unless the Sync-Freeze mode is active). - true: The event is triggered after every cycle.	

IsSendSyncEventInDefaultModeEnabled { get; set; }

Returns or sets the SendSyncEventInDefaultMode mode. In this mode the OnSyncPointReached event is triggered after each cycle end for every operating mode. If

9.7 Cycle control

the event is also to be received in the Default operating mode, set the return value to `true`. See `OnSyncPointReached` (Page 220).

Table 9-165 `IsSendSyncEventInDefaultModeEnabled` { get; set; } - .NET (C#)

Syntax	<code>bool IsSendSyncEventInDefaultModeEnabled { get; set; }</code>	
Parameters	None	
Return values	<code>false</code>: The event is not triggered (unless the Sync-Freeze mode is active). <code>true</code>: The event is triggered after every cycle.	
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.

`GetOverwrittenMinimalCycleTime_ns()`

Returns the overwritten minimum cycle time (in nanoseconds) that is used in the `SingleStep_CT` and `SingleStep_CPT` operating modes.

Table 9-166 `GetOverwrittenMinimalCycleTime_ns()` - Native C++

Syntax	<code>INT64 GetOverwrittenMinimalCycleTime_ns();</code>
Parameters	None
Return values	<code>INT64</code> : The overwritten minimum cycle time in nanoseconds.

`SetOverwrittenMinimalCycleTime_ns()`

Sets the overwritten minimum cycle time (in nanoseconds) that is used in the `SingleStep_CT` and `SingleStep_CPT` operating modes.

A value between 0 and 6000000000 is valid. The default setting is 100 ms.

A change in the value during runtime only takes effect at the cycle control point.

Table 9-167 `SetOverwrittenMinimalCycleTime_ns()` - Native C++

Syntax	<code>ERuntimeErrorCode SetOverwrittenMinimalCycleTime_ns(INT64 in_CycleTime_ns);</code>	
Parameters	<code>INT64 in_CycleTime_ns</code>: The overwritten minimum cycle time in nanoseconds.	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_INTERFACE_REMOVED</code>	The instance is not registered in Runtime Manager.
	<code>SREC_TIMEOUT</code>	The function does not return on time.
	<code>SREC_WRONG_ARGUMENT</code>	The value is outside the limits.

OverwrittenMinimalCycleTime_ns { get; set; }

Returns or sets the overwritten minimum cycle time in nanoseconds that is used in the SingleStep_CT and SingleStep_CPT operating modes.

A value between 0 and 6000000000 is valid. The default setting is 100 ms.

A change in the value during runtime only takes effect at the cycle control point.

Table 9-168 OverwrittenMinimalCycleTime_ns { get; set; } - .NET (C#)

Syntax	Int64 OverwrittenMinimalCycleTime_ns { get; set; }	
Parameters	None	
Return values	Int64: The overwritten minimum cycle time in nanoseconds.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.WrongArgument	The value is outside the limits.

RunToNextSyncPoint()

If the simulated PLC instance is running in a SingleStep operating mode, it is stopped at the synchronization point (Freeze state). The RunToNextSyncPoint() function cancels the freeze state. The simulated PLC instance continues to run until the next synchronization point.

Table 9-169 RunToNextSyncPoint() - Native C++

Syntax	ERuntimeErrorCode RunToNextSyncPoint(); ERuntimeErrorCode RunToNextSyncPoint(UINT32 in_IoSystemId, UINT64 in_BusTimeStamp_ns);	
Parameters	The call with parameters is only valid for the SingleStep_Bus operating mode. - UINT32 in_IoSystemId: Number of the I/O system (PROFIBUS 1..32, PROFINET 100..115) that is to be used for the cycle-controlled synchronization of the simulated PLC instance. - UINT64 in_BusTimeStamp_ns: Time stamp in nanoseconds of the respective I/O system at the start of each cycle.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_INVALID_OPERATING_STATE	The instance is in an invalid operating state for processing the call (BOOTING, SHUTTING_DOWN, OFF)
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_WRONG_ARGUMENT	For the SingleStep_Bus operating mode: The ID of the I/O system is invalid. For all other operating modes: The function call with parameters is invalid.

9.7 Cycle control

Table 9-170 RunToNextSyncPoint() - .NET (C#)

Syntax	<pre>void RunToNextSyncPoint(); void RunToNextSyncPoint(UINT32 in_IoSystemId, UINT64 in_BusTimeStamp_ns);</pre>	
Parameters	The call with parameters is only valid for the <code>SingleStep_Bus</code> operating mode. UINT32 in_IoSystemId: Number of the I/O system (PROFIBUS 1..32, PROFINET 100..115) that is to be used for the cycle-controlled synchronization of the simulated PLC instance. UINT64 in_BusTimeStamp_ns: Time stamp in nanoseconds of the respective I/O system at the start of each cycle.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InvalidOperatingState	The instance is in an invalid operating state for processing the call (BOOTING, SHUTTING_DOWN, OFF)
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.WrongArgument	For the <code>SingleStep_Bus</code> operating mode: The ID of the I/O system is invalid. For all other operating modes: The function call with parameters is invalid.

StartProcessing()

If the simulated PLC instance is running in a `TimespanSynchronized` operating mode, it is stopped at the synchronization point (Freeze state). The `StartProcessing()` function cancels the freeze state. The simulated PLC instance will now run for at least the requested time before it changes to Freeze state at the next synchronization point.

NOTE

StartProcessing() time value represents simulation time

The specified time-value parameter in `StartProcessing()` is not the normal PC time. This time value specifies the simulation time in S7-PLCSIM Advanced. The simulation time differs from PC time because it pauses when waiting for the Windows scheduler to allocate calculation time to the S7-PLCSIM Advanced process. Additionally, it takes some time until the next SyncPoint is reached. This means that the longer the application runs, the time difference between the console application and S7-PLCSIM Advanced increases.

Table 9-171 StartProcessing() - Native C++

Syntax	<pre>ERuntimeErrorCode StartProcessing(INT64 in_MinimalTimeToRun_ns);</pre>	
Parameters	INT64 in_MinimalTimeToRun_ns: The minimum virtual time (in nanoseconds) that the simulated PLC instance runs before it changes to Freeze state.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.

Return values	SREC_TIMEOUT	The function does not return on time.
	SREC_INSTANCE_NOT_RUNNING	The process of the simulated PLC instance is not running.
	SREC_WRONG_ARGUMENT	The value is less than 0.

Table 9-172 StartProcessing() - .NET (C#)

Syntax	void StartProcessing(Int64 in_MinimalTimeToRun_ns);	
Parameters	Int64 in_MinimalTimeToRun_ns: The minimum virtual time (in nanoseconds) that the simulated PLC instance runs before it changes to Freeze state.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.InstanceNotRunning	The process of the simulated PLC instance is not running.
	ERuntimeErrorCode.WrongArgument	The value is less than 0.

More information

For more information, see sections Virtual and real time (Page 179), Stop simulation (Page 110).

9.8 Virtual and real time

The simulated PLC instance uses two types of internal clocks for the simulation: A virtual clock and a real clock. The virtual clock is always the basis for the user program. It is used by components that are relevant for running the STEP 7 user program, such as cyclic OBs, cycle time monitoring, minimum cycle time, virtual system time and time calculations. Also, the time between two cycle control or single step control points is measured in virtual time.

The virtual time can be accelerated or slowed for simulation and optimization purposes.

The real clock always runs unchanged. It is used by components that are not subject to control processes, for example, communication with STEP 7.

Interruption of the process

Since S7-PLCSIM Advanced runs in a Windows environment, Windows might temporarily suspend the simulated PLC instance process. In such a case, both the virtual and the real clock stop in the simulated PLC instance. They only continue to run when Windows resumes processing.

Virtual system time

When you start S7-PLCSIM Advanced, the virtual system time of the instance starts with the system time of Windows.

The virtual system time is based on the virtual clock. If a scaling factor is used, the system time runs correspondingly faster or slower.

All events from the simulated PLC instance provide a time stamp based on the system time.

NOTE**Difference between system time and local time**

- System time: UTC $\pm$ 0 with daylight saving / standard time
 - Local time: UTC $\pm$ time zone with daylight saving time / standard time
-

Time offset

NOTE

Keep in mind that the time information of virtual system time and real local time differs by the time offset that is formed in addition to the selected scaling factor from the time zone offset and the daylight saving time/standard time offset.

Scaling factor

Using a scaling factor, you can speed up or slow down the virtual clock of the simulated PLC instance for simulations.

To set the required scaling factor, enable the grayed out icon  on the control panel to the right of your S7-PLCSIM Advanced instance. The symbol then becomes active  and you can use the scaling function.

- The default setting is 1, i.e. the virtual time passes the same as the real time.
- **Fast forward:** A scaling factor greater than 1 accelerates the virtual clock.
Example: Scaling factor 2.0 → The virtual time is running twice as fast.
- **Slow motion:** A scaling factor less than 1 decelerates the virtual clock.
Example: Scaling factor 0.5 → The progress of the virtual time slows down to 50%.

SetCycleTimeMonitoringMode()

With this function you can change the source of the timer for the maximum cycle time monitoring.

Table 9-173 SetCycleTimeMonitoringMode() - Native C++

Syntax	<pre>ERuntimeErrorCode SetCycleTimeMonitoringMode(ECycleTimeMonitoringMode in_CycleTimeMonitoringMode) ERuntimeErrorCode SetCycleTimeMonitoringMode(ECycleTimeMonitoringMode in_CycleTimeMonitoringMode, INT64 in_MaxCycleTime_ns)</pre>	
Parameters	- ECycleTimeMonitoringMode in_CycleTimeMonitoringMode: Select one of the following options for the maximum cycle time monitoring: - SRCTMM_DOWNLOADED: The maximum cycle time from the project that was downloaded from STEP 7 is used as maximum cycle time monitoring. - SRCTMM_IGNORED (default): A timer value of one minute is used as maximum cycle time monitoring to prevent a potential error in case of an overflow of cyclic events. See Monitoring overflow (Page 183). - SRCTMM_SPECIFIED: A value that is specified with the in_MaxCycleTime_ns parameter is used as maximum cycle time monitoring. Default: 150 ms. - INT64 in_MaxCycleTime_ns: The user-specific value for the maximum cycle time monitoring. A value between 1000000 and 600000000000 ns (1 millisecond to 1 minute) is valid. If no value is specified in the API, the default value of 150 ms applies.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.
	SREC_WRONG_ARGUMENT	The cycle time monitoring mode is invalid.
	SREC_INDEX_OUT_OF_RANGE	The user-specific value for the maximum cycle time monitoring is outside the limits.

Table 9-174 SetCycleTimeMonitoringMode() - .NET (C#)

Syntax	<pre>void SetCycleTimeMonitoringMode(ECycleTimeMonitoringMode in_CycleTimeMonitoringMode) void SetCycleTimeMonitoringMode(ECycleTimeMonitoringMode in_CycleTimeMonitoringMode, Int64 in_MaxCycleTime_ns)</pre>	
Parameters	- ECycleTimeMonitoringMode in_CycleTimeMonitoringMode: Select one of the following options for the maximum cycle time monitoring: - ECycleTimeMonitoringMode.Downloaded: The maximum cycle time from the project that was downloaded from STEP 7 is used as maximum cycle time monitoring. - ECycleTimeMonitoringMode.Ignored (default): A timer value of one minute is used as maximum cycle time monitoring to prevent a potential error in case of an overflow of cyclic events. See Monitoring overflow (Page 183). - ECycleTimeMonitoringMode.Specified: A value that is specified with the in_MaxCycleTime_ns parameter is used as maximum cycle time monitoring. Default: 150 ms. - Int64 in_MaxCycleTime_ns: The user-specific value for the maximum cycle time monitoring. A value between 1000000 and 600000000000 ns (1 millisecond to 1 minute) is valid. If no value is specified in the API, the default value of 150 ms applies.	

Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.
	ERuntimeErrorCode.WrongArgument	The cycle time monitoring mode is invalid.
	ERuntimeErrorCode.IndexOutOfRange	The user-specific value for the maximum cycle time monitoring is outside the limits.

GetCycleTimeMonitoringMode()

This function returns information on the source of the timer for the maximum cycle time monitoring.

Table 9-175 GetCycleTimeMonitoringMode() - Native C++

Syntax	ERuntimeErrorCode GetCycleTimeMonitoringMode(ECycleTimeMonitoringMode* out_CycleTimeMonitoringMode, INT64* out_MaxCycleTime_ns)	
Parameters	- ECycleTimeMonitoringMode* out_CycleTimeMonitoringMode: The configured mode for cycle time monitoring. The default setting is SRCTM_IGNORED. - INT64 in_MaxCycleTime_ns: The user-specific value for the maximum cycle time monitoring. If no value is specified in the API, the default value of 150 ms is returned.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.

Table 9-176 GetCycleTimeMonitoringMode() - .NET (C#)

Syntax	void GetCycleTimeMonitoringMode(out ECycleTimeMonitoringMode out_CycleTimeMonitoringMode, out Int64 out_MaxCycleTime_ns)	
Parameters	- ECycleTimeMonitoringMode out_CycleTimeMonitoringMode: The configured mode for cycle time monitoring. The default setting is ECycleTimeMonitoringMode.Ignored. - Int64 in_MaxCycleTime_ns: The user-specific value for the maximum cycle time monitoring. If no value is specified in the API, the default value of 150 ms is returned.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.

9.9 Monitoring overflow

Monitoring of main cycle

The maximum cycle time monitoring for S7-PLCSIM Advanced is one minute.

Monitoring of cyclical events

If your simulation contains cyclic interrupts, the queue of S7-PLCSIM Advanced can overflow for cyclic events. Due to the runtime speed of S7-PLCSIM Advanced compared to real hardware, the time needed to create the diagnostic buffer entry might be longer than the time until the next cyclic interrupt.

In this case, an additional entry is placed in the queue, causing another overflow. In the event of an overflow, S7-PLCSIM Advanced provides diagnostics buffer messages and a red error icon in the project tree in STEP 7.

9.10 Display functions and events

9.10.1 Display functions

The display functionality allows you to access the display content of a simulated PLC that is equipped with an integrated display.

For unspecified instances or PLC types that do not have an integrated display in their physical hardware, the API returns simulation-specific display data indicating the absence of a display.

The output is returned as raw bitmap data, which you can then integrate and display in your applications.

NOTE

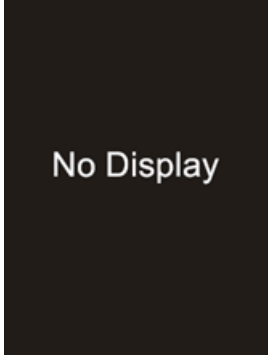
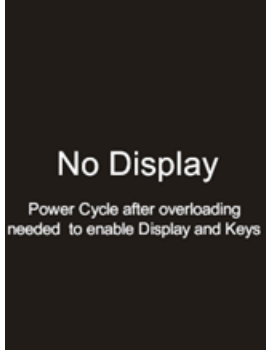
Display functionality is only available for local use and cannot be utilized for remote connections.

NOTE**Restrictions in Softbus mode**

It is not possible to access address settings in the display when using Softbus mode.

Display content scenarios

The following display data is returned based on the state of the instance:

Instance has a display and is switched on	Instance does not have a display and is switched on	Instance is unspecified and switched on	Instance is switched off
Returned data contains the current display content, which is identical to the one a real PLC would show.	Simulation-specific display content is returned, indicating that the PLC does not have a display.	Simulation-specific display content is returned, indicating that the PLC does not have a display and that overload behavior must be considered.	Simulation-specific display content with a PLCSIM Advanced logo on a black background is returned, indicating that the PLC is off.
			

NOTE

To obtain the correct display behavior for an unspecified PLC that has been overloaded with a specified PLC type, a restart of the PLC must be performed. Without restarting, the display behavior stays as described for unspecified instances.

GetDisplayData()

Returns the display data.

The display data does not always occupy the full length of the provided `displayData` buffer within the `SDisplayData` struct. Therefore, a second parameter named `dataLength` within the structure indicates the current length of the valid display data.

The data stored in this struct constitutes the raw bitmap data.

Table 9-177 GetDisplayData() - Native C++

Syntax	<code>ERuntimeErrorCode GetDisplayData(SDisplayData& displayData);</code>
Parameters	<code>SDisplayData& displayData:</code> • <code>struct SDisplayData</code> (Page 264) { <code>uint8_t displayData[MAX_SIZE_DISPLAY_DATA]; uint32_t dataLength;</code> }; – Struct where the display data gets written into – Contains current bitmap data with a maximum length of <code>MAX_SIZE_DISPLAY_DATA</code> (=153666) – <code>dataLength</code> tells the length of the data inside "displayData"

Return Values	Runtime error code	Condition
	SREC_OK	The function is successful.
	SREC_TIMEOUT	The function does not return on time.

Table 9-178 GetDisplayData() - .NET C#

Syntax	SDisplayData GetDisplayData()	
Parameters	None	
Return Values	SDisplayData& displayData: • struct SDisplayData (Page 264){ uint8_t displayData[MaxSizeDisplayData]; uint32_t dataLength; }; – Struct where the display data gets written into – Contains current bitmap data with a maximum length of MaxSizeDisplayData (=153666) – dataLength tells the length of the data inside "displayData"	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Condition
	SREC_TIMEOUT	The function does not return on time.

DisplayKeyPress()

Used for performing display key presses on the PLC.

If the PLC does not have a display, is powered off, or is unspecified, key presses are not supported.

Table 9-179 DisplayKeyPress() Native C++

Syntax	ERuntimeErrorCode DisplayKeyPress(EDisplayKey keyId);	
Parameters	EDisplayKey keyId: • enum EDisplayKey (Page 282) { SRFK_UP, SRFK_DOWN, SRFK_LEFT, SRFK_RIGHT, SRFK_OK, SRFK_ESC, SRFK_LONGESC }; • Enum containing the key to be pressed	
Return Values	Runtime error code	Condition
	SREC_OK	The Key press was executed successfully.
	SREC_NOT_SUPPORTED	The PLC does not support key presses.
	SREC_INSTANCE_NOT_RUNNING	The PLC is not powered on.

Table 9-180 DisplayKeyPress() .NET C#

Syntax	ERuntimeErrorCode DisplayKeyPress(EDisplayKey keyId);	
Parameters	EDisplayKey keyId: - enum EDisplayKey (Page 282) { SRFK_UP, SRFK_DOWN, SRFK_LEFT, SRFK_RIGHT, SRFK_OK, SRFK_ESC, SRFK_LONGESC }; - Enum containing the key to be pressed	
Return Values	Runtime error code	Condition
	SREC_OK	The Key press was executed succesfully.
	SREC_NOT_SUPPORTED	The PLC does not support key presses.
	SREC_INSTANCE_NOT_RUNNING	The PLC is not powered on.
Exceptions	None	

HasDisplay()

Informs you as to whether an instance has a display.

A return value of `true` confirms the presence of a display, while `false` indicates that the instance does not have a display.

To use this functionality, the PLC must be powered on.

Table 9-181 HasDisplay() Native C++

Syntax	ERuntimeErrorCode HasDisplay(bool& hasDisplay)	
Parameters	bool& hasDisplay: Reference parameter that indicates whether the PLC has a display	
Return Values	Runtime error code	Condition
	SREC_OK	The function finished succesfully.
	SREC_INSTANCE_NOT_RUNNING	The PLC is not powered on.

Table 9-182 HasDisplay() .NET C#

Syntax	bool HasDisplay();	
Parameters	None	
Return values	bool: Returns <code>true</code> if the PLC has a display Returns <code>false</code> if the PLC does not have a display	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Condition
	SREC_INSTANCE_NOT_RUNNING	The PLC is not powered on.

9.10.2 OnDisplayDataChanged events

The subscription to the callback can be made as soon as the PLC exists, regardless of whether it is powered on or what its display integration state is.

When the display data changes, the registered callback is triggered. The call of the callback informs that the display data has been changed and new bitmap data is accessible.

The display data is not delivered by the callback itself.

OnDisplayDataChanged()

Registers or unregisters an event handler method.

After registration an event will be fired when the Display Data has changed.

NOTE

The event handler method runs in a separate thread.

Table 9-183 OnDisplayDataChanged() .NET C#

Syntax	event Delegate_DisplayDataChanged OnDisplayDataChanged;	
Parameters	Delegate_DisplayDataChanged	
	Syntax	delegate void Delegate_DisplayDataChanged (IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, System::DateTime in_DateTime);
	Parameters	- IInstance in_Sender: An interface of the instance that receives this event. - ERuntimeErrorCode in_ErrorCode: A possible error code. - DateTime in_DateTime: The virtual system time of the virtual controller at the time when this event was triggered.
	Return Values	None
Return Values	None	
Exceptions	None	

RegisterOnDisplayDataChangedCallback()

When the event occurs, the registered callback function is called. Only one callback function can be registered for the event. Registering a new callback function causes the previous callback function to be deleted.

NOTE

The event handler method runs in a separate thread.

Table 9-184 RegisterOnDisplayDataChangedCallback() Native C++

Syntax	void RegisterOnDisplayDataChangedCallback (EventCallback_II_SREC_ST in_CallbackFunction);	
Parameters	EventCallback_II_SREC_ST in_CallbackFunction: • A callback function that subscribes to an event	
	Syntax	typedef void (*EventCallback_II_SREC_ST){ IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime };
	Parameters	- IInstance* in_Sender: An interface of the instance that receives this event. - ERuntimeErrorCode in_ErrorCode: A possible error code. - SYSTEMTIME in_SystemTime: The virtual system time of the virtual controller at the time when this event was triggered.
	Return Values	None
Return Values	Runtime error code	Condition
	SREC_OK	The function is successful.
	SREC_TIMEOUT	The function does not return on time.

RegisterOnDisplayDataChangedEvent()

When the event occurs, the registered event object is set to the signaled state. Only one event object can be registered for the event. Registration of a new event object causes the previous event object to be deleted.

The event signals that the display data changed.

Table 9-185 RegisterOnDisplayDataChangedEvent() - Native C++

Syntax	void RegisterOnDisplayDataChangedEvent (); void RegisterOnDisplayDataChangedEvent (HANDLE* in_Event);	
Parameters	- None: An internal event object is registered. - HANDLE* in_Event: A handle for a user-specific event object. The event object is registered.	
Return Values	None	

Table 9-186 RegisterOnDisplayDataChangedEvent() - .NET C#

Syntax	void UnregisterOnDisplayDataChangedEvent ();	
Parameters	None	
Return Values	None	

UnregisterOnDisplayDataChangedCallback()

Unregisters the callback function. When the event occurs, no callback function is called.

Table 9-187 UnregisterOnDisplayDataChangedCallback() - Native C++

Syntax	<code>void UnregisterOnDisplayDataChangedCallback();</code>
Parameters	None
Return Values	None

UnregisterOnDisplayDataChangedEvent()

Unregisters the event object. The information that the display data got changed is now not signaled anymore.

Table 9-188 UnregisterOnDisplayDataChangedEvent() - Native C++

Syntax	<code>void UnregisterOnDisplayDataChangedEvent();</code>
Parameters	None
Return Values	None

Table 9-189 UnregisterOnDisplayDataChangedEvent() - .NET C#

Syntax	<code>void UnregisterOnDisplayDataChangedEvent();</code>
Parameters	None
Return Values	None

WaitForOnDisplayDataChangedEvent()

The function blocks the program until the registered event object is set to the signaled state or the timeout interval is exceeded.

This means the program can be blocked until a change of display data occurs.

Table 9-190 WaitForOnDisplayDataChangedEvent() - Native C++

Syntax	<code>bool WaitForOnDisplayDataChangedEvent();</code> <code>bool WaitForOnDisplayDataChangedEvent(UINT32 in_Time_ms);</code>
Parameters	- None: The time limit is set to INFINITE. <code>UINT32 in_Time_ms</code>: Value for the time limit in milliseconds.
Return Values	- true: If the event object was set to the signaled state. - false: If no event was received during the defined time limit.

9.11 Acyclic services

Table 9-191 WaitForOnDisplayDataChangedEvent() - .NET #

Syntax	<code>bool WaitForOnDisplayDataChangedEvent();</code> <code>bool WaitForOnDisplayDataChangedEvent(UINT32 in_Time_ms);</code>
Parameters	- None: The time limit is set to INFINITE. - UINT32 in_Time_ms: Value for the time limit in milliseconds.
Return Values	- true: If the event object was set to the signaled state. - false: If no event was received during the defined time limit.

9.11 Acyclic services

9.11.1 Overview

The acyclic services of S7-PLCSIM Advanced include:

- Read and write processes of parameter and status data from the user program of the PLC to the I/O modules
- Interrupt and event information which the I/O modules send to the CPU.

Read and write operations

Events triggered by the STEP 7 user program that have logged on for the notification:

Table 9-192 Events: Read and write operations

SFB	Name	API method (alarm)	API event for triggering the SFB
52	RDREC	ReadRecordDone (Page 192)	OnDataRecordRead (Page 225)
53	WRREC	WriteRecordDone (Page 192)	OnDataRecordWrite (Page 225)

Flowchart

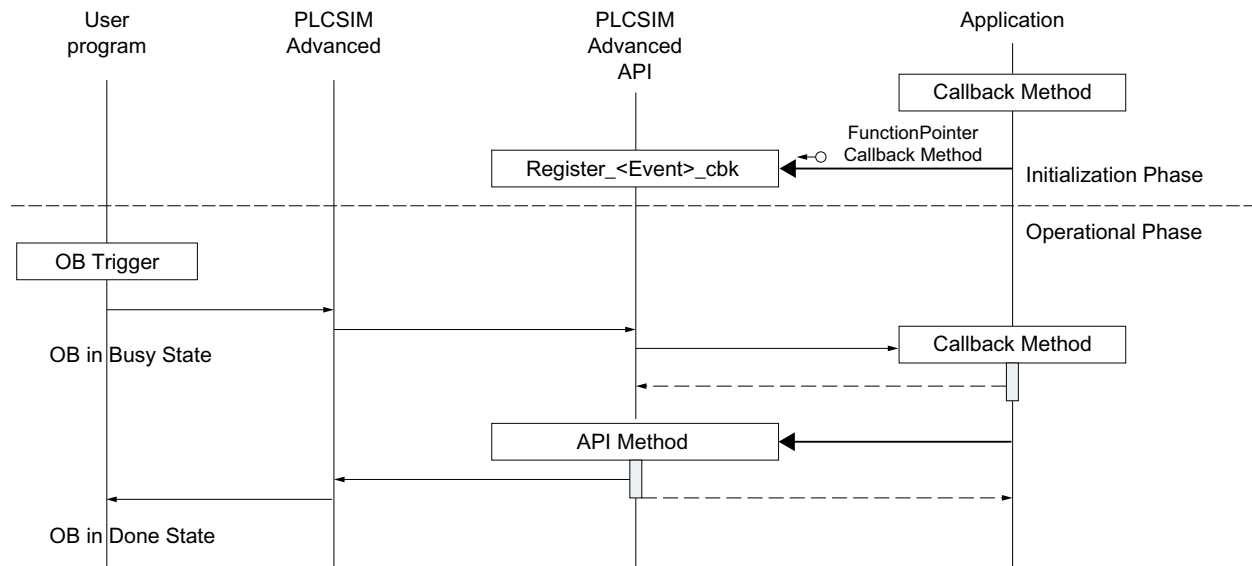


Figure 9-3 Read and write operations flowchart

API methods and associated events

Events which are triggered by the I/O modules and associated API methods:

Table 9-193 API methods and associated events

OB	Name	API methods for triggering the OB (query)	API event after OB execution (alarm)
82	Diagnostic error Interrupt	AlarmNotification (Page 194)	OnAlarmNotificationDone (Page 227)
4x	Hardware Interrupt	ProcessEvent (Page 196)	OnProcessEventDone (Page 228)
83	Pull or Plug of module	PullOrPlugEvent (Page 197)	OnPullOrPlugEventDone (Page 229)
55	Status	StatusEvent (Page 199)	OnStatusEventDone (Page 230)
57	Profile	ProfileEvent (Page 200)	OnProfileEventDone (Page 231)
56	Update	UpdateEvent (Page 201)	OnUpdateEventDone (Page 232)
86	Rack or station failure	RackOrStationFaultEvent (Page 202)	OnRackOrStationFaultEventDone (Page 233)

Flowchart

Flowchart for the simulation of events which are triggered by the I/O modules.

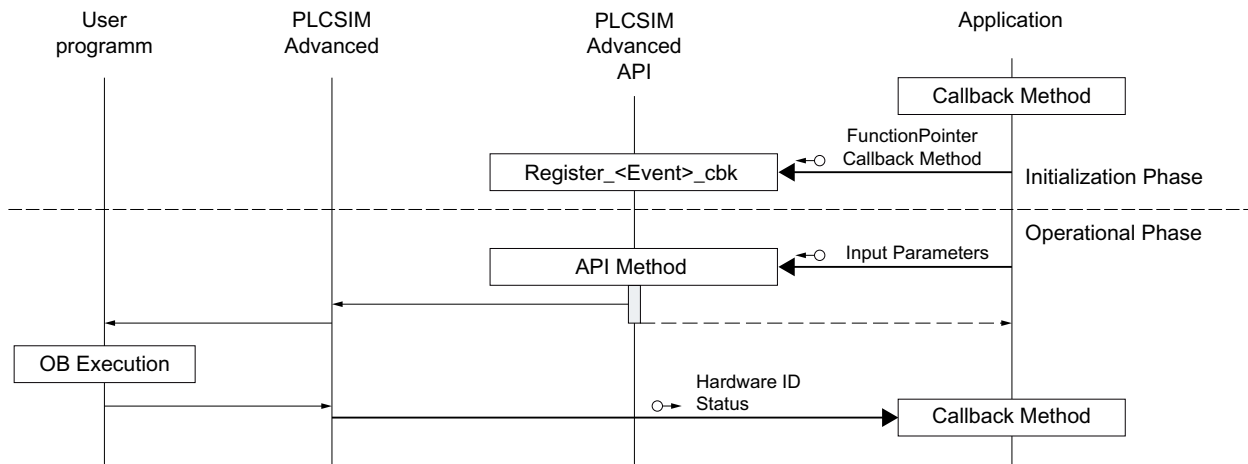


Figure 9-4 Flowchart for the simulation of events

9.11.2 ReadRecordDone/WriteRecordDone

ReadRecordDone()

With this API method, the simulation of an I/O module signals to the CPU that the asynchronous reading of a data record has been completed. The simulation hereby makes the read information available.

Table 9-194 ReadRecordDone() - Native C++

Syntax	ERuntimeErrorCode ReadRecordDone(SDataRecordInfo in_RecordInfo, BYTE* in_Data, UINT32 in_Status);	
Parameters	- SDataRecordInfo in_RecordInfo: Structure which contains the data record information. See SDataRecordInfo (Page 270). - BYTE* in_Data: Byte array of the read data record with the length defined by DataSize in the structure SDataRecordInfo. - UINT32 in_Status: Status of the job execution	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_DOES_NOT_EXIST	No hardware identifier of the module.
	SREC_INDEX_OUT_OF_RANGE	The byte array of the read data record exceeds the length DDATARECORD_MAX_SIZE = 64000.
	SREC_TIMEOUT	The function does not return on time.

Table 9-195 ReadRecordDone() - .NET (C#)

Syntax	<code>void ReadRecordDone(SDataRecordInfo in_RecordInfo, BYTE[] in_Data, UInt32 in_Status);</code>	
Parameters	<code>SDataRecordInfo in_RecordInfo</code>: Structure which contains the data record information. See SDataRecordInfo (Page 270). <code>BYTE[] in_Data</code>: Byte array of the read data record with the length defined by <code>DataSize</code> in the structure <code>SDataRecordInfo</code>. <code>UInt32 in_Status</code>: Status of the job execution	
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.DoesNotExist</code>	No hardware identifier of the module.
	<code>ERuntimeErrorCode.IndexOutOfRange</code>	The byte array of the read data record exceeds the length <code>DataRecordMaxSize = 64000</code> .
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.

WriteRecordDone()

With this API method, the simulation of an I/O module signals to the CPU that the asynchronous writing of a data record has been completed.

Table 9-196 WriteRecordDone() - Native C++

Syntax	<code>ERuntimeErrorCode WriteRecordDone(SDataRecordInfo in_RecordInfo, UINT32 in_Status);</code>	
Parameters	<code>SDataRecordInfo in_RecordInfo</code>: Structure which contains the data record information. See SDataRecordInfo (Page 270). <code>UINT32 in_Status</code>: Status of the job execution	
Return values	Runtime error code	Description
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_INTERFACE_REMOVED</code>	The instance is not registered in Runtime Manager.
	<code>SREC_DOES_NOT_EXIST</code>	No hardware identifier of the module.
	<code>SREC_TIMEOUT</code>	The function does not return on time.

Table 9-197 WriteRecordDone() - .NET (C#)

Syntax	<code>void WriteRecordDone(SDataRecordInfo in_RecordInfo, UInt32 in_Status);</code>	
Parameters	<code>SDataRecordInfo in_RecordInfo</code>: Structure which contains the data record information. See SDataRecordInfo (Page 270). <code>UInt32 in_Status</code>: Status of the job execution	
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description

9.11 Acyclic services

Exceptions	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.DoesNotExist	No hardware identifier of the module.
	ERuntimeErrorCode.Timeout	The function does not return on time.

NOTE**Return values of a written data record**

The return values of the status of a written data record in S7-PLCSIM Advanced can differ from the return values of a real hardware CPU.

9.11.3 AlarmNotification

AlarmNotification()

This function triggers diagnostic alarms according to the PROFINET standard.

Each call of this function calls the diagnostic interrupt OB (OB 82) once, regardless of the number and the severity level of the transferred diagnostic entries.

Table 9-198 AlarmNotification() - Native C++

Syntax	ERuntimeErrorCode AlarmNotification(UINT16 in_HardwareIdentifier, UINT16 in_ModuleState, UINT16 in_NumberOfDiagnosisEvents, SDiagExtChannelDescription* in_ArrayOfDiagnosisEvents, UINT16* out_SequenceNumber);	
Parameters	- UINT16 in_HardwareIdentifier: The hardware identifier of the module or submodule which sends the diagnostics entry. The identifier must belong to a hardware component in the currently loaded project. - UINT16 in_ModuleState: Module status. The following statuses are valid: DMODULE_STATE_OK = 0, DMODULE_STATE_ERROR = 1, DMODULE_STATE_MAINT_DEMANDED = 2, DMODULE_STATE_MAINT_REQUIRED = 4 The in_ModuleState parameter is derived from the sum (ORed) of the severity level in the SDiagExtChannelDescription field. If a diagnostic interrupt should be generated for both "Maintenance demanded" as well as "Maintenance required", select "6" as the module status. - UINT16 in_NumberOfDiagnosisEvents: Multiple diagnostic entries can be sent to the CPU with a single API call. Valid range: 0 to 16. 0 means that no diagnostics entry should appear for the submodule or the channel. - SDiagExtChannelDescription* in_ArrayOfDiagnosisEvents: Pointer to a field with diagnostic entries. The field must match the number of diagnostic entries. It can also be a null pointer. For definitions, see SDiagExtChannelDescription (Page 272). - UINT16* out_SequenceNumber: S7-PLCSIM Advanced assigns a unique consecutive number to each interrupt event. According to PROFINET standard the sequence number is 10 bits wide (1 to 7FF_H). When the highest number is reached the numbering starts again at 1.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_WRONG_MODULE_STATE	The module is currently unplugged.

Return values	SREC_DOES_NOT_EXIST	No hardware identifier of the module.
	SREC_WRONG_MODULE_TYPE	The channel number does not exist for the module.
	SREC_WRONG_ARGUMENT	The value for the module status is invalid.
	SREC_TIMEOUT	The function does not return on time.

Table 9-199 AlarmNotification() - .NET (C#)

Syntax	void AlarmNotification(ushort in_HardwareIdentifier, ushort in_ModuleState, ushort in_NumberOfDiagnosisEvents, SDiagExtChannelDescription [] in_ArrayOfDiagnosisEvents, Out ushort out_SequenceNumber);	
Parameters	- ushort in_HardwareIdentifier: The hardware identifier of the module or submodule which sends the diagnostics entry. The identifier must belong to a hardware component in the currently loaded project. - ushort in_ModuleState: Module status. The following statuses are valid: ModuleState.Ok = 0, ModuleState.Error = 1, ModuleState.MaintenanceDemanded = 2, ModuleState.MaintenanceRequired = 4 The in_ModuleState parameter is derived from the sum (ORed) of the severity level in the SDiagExtChannelDescription field. If a diagnostic interrupt should be generated for both "Maintenance demanded" as well as "Maintenance required", select "6" as the module status. - ushort in_NumberOfDiagnosisEvents Multiple diagnostic entries can be sent to the CPU with a single API call. Valid range: 0 to 16. 0 means that no diagnostics entry should appear for the submodule or the channel. - SDiagExtChannelDescription [] in_ArrayOfDiagnosisEvents: Pointer to a field with diagnostic entries. The field must match the number of diagnostic entries. It can also be a null pointer. For definitions, see SDiagExtChannelDescription (Page 272). - Out ushort out_SequenceNumber: S7-PLCSIM Advanced assigns a unique consecutive number to each alarm event. According to PROFINET standard the sequence number is 10 bits wide (1 to 7FF_H). When the highest number is reached the numbering starts again at 1.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.WrongModuleState	The module is currently unplugged.
	ERuntimeErrorCode.DoesNotExist	No hardware identifier of the module.
	ERuntimeErrorCode.WrongArgument	The value for the module status is invalid.
	ERuntimeErrorCode.WrongModuleType	The channel number does not exist for the module.
	ERuntimeErrorCode.Timeout	The function does not return on time.
Example	<pre> ushort seqNumber; var In_ArrayOfDiagnosisEvent = new SDiagExtChannelDescription[] { new SDiagExtChannelDescription() {ChannelNumber = 0x8000, ErrorType = 0x0001, ExtErrorType = 0, Direction = EDiagProperty.Appear, Severity =EDiagSeverity.MaintDemanded}, new SDiagExtChannelDescription() {ChannelNumber = 0x8000, ErrorType = 0x0002, ExtErrorType = 0, Direction = EDiagProperty.Appear, Severity =EDiagSeverity.Failure}, </pre>	

9.11 Acyclic services

	<pre> new SDiagExtChannelDescription() {ChannelNumber = 0x8000, ErrorType = 0x0003, ExtErrorType = 0, Direction = EDiagProperty.Appear,Severity =EDiagSeverity.MaintRequired}, Instance.AlarmNotification(269, 7, 3, In_ArrayOfDiagnosisEvent, out seqNumber); //ModuleState parameter is sum of the severities in the SDiagExtChannelDescription array above: 4+2+1 </pre>
--	--

9.11.4 ProcessEvent

ProcessEvent()

Process events from central and distributed input modules can be simulated with this function.

Table 9-200 ProcessEvent() - Native C++

Syntax	ERuntimeErrorCode ProcessEvent(UINT16 in_HardwareIdentifier, UINT16 in_Channel, EProcessEvent in_ProcessEventType, UINT16* out_SequenceNumber);	
Parameters	- UINT16 in_HardwareIdentifier: The hardware identifier of the module or submodule which sends the process event. The identifier must belong to a hardware component in the currently loaded project. - UINT16 in_Channel: The channel of the I/O module which sends the process event. - EProcessEvent in_ProcessEventType: A value from the list of predefined types of events for S7 modules, see EProcessEvent (Page 291). - UINT16* out_SequenceNumber: S7-PLCSIM Advanced assigns a unique consecutive number to each interrupt event. According to PROFINET standard the sequence number is 10 bits wide (1 to 7FF_H). When the highest number is reached the numbering starts again at 1.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_WRONG_MODULE_STATE	The module is currently unplugged.
	SREC_DOES_NOT_EXIST	No hardware identifier of the module.
	SREC_NOT_SUPPORTED_BY_MODULE	The module is not supported by this user action.
	SREC_TIMEOUT	The function does not return on time.

Table 9-201 ProcessEvent() - .NET (C#)

Syntax	void ProcessEvent(ushort in_HardwareIdentifier, ushort in_Channel, EProcessEvent in_ProcessEventType, Out ushort out_SequenceNumber);	
Parameters	- ushort in_HardwareIdentifier: The hardware identifier of the module or submodule which generates the process event. The identifier must belong to a hardware component in the currently loaded project. - ushort in_Channel: The channel of the I/O module which generates the process event.	

	<code>EProcessEventType</code> in <code>ProcessEventType</code>: A value from the list of predefined types of events for S7 modules, see <code>EProcessEventType</code> (Page 291). <code>Out ushort out_SequenceNumber</code>: S7-PLCSIM Advanced assigns a unique consecutive number to each interrupt event. According to PROFINET standard the sequence number is 10 bits wide (1 to 7FF_H). When the highest number is reached the numbering starts again at 1.	
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.WrongModuleState</code>	The module is currently unplugged.
	<code>ERuntimeErrorCode.DoesNotExist</code>	No hardware identifier of the module.
	<code>ERuntimeErrorCode.NotSupportedByModule</code>	The module is not supported by this user action.
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.

Priority for hardware interrupt OB

The hardware interrupts triggered via the S7-PLCSIM Advanced API are transmitted in sequence to the user program.

The priority of the assigned hardware interrupt OB determines the sequence of execution only if events occur simultaneously.

9.11.5 PullOrPlugEvent

PullOrPlugEvent()

This function triggers pull/plug events. The interrupt OB (OB 83) "Pull or plug of modules" is executed for these events.

NOTE

Removing and inserting head modules of distributed I/O systems and R1 devices

An actual PLC cannot detect a pull or plug of a head module in a distributed I/O system, except for R1 devices.

In contrast, a simulated PLC in S7-PLCSIM Advanced does allow the removal and insertion of head modules in distributed I/O systems and R1 devices. Your program can use this event for such a case, but it does not correspond to an event that would occur with an actual PLC.

NOTE

PN/PC Coupler devices

With a physical PN/PN Coupler device, you cannot pull or plug the module.

S7-PLCSIM Advanced allows a pull or plug of PN/PN Couplers if Proxy HwID is used as input for pull/plug events.

9.11 Acyclic services

Table 9-202 PullOrPlugEvent() - Native C++

Syntax	<code>ERuntimeErrorCode PullOrPlugEvent(UINT16 in_HardwareIdentifier, EPullOrPlugEventType in_PullOrPlugEventType, UINT16* out_SequenceNumber);</code>	
Parameters	UINT16 in_HardwareIdentifier: The hardware identifier of the module or submodule which generates the pull/plug event. The identifier must belong to a hardware component in the currently loaded project. EPullOrPlugEventType in_PullOrPlugEventType: A value from the list of predefined types of pull/plug events, see EPullOrPlugEventType (Page 291). UINT16* out_SequenceNumber: S7-PLCSIM Advanced assigns a unique consecutive number to each interrupt event. According to PROFINET standard the sequence number is 10 bits wide (1 to 7FF_H). When the highest number is reached the numbering starts again at 1.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_WRONG_MODULE_STATE	The module is currently unplugged.
	SREC_WRONG_MODULE_TYPE	The wrong module type was selected, e.g. if an onboard I/O of a compact CPU is to be pulled.
	SREC_NOT_SUPPORTED_BY_MODULE	The module is not supported by this user action.
	SREC_DOES_NOT_EXIST	No hardware identifier of the module.
	SREC_TIMEOUT	The function does not return on time.

Table 9-203 PullOrPlugEvent() - .NET (C#)

Syntax	<code>void PullOrPlugEvent(ushort in_HardwareIdentifier, EPullOrPlugEventType in_PullOrPlugEventType, Out ushort out_SequenceNumber);</code>	
Parameters	ushort in_HardwareIdentifier: The hardware identifier of the module or submodule which generates the pull/plug event. The identifier must belong to a hardware component in the currently loaded project. EPullOrPlugEventType in_PullOrPlugEventType: A value from the list of predefined types of pull/plug events, see EPullOrPlugEventType (Page 291). Out ushort out_SequenceNumber S7-PLCSIM Advanced assigns a unique consecutive number to each interrupt event. According to PROFINET standard the sequence number is 10 bits wide (1 to 7FF_H). When the highest number is reached the numbering starts again at 1.	
Return values	None	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.WrongModuleState	The module is currently unplugged.
	ERuntimeErrorCode.WrongModuleType	The wrong module type was selected, e.g. if an onboard I/O of a compact CPU is to be pulled.
	ERuntimeErrorCode.DoesNotExist	No hardware identifier of the module.
	ERuntimeErrorCode.NotSupportedByModule	The module is not supported by this user action.
	ERuntimeErrorCode.Timeout	The function does not return on time.

9.11.6 StatusEvent

StatusEvent()

This function is used to trigger the status event OB (OB 55). Status events are only supported for modules in a distributed I/O system.

Table 9-204 StatusEvent() - Native C++

Syntax	<code>ERuntimeErrorCode StatusEvent(UINT16 in_HardwareIdentifier, UINT16 in_Specifier);</code>	
Parameters	UINT16 in_HardwareIdentifier: The hardware identifier of the module that generates the status event. The identifier must belong to a hardware component in the currently loaded project. UINT16 in_Specifier: The parameter is transferred to the interrupt frame as interrupt specifier. It is available as input parameter of the OB 55 call.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_WRONG_MODULE_STATE	The module is currently unplugged.
	SREC_NOT_SUPPORTED_BY_MODULE	The module is not supported by this user action.
	SREC_DOES_NOT_EXIST	No hardware identifier of the module.
	SREC_TIMEOUT	The function does not return on time.

Table 9-205 StatusEvent() - .NET (C#)

Syntax	<code>void StatusEvent(ushort in_HardwareIdentifier, ushort in_Specifier);</code>	
Parameters	ushort in_HardwareIdentifier: The hardware identifier of the module that generates the status event. The identifier must belong to a hardware component in the currently loaded project. ushort in_Specifier: The parameter is transferred to the interrupt frame as interrupt specifier. It is available as input parameter of the OB 55 call.	
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.WrongModuleState</code>	The module is currently unplugged.
	<code>ERuntimeErrorCode.DoesNotExist</code>	No hardware identifier of the module.
	<code>ERuntimeErrorCode.NotSupportedByModule</code>	The module is not supported by this user action.
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.

9.11.7 ProfileEvent

ProfileEvent()

This function is used to trigger the Profile event OB (OB 57). Profile events are only supported for modules in a distributed I/O system.

Table 9-206 ProfileEvent() - Native C++

Syntax	<code>ERuntimeErrorCode ProfileEvent(UINT16 in_HardwareIdentifier, UINT16 in_Specifier);</code>	
Parameters	UINT16 in_HardwareIdentifier: The hardware identifier of the module that generates the profile event. The identifier must belong to a hardware component in the currently loaded project. UINT16 in_Specifier: The parameter is transferred to the interrupt frame as interrupt specifier. It is available as input parameter of the OB 57 call.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_WRONG_MODULE_STATE	The module is currently unplugged.
	SREC_NOT_SUPPORTED_BY_MODULE	The module is not supported by this user action.
	SREC_DOES_NOT_EXIST	No hardware identifier of the module.
	SREC_TIMEOUT	The function does not return on time.

Table 9-207 ProfileEvent() - .NET (C#)

Syntax	<code>void ProfileEvent(ushort in_HardwareIdentifier, ushort in_Specifier);</code>	
Parameters	ushort in_HardwareIdentifier: The hardware identifier of the module that generates the profile event. The identifier must belong to a hardware component in the currently loaded project. ushort in_Specifier: The parameter is transferred to the interrupt frame as interrupt specifier. It is available as input parameter of the OB 57 call.	
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.WrongModuleState</code>	The module is currently unplugged.
	<code>ERuntimeErrorCode.DoesNotExist</code>	No hardware identifier of the module.
	<code>ERuntimeErrorCode.NotSupportedByModule</code>	The module is not supported by this user action.
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.

9.11.8 UpdateEvent

UpdateEvent()

This function is used to trigger the Update event OB (OB 56). Update events are only supported for modules in a distributed I/O system.

Table 9-208 UpdateEvent() - Native C++

Syntax	<code>ERuntimeErrorCode UpdateEvent(UINT16 in_HardwareIdentifier, UINT16 in_Specifier);</code>	
Parameters	UINT16 in_HardwareIdentifier: The hardware identifier of the module that triggers the update event. The identifier must belong to a hardware component in the currently loaded project. UINT16 in_Specifier: The parameter is transferred to the interrupt frame as interrupt specifier. It is available as input parameter of the OB 56 call.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_WRONG_MODULE_STATE	The module is currently unplugged.
	SREC_NOT_SUPPORTED_BY_MODULE	The module is not supported by this user action.
	SREC_DOES_NOT_EXIST	No hardware identifier of the module.
	SREC_TIMEOUT	The function does not return on time.

Table 9-209 UpdateEvent() - .NET (C#)

Syntax	<code>void UpdateEvent(ushort in_HardwareIdentifier, ushort in_Specifier);</code>	
Parameters	ushort in_HardwareIdentifier: The hardware identifier of the module that triggers the update event. The identifier must belong to a hardware component in the currently loaded project. ushort in_Specifier: The parameter is transferred to the interrupt frame as interrupt specifier. It is available as input parameter of the OB 56 call.	
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.WrongModuleState</code>	The module is currently unplugged.
	<code>ERuntimeErrorCode.DoesNotExist</code>	No hardware identifier of the module.
	<code>ERuntimeErrorCode.NotSupportedByModule</code>	The module is not supported by this user action.
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.

9.11.9 GetConfiguredProcessEvent

GetConfiguredProcessEvents()

With this API method, the process events configured in the TIA Portal can be read out during runtime.

If no process events are present, SREC_OK is returned. The value for EventsCount is then 0.

Table 9-210 GetConfiguredProcessEvents() - Native C++

Syntax	<code>ERuntimeErrorCode GetConfiguredProcessEvents(UINT16* out_EventsCount,);</code>	
Parameters	<code>SConfiguredProcessEvents* inout_ProcessEvents:</code> Pointer or reference to a user-defined memory which contains the field with the downloaded configured process events. The structure <code>SConfiguredProcessEvents</code> (Page 271) contains information about these process events. <code>UINT16* out_EventsCount:</code> Pointer or reference to a tag which contains the number of configured process events.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.

Table 9-211 GetConfiguredProcessEvents() - .NET (C#)

Syntax	<code>SConfiguredProcessEvents [] GetConfiguredProcessEvents();</code>	
Parameters	None	
Return values	Field with configured process events and field size provide the number of configured process events.	
Exceptions	<code>Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException</code>	
	Runtime error code	Description
	<code>ERuntimeErrorCode.InterfaceRemoved</code>	The instance is not registered in Runtime Manager.
	<code>ERuntimeErrorCode.Timeout</code>	The function does not return on time.

9.11.10 RackOrStationFaultEvent

Description

This function is used to trigger the RackOrStationFault event OB (OB 86). These events are only supported for distributed devices.

Table 9-212 RackOrStationFaultEvent() - Native C++

Syntax	<code>ERuntimeErrorCode RackOrStationFaultEvent(UINT16 in_HardwareIdentifier, ERackOrStationFaultType in_EventType);</code>	
Parameter	<code>UINT16 in_HardwareIdentifier:</code> The hardware identifier of the device that sends the event. Use the hardware identifier of the <code>Hw_Device</code> type. <code>ERackOrStationFaultType in_EventType:</code> A value from the list of predefined types of events, see <code>ERackOrStationFaultType</code> (Page 293).	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The instance is not registered in Runtime Manager.

Return values	SREC_WRONG_MODULE_TYPE	The specified HW identifier is not that of a distributed device.
	SREC_WRONG_MODULE_STATE	The device with the specified HW identifier already reports the status Fault/Return.
	SREC_DOES_NOT_EXIST	The specified HW identifier of the device does not exist.
	SREC_TIMEOUT	The function does not return on time.

Table 9-213 RackOrStationFaultEvent() - .NET (C#)

Syntax	void RackOrStationFaultEvent(ushort in_HardwareIdentifier, ERackOrStationFaultType in_EventType);	
Parameter	- ushort in_HardwareIdentifier: The hardware identifier of the device that sends the event. Use the hardware identifier of the Hw_Device type. - ERackOrStationFaultType in_EventType: A value from the list of predefined types of events, see ERackOrStationFaultType (Page 293).	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The instance is not registered in Runtime Manager.
	ERuntimeErrorCode.DoesNotExist	The specified HW identifier of the device does not exist.
	ERuntimeErrorCode.WrongModuleType	The specified HW identifier is not that of a distributed device.
	ERuntimeErrorCode.WrongModuleState	The device with the specified HW identifier already reports the status Fault/Return.
	ERuntimeErrorCode.Timeout	The function does not return on time.

More information

You can find more information on the HW identifier in the STEP 7 online help.

9.12 Events for Instances

9.12.1 Events for operating state and cycle control

The following events are triggered for the Instances interface:

Table 9-214 Events for Instances

Event	Cause
OnOperatingStateChanged (Page 210)	The operating state of the simulated PLC instance has changed.
OnLedChanged (Page 213)	The LED display of the simulated PLC instance has changed.
OnSoftwareConfigurationChanged (- Page 216)	The software configuration of the simulated PLC instance changed: - After power up from the Virtual SIMATIC Memory Card - At the end of a download with changed software configuration When this event is triggered, the stored tag list is reset.

Event	Cause
OnIPAddressChanged (Page 218)	The IP Address of the simulated PLC has changed.
OnSyncPointReached (Page 220)	The simulated PLC instance has reached a synchronization point. If the simulated PLC instance is being operated in <code>Default</code> mode, the <code>SendSyncEventInDefaultMode</code> flag must be set to receive the event. See <code>SendSyncEventInDefaultMode</code> (Page 174).
OnHardwareConfigChanged (Page 222)	When the hardware configuration of the simulated PLC instance has changed, the event is triggered at the following times: - After power up from the Virtual SIMATIC Memory Card - During the Hardware Config download

9.12.1.1 Synchronize simulation partner

Synchronize simulation partner cycle-controlled

SingleStep operating modes

The SingleStep modes of operation of the simulated PLC instance are used to synchronize multiple simulation partners (clients) using synchronization points. The operating modes define the synchronization point at which the instance changes to the freeze state and sends the `OnSyncPointReached` event.

Table 9-215 Cycle-controlled operating modes (SingleStep)

Operating mode ¹	Synchronization point		Minimum cycle time ²	Send clock "Bus" ³
	Cycle control point	Before reading in the process image partition		
	"C"	"P"		
SingleStep_C ⁴	✓			
SingleStep_P		✓		
SingleStep_CP ⁴	✓	✓		
SingleStep_CT ⁴	✓		✓	
SingleStep_CPT ⁴	✓	✓	✓	
SingleStep_Bus	✓			✓

¹ A change from one SingleStep operating mode to another during operation would result in the input parameter `TimeSinceSameSyncPoint_ns` / `TimeSinceAnySyncPoint_ns` being set to 0 in the first `SyncPointReached` callback event in the new operating mode. We recommend that the SingleStep operating mode not be changed during operation.

² In addition, the minimum scan cycle time of OB 1 is overwritten in this operating mode. When you define a minimum cycle time of 200 ms, the minimum distance between two cycle control points is 200 virtual milliseconds. The default setting is 100 ms.

³ Send clock of the I/O system (PROFIBUS or PROFINET) that is to be used for the cycle-controlled synchronization of the simulated PLC instance. You set the send clock of the respective I/O system in the STEP 7 properties, for example, in the properties of the PROFINET interface of the CPU (Advanced options > Real time settings > I/O communication > Send clock).

⁴ During the execution of these operating modes, the input parameter of the first callback event returns the value 0 for `TimeSinceSameSyncPoint_ns` / `TimeSinceAnySyncPoint_ns`. Subsequent callback events for `TimeSinceSameSyncPoint_ns` / `TimeSinceAnySyncPoint_ns` then return values other than 0.

API functions / events

- `GetOverwrittenMinimalCycleTime_ns()` (Page 174)
- `SetOverwrittenMinimalCycleTime_ns()` (Page 174)
- `OverwrittenMinimalCycleTime_ns { get; set; }` (Page 174)
- `RunToNextSyncPoint()` (Page 220)
- `OnSyncPointReached` (Page 220)
- `EventCallback_II_SREC_ST_UINT32_UINT32_EPET_UINT32` (Page 246) /
`Delegate_II_EREC_DT_UINT32_INT64_UINT32` (Page 251)

Conditions for the `SingleStep_Bus` operating mode:

The following conditions must be met for the `SingleStep_Bus` operating mode:

- You must have configured outputs on the controller.
- To synchronize the process image of the outputs, you must call the `SYNC_PO` instruction in the isochronous mode interrupt OB.

NOTE

In addition to the send clock of the bus, the cycle control point is also used as a synchronization point. In the cycle control point, the transferred ID has the value `0xFFFF`. The `in_parameters` "in_TimeSinceSameSyncPoint_ns" and "in_TimeSinceAnySyncPoint_ns" have the value 0.

A synchronization of OBs and process image partitions is only possible if the OBs call the instructions "SYNC_PI" and "SYNC_PO". To use process image partition 1 (PIP 1) for an F-runtime group, configure the "Pre/Post processing of the F-runtime group" in the Safety Administration editor.

To configure the preprocessing and postprocessing of the F-runtime group you need new standard FCs for "SYNC_PI" and "SYNC_PO" with the following properties:

- Name of the FC: `SYNC_PI`
`#RET_VAL_PI := SYNC_PI(PART := "PIP 1", FLADDR => #FLADDR_PI);`
- Name of the FC: `SYNC_PO`
`#RET_VAL_PO := SYNC_PO(PART := "PIP 1", FLADDR => #FLADDR_PO);`

To synchronize the process image partition 1 (PIP 1), add the FCs in the Safety Administration editor under "Pre/Post processing of the F-runtime group".

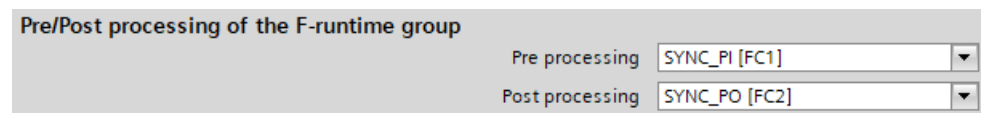


Figure 9-5 Safety Administration Editor

After compiling these changes, process image partition 1 (PIP 1) is also available for the fail-safe organization block (FOB).

Notes regarding the SingleStep_P, SingleStep_Bus, and SingleStep_CP operating modes

NOTE

Loss of a OnSyncPointReached event when downloading a clock-synchronous OB in RUN mode.

If you are registered for the OnSyncPointReached event in the SingleStep_P and SingleStep_Bus operating modes, the simulated PLC instance does not send this event to the API client when downloading a clock-synchronous OB.

Solution:

1. Use the OnSoftwareConfigurationChanged event to determine the start and end of a download
 2. Register and log off the OnSyncPointReached event, or handle the loss of the OnSyncPointReached event accordingly.
-

NOTE

Note on SingleStep_CP operating mode

Downloading software changes in RUN mode.

If you perform a software download in RUN mode in SingleStep_CP operating mode, the download may not be possible and the simulation may stop. An overload during the download in RUN may be the reason for this.

Solution:

1. Use the "Virtual Time Scaling" function and select a low scaling factor (e.g. scaling factor of 0.25).
 2. Provide sufficient resources to increase the runtime performance and the S7-PLCSIM Advanced communication performance.
-

Terminating the freeze state

The RunToNextSyncPoint() function cancels the freeze state and induces the simulated PLC instance to continue running until the next synchronization point.

Switching to the Default operating mode also terminates the freeze state.

Example

The figure schematically shows the sequence in the SingleStep_CP operating mode.

In addition to the OnSyncPointReached event the simulated PLC instance also sends the virtual time since the last synchronization point of the same process image partition ID or of any process image partition ID has been reached (TimeSinceSameSyncPoint_ns / TimeSinceAnySyncPoint_ns).

The `RunToNextSyncPoint()` function cancels the freeze state.

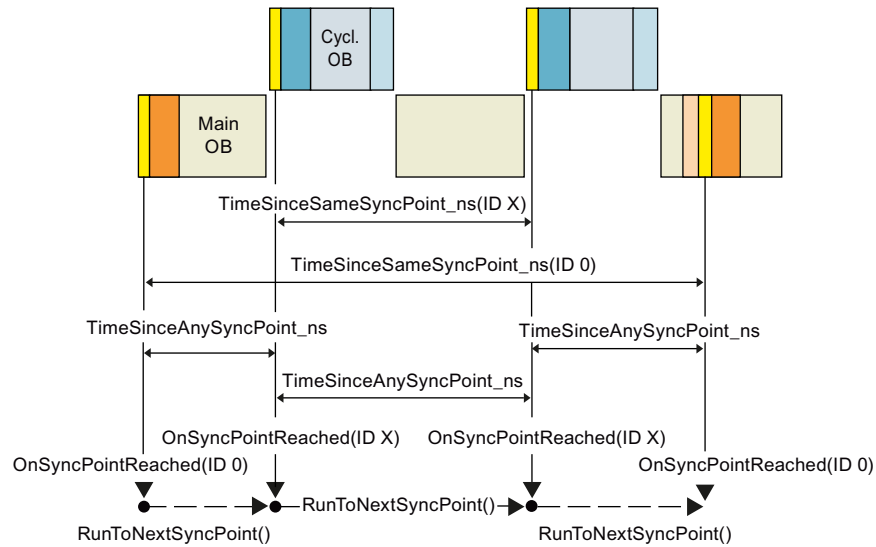


Figure 9-6 Example: Sequence in the SingleStep_CP operating mode

Changing the settings in the watch table

NOTE

Selecting triggers for monitoring of tags in the SingleStep operating modes

In TIA Portal the watch table in basic mode shows the values for outputs and bit memories **before** the processing.

To display the tag values **after** processing, proceed as follows:

1. Select the extended mode for the watch table.
2. In the "Monitor with trigger" column, select "End of cycle, permanent".

Synchronize simulation partner time-controlled

TimespanSynchronized operating modes

Several simulation partners (clients) are synchronized time-controlled with the TimespanSynchronized operating modes of the simulated PLC instance. The operating modes define the synchronization point at which the instance changes to the freeze state and sends the OnSyncPointReached event.

Table 9-216 Time-controlled operating modes (TimespanSynchronized)

Operating mode	Synchronization point	
	Cycle control point	Before reading in the process image partition
	"C"	"P"
TimespanSynchronized_C	✓	
TimespanSynchronized_CP	✓	✓
TimespanSynchronized_P		✓

API functions / events

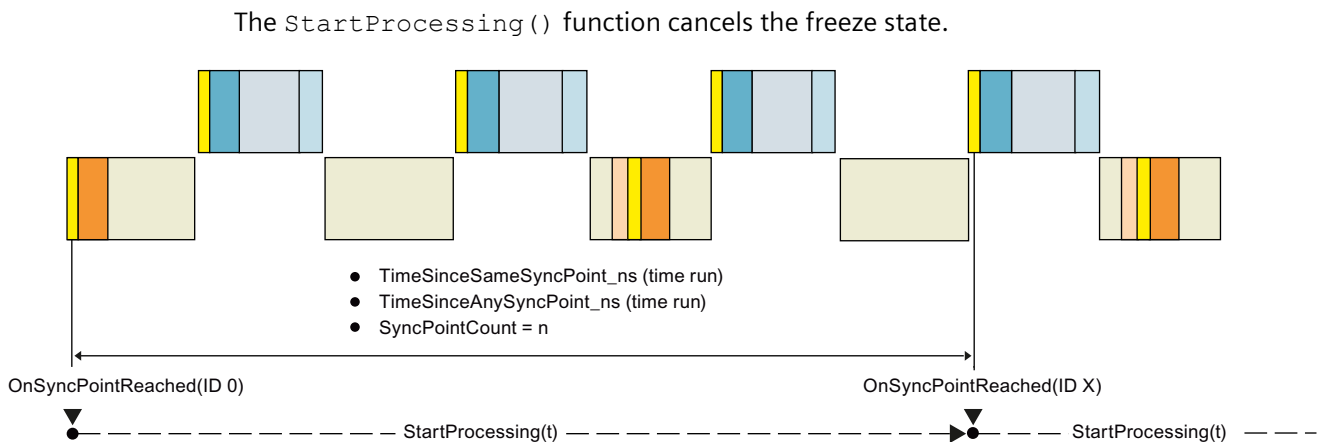
- Settings for cycle control (Page 174)
- StartProcessing() (Page 174)
- OnSyncPointReached (Page 220)

Terminating the freeze state

The StartProcessing(t) function cancels the freeze state and induces the simulated PLC instance to continue running at least as long as required (on the basis of the virtual time) before it changes back to the freeze state at the next synchronization point.
Switching to the Default operating mode also terminates the freeze state.

Example

The figure schematically shows the sequence in the TimespanSynchronized_CP operating mode.
In addition to the OnSyncPointReached event the simulated PLC instance also sends the runtime since the last call of the StartProcessing(t) (TimeSinceSameSyncPoint_ns / TimeSinceAnySyncPoint_ns) function.



Description

9.12.1.2 OnOperatingStateChanged events

OnOperatingStateChanged

Registers or unregisters an event handler method.

Table 9-217 OnOperatingStateChanged - .NET (C#)

Syntax	<code>event Delegate_II_EREC_DT_EOS_EOS OnOperatingStateChanged;</code>
Parameters	None. See <code>Delegate_II_EREC_DT_EOS_EOS</code> (Page 250).
Return values	None
Exceptions	None
Note	The event handler method runs in a separate thread.

RegisterOnOperatingStateChangedCallback()

When the event occurs, the registered callback function is called. Multiple callback functions can be registered for the event.

Table 9-218 RegisterOnOperatingStateChangedCallback() - Native C++

Syntax	<code>void RegisterOnOperatingStateChangedCallback(EventCallback_II_SREC_ST_SROS_S- ROS in_CallbackFunction);</code>
Parameters	<code>EventCallback_II_SREC_ST_SROS_SROS in_CallbackFunction:</code> A callback function that subscribes to the event. See <code>EventCallback_II_SREC_ST_SROS_SROS</code> (Page 242).
Return values	None
Note	The callback function runs in a separate thread.

RegisterOnOperatingStateChangedEvent()

When the event occurs, the registered event object is set to the signaled state. Only one event object can be registered for the event. Registering a new event object causes the previous event object to be deleted.

Table 9-219 RegisterOnOperatingStateChangedEvent() - Native C++

Syntax	<pre>void RegisterOnOperatingStateChangedEvent(); void RegisterOnOperatingStateChangedEvent(HANDLE* in_Event);</pre>
Parameters	- None: An internal event object is registered. - HANDLE* in_Event: A handle for a user-specific event object. The event object is registered.
Return values	None
Example C++	<pre>// Thread 1 ----- ISimulationRuntimeManager * api = NULL; ERuntimeErrorCode result = InitializeApi(&api); IInstance* psa = NULL; if (result == SREC_OK) { result = api->RegisterInstance(&psa); } // Register the internal event object psa->RegisterOnOperatingStateChangedEvent(); // Thread 2 ----- while (condition) { // Wait for the event to be set (timeout after 10s) bool isEventSet = psa->WaitForOnOperatingStateChangedEvent(10000); if (isEventSet) { // Do Something ... } }</pre>
Example C++	<pre>// Thread 1 ----- ISimulationRuntimeManager * api = NULL; ERuntimeErrorCode result = InitializeApi(&api); IInstance* psa = NULL; if (result == SREC_OK) { result = api->RegisterInstance(&psa); } // Create an event object HANDLE eventHandle = CreateEvent(NULL, FALSE, FALSE, NULL); // Register the user created event object psa->RegisterOnOperatingStateChangedEvent(&eventHandle); // Do Something ... // Clean up the handle CloseHandle(eventHandle); // Thread 2 ----- while (condition) { // Wait for the event to be set //OR: WaitForSingleObject(eventHandle, INFINITE); //psa- >WaitForOnOperatingStateChangedEvent(); }</pre>

9.12 Events for Instances

	<pre>// Do Something ... }</pre>
--	----------------------------------

Table 9-220 RegisterOnOperatingStateChangedEvent() – .NET (C#)

Syntax	<code>void RegisterOnOperatingStateChangedEvent ();</code>
Parameters	None
Return values	None

UnregisterOnOperatingStateChangedCallback()

Unregisters the callback function.

Table 9-221 UnregisterOnOperatingStateChangedCallback() - Native C++

Syntax	<pre>void UnregisterOnOperatingStateChangedCallback(EventCallback_II_SREC_ST_SROS- _SROS in_CallbackFunction = nullptr);</pre>
Parameters	Optional parameter of the same type as for the Register*() function If you use the parameter, it uninstalls the specific function; otherwise, the last registered callback function is unregistered.
Return values	None

UnregisterOnOperatingStateChangedEvent()

Unregisters the event object.

Table 9-222 UnregisterOnOperatingStateChangedEvent() - Native C++

Syntax	<code>void UnregisterOnOperatingStateChangedEvent ();</code>
Parameters	None
Return values	None

Table 9-223 UnregisterOnOperatingStateChangedEvent() - .NET (C#)

Syntax	<code>void UnregisterOnOperatingStateChangedEvent ();</code>
Parameters	None
Return values	None

WaitForOnOperatingStateChangedEvent()

The function blocks the program until the registered event object is in the signaled state or the timeout interval is exceeded.

Table 9-224 WaitForOnOperatingStateChangedEvent() - Native C++

Syntax	<pre>bool WaitForOnOperatingStateChangedEvent(); bool WaitForOnOperatingStateChangedEvent(UINT32 in_Time_ms);</pre>
Parameters	- None: The time limit is set to INFINITE. - UINT32 in_Time_ms: Value for the time limit in milliseconds.
Return values	- true: If the event object was set to the signaled state. - false: If no event was received during the defined time limit.

Table 9-225 WaitForOnOperatingStateChangedEvent() - .NET (C#)

Syntax	<pre>bool WaitForOnOperatingStateChangedEvent(); bool WaitForOnOperatingStateChangedEvent(UInt32 in_Time_ms);</pre>
Parameters	- None: The time limit is set to INFINITE. - UInt32 in_Time_ms: Value for the time limit in milliseconds.
Return values	- true: If the event object was set to the signaled state. - false: If no event was received during the defined time limit.

9.12.1.3 OnLedChanged events

OnLedChanged

Registers or unregisters an event handler method.

Table 9-226 OnLedChanged - .NET (C#)

Syntax	<pre>event Delegate_II_EREC_DT_ELT_ELM OnLedChanged;</pre>
Parameters	None. See Delegate_II_EREC_DT_ELT_ELM (Page 250).
Return values	None
Exceptions	None
Note	The event handler method runs in a separate thread.

RegisterOnLedChangedCallback()

When the event occurs, the registered callback function is called. Multiple callback functions can be registered for the event.

Table 9-227 RegisterOnLedChangedCallback() - Native C++

Syntax	<code>void RegisterOnLedChangedCallback(EventCallback_II_SREC_ST_SRLT_SRLM in_CallbackFunction);</code>
Parameters	EventCallback_II_SREC_ST_SRLT_SRLM in_CallbackFunction: A callback function that subscribes to an event. See EventCallback_II_SREC_ST_SRLT_SRLM (Page 244).
Return values	None
Note	The callback function runs in a separate thread.

RegisterOnLedChangedEvent()

When the event occurs, the registered event object is set to the signaled state. Only one event object can be registered for the event. Registering a new event object causes the previous event object to be deleted.

Table 9-228 RegisterOnLedChangedEvent() - Native C++

Syntax	<code>void RegisterOnLedChangedEvent();</code> <code>void RegisterOnLedChangedEvent(HANDLE* in_Event);</code>
Parameters	None: An internal event object is registered. HANDLE* in_Event: A handle for a user-specific event object. The event object is registered.
Return values	None

Table 9-229 RegisterOnLedChangedEvent() – .NET (C#)

Syntax	<code>void RegisterOnLedChangedEvent();</code>
Parameters	None
Return values	None

UnregisterOnLedChangedCallback()

Unregisters the callback function.

Table 9-230 UnregisterOnLedChangedCallback() - Native C++

Syntax	<code>void UnregisterOnLedChangedCallback();</code>
Parameters	Optional parameter of the same type as for the Register*() function If you use the parameter, it uninstalls the specific function; otherwise, the last registered callback function is unregistered.
Return values	None

UnregisterOnLedChangedEvent()

Unregisters the event object.

Table 9-231 UnregisterOnLedChangedEvent() - Native C++

Syntax	<code>void UnregisterOnLedChangedEvent();</code>
Parameters	None
Return values	None

Table 9-232 UnregisterOnLedChangedEvent() - .NET (C#)

Syntax	<code>void UnregisterOnLedChangedEvent();</code>
Parameters	None
Return values	None

WaitForOnLedChangedEvent()

The function blocks the program until the registered event object is in the signaled state or the timeout interval is exceeded.

Table 9-233 WaitForOnLedChangedEvent() - Native C++

Syntax	<code>bool WaitForOnLedChangedEvent();</code> <code>bool WaitForOnLedChangedEvent(UINT32 in_Time_ms);</code>
Parameters	- None: The time limit is set to INFINITE. - UINT32 in_Time_ms: Value for the time limit in milliseconds.
Return values	- true: If the event object was set to the signaled state. - false: If no event was received during the defined time limit.

Table 9-234 WaitForOnLedChangedEvent() - .NET (C#)

Syntax	<code>bool WaitForOnLedChangedEvent();</code> <code>bool WaitForOnLedChangedEvent(UInt32 in_Time_ms);</code>
Parameters	- None: The time limit is set to INFINITE. - UInt32 in_Time_ms: Value for the time limit in milliseconds.
Return values	- true: If the event object was set to the signaled state. - false: If no event was received during the defined time limit.

9.12.1.4 OnSoftwareConfigurationChanged events

OnSoftwareConfigurationChanged()

Reports software changes in STOP and RUN modes.

Table 9-235 OnSoftwareConfigurationChanged

Syntax	<code>event Delegate_SRSCC OnSoftwareConfigurationChanged();</code>
Parameters	Instance, ERuntimeErrorCode, SYSTEMTIME, ESoftwareConfigChanged
Return values	None
Exceptions	None
Note	The event handler method runs in a separate thread.

RegisterOnSoftwareConfigurationChangedCallback()

When the event occurs, the registered callback function is called. Only one callback function can be registered for the event. The registration of a new callback function causes the previous callback function to be deleted.

Table 9-236 RegisterOnSoftwareConfigurationChangedCallback() - Native C++

Syntax	<code>void RegisterOnSoftwareConfigurationChangedCallback();</code>
Parameters	CallbackFunction
Return values	None
Note	The event handler method runs in a separate thread.

RegisterOnSoftwareConfigurationChangedEvent()

When the event occurs, the registered event object is set to the signaled state. Only one event object can be registered for the event. Registration of a new event object causes the previous event object to be deleted.

Table 9-237 RegisterOnSoftwareConfigurationChangedEvent() - Native C++

Syntax	<code>void RegisterOnSoftwareConfigurationChangedEvent();</code>
Parameters	- None An internal event object is registered. - HANDLE* A handle for a user-specific event object. The event object is registered.
Return values	None

UnregisterOnSoftwareConfigurationChangedCallback()

Unregisters the callback function. When the event occurs, no callback function is called.

Table 9-238 UnregisterOnSoftwareConfigurationChangedCallback() - Native C++

Syntax	<code>void UnregisterOnSoftwareConfigurationChangedCallback();</code>
Parameters	None
Return values	None

UnregisterOnSoftwareConfigurationChangedEvent()

Unregisters the event object.

Table 9-239 UnregisterOnSoftwareConfigurationChangedEvent() - Native C++

Syntax	<code>void UnregisterOnSoftwareConfigurationChangedEvent();</code>
Parameters	None
Return values	None

Table 9-240 UnregisterOnSoftwareConfigurationChangedEvent() - .NET (C#)

Syntax	<code>void UnregisterOnSoftwareConfigurationChangedEvent();</code>
Parameters	None
Return values	None

WaitForOnSoftwareConfigurationChangedEvent()

The function blocks the program until the registered event object is set to the signaled state or the timeout interval is exceeded.

Table 9-241 WaitForOnSoftwareConfigurationChangedEvent() - Native C++

Syntax	<code>void WaitForOnSoftwareConfigurationChangedEvent();</code>
Parameters	- None: The time limit is set to INFINITE. - UINT32 in_Time_ms: Value for the time limit in milliseconds.
Return values	- true: If the event object was set to the signaled state. - false: If no event was received during the defined time limit.
Note	The function blocks the CoSimulation program until a software download is set to the signaled state or the timeout interval is exceeded.

9.12 Events for Instances

Table 9-242 WaitForOnSoftwareConfigurationChangedEvent() - .NET (C#)

Syntax	<code>void WaitForOnSoftwareConfigurationChangedEvent();</code>
Parameters	• None: The time limit is set to INFINITE. • UINT32 in_Time_ms: Value for the time limit in milliseconds.
Return values	• true: If the event object was set to the signaled state. • false: If no event was received during the defined time limit.
Note	The function blocks the CoSimulation program until a software download is set to the signaled state or the timeout interval is exceeded.

9.12.1.5 OnIPAddressChanged events

RegisterOnIPAddressChangedCallback()

Register function for receiving callback functions when an IP address has changed.

Table 9-243 RegisterOnIPAddressChangedCallback() - Native C++

Syntax	<code>RegisterOnIPAddressChangedCallback();</code>
Parameters <i>in_CallbackFunction</i>	• IInstance* in_Sender Instance object • ERuntimeErrorCode in_ErrorCode Result of operation • SYSTEMTIME in_SystemTime CPU system time of the event • UINT8 in_InterfaceId ID of the CPU interface involved • const SIPSuite4& in_SIP Reference of the SIPSuite4 structure
Return values	None
Note	The event handler method runs in a separate thread.

UnregisterOnIPAddressChangedCallback()

Unregister function for receiving callback functions when an IP address has changed.

Table 9-244 UnregisterOnIPAddressChangedCallback() - Native C++

Syntax	<code>void UnregisterOnIPAddressChangedCallback();</code>
Parameters <i>in_CallbackFunction</i>	Address of the specific function or nullptr for removing the last registered callback function
Return values	None

RegisterOnIPAddressChangedEvent()

Register function for receiving events when an IP address has changed.

Table 9-245 RegisterOnIPAddressChangedEvent() - Native C++

Syntax	void IInstance::RegisterOnIPAddressChangedEvent() void IInstance::RegisterOnIPAddressChangedEvent(HANDLE* in_Event);
Parameters in_Event	HANDLE* (a user-specific event object is registered)
Return values	None

UnregisterOnIPAddressChangedEvent()

Unregister function for receiving events when an IP address has changed.

Table 9-246 UnregisterOnIPAddressChangedEvent() - Native C++

Syntax	void IInstance::UnregisterOnIPAddressChangedEvent();
Parameters	None
Return values	None

OnIPAddressChanged()

The event is called when the IP configuration of the CPU interface has changed.

Table 9-247 OnIPAddressChanged() - .NET (C#)

Syntax	void IInstance::OnIPAddressChanged();
Parameters	- IInstance in_Sender Instance object - ERuntimeErrorCode in_ErrorCode Result of operation - DateTime in_SystemTime CPU system time of the event - UINT8 in_InterfaceId ID of the CPU interface involved - SIPSuite4 in_SIP IP address of the interface
Return values	None
Note	The event handler method runs in a separate thread.

WaitForOnIPAddressChangedEvent()

Blocking function up to the change of the IP address.

Table 9-248 WaitForOnIPAddressChangedEvent() - Native C++

Syntax	bool IInstance::WaitForOnIPAddressChangedEvent(); bool IInstance::WaitForOnIPAddressChangedEvent(UINT32 in_Time_ms)
Parameters in_Time_ms	Time in milliseconds
Return values	Returns True if the change was made within a set period, otherwise False
Note	None

9.12 Events for Instances

Table 9-249 WaitForOnIPAddressChangedEvent() - .NET (C#)

Syntax	<code>bool IInstance::WaitForOnIPAddressChangedEvent();</code> <code>bool IInstance::WaitForOnIPAddressChangedEvent(uint in_Time_ms);</code>
Parameters <code>in_Time_ms</code>	Time in milliseconds
Return values	Returns <code>True</code> if the change was made within a set period, otherwise <code>False</code>
Note	The event handler method runs in a separate thread.

9.12.1.6 OnSyncPointReached events

OnSyncPointReached

Registers or unregisters an event handler method.

Table 9-250 OnSyncPointReached - .NET (C#)

Syntax	<code>event Delegate_II_EREC_DT_UINT32_INT64_UINT32 OnSyncPointReached;</code>
Parameters	None. See <code>Delegate_II_EREC_DT_UINT32_INT64_UINT32</code> (Page 251).
Return values	None
Exceptions	None
Note	The event handler method runs in a separate thread.

RegisterOnSyncPointReachedCallback()

When the event occurs, the registered callback function is called. Multiple callback functions can be registered for the event.

Table 9-251 RegisterOnSyncPointReachedCallback() - Native C++

Syntax	<code>void</code> <code>RegisterOnSyncPointReachedCallback(EventCallback_II_SREC_ST_UINT32_INT64_</code> <code>4_INT64_UINT32 in_CallbackFunction);</code>
Parameters	<code>EventCallback_II_SREC_ST_UINT32_INT64_INT64_UINT32</code> <code>in_CallbackFunction:</code> A callback function that subscribes to an event. See <code>EventCallback_II_SREC_ST_UINT32_INT64_UINT32</code> (Page 243).
Return values	None
Note	The callback function runs in a separate thread.

RegisterOnSyncPointReachedEvent()

When the event occurs, the registered event object is set to the signaled state. Only one event object can be registered for the event. Registering a new event object causes the previous event object to be deleted.

Table 9-252 RegisterOnSyncPointReachedEvent() - Native C++

Syntax	<pre>void RegisterOnSyncPointReachedEvent(); void RegisterOnSyncPointReachedEvent(HANDLE* in_Event);</pre>
Parameters	- None: An internal event object is registered. - HANDLE* in_Event: A handle for a user-specific event object. The event object is registered.
Return values	None

Table 9-253 RegisterOnSyncPointReachedEvent() – .NET (C#)

Syntax	<pre>RegisterOnSyncPointReachedEvent() - .NET (C#)</pre>
Parameters	None
Return values	None

UnregisterOnSyncPointReachedCallback()

Unregisters the callback function.

Table 9-254 UnregisterOnSyncPointReachedCallback() - Native C++

Syntax	<pre>void UnregisterOnSyncPointReachedCallback();</pre>
Parameters	Optional parameter of the same type as for the Register*() function If you use the parameter, it uninstalls the specific function; otherwise, the last registered callback function is unregistered.
Return values	None

UnregisterOnSyncPointReachedEvent()

Unregisters the event object.

Table 9-255 UnregisterOnSyncPointReachedEvent() - Native C++

Syntax	<pre>void UnregisterOnSyncPointReachedEvent();</pre>
Parameters	None
Return values	None

Table 9-256 UnregisterOnSyncPointReachedEvent() - .NET (C#)

Syntax	<pre>void UnregisterOnSyncPointReachedEvent();</pre>
Parameters	None
Return values	None

WaitForOnSyncPointReachedEvent()

The function blocks the program until the registered event object is in the signaled state or the timeout interval is exceeded.

Table 9-257 WaitForOnSyncPointReachedEvent() - Native C++

Syntax	<pre>SOnSyncPointReachedResult WaitForOnSyncPointReachedEvent(); SOnSyncPointReachedResult WaitForOnSyncPointReachedEvent (UINT32 in_Time_ms);</pre>
Parameters	- None: The time limit is set to INFINITE. - UINT32 in_Time_ms: Value for the time limit in milliseconds.
Return values	SOnSyncPointReachedResult: A structure that supplies information about the event. See SOnSyncPointReachedResult (Page 268).

Table 9-258 WaitForOnSyncPointReachedEvent() - .NET (C#)

Syntax	<pre>SOnSyncPointReachedResult WaitForOnSyncPointReachedEvent(); SOnSyncPointReachedResult WaitForOnSyncPointReachedEvent (UInt32 in_Time_ms);</pre>
Parameters	- None: The time limit is set to INFINITE. - UInt32 in_Time_ms: Value for the time limit in milliseconds.
Return values	SOnSyncPointReachedResult: A structure that supplies information about the event. See SOnSyncPointReachedResult (Page 268).

9.12.1.7 OnHardwareConfigChanged events

OnHardwareConfigChanged

Registers or unregisters an event handler method.

Table 9-259 OnHardwareConfigChanged - .NET (C#)

Syntax	<pre>event Delegate_II_EREC_DT OnHardwareConfigChanged;</pre>
Parameters	None; see Delegate_II_EREC_DT
Return values	None
Exceptions	None
Note	The event handler method runs in a separate thread.

RegisterOnHardwareConfigChangedCallback()

When the event occurs, the registered callback function is called. Multiple callback functions can be registered for the event.

Table 9-260 Table: RegisterOnHardwareConfigChangedCallback() – Native C++

Syntax	<code>void RegisterOnHardwareConfigChangedCallback(EventCallback_II_SREC_ST in_CallbackFunction);</code>
Parameters	<code>EventCallback_II_SREC_ST in_CallbackFunction</code> A callback function that subscribes to an event; see <code>EventCallback_II_SREC_ST()</code>
Return values	None
Note	The callback function runs in a separate thread.

RegisterOnHardwareConfigChangedEvent()

When the event occurs, the registered event object is set to the signaled state. Only one event object can be registered for the event. Registration of a new event object causes the previous event object to be deleted.

Table 9-261 RegisterOnHardwareConfigChangedEvent() – Native C++

Syntax	<code>void RegisterOnHardwareConfigChangedEvent();</code> <code>void RegisterOnHardwareConfigChangedEvent(Handle* in_Event);</code>
Parameters	<code>None</code>: An internal event object is registered. <code>Handle* in_Event</code>: A handle for a user-specific event object. The event object is registered.
Return values	None

Table 9-262 RegisterOnHardwareConfigChangedEvent() – .NET (C#)

Syntax	<code>void RegisterOnHardwareConfigChangedEvent();</code>
Parameters	None
Return values	None

UnregisterOnHardwareConfigChangedCallback()

`UnregisterOnHardwareConfigChangedCallback()` Unregisters the callback function.

Table 9-263 UnregisterOnHardwareConfigChangedCallback() – Native C++

Syntax	<code>void UnregisterOnHardwareConfigChangedCallback();</code>
Parameters	Optional parameter of the same type as for the <code>Register*()</code> function If you use the parameter, it uninstalls the specific function; otherwise, the last registered callback function is unregistered.
Return values	None

UnregisterOnHardwareConfigChangedEvent()

Unregisters the event object.

Table 9-264 UnregisterOnHardwareConfigChangedEvent() – Native C++

Syntax	<code>void UnregisterOnHardwareConfigChangedEvent();</code>
Parameters	None
Return values	None

Table 9-265 UnregisterOnHardwareConfigChangedEvent() – .NET (C#)

Syntax	<code>void UnregisterOnHardwareConfigChangedEvent();</code>
Parameters	None
Return values	None

WaitForOnHardwareConfigChangedEvent()

The function blocks the program until the registered event object is in the signaled state or the timeout interval is exceeded.

Table 9-266 WaitForOnHardwareConfigChangedEvent() – Native C++

Syntax	<code>bool WaitForOnHardwareConfigChangedEvent();</code> <code>bool WaitForOnHardwareConfigChangedEvent (in_Time_ms);</code>
Parameters	- None: The time limit is set to INFINITE. - UINT32 in_Time_ms: Value for the time limit in milliseconds.
Return values	- true: If the event object was set to the signaled state. - false: If no event was received during the defined time limit.

Table 9-267 WaitForOnHardwareConfigChangedEvent() – .NET (C#)

Syntax	<code>bool WaitForOnHardwareConfigChangedEvent();</code> <code>bool WaitForOnHardwareConfigChangedEvent (UINT32 in_Time_ms);</code>
Parameters	- None: The time limit is set to INFINITE. - UINT32 in_Time_ms: Value for the time limit in milliseconds.
Return values	- true: If the event object was set to the signaled state. - false: If no event was received during the defined time limit.

9.12.2 Events for acyclic services

OnDataRecordRead

Registers or unregisters an event handler method.

Table 9-268 OnDataRecordRead - .NET (C#)

Syntax	<code>event Delegate_II_EREC_DT_SDRI OnDataRecordRead;</code>
Parameter	None. See Delegate_II_EREC_DT_SDRI (Page 252) .
Return values	None
Exceptions	None
Note	The Event-Handler Methode runs in a separate thread.

OnDataRecordWrite

Registers or unregisters an event handler method.

Table 9-269 OnDataRecordWrite - .NET (C#)

Syntax	<code>event Delegate_II_EREC_DT_SDR OnDataRecordWrite;</code>
Parameter	None. See Delegate_II_EREC_DT_SDR (Page 252) .
Return values	None
Exceptions	None
Note	The event handler method runs in a separate thread.

RegisterOnDataRecordReadCallback()

When the event occurs, the registered callback function is called. Multiple callback functions can be registered for the event.

Table 9-270 RegisterOnDataRecordReadCallback() - Native C++

Syntax	<code>void RegisterOnDataRecordReadCallback (Event Callback_II_SREC_ST_SDRI in_CallbackFunction);</code>
Parameters	EventCallback_II_SREC_ST_SDRI in_CallbackFunction: A callback function that subscribes to the event. See EventCallback_II_SREC_ST_SDRI (Page 244).
Return values	None
Note	The callback function runs in a separate thread.

UnregisterOnDataRecordReadCallback()

Unregisters the callback function.

Table 9-271 UnregisterOnDataRecordReadCallback() - Native C++

Syntax	<code>void UnregisterOnDataRecordReadCallback();</code>
Parameters	Optional parameter of the same type as for the Register*() function If you use the parameter, it uninstalls the specific function; otherwise, the last registered callback function is unregistered.
Return values	None

RegisterOnDataRecordWriteCallback()

When the event occurs, the registered callback function is called. Multiple callback functions can be registered for the event.

Table 9-272 RegisterOnDataRecordWriteCallback() - Native C++

Syntax	<code>void RegisterOnDataRecordWriteCallback (EventCallback_II_SREC_ST_SDRI_BYTE in_CallbackFunction);</code>
Parameters	EventCallback_II_SREC_ST_SDRI_BYTE in_CallbackFunction: A callback function that subscribes to the event. See EventCallback_II_SREC_ST_SDRI_BYTE (Page 245).
Return values	None
Note	The callback function runs in a separate thread.

UnregisterOnDataRecordWriteCallback()

Unregisters the callback function. When the event occurs, no callback function is called.

Table 9-273 UnregisterOnDataRecordWriteCallback() - Native C++

Syntax	<code>void UnregisterOnDataRecordWriteCallback();</code>
Parameters	Optional parameter of the same type as for the Register*() function If you use the parameter, it uninstalls the specific function; otherwise, the last registered callback function is unregistered.
Return values	None

9.12.2.1 OnAlarmNotification events

OnAlarmNotificationDone()

Registers or unregisters an event handler method.

Table 9-274 OnAlarmNotificationDone() - .NET (C#)

Syntax	<code>event Delegate_SREC_ST_UINT32_UINT32 OnAlarmNotificationDone;</code>
Parameters	None. See <code>Delegate_SREC_ST_UINT32_UINT32</code> (Page 254).
Return values	None
Exceptions	None
Note	The event handler method runs in a separate thread.

RegisterOnAlarmNotificationDoneCallback()

When the event occurs, the registered callback function is called. Multiple callback functions can be registered for the event.

Table 9-275 RegisterOnAlarmNotificationDoneCallback() - Native C++

Syntax	<code>void RegisterOnAlarmNotificationDoneCallback (Event Callback_II_SREC_ST_SDRI in_CallbackFunction);</code>
Parameters	<code>EventCallback_II_SREC_ST_UINT32_UINT32 in_CallbackFunction</code>: A callback function that subscribes to the event. See <code>EventCallback_II_SREC_ST_UINT32_UINT32</code> (Page 245).
Return values	None
Note	The callback function runs in a separate thread.

UnregisterOnAlarmNotificationDoneCallback()

Unregisters the callback function.

Table 9-276 UnregisterOnAlarmNotificationDoneCallback() - Native C++

Syntax	<code>void UnregisterOnAlarmNotificationDoneCallback ();</code>
Parameters	Optional parameter of the same type as for the <code>Register*()</code> function If you use the parameter, it uninstalls the specific function; otherwise, the last registered callback function is unregistered.
Return values	None

9.12.2.2 OnProcessEvent events

OnProcessEventDone()

Registers or unregisters an event handler method.

Table 9-277 OnProcessEventDone() - .NET (C#)

Syntax	<code>event Delegate_SREC_ST_UINT32_UINT32_EPET_UINT32 OnProcessEventDone;</code>
Parameters	None. See <code>Delegate_SREC_ST_UINT32_UINT32_EPET_UINT32</code> . (Page 253)
Return values	None
Exceptions	None
Note	The event handler method runs in a separate thread.

RegisterOnProcessEventDoneCallback()

When the event occurs, the registered callback function is called. Multiple callback functions can be registered for the event.

Table 9-278 RegisterOnProcessEventDoneCallback() - Native C++

Syntax	<code>void RegisterOnProcessEventDoneCallback (EventCallback_II_SREC_ST_UINT32_UINT32_EPET_UINT32 in_CallbackFunction);</code>
Parameters	<code>EventCallback_II_SREC_ST_UINT32_UINT32_EPET_UINT32 in_CallbackFunction:</code> A callback function that subscribes to the event. See <code>EventCallback_II_SREC_ST_UINT32_UINT32_EPET_UINT32</code> (Page 246)
Return values	None
Note	The callback function runs in a separate thread.

UnregisterOnProcessEventDoneCallback()

Unregisters the callback function.

Table 9-279 UnregisterOnProcessEventDoneCallback() - Native C++

Syntax	<code>void UnregisterOnProcessEventDoneCallback ();</code>
Parameters	Optional parameter of the same type as for the <code>Register*()</code> function If you use the parameter, it uninstalls the specific function; otherwise, the last registered callback function is unregistered.
Return values	None

9.12.2.3 OnPullOrPlugEvent events

OnPullOrPlugEventDone()

Registers or unregisters an event handler method.

Table 9-280 OnPullOrPlugEventDone() - .NET (C#)

Syntax	<code>event Delegate_SREC_ST_UINT32_EPPET_UINT32 OnPullOrPlugEventDone;</code>
Parameters	None. See Delegate_SREC_ST_UINT32_EPPET_UINT32 (Page 253).
Return values	None
Exceptions	None
Note	The event handler method runs in a separate thread.

RegisterOnPullOrPlugEventDoneCallback()

When the event occurs, the registered callback function is called. Multiple callback functions can be registered for the event.

Table 9-281 RegisterOnPullOrPlugEventDoneCallback() - Native C++

Syntax	<code>void RegisterOnPullOrPlugEventDoneCallback (EventCallback_II_SREC_ST_UINT32_EPPET_UINT32 in_CallbackFunction);</code>
Parameters	EventCallback_II_SREC_ST_UINT32_EPPET_UINT32 in_CallbackFunction: A callback function that subscribes to the event. See EventCallback_II_SREC_ST_UINT32_EPPET_UINT32 (Page 246).
Return values	None
Note	The callback function runs in a separate thread.

UnregisterOnPullOrPlugEventDoneCallback()

Unregisters the callback function.

Table 9-282 UnregisterOnPullOrPlugEventDoneCallback() - Native C++

Syntax	<code>void UnregisterOnPullOrPlugEventDoneCallback ();</code>
Parameters	Optional parameter of the same type as for the Register*() function If you use the parameter, it uninstalls the specific function; otherwise, the last registered callback function is unregistered.
Return values	None

9.12.2.4 OnStatusEvent events

OnStatusEventDone()

Registers or unregisters an event handler method.

Table 9-283 OnStatusEventDone() - .NET (C#)

Syntax	<code>event Delegate_SREC_ST_UINT32 OnStatusEventDone;</code>
Parameters	None. See Delegate_SREC_ST_UINT32 (Page 254).
Return values	None
Exceptions	None
Note	The event handler method runs in a separate thread.

RegisterOnStatusEventDoneCallback()

When the event occurs, the registered callback function is called. Multiple callback functions can be registered for the event.

Table 9-284 RegisterOnStatusEventDoneCallback() - Native C++

Syntax	<code>void RegisterOnStatusEventDoneCallback (EventCallback_II_SREC_ST_UINT32 in_CallbackFunction);</code>
Parameters	EventCallback_II_SREC_ST_UINT32 in_CallbackFunction: A callback function that subscribes to the event. See EventCallback_II_SREC_ST_UINT32 (Page 247).
Return values	None
Note	The callback function runs in a separate thread.

UnregisterOnStatusEventDoneCallback()

Unregisters the callback function.

Table 9-285 UnregisterOnStatusEventDoneCallback() - Native C++

Syntax	<code>void UnregisterOnStatusEventDoneCallback ();</code>
Parameters	Optional parameter of the same type as for the Register*()function. If you use the parameter, it uninstalls the specific function; otherwise, the last registered callback function is unregistered.
Return values	None

9.12.2.5 OnProfileEvent events

OnProfileEventDone()

Registers or unregisters an event handler method.

Table 9-286 OnProfileEventDone() - .NET (C#)

Syntax	<code>event Delegate_SREC_ST_UINT32 OnProfileEventDone;</code>
Parameters	None. See Delegate_SREC_ST_UINT32 (Page 254).
Return values	None
Exceptions	None
Note	The event handler method runs in a separate thread.

RegisterOnProfileEventDoneCallback()

When the event occurs, the registered callback function is called. Multiple callback functions can be registered for the event.

Table 9-287 RegisterOnProfileEventDoneCallback() - Native C++

Syntax	<code>void RegisterOnProfileEventDoneCallback (EventCallback_II_SREC_ST_UINT32 in_CallbackFunction);</code>
Parameters	EventCallback_II_SREC_ST_UINT32 in_CallbackFunction: A callback function that subscribes to the event. See EventCallback_II_SREC_ST_UINT32 (Page 247).
Return values	None
Note	The callback function runs in a separate thread.

UnregisterOnProfileEventDoneCallback()

Unregisters the callback function. When the event occurs, no callback function is called.

Table 9-288 UnregisterOnProfileEventDoneCallback() - Native C++

Syntax	<code>void UnregisterOnProfileEventDoneCallback ();</code>
Parameters	Optional parameter of the same type as for the Register*() function If you use the parameter, it uninstalls the specific function; otherwise, the last registered callback function is unregistered.
Return values	None

9.12.2.6 OnUpdateEvent events

OnUpdateEventDone()

Registers or unregisters an event handler method.

Table 9-289 OnUpdateEventDone() - .NET (C#)

Syntax	<code>event Delegate_SREC_ST_UINT32 OnUpdateEventDone;</code>
Parameters	None. See Delegate_SREC_ST_UINT32 (Page 254).
Return values	None
Exceptions	None
Note	The event handler method runs in a separate thread.

RegisterOnUpdateEventDoneCallback()

When the event occurs, the registered callback function is called. Multiple callback functions can be registered for the event.

Table 9-290 RegisterOnUpdateEventDoneCallback() - Native C++

Syntax	<code>void RegisterOnUpdateEventDoneCallback (EventCallback_II_SREC_ST_UINT32 in_CallbackFunction);</code>
Parameters	EventCallback_II_SREC_ST_UINT32 in_CallbackFunction: A callback function that subscribes to the event. See EventCallback_II_SREC_ST_UINT32 (Page 247).
Return values	None
Note	The callback function runs in a separate thread.

UnregisterOnUpdateEventDoneCallback()

Unregisters the callback function.

Table 9-291 UnregisterOnUpdateEventDoneCallback() - Native C++

Syntax	<code>void UnregisterOnUpdateEventDoneCallback ();</code>
Parameters	Optional parameter of the same type as for the Register*() function If you use the parameter, it uninstalls the specific function; otherwise, the last registered callback function is unregistered.
Return values	None

9.12.2.7 RackOrStationFault events

OnRackOrStationFaultEvent

Registers or unregisters an event handler method.

Table 9-292 OnRackOrStationFaultEvent - .NET (C#)

Syntax	<code>event Delegate_SREC_ST_UINT32_ERSFET OnRackOrStationFault;</code>
Parameter	None. See <code>Delegate_SREC_ST_UINT32_ERSFET</code> (Page 255).
Return values	None
Exceptions	None
Note	The event handler method runs in a separate thread.

RegisterOnRackOrStationFaultEventCallback()

When the event occurs, the registered callback function is called. Multiple callback functions can be registered for the event.

Table 9-293 RegisterOnRackOrStationFaultEventCallback() - Native C++

Syntax	<code>void RegisterOnRackOrStationFaultEventCallback (EventCallback_II_SREC_ST_UINT32_ERSFET in_CallbackFunction);</code>
Parameter	<code>EventCallback_II_SREC_ST_UINT32_ERSFET in_CallbackFunction</code> . A callback function that subscribes to the event. See <code>EventCallback_II_SREC_ST_UINT32_ERSFET</code> (Page 247)
Return values	None
Note	The callback function runs in a separate thread.

UnregisterOnRackOrStationFaultEventCallback()

Unregisters the callback function.

Table 9-294 UnregisterOnRackOrStationFaultEventCallback() - Native C++

Syntax	<code>void UnregisterOnRackOrStationFaultEventCallback ();</code>
Parameter	Optional parameter of the same type as for the <code>Register*()</code> function If you use the parameter, it uninstalls the specific function; otherwise, the last registered callback function is unregistered.
Return values	None

IRemoteRuntimeManager

10.1 Interfaces - Information and settings

Dispose()

Deletes the managed interface and unloads the native components of the user interfaces.

NOTE

When the interface of the Remote Runtime Manager is deleted, no `IInstance` interface which was generated by the `IRemoteRuntimeManager` interface can be used.

The .NET Garbage Collector clears its `IRemoteRuntimeManager` interface when no active references are present.

Table 10-1 Dispose() - .NET (C#)

Syntax	<code>void Dispose()</code>
Parameters	None
Return values	None

GetIP() / IP { get; }

Returns the IP address of the PC on which the remote Runtime Manager is running. If the function fails, the return value is 0.

Table 10-2 GetIP() - Native C++

Syntax	<code>UIP GetIP();</code>
Parameters	None
Return values	<code>UIP</code> : Returns the IP address of the PC on which the Runtime Manager is running.

Table 10-3 IP { get; } - .NET (C#)

Syntax	<code>SIP IP { get; }</code>
Parameters	None
Return values	<code>SIP</code> : Returns the IP address of the PC on which the Runtime Manager is running.

GetRemoteComputerName() / RemoteComputerName { get; }

This method returns the NetBIOS name of the PC on which the remote Runtime Manager is running.

When the name of the PC cannot be identified, the method returns the IP address.

Table 10-4 GetRemoteComputerName() - Native C++

Syntax	ERuntimeErrorCode GetRemoteComputerName (WCHAR* inout_Name, UINT32 in_ArrayLength);	
Parameters	- WCHAR* inout_Name: A user-allocated array for the computer name. - UINT32 in_ArrayLength: The array length. The array should be longer than MAX_COMPUTERNAME_LENGTH.	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The interface is disconnected from the remote Runtime Manager.
	SREC_INDEX_OUT_OF_RANGE	The array is too small to accommodate the computer name.

Table 10-5 RemoteComputerName { get; } - .NET (C#)

Syntax	string RemoteComputerName { get; }	
Parameters	None	
Return values	string: Name of the PC on which the remote Runtime Manager is running.	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The interface is disconnected from the remote Runtime Manager.
	ERuntimeErrorCode.IndexOutOfRangeException	The array is too small to accommodate the computer name.

Disconnect()

Closes the connection to the remote Runtime Manager.

NOTE

All applications that are connected to the remote Runtime Manager lose this connection.

Table 10-6 Disconnect() - Native C++

Syntax	ERuntimeErrorCode Disconnect();	
Parameters	None	
Return values	Runtime error code	Description
	SREC_OK	The function is successful.
	SREC_INTERFACE_REMOVED	The interface is disconnected from the remote Runtime Manager.
	SREC_TIMEOUT	The function does not return on time.

Table 10-7 Disconnect() - .NET (C#)

Syntax	void Disconnect();	
Parameters	None	
Return values	None	
Exceptions	Siemens.Simatic.Simulation.Runtime.SimulationRuntimeException	
	Runtime error code	Description
	ERuntimeErrorCode.InterfaceRemoved	The interface is disconnected from the remote Runtime Manager.
	ERuntimeErrorCode.Timeout	The function does not return on time.

GetSecurityMode()

GetSecurityMode returns the current ESecurityMode of the opened connection.

Table 10-8 GetSecurityMode - Native C++

Syntax	ESecurityMode IRemoteRuntimeManager::GetSecurityMode();	
Parameters	None	
Return values	Runtime error code	Description
	ESecurityMode::UNSECURED	No security is used.
	ESecurityMode::ENCRYPTED_COMMUNICATION	Secure communication
	ESecurityMode::UNKNOWN	No connection is opened.

Table 10-9 GetSecurityMode - .NET (C#)

Syntax	ESecurityMode RemoteRuntimeManager::SecurityMode { get; };	
Parameters	None	
Return values	Runtime error code	Description
	ESecurityMode::Unsecured	No security is used.
	ESecurityMode::EncryptedCommunication	Secure communication
	ESecurityMode::Unknown	No connection is opened.

10.2 Events for IRemoteRuntimeManager

10.2.1 OnConnectionLost events

Description

The event is triggered when the connection to the Remote Runtime Manager has been terminated.

OnConnectionLost

Registers or unregisters an event handler method.

Table 10-10 OnConnectionLost - .NET (C#)

Syntax	<code>event Delegate_IRRTM OnConnectionLost;</code>
Parameters	None. See Delegate_IRRTM (Page 251)
Return values	None
Exceptions	None
Note	The event handler method runs in a separate thread.

RegisterOnConnectionLostCallback()

When the event occurs, the registered callback function is called. Multiple callback functions can be registered for the event.

Table 10-11 RegisterOnConnectionLostCallback() - Native C++

Syntax	<code>void RegisterOnConnectionLostCallback(EventCallback_IRRTM in_CallbackFunction);</code>
Parameters	EventCallback_IRRTM in_CallbackFunction: A callback function that subscribes to an event. See EventCallback_IRRTM (Page 242).
Return values	None
Note	The callback function runs in a separate thread.

RegisterOnConnectionLostEvent()

When the event occurs, the registered event object is set to the signaled state. Only one event object can be registered for the event. Registration of a new event object causes the previous event object to be deleted.

Table 10-12 RegisterOnConnectionLostEvent() - Native C++

Syntax	<code>void RegisterOnConnectionLostEvent();</code> <code>void RegisterOnConnectionLostEvent(HANDLE* in_Event);</code>
Parameters	- None: An internal event object is registered. - HANDLE* in_Event: A handle for a user-specific event object. The event object is registered.
Return values	None

Table 10-13 RegisterOnConnectionLostEvent() - .NET (C#)

Syntax	<code>void RegisterOnConnectionLostEvent();</code>
Parameters	None
Return values	None

UnregisterOnConnectionLostCallback()

Unregisters the callback function. When the event occurs, no callback function is called.

Table 10-14 UnregisterOnConnectionLostCallback() - Native C++

Syntax	<code>void UnregisterOnConnectionLostCallback();</code>
Parameters	Optional parameter of the same type as for the Register*() function If you use the parameter, it uninstalls the specific function; otherwise, the last registered callback function is unregistered.
Return values	None

UnregisterOnConnectionLostEvent()

Unregisters the event object.

Table 10-15 UnregisterOnConnectionLostEvent() - Native C++

Syntax	<code>void UnregisterOnConnectionLostEvent();</code>
Parameters	None
Return values	None

Table 10-16 UnregisterOnConnectionLostEvent() - .NET (C#)

Syntax	<code>void UnregisterOnConnectionLostEvent();</code>
Parameters	None
Return values	None

WaitForOnConnectionLostEvent()

The function blocks the program until the registered event object is in the signaled state or the timeout interval is exceeded.

Table 10-17 WaitForOnConnectionLostEvent() - Native C++

Syntax	<code>bool WaitForOnConnectionLostEvent();</code> <code>bool WaitForOnConnectionLostEvent(UINT32 in_Time_ms);</code>
Parameters	- None: The time limit is set to INFINITE. - UINT32 in_Time_ms: Value for the time limit in milliseconds.
Return values	- true: If the event object was set to the signaled state. - false: If no event was received during the defined time limit.

Table 10-18 WaitForOnConnectionLostEvent() - .NET (C#)

Syntax	<code>bool WaitForOnConnectionLostEvent();</code> <code>bool WaitForOnConnectionLostEvent(UInt32 in_Time_ms);</code>
Parameters	- None: The time limit is set to INFINITE. - UInt32 in_Time_ms: Value for the time limit in milliseconds.
Return values	- true: If the event object was set to the signaled state. - false: If no event was received during the defined time limit.

Data types

Supported data types

The S7-PLCSIM Advanced Runtime API supports the data types of S7-1500 CPUs.

Converting data types

When writing, data types are not transferred BCD-coded but mapped onto primitive data types.

The data types Counter, Date and Time must be transferred to the API BCD-coded so that the values are written to the counter and no incorrect values are returned when reading.

For these data types, you must perform a BCD conversion before writing and a BCD back-conversion after reading.

Example:

If the value 999 is transferred to the API as 2457_H, then `Write` modifies the value 2457_H to 999. Without BCD conversion, there is no `UInt16` value and `Write` writes no value at all.

Additional information

For information on data types and conversion, refer to section "Data types" in the SIMATIC STEP 7 Basic/Professional (<https://support.industry.siemens.com/cs/ww/en/view/109755202>) System Manual.

11.1 DLL import functions (Native C++)

11.1.1 ApiEntry_Initialize

Description

Type of the central entry point for the API library (DLL).

Table 11-1 ApiEntry_Initialize - Native C++

Syntax	<pre>typedef ERuntimeErrorCode(*ApiEntry_Initialize)(ISimulationRuntimeManager** out_RuntimeManagerInterface);</pre>	
Parameters	<code>ISimulationRuntimeManager** out_SimulationRuntimeManagerInterface</code>: Pointer to a Runtime Manager interface pointer. The pointer must be initialized with NULL. The interface is created within the function. <code>UINT32 in_InterfaceVersion</code>: Version of the API interface to be downloaded: <code>API_DLL_INTERFACE_VERSION</code>.	
Return values	Runtime error code	Condition

11.2 Event callback functions (Native C++)

Return values	SREC_OK	The function is successful.
	SREC_WRONG_ARGUMENT	The pointer to the Runtime Manager interface is NULL.
	SREC_WRONG_VERSION	The version of the interface in use does not match the version of the API library (DLL).
	SREC_CONNECTION_ERROR	Unable to establish a connection to the Runtime Manager.
	SREC_CONFIG_FILE_ERROR	Operation regarding the configuration file "UserInterfaceConfiguration.xml" has failed, for example, create, read, write.

11.1.2 ApiEntry_DestroyInterface

Description

Type of the entry point for DestroyInterface [\(Page 44\)](#).

Table 11-2 ApiEntry_DestroyInterface - Native C++

Syntax	typedef ERuntimeErrorCode (*ApiEntry_DestroyInterface) (IBaseInterface* in_Interface);	
Parameters	IBaseInterface* in_Interface: The interface to be deleted.	
Return values	Runtime error code	Condition
	SREC_OK	The function is successful.
	SREC_WRONG_ARGUMENT	The pointer to the interface is NULL.

11.2 Event callback functions (Native C++)

11.2.1 EventCallback_VOID

Description

Table 11-3 EventCallback_VOID - Native C++

Syntax	typedef void (*EventCallback_VOID) ();
Parameters	None
Return values	None

11.2.2 EventCallback_SRCC_UINT32_UINT32_INT32

Description

Table 11-4 EventCallback_SRCC_UINT32_UINT32_INT32 - Native C++

Syntax	ERuntimeConfigChanged in_RuntimeConfigChanged, UINT32 in_Param1, UINT32 in_Param2, INT32 in_Param3);			
Parameters	ERuntimeConfigChanged in_RuntimeConfigChanged	UInt32 in_Param1	UInt32 in_Param2	Int32 in_Param3
	SRCC_INSTANCE_REGISTERED	-	-	ID of the registered instance
	SRCC_INSTANCE_UNREGISTERED	-	-	ID of the unregistered instance
	SRCC_CONNECTION_OPENED	IP of the remote Runtime Manager	Port of the remote Runtime Manager	-
	SRCC_CONNECTION_CLOSED	IP of the remote Runtime Manager	Port of the remote Runtime Manager	-
	SRCC_PORT_OPENED	The open port	-	-
	SRCC_PORT_CLOSED	-	-	-
Return values	None			

11.2.3 EventCallback_SRRSI_AD

Description

Table 11-5 EventCallback_SRRSI_AD - Native C++

Syntax	typedef void (*EventCallback_SRRSI_AD) (EAutodiscoverType in_AutodiscoverMsg, SAutodiscoverData in_AutodiscoverData);
Parameters	- in_AutodiscoverMsg: A value from the list of predefined types of events, see EAutodiscoverType (Page 294). - SRRSI_DISCOVER_STARTED, if the identification process was started by successfully calling the function RunAutodiscover(). - SRRSI_DISCOVER_DATA, if a Runtime Manager in the network was determined by the identification process. For detailed information about the found Runtime Manager, see parameter in_AutodiscoverData. - SRRSI_DISCOVER_FINISHED, if the identification process was completed after the time defined by the "in_Timeout" parameter had elapsed. - SRRSI_DISCOVER_STARTED and SRRSI_DISCOVER_FINISHED are always triggered, even if no data is received. - in_AutodiscoverData: Data from the Remote Runtime Manager. The parameter contains valid data only if in_AutodiscoverMsg = SRRSI_DISCOVER_DATA. Otherwise it is initialized with 0. See SAutodiscoverData (Page 273).
Return values	None

11.2.4 EventCallback_IRRTM

Description

Table 11-6 EventCallback_IRRTM - Native C++

Syntax	<code>typedef void (*EventCallback_IRRTM) (IRemoteRuntimeManager* in_Sender);</code>
Parameters	<code>IRemoteRuntimeManager* in_Sender</code>: An interface of the remote Runtime Manager that receives this event.
Return values	None

11.2.5 EventCallback_II_SREC_ST_SROS_SROS

Description

Table 11-7 EventCallback_II_SREC_ST_SROS_SROS - Native C++

Syntax	<code>typedef void (*EventCallback_II_SREC_ST_SROS_SROS) (IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, EOperatingState in_PrevState, EOperatingState in_OperatingState);</code>	
Parameters	<code>IInstance* in_Sender</code>: An interface of the instance that receives this event. <code>ERuntimeErrorCode in_ErrorCode</code>: A possible error code. <code>SYSTEMTIME in_SystemTime</code>: The virtual system time of the simulated PLC instance at the time when this event was triggered. <code>EOperatingState in_PrevState</code>: The operating state before the change. <code>EOperatingState in_OperatingState</code>: The current operating state.	
Return values	None	
Error codes	Runtime error code	Condition
	<code>SREC_OK</code>	The function is successful.
	<code>SREC_WARNING_TRIAL_MODE_ACTIVE</code>	No license available. You can use the instance without restrictions with the Trial License. Afterwards, the instance is shut down.
	<code>SREC_LICENSE_NOT_FOUND</code>	Test mode has expired.
	<code>SREC_COMMUNICATION_INTERFACE_NOT_AVAILABLE</code>	A problem has occurred with the selected communication interface. Check your settings.

11.2.6 EventCallback_II_SREC_ST_UINT32_INT64_INT64_UINT32

Description

Table 11-8 EventCallback_II_SREC_ST_UINT32_INT64_INT64_UINT32 - Native C++

Syntax	<pre>typedef void (*EventCallback_II_SREC_ST_UINT32_INT64_INT64_UINT32) (IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, UINT32 in_Id, INT64 in_TimeSinceSameSyncPoint_ns, INT64 in_TimeSinceAnySyncPoint_ns, UINT32 in_SyncPointCount);</pre>
Parameters	IInstance* in_Sender: An interface of the instance that receives this event. ERuntimeErrorCode in_ErrorCode: A possible error code. SYSTEMTIME in_SystemTime: The virtual system time of the simulated PLC instance at the time when this event was triggered. UINT32 in_Id: For the SingleStep_Bus operating mode: The ID of the I/O system that is used for cycle-controlled synchronization of the simulated PLC instance. For all other operating modes: The ID of the process image partition (PIP) that triggers this event. 0 for the cycle control point (End of cycle). INT64 in_TimeSinceSameSyncPoint_ns: The virtual time (in nanoseconds) since the last synchronization point of the same process image partition ID was reached. For the time-controlled operating modes: The runtime since the last call of the StartProcessing() function. INT64 in_TimeSinceAnySyncPoint_ns: The virtual time (in nanoseconds) since the last synchronization point of any process image partition ID was reached. For the time-controlled operating modes: The runtime since the last call of the StartProcessing() function. UINT32 in_SyncPointCount: The number of synchronization points since the last event. If the events are triggered faster than they are received, multiple events are combined into one event. In this case, this value contains the number of cycles since the last event was received.
Return values	None

11.2.7 EventCallback_II_SREC_ST

Description

Table 11-9 EventCallback_II_SREC_ST - Native C++

Syntax	<pre>typedef void (*EventCallback_II_SREC_ST) (IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime);</pre>
Parameters	IInstance* in_Sender: An interface of the instance that receives this event. ERuntimeErrorCode in_ErrorCode: A possible error code. SYSTEMTIME in_SystemTime: The virtual system time of the simulated PLC instance at the time when this event was triggered.
Return values	None

11.2.8 EventCallback_II_SREC_ST_SRLT_SRLM

Description

Table 11-10 EventCallback_II_SREC_ST_SRLT_SRLM - Native C++

Syntax	<pre>typedef void (*EventCallback_II_SREC_ST_SRLT_SRLM)(IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, ELEDType in_LEDType, ELEDMode in_LEDMode);</pre>
Parameters	<code>IInstance* in_Sender</code>: An interface of the instance that receives this event. <code>ERuntimeErrorCode in_ErrorCode</code>: A possible error code. <code>SYSTEMTIME in_SystemTime</code>: The virtual system time of the simulated PLC instance at the time when this event was triggered. <code>ELEDType in_LEDType</code>: The LED type that changed its state. <code>ELEDMode in_LEDMode</code>: The new state of the LED display.
Return values	None

11.2.9 EventCallback_II_SREC_ST_SDRI

Description

Table 11-11 EventCallback_II_SREC_ST_SDRI - Native C++

Syntax	<pre>typedef void (*EventCallback_II_SREC_ST_SDRI)(IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, SDataRecordInfo in_DataRecordInfo);</pre>
Parameters	<code>IInstance* in_Sender</code>: An interface of the instance that receives this event. <code>ERuntimeErrorCode in_ErrorCode</code>: A possible error code. <code>SYSTEMTIME in_SystemTime</code>: The virtual system time of the simulated PLC instance at the time when this event was triggered. <code>SDataRecordInfo in_DataRecordInfo</code>: The structure <code>SDataRecordInfo</code> contains the following information: - The HW identifier from which the CPU is to read the data record - The index of the collected data record - The maximum size of the data record which the IO device can transfer.
Return values	None

11.2.10 EventCallback_II_SREC_ST_SDRI_BYTE

Description

Table 11-12 EventCallback_II_SREC_ST_SDRI_BYTE - Native C++

Syntax	<pre>typedef void (*EventCallback_II_SREC_ST_SDRI_BYTE)(IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, SDataRecordInfo in_DataRecordInfo const BYTE* in_Data);</pre>
Parameters	IInstance* in_Sender: An interface of the instance that receives this event. ERuntimeErrorCode in_ErrorCode: A possible error code. SYSTEMTIME in_SystemTime: The virtual system time of the simulated PLC instance at the time when this event was triggered. SDataRecordInfo in_DataRecordInfo: The structure SDataRecordInfo contains the following information: – The HW identifier to which the CPU wants to write the data record – The index of the supplied data record – Size of data record const BYTE* in_Data: The data record. This pointer becomes invalid after the callback function has returned.
Return values	None

11.2.11 EventCallback_II_SREC_ST_UINT32_UINT32

Description

Table 11-13 EventCallback_II_SREC_ST_UINT32_UINT32 - Native C++

Syntax	<pre>typedef void (*EventCallback_II_SREC_ST_UINT32_UINT32)(IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, UINT32 in_HardwareIdentifier), UINT32 in_SequenceNumber);</pre>
Parameters	IInstance* in_Sender: An interface of the instance that receives this event. ERuntimeErrorCode in_ErrorCode: A possible error code. SYSTEMTIME in_SYSTEMTIME: The virtual system time of the simulated PLC instance at the time when this event was triggered. UINT32 in_HardwareIdentifier: The hardware identifier of the module or submodule which sends the diagnostics event. UINT32 in_SequenceNumber: S7-PLCSIM Advanced assigns a unique consecutive number to each interrupt event. According to PROFINET standard the sequence number is 10 bits wide (1 to 7FF_H). When the highest number is reached the numbering starts again at 1. Note In a real hardware system the IO controller uses the sequence number to check if it has lost a hardware interrupt. During the simulation, the sequence number creates the relation between interrupt request and the associated acyclic alarm.
Return values	None

11.2.12 EventCallback_II_SREC_ST_UINT32_UINT32_EPET_UINT32

Description

Table 11-14 EventCallback_II_SREC_ST_UINT32_UINT32_EPET_UINT32 - Native C++

Syntax	<pre>typedef void (*EventCallback_II_SREC_ST_UINT32_UINT32_EPET_UINT32) (IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, UINT32 in_HardwareIdentifier, UINT32 in_Channel, EProcessEventType in_ProcessEventType, UINT32 in_SequenceNumber);</pre>
Parameters	IInstance* in_Sender: An interface of the instance that receives this event. ERuntimeErrorCode in_ErrorCode: A possible error code. SYSTEMTIME in_SystemTime: The virtual system time of the simulated PLC instance at the time when this event was triggered. UINT32 in_HardwareIdentifier: The hardware identifier of the I/O module that sends the process event. UINT32 in_Channel: The channel of the I/O module which sends the process event. EProcessEventType in_ProcessEventType: A value from the list of predefined types of events for S7 modules, see EProcessEventType (Page 291). UINT32 in_SequenceNumber: S7-PLCSIM Advanced assigns a unique consecutive number to each interrupt event. According to PROFINET standard the sequence number is 10 bits wide (1 to 7FF_H). When the highest number is reached the numbering starts again at 1. Note In a real hardware system the I/O controller uses the sequence number to check if it has lost a hardware interrupt. During the simulation, the sequence number creates the relation between interrupt request and the associated acyclic alarm.
Return values	None

11.2.13 EventCallback_II_SREC_ST_UINT32_EPPET_UINT32

Description

Table 11-15 EventCallback_II_SREC_ST_UINT32_EPPET_UINT32 - Native C++

Syntax	<pre>typedef void (*EventCallback_II_SREC_ST_UINT32_EPPET_UINT32) (IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SYSTEMTIME, UINT32 in_HardwareIdentifier, EPullOrPlugEventType in_PullOrPlugEventType, UINT32 in_SequenceNumber);</pre>
Parameters	IInstance* in_Sender: An interface of the instance that receives this event. ERuntimeErrorCode in_ErrorCode: A possible error code. SYSTEMTIME in_SYSTEMTIME: The virtual system time of the simulated PLC instance at the time when this event was triggered. UINT32 in_HardwareIdentifier: The hardware identifier of the module or submodule which sends the pull/plug event. EPullOrPlugEventType in_PullOrPlugEventType: A value from the list of predefined types of events for S7 modules, see EPullOrPlugEventType (Page 291). UINT32 in_SequenceNumber: S7-PLCSIM Advanced assigns a unique consecutive number to each interrupt event.

	According to PROFINET standard the sequence number is 10 bits wide (1 to 7FF_H). When the highest number is reached the numbering starts again at 1. Note In a real hardware system the IO controller uses the sequence number to check if it has lost a hardware interrupt. During the simulation, the sequence number creates the relation between interrupt request and the associated acyclic alarm.
Return values	None

11.2.14 EventCallback_II_SREC_ST_UINT32_ERSFET

Description

Table 11-16 EventCallback_II_SREC_ST_UINT32_ERSFET - Native C++

Syntax	<pre>typedef void (*EventCallback_II_SREC_ST_UINT32_ERSFET)(IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, UINT32 in_HardwareIdentifier, ERackOrStationFaultType in_EventType);</pre>
Parameters	IInstance* in_Sender: An interface of the instance that receives this event. ERuntimeErrorCode in_ErrorCode: A possible error code. SYSTEMTIME in_SystemTime: The virtual system time of the simulated PLC instance at the time when this event was triggered. UINT32 in_HardwareIdentifier: The hardware identifier of the module or submodule which sends the diagnostics event. ERackOrStationFaultType in_EventType: A value from the list of predefined RackOrStationFault event types. See ERackOrStationFaultType (Page 293).
Return values	None

11.2.15 EventCallback_II_SREC_ST_UINT32

Description

Table 11-17 EventCallback_II_SREC_ST_UINT32 - Native C++

Syntax	<pre>typedef void (*EventCallback_II_SREC_ST_UINT32)(IInstance* in_Sender, ERuntimeErrorCode in_ErrorCode, SYSTEMTIME in_SystemTime, UINT32 in_HardwareIdentifier);</pre>
Parameters	IInstance* in_Sender: An interface of the instance that receives this event. ERuntimeErrorCode in_ErrorCode: A possible error code. SYSTEMTIME in_SystemTime: The virtual system time of the simulated PLC instance at the time when this event was triggered. UINT32 in_HardwareIdentifier: The hardware identifier of the module or submodule which generates the status, update or Profile event. The identifier must belong to a hardware component in the currently loaded project.
Return values	None

11.3 Delegate definitions (managed code)

11.3.1 Delegate_Void

Description

Table 11-18 Delegate_Void - .NET (C#)

Syntax	<code>delegate void Delegate_Void();</code>
Parameters	None
Return values	None

11.3.2 Delegate_DisplayDataChanged

Description

Table 11-19 Delegate_DisplayDataChanged - .NET C#

Syntax	<code>delegate void Delegate_DisplayDataChanged (IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, System::DateTime in_DateTime);</code>		
Parameters	<code>IInstance in_Sender</code>: An interface of the instance that receives this event. <code>ERuntimeErrorCode in_ErrorCode</code>: A possible error code. <code>DateTime in_DateTime</code>: The virtual system time of the virtual controller at the time when this event was triggered.		
Return values	None		

11.3.3 Delegate_SRCC_UINT32_UINT32_INT32

Description

Table 11-20 Delegate_SRCC_UINT32_UINT32_INT32 - .NET (C#)

Syntax	<code>delegate void Delegate_SRCC_UINT32_UINT32_INT32 (ERuntimeConfigChanged in_RuntimeConfigChanged, UInt32 in_Param1, UInt32 in_Param2, Int32 in_Param3);</code>			
Parameters	<code>ERuntimeConfigChanged in_RuntimeConfigChanged</code>	<code>UInt32 in_Param1</code>	<code>UInt32 in_Param2</code>	<code>Int32 in_Param3</code>
	<code>InstanceRegistered</code>	-	-	ID of the registered instance
	<code>InstanceUnregistered</code>	-	-	ID of the unregistered instance
	<code>ConnectionOpened</code>	IP of the Remote Runtime Manager	Port of the remote Runtime Manager	-
	<code>ConnectionClosed</code>	IP of the Remote Runtime Manager	Port of the remote Runtime Manager	-

11.3 Delegate definitions (managed code)

Parameters	PortOpened	The open port	-	-
	PortClosed	-	-	-
Return values	None			

11.3.4 Delegate_SRRSI_AD

Description

Table 11-21 Delegate_SRRSI_AD - .NET (C#)

Syntax	<code>delegate void Delegate_SRRSI_AD(EAutodiscoverType in_AutodiscoverType, SAutodiscoverData in_AutodiscoverData);</code>
Parameters	in_AutodiscoverType A value from the list of predefined types of events, see EAutodiscoverType (Page 294). - AutodiscoverStarted, if the identification process was started by successfully calling the function RunAutodiscover(). - AutodiscoverData, if a Runtime Manager in the network was determined by the identification process. For detailed information about the found Runtime Manager, see parameter in_AutodiscoverData. - AutodiscoverFinished, if the identification process is completed after the time defined by the parameter "in_Timeout" has elapsed. - AutodiscoverStarted and AutodiscoverFinished are always triggered, even if no data is received. in_AutodiscoverData Data from the Remote Runtime Manager. The parameter contains valid data only if in_AutodiscoverType = AutodiscoverData. Otherwise it is initialized with 0. See SAutodiscoverData (Page 273).
Return values	None

11.3.5 Delegate_II_EREC_DT

Description

Table 11-22 Delegate_II_EREC_DT - .NET (C#)

Syntax	<code>delegate void Delegate_II_EREC_DT (IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime);</code>
Parameters	IInstance in_Sender: An interface of the instance that receives this event. ERuntimeErrorCode in_ErrorCode: A possible error code. DateTime in_DateTime: The virtual system time of the simulated PLC instance at the time when this event was triggered.
Return values	None

11.3.6 Delegate_II_EREC_DT_EOS_EOS

Description

Table 11-23 Delegate_II_EREC_DT_EOS_EOS - .NET (C#)

Syntax	delegate void Delegate_II_EREC_DT_EOS_EOS(IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, EOperatingState in_PrevState, EOperatingState in_OperatingState);	
Parameters	- IInstance in_Sender: An interface of the instance that receives this event. - ERuntimeErrorCode in_ErrorCode: A possible error code. - DateTime in_DateTime: The virtual system time of the simulated PLC instance at the time when this event was triggered. - EOperatingState in_PrevState: The operating state before the change. - EOperatingState in_OperatingState: The current operating state.	
Return values	None	
Error codes	Runtime error code	Condition
	ERuntimeErrorCode.OK	The function is successful.
	ERuntimeErrorCode.WarningTrialMode-Active	No license available. You can use the instance without restrictions with the Trial License. Afterwards, the instance is shut down.
	ERuntimeErrorCode.LicenseNotFound	Test mode has expired.
	ERuntimeErrorCode.CommunicationInterfaceNotAvailable	A problem has occurred with the selected communication interface. Check your settings.

11.3.7 Delegate_II_EREC_DT_ELT_ELM

Description

Table 11-24 Delegate_II_EREC_DT_ELT_ELM - .NET (C#)

Syntax	delegate void Delegate_II_EREC_DT_ELT_ELM(IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, ELEDType in_LEDType, ELEDMode in_LEDMode);	
Parameters	- IInstance in_Sender: An interface of the instance that receives this event. - ERuntimeErrorCode in_ErrorCode: A possible error code. - DateTime in_DateTime: The virtual system time of the simulated PLC instance at the time when this event was triggered. - ELEDType in_LEDType: The LED type that changed its state. - ELEDMode in_LEDMode: The new state of the LED display.	
Return values	None	

11.3.8 Delegate_II_EREC_DT_UINT32_INT64_INT64_UINT32

Description

Table 11-25 Delegate_II_EREC_DT_UINT32_INT64_INT64_UINT32 - .NET (C#)

Syntax	<code>delegate void Delegate_II_EREC_DT_UINT32_INT64_INT64_UINT32 (IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, UInt32 in_Id, Int64 in_TimeSinceSameSyncPoint_ns, Int64 in_TimeSinceAnySyncPoint_ns, UInt32 in_SyncPointCount);</code>
Parameters	IInstance in_Sender: An interface of the instance that receives this event. ERuntimeErrorCode in_ErrorCode: A possible error code. DateTime in_DateTime: The virtual system time of the simulated PLC instance at the time when this event was triggered. UInt32 in_Id: For the SingleStep_Bus operating mode: The ID of the I/O system that is used for cycle-controlled synchronization of the simulated PLC instance. For all other operating modes: The ID of the process image partition (PIP) that triggers this event. 0 for the cycle control point (End of cycle). Note: In the SingleStep operating modes, the transferred ID "in_Id" for the cycle control point has the value 0xFFFF. In this case, the time values are set to 0. Int64 in_TimeSinceSameSyncPoint_ns: The virtual time (in nanoseconds) since the last synchronization point of the same process image partition ID was reached. Or the process time for the time-controlled operating modes. Int64 in_TimeSinceAnySyncPoint_ns: The virtual time (in nanoseconds) since the last synchronization point of any process image partition ID was reached. Or the process time for the time-controlled operating modes. UInt32 in_SyncPointCount: The number of synchronization points since the last event. If the events are triggered faster than they are received, multiple events are combined into one event. In this case, this value contains the number of cycles since the last event was received.
Return values	None

11.3.9 Delegate_IRRTM

Description

Table 11-26 Delegate_IRRTM - .NET (C#)

Syntax	<code>delegate void Delegate_IRRTM(IRemoteRuntimeManager in_Sender);</code>
Parameters	IRemoteRuntimeManager in_Sender: An interface of the remote Runtime Manager that receives this event.
Return values	None

11.3.10 Delegate_II_EREC_DT_SDRI

Description

Table 11-27 Delegate_II_EREC_DT_SDRI - .NET (C#)

Syntax	<code>delegate void Delegate_II_EREC_DT_SDRI (IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, SDataRecordInfo in_DataRecordInfo);</code>
Parameters	<code>IInstance in_Sender</code>: An interface of the instance that receives this event. <code>ERuntimeErrorCode in_ErrorCode</code>: A possible error code. <code>DateTime in_DateTime</code>: The virtual system time of the simulated PLC instance at the time when this event was triggered. <code>SDataRecordInfo in_DataRecordInfo</code>: The structure <code>SDataRecordInfo</code> contains the following information: - The HW identifier to which the CPU wants to write the data record - The index of the supplied data record - Size of data record - The data record
Return values	None

11.3.11 Delegate_II_EREC_DT_SDR

Description

Table 11-28 Delegate_II_EREC_DT_SDR - .NET (C#)

Syntax	<code>delegate void Delegate_II_EREC_DT_SDR (IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, SDataRecord in_DataRecord);</code>
Parameters	<code>IInstance in_Sender</code>: An interface of the instance that receives this event. <code>ERuntimeErrorCode in_ErrorCode</code>: A possible error code. <code>DateTime in_DateTime</code>: The virtual system time of the simulated PLC instance at the time when this event was triggered. <code>SDataRecord in_DataRecord</code>: The structure <code>SDataRecord</code> contains the following information: - The HW identifier to which the CPU writes the data record - The index of the supplied data record - Size of the data record - The data record
Return values	None

11.3.12 Delegate_SREC_ST_UINT32_EPET_UINT32

Description

Table 11-29 Delegate_SREC_ST_UINT32_EPET_UINT32 - .NET (C#)

Syntax	<pre>delegate void Delegate_SREC_ST_UINT32 (IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, UInt32 in_HardwareIdentifier, EPullOrPlugEventType in_PullOrPlugEventType, UInt32 in_SequenceNumber);</pre>
Parameters	IInstance in_Sender: An interface of the instance that receives this event. ERuntimeErrorCode in_ErrorCode: A possible error code. DateTime in_DateTime: The virtual system time of the simulated PLC instance at the time when this event was triggered. UInt32 in_HardwareIdentifier: The hardware identifier of the module or submodule that generated the pull/plug event. EPullOrPlugEventType in_PullOrPlugEventType: A value from the list of predefined types of events for S7 modules (Page 291). UInt32 in_SequenceNumber: S7-PLCSIM Advanced assigns a unique consecutive number to each interrupt event. According to the PROFINET standard, the sequence number is only 10 bits wide, from 1 to 0x7FF. When the highest number is reached the numbering starts again at 1. Note In a real hardware system the IO controller uses the sequence number to check if it has lost a hardware interrupt. During the simulation, the sequence number creates the relation between interrupt request and the associated acyclic alarm.
Return values	None

11.3.13 Delegate_SREC_ST_UINT32_UINT32_EPET_UINT32

Description

Table 11-30 Delegate_SREC_ST_UINT32_UINT32_EPET_UINT32 - Native C++

Syntax	<pre>delegate void Delegate_SREC_ST_UINT32_UINT32_EPET_UINT32(IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, UInt32 in_HardwareIdentifier, UInt32 in_Channel, EProcessEventType in_ProcessEventType, UInt32 in_SequenceNumber);</pre>
Parameters	IInstance in_Sender: An interface of the instance that receives this event. ERuntimeErrorCode in_ErrorCode: A possible error code. DateTime in_DateTime: The virtual system time of the simulated PLC instance at the time when this event was triggered. UInt32 in_HardwareIdentifier: The hardware identifier of the module or submodule that generates the process event. UInt32 in_Channel: The channel of the I/O module which sends the process event. EProcessEventType in_ProcessEventType: A value from the list of predefined types of events for S7 modules, see EProcessEventType (Page 291).

11.3 Delegate definitions (managed code)

	• UInt32 in SequenceNumber: S7-PLCSIM Advanced assigns a unique consecutive number to each interrupt event. According to PROFINET standard the sequence number is 10 bits wide (1 to 7FF_H). When the highest number is reached the numbering starts again at 1. Note In a real hardware system the I/O controller uses the sequence number to check if it has lost a hardware interrupt. During the simulation, the sequence number creates the relation between interrupt request and the associated acyclic alarm.
Return values	None

11.3.14 Delegate_SREC_ST_UINT32

Description

Table 11-31 Delegate_SREC_ST_UINT32 - .NET (C#)

Syntax	<code>delegate void Delegate_SREC_ST_UINT32 (IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, UInt32 in_HardwareIdentifier);</code>
Parameters	• IInstance in_Sender: An interface of the instance that receives this event. • ERuntimeErrorCode in_ErrorCode: A possible error code. • DateTime in_DateTime: The virtual system time of the simulated PLC instance at the time when this event was triggered. • UInt32 in_HardwareIdentifier: The ID of the module that generates the status event, update event, or profile event.
Return values	None

11.3.15 Delegate_SREC_ST_UINT32_UINT32

Description

Table 11-32 Delegate_SREC_ST_UINT32_UINT32 - .NET (C#)

Syntax	<code>delegate void Delegate_SREC_ST_UINT32 (IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, UInt32 in_HardwareIdentifier, UInt32 in_SequenceNumber);</code>
Parameters	• IInstance in_Sender: An interface of the instance that receives this event. • ERuntimeErrorCode in_ErrorCode: A possible error code. • DateTime in_DateTime: The virtual system time of the simulated PLC instance at the time when this event was triggered. • UInt32 in_HardwareIdentifier: The hardware identifier of the module or submodule which sends the diagnostics entry. • UInt32 in_SequenceNumber: S7-PLCSIM Advanced assigns a unique consecutive number to each interrupt event. According to PROFINET standard the sequence number is 10 bits wide (1 to 7FF_H). When the highest number is reached the numbering starts again at 1. Note In a real hardware system the IO controller uses the sequence number to check if it has lost a hardware interrupt.

	During the simulation, the sequence number creates the relation between interrupt request and the associated acyclic alarm.
Return values	None

11.3.16 Delegate_SREC_ST_UINT32_ERSFET

Description

Table 11-33 Delegate_SREC_ST_UINT32_ERSFET - .NET (C#)

Syntax	<code>delegate void Delegate_SREC_ST_UINT32_ERSFET(IInstance in_Sender, ERuntimeErrorCode in_ErrorCode, DateTime in_DateTime, UInt32 in_HardwareIdentifier, ERackOrStationFaultType in_EventType);</code>
Parameters	IInstance in_Sender: An interface of the instance that receives this event. ERuntimeErrorCode in_ErrorCode: A possible error code. DateTime in_DateTime: The virtual system time of the simulated PLC instance at the time when this event was triggered. UInt32 in_HardwareIdentifier: The hardware identifier of the module or submodule which sends the diagnostics entry. ERackOrStationFaultType in_EventType: A value from the list of predefined RackOrStationFault event types (Page 293).
Return values	None

11.4 Definitions and constants

The following identifiers are used in the API:

Table 11-34 Definitions - Native C++

Identifier	Value	Description
DINSTANCE_NAME_MAX_LENGTH	64	The unique name of an instance must be less than this value.
DSTORAGE_PATH_MAX_LENGTH	130	The maximum path length to the virtual memory card. Including NULL termination.
DTAG_NAME_MAX_LENGTH	300	The maximum length of the name of a PLC tag. Including NULL termination.
DTAG_ARRAY_DIMENSION	6	The maximum number of dimensions for a multi-dimensional field.
DCONTROLLER_NAME_MAX_LENGTH	128	The maximum length of the controller name. Including NULL termination.
DCONTROLLER_SHORT_DESIGNATION_MAX_LENGTH	32	The maximum length of the abbreviation of the controller (CPU type). Including NULL termination.
DALARM_NOTIFICATION_MAX_DIAG_EVENTS	100	The maximum number of diagnostic events that are sent in a diagnostic alarm.
DPROCESS_EVENT_NAME_MAX_LENGTH	123	The maximum length of the name for the process event. Including NULL termination.
DPROCESS_EVENTS_MAX_ITEMS	256	The maximum number of configurable process events.
DMODULE_STATE_OK	0	AlarmNotification: Module status OK

11.5 Unions (Native C++)

Identifier	Value	Description
DMODULE_STATE_ERROR	1	AlarmNotification: Module status faulty
DMODULE_STATE_MAINT_DEMANDED	2	AlarmNotification: Maintenance demanded
DMODULE_STATE_MAINT_REQUIRED	4	AlarmNotification: Maintenance required
MAX_SIZE_DISPLAY_DATA	153666	Maximum length of display bitmap data

Table 11-35 Constants - .NET (C#)

Identifier	Value	Description
RuntimeConstants.InstanceNameLength	64	The unique name of an instance must be less than this value.
RuntimeConstants.StoragePath-MaxLength	130	The maximum path length to the virtual memory card. Including NULL termination.
RuntimeConstants.TagNameMaxLength	300	The maximum length of the name of a PLC tag. Including NULL termination.
RuntimeConstants.TagArrayDimension	6	The maximum number of dimensions for a multi-dimensional field.
RuntimeConstants.Controller-NameMaxLength	128	The maximum length of the controller name. Including NULL termination.
RuntimeConstants.ControllerShortDesignationMaxLength	32	The maximum length of the abbreviation of the controller (CPU type). Including NULL termination.
ModuleState.Ok	0	AlarmNotification: Module status OK
ModuleState.Error	1	AlarmNotification: Module status faulty
ModuleState.MaintenanceDemanded	2	AlarmNotification: Maintenance demanded
ModuleState.MaintenanceRequired	4	AlarmNotification: Maintenance required
RuntimeConstants.MaxSizeDisplayData	153666	Maximum length of display bitmap data

11.5 Unions (Native C++)

11.5.1 UIP

Description

Contains an IPv4 address.

Table 11-36 UIP - Native C++

Syntax	<pre>union UIP { DWORD IP; BYTE IPs[4]; };</pre>
Member	- DWORD IP: The IP address in a single DWORD - BYTE IPs[4]: The four elements of IP in descending order
Example	Example for an IP address: 192.168.0.1 UIP.IP = 0xCOA80001 UIP.IPs[3] = 192, UIP.IPs[2] = 168, UIP.IPs[1] = 0, UIP.IPs[0] = 1

11.5.2 UDataValue

Description

Contains the value of a PLC tag.

Table 11-37 UDataValue - Native C++

Syntax	<pre> union UDataValue { bool Bool; INT8 Int8; INT16 Int16; INT32 Int32; INT64 Int64; UINT8 UInt8; UINT16 UInt16; UINT32 UInt32; UINT64 UInt64; float Float; double Double; CHAR Char; WCHAR WChar; }; </pre>
Member	• bool Bool: 1 byte boolean value • INT8 Int8: 1 byte integer with sign • INT16 Int16: 2 byte integer with sign • INT32 Int32: 4 byte integer with sign • INT64 Int64: 8 byte integer with sign • UINT8 UInt8: 1 byte integer without sign • UINT16 UInt16: 2 byte integer without sign • UINT32 UInt32: 4 byte integer without sign • UINT64 UInt64: 8 byte integer without sign • float Float: 4 byte floating-point value • double Double: 8 byte floating-point value • CHAR Char: 1 byte value character • WCHAR WChar: 2 byte value character

11.6 Structures

The following structures are available:

- SDataValue (Page 258)
- SDVBNI (Page 259)
- SDataValueByAddress (Page 260)
- SDataValueByAddressWithCheck (Page 261)
- SDataValueByName (Page 262)
- SDataValueByNameWithCheck (Page 263)
- SConnectionInfo (Page 263)
- SInstanceInfo (Page 264)
- SDimension (Page 265)
- STagInfo (Page 265)
- SIP (Page 267)
- SIPSuite4 (Page 267)
- SOnSyncPointReachedResult (Page 268)
- SDataRecordInfo (Page 270)
- SDataRecord (Page 270)
- SConfiguredProcessEvents (Page 271)
- SdiagExtChannelDescription (Page 272)
- SAutodiscoverData (Page 273)

11.6.1 SDataValue

Description

The structure contains the value and type of a PLC tag.

Table 11-38 SDataValue - Native C++

Syntax	<pre>struct SDataValue { UDataValue Value; EPrimitiveDataType Type; };</pre>
Member	• UDataValue Value: The value of the PLC tags • EPrimitiveDataType Type: Type of PLC tag

Table 11-39 SDataValue - .NET (C#)

Syntax	<pre> struct SDataValue { bool Bool { get; set; } Int8 Int8 { get; set; } Int16 Int16 { get; set; } Int32 Int32 { get; set; } Int64 Int64 { get; set; } UInt8 UInt8 { get; set; } UInt16 UInt16 { get; set; } UInt32 UInt32 { get; set; } UInt64 UInt64 { get; set; } float Float { get; set; } double Double { get; set; } sbyte Char { get; set; } char WChar { get; set; } EPrimitiveDataType Type { get; set; } } </pre>
Member	• bool Bool: 1 byte boolean value • Int8 Int8: 1 byte integer with sign • Int16 Int16: 2 byte integer with sign • Int32 Int32: 4 byte integer with sign • Int64 Int64: 8 byte integer with sign • UInt8 UInt8: 1 byte integer without sign • UInt16 UInt16: 2 byte integer without sign • UInt32 UInt32: 4 byte integer without sign • UInt64 UInt64: 8 byte integer without sign • float Float: 4 byte floating-point value • double Double: 8 byte floating-point value • sbyte Char: 1 byte value character • char WChar: 2 byte value character • EPrimitiveDataType Type: Type of PLC tag

11.6.2 SDVBNI

Description

This structure is for internal use only. Do not change this structure.

Table 11-40 SDVBNI - Native C++

Syntax	<pre> struct SDVBNI </pre>
--------	----------------------------

Table 11-41 SDVBNI - .NET (C#)

Syntax	<pre> struct SDVBNI </pre>
--------	----------------------------

11.6.3 SDataValueByAddress

Description

This structure represents a PLC tag that is accessed via its address.

Table 11-42 SDataValueByAddress - Native C++

Syntax	<pre>struct SDataValueByAddress { UINT32 Offset; UINT8 Bit; SDataValue DataValue; ERuntimeErrorCode ErrorCode; }</pre>
Member	• UINT32 Offset The byte offset of the tag if it is not located in a data block. • UINT8 Bit The bit offset of the tag if it is not located in a data block. • SDataValue DataValue The value of the PLC tags • ERuntimeErrorCode ErrorCode – SREC_TIMEOUT if no event was triggered during the defined time interval. – SREC_WARNING_INVALID_CALL if no function RegisterOnSyncPointReachedEvent was called before, see ERuntimeErrorCode.

Table 11-43 SDataValueByAddress - .NET (C#)

Syntax	<pre>struct SDataValueByAddress { UInt32 Offset; UInt8 Bit; SDataValue DataValue; ERuntimeErrorCode ErrorCode; }</pre>
Member	• UINT32 Offset The byte offset of the tag if it is not located in a data block. • UINT8 Bit The bit offset of the tag if it is not located in a data block. • SDataValue DataValue The value of the PLC tags • ERuntimeErrorCode ErrorCode – SREC_TIMEOUT if no event was triggered during the defined time interval. – SREC_WARNING_INVALID_CALL if no function RegisterOnSyncPointReachedEvent was called before, see ERuntimeErrorCode.

11.6.4 SDataValueByAddressWithCheck

Description

This structure represents a PLC tag that is accessed via its address.

Table 11-44 SDataValueByAddressWithCheck - Native C++

Syntax	<pre>struct SDataValueByAddressWithCheck { UINT32 Offset; UINT8 Bit; SDataValue DataValue; ERuntimeErrorCode ErrorCode; bool ValueHasChanged; }</pre>
Member	• <code>UINT32 Offset</code> The byte offset of the tag if it is not located in a data block. • <code>UINT8 Bit</code> The bit offset of the tag if it is not located in a data block. • <code>SDataValue DataValue</code> The value of the PLC tags • <code>ERuntimeErrorCode ErrorCode</code> – <code>SREC_TIMEOUT</code> if no event was triggered during the defined time interval. – <code>SREC_WARNING_INVALID_CALL</code> if no function <code>RegisterOnSyncPointReachedEvent</code> was called before, see <code>ERuntimeErrorCode</code>. • <code>bool ValueHasChanged</code>

Table 11-45 SDataValueByAddressWithCheck - .NET (C#)

Syntax	<pre>struct SDataValueByAddressWithCheck { UInt32 Offset; UInt8 Bit; SDataValue DataValue; ERuntimeErrorCode ErrorCode; bool ValueHasChanged; }</pre>
Member	• <code>UINT32 Offset</code> The byte offset of the tag if it is not located in a data block. • <code>UINT8 Bit</code> The bit offset of the tag if it is not located in a data block. • <code>SDataValue DataValue</code> The value of the PLC tags • <code>ERuntimeErrorCode ErrorCode</code> – <code>SREC_TIMEOUT</code> if no event was triggered during the defined time interval. – <code>SREC_WARNING_INVALID_CALL</code> if no function <code>RegisterOnSyncPointReachedEvent</code> was called before, see <code>ERuntimeErrorCode</code>. • <code>bool ValueHasChanged</code>

11.6.5 SDataValueByName

Description

This structure represents a PLC tag that is called by its name.

Table 11-46 SDataValueByName - Native C++

Syntax	<pre>struct SDataValueByName { WCHAR Name[DTAG_NAME_MAX_LENGTH]; SDataValue DataValue; ERuntimeErrorCode ErrorCode; SDVBNI Internal; }</pre>
Member	• WCHAR Name[DTAG_NAME_MAX_LENGTH] The name of the tag • SDataValue DataValue The value of the PLC tags • ERuntimeErrorCode ErrorCode – SREC_TIMEOUT if no event was triggered during the defined time interval. – SREC_WARNING_INVALID_CALL if no function RegisterOnSyncPointReachedEvent was called before, see ERuntimeErrorCode. • SDVBNI Internal

Table 11-47 SDataValueByName - .NET (C#)

Syntax	<pre>struct SDataValueByName { String Name; SDataValue DataValue; ERuntimeErrorCode ErrorCode; SDVBNI Internal; }</pre>
Member	• String Name The name of the tag • SDataValue DataValue The value of the PLC tags • ERuntimeErrorCode ErrorCode – SREC_TIMEOUT if no event was triggered during the defined time interval. – SREC_WARNING_INVALID_CALL if no function RegisterOnSyncPointReachedEvent was called before, see ERuntimeErrorCode. • SDVBNI Internal

11.6.6 SDataValueByNameWithCheck

Description

This structure represents a PLC tag that is called by its name.

Table 11-48 SDataValueByNameWithCheck - Native C++

Syntax	<pre>struct SDataValueByNameWithCheck { WCHAR Name[DTAG_NAME_MAX_LENGTH]; SDataValue DataValue; ERuntimeErrorCode ErrorCode; SDVBNI Internal; bool ValueHasChanged; };</pre>
--------	--

Table 11-49 SDataValueByNameWithCheck - .NET (C#)

Syntax	<pre>struct SDataValueByNameWithCheck { String Name; SDataValue DataValue; ERuntimeErrorCode ErrorCode; SDVBNI Internal; bool ValueHasChanged; }</pre>
--------	--

11.6.7 SConnectionInfo

Description

This structure contains the IP address and port of a remote runtime connection.

Table 11-50 SConnectionInfo - Native C++

Syntax	<pre>struct SConnectionInfo { UIP IP; UINT16 Port; };</pre>
--------	---

Table 11-51 SConnectionInfo - .NET (C#)

Syntax	<pre>struct SConnectionInfo { SIP IP; UInt16 Port; }</pre>
--------	--

11.6.8 SDisplayData

Description

Struct to manage the display data. The data stored in this struct constitutes the raw bitmap data.

Table 11-52 SDisplayData() - Native C++

Syntax	<pre>struct SDisplayData { uint8_t displayData[MAX_SIZE_DISPLAY_DATA]; uint32_t dataLength; };</pre>
Member	• <code>uint8_t displayData[MAX_SIZE_DISPLAY_DATA]</code>: Current bitmap data with a maximum length of <code>MAX_SIZE_DISPLAY_DATA</code> (=153666) • <code>uint32_t dataLength</code>: Tells the length of the data inside <code>displayData</code>

Table 11-53 SDisplayData() - .NET C#

Syntax	<pre>struct SDisplayData { byte[] displayData; UInt32 dataLength; };</pre>
Member	• <code>byte[] displayData</code>: Current bitmap data with a maximum length of <code>MaxSizeDisplayData</code> (=153666) • <code>UInt32 dataLength</code>: Tells the length of the data inside <code>displayData</code>

11.6.9 SInstanceInfo

Description

This structure contains an IPv4 address.

Table 11-54 SInstanceInfo - Native C++

Syntax	<pre>struct SInstanceInfo { INT32 ID; WCHAR Name[DINSTANCE_NAME_MAX_LENGTH]; };</pre>
Member	• <code>INT32 ID</code>: The ID of the instance • <code>WCHAR Name[DINSTANCE_NAME_MAX_LENGTH]</code>: The name of the instance

Table 11-55 SInstanceInfo - .NET (C#)

Syntax	<pre>struct SInstanceInfo { Int32 ID; String Name; }</pre>
Member	- Int32 ID: The ID of the instance - String name: The name of the instance

11.6.10 SDimension

Description

This structure contains information about the dimension of a field.

Table 11-56 SDimension - Native C++

Syntax	<pre>struct SDimension { INT32 StartIndex; UINT32 Count; };</pre>
--------	---

Table 11-57 SDimension - .NET (C#)

Syntax	<pre>struct SDimension { Int32 StartIndex; UInt32 Count; }</pre>
--------	--

11.6.11 STagInfo

Description

This structure contains information about a PLC tag.

Table 11-58 STagInfo - Native C++

Syntax	<pre>struct STagInfo { WCHAR Name[DTAG_NAME_MAX_LENGTH]; EArea Area; EDataType DataType; EPrimitiveDataType PrimitiveDataType; UINT16 Size; UINT32 Offset; UINT8 Bit; UINT8 DimensionCount; UINT32 Index; UINT32 ParentIndex; SDimension Dimension[DTAG_ARRAY_DIMENSION]; };</pre>
--------	--

Member	• <code>WCHAR Name[DTAG_NAME_MAX_LENGTH]</code> : The name of the tag • <code>EArea area</code> : The CPU area where the tag is located. • <code>EDataType DataType</code> : The CPU data type of the tag • <code>EPrimitiveDataType PrimitiveDataType</code> : The primitive data type of the tag • <code>UINT16 size</code> : The size of the tag in bytes • <code>UINT32 offset</code> : The byte offset of the tag if it is not located in a data block. • <code>UINT8 bit</code> : The bit offset of the tag if it is not located in a data block. • <code>UINT8 DimensionCount</code> : The number of dimensions of the array. 0 if the tag is not a field. • <code>UINT32 index</code> : The index of the tag • <code>UINT32 ParentIndex</code> : If this tag is embedded in another tag (for example, an element of a structure), this value then displays the index of the parent tag. The value is 0 if the tag has no parent tag. • <code>SDimension Dimension[DTAG_ARRAY_DIMENSION]</code> : Information about each dimension of the field
--------	--

Table 11-59 STagInfo - .NET (C#)

Syntax	<pre>public struct STagInfo { String Name; EArea Area; EDataType DataType; EPrimitiveDataType PrimitiveDataType; UInt16 Size; UInt32 Offset; UInt8 Bit; UInt32 Index; UInt32 ParentIndex; SDimension[] Dimension; }</pre>
Member	• <code>String name</code> : The name of the tag • <code>EArea area</code> : The CPU area where the tag is located. • <code>EDataType DataType</code> : The CPU data type of the tag • <code>EPrimitiveDataType PrimitiveDataType</code> : The primitive data type of the tag • <code>UInt16 size</code> : The size of the tag in bytes. • <code>UInt32 offset</code> : The byte offset of the tag if it is not located in a data block. • <code>UInt8 bit</code> : The bit offset of the tag if it is not located in a data block. • <code>UInt32 index</code> : The index of the tag • <code>UInt32 ParentIndex</code> : If this tag is embedded in another tag (for example, an element of a structure), this value then displays the index of the parent tag. The value is 0 if the tag has no parent tag. • <code>SDimension[] Dimension</code> : Information about each dimension of the field. Empty, if the tag is not an array.

11.6.12 SIP

Description

This structure contains an IPv4 address.

Table 11-60 SIP - .NET (C#)

Syntax	<pre>struct SIP { byte[] IPArray { get; set; } UInt32 IPDWord { get; set; } string IPString { get; set; } }</pre>
Member	• UInt32 IPDWord: The IP address in a single DWORD • byte[] IPArray: The four elements of IP in descending order • string IPString: The IPv4 address as a string
Example	Example for an IP address: 192.168.0.1 SIP.IPDWord = 0xCOA80001 SIP.IPArray[3] = 192, SIP.IPArray[2] = 168, SIP.IPArray[1] = 0, SIP.IPArray[0] = 1 SIP.IPString = "192.168.0.1"

11.6.13 SIPSuite4

Description

This structure contains an IPv4 suite.

Table 11-61 SIPSuite4 - Native C++

Syntax	<pre>struct SIPSuite4 { UIP IPAddress; UIP SubnetMask; UIP DefaultGateway; };</pre>
Member	• UIP IPAddress: The IP address • UIP SubnetMask: The subnet mask • UIP DefaultGateway: The standard gateway

Table 11-62 SIPSuite4 - .NET (C#)

Syntax	<pre>struct SIPSuite4 { SIP IPAddress; SIP SubnetMask; SIP DefaultGateway; }</pre>
Member	• SIP IPAddress: The IP address • SIP SubnetMask: The subnet mask • SIP DefaultGateway: The standard gateway

11.6.14 SOnSyncPointReachedResult

Description

This structure contains the results of the OnSyncPointReached event.

Table 11-63 SOnSyncPointReachedResult - Native C++

Syntax	<pre>struct SOnSyncPointReachedResult { ERuntimeErrorCode ErrorCode; SYSTEMTIME SystemTime; UINT32 Id; INT64 TimeSinceSameSyncPoint_ns; INT64 TimeSinceAnySyncPoint_ns; UINT32 SyncPointCount; };</pre>
Member	• ERuntimeErrorCode ErrorCode: – SREC_TIMEOUT if no event was triggered during the defined time interval. In such a case, the struct values are not used. – SREC_WARNING_INVALID_CALL, if no function RegisterOnSyncPointReachedEvent was called before. See ERuntimeErrorCode. • SYSTEMTIME SystemTime: The virtual system time of the simulated PLC instance at the time when this event was triggered. • UINT32 Id: For the SingleStep_Bus operating mode: The ID of the I/O system that is used for cycle-controlled synchronization of the simulated PLC instance. For all other operating modes: The ID of the process image partition (PIP) that triggers this event. 0 for the cycle control point (End of cycle). Note: In the SingleStep operating modes, the transferred ID for the cycle control point has the value 0xFFFF. In this case, the time values are set to 0. • INT64 TimeSinceSameSyncPoint_ns: The virtual time (in nanoseconds) since the last synchronization point of the same process image partition ID was reached. For the time-controlled operating modes: The runtime since the last call of the StartProcessing() function. • INT64 TimeSinceAnySyncPoint_ns: The virtual time (in nanoseconds) since the last synchronization point of any process image partition ID was reached. For the time-controlled operating modes: The runtime since the last call of the StartProcessing() function. • UINT32 SyncPointCount: The number of synchronization points since the last event. If the events are triggered faster than they are received, multiple events are combined into one event. In this case, this value contains the number of cycles since the last event was received.

Table 11-64 SOnSyncPointReachedResult - .NET (C#)

Syntax	<pre> struct SOnSyncPointReachedResult { ERuntimeErrorCode ErrorCode; DateTime SystemTime; UInt32 Id; Int64 TimeSinceSameSyncPoint_ns; Int64 TimeSinceAnySyncPoint_ns; UInt32 SyncPointCount; } </pre>
Member	• <code>ERuntimeErrorCode ErrorCode</code>: – <code>ERuntimeErrorCode.Timeout</code>, if no event was triggered during the defined time interval. – <code>WarningInvalidCall</code>, if no function <code>RegisterOnSyncPointReachedEvent</code> was called before. See <code>ERuntimeErrorCode</code>. • <code>DateTime DateTime</code>: The virtual system time of the simulated PLC instance at the time when this event was triggered. • <code>UInt32 Id</code>: For the <code>SingleStep_Bus</code> operating mode: The ID of the I/O system that is used for cycle-controlled synchronization of the simulated PLC instance. For all other operating modes: The ID of the process image partition (PIP) that triggers this event. 0 for the cycle control point (End of cycle). • <code>Int64 TimeSinceSameSyncPoint_ns</code>: The virtual time (in nanoseconds) since the last synchronization point of the same process image partition ID was reached. For the time-controlled operating modes: The runtime since the last call of the <code>StartProcessing()</code> function. • <code>Int64 TimeSinceAnySyncPoint_ns</code>: The virtual time (in nanoseconds) since the last synchronization point of any process image partition ID was reached. For the time-controlled operating modes: The runtime since the last call of the <code>StartProcessing()</code> function. • <code>UInt32 SyncPointCount</code>: The number of synchronization points since the last event. If the events are triggered faster than they are received, multiple events are combined into one event. In this case, this value contains the number of cycles since the last event was received.

11.6.15 SDataRecordInfo

Description

This structure contains read/write data record information.

Table 11-65 SDataRecordInfo - Native C++

Syntax	<pre>struct SDataRecordInfo { UINT32 HardwareId; UINT32 RecordIdx; UINT32 DataSize; };</pre>
Member	• UINT32 HardwareId: The ID of the hardware module (hardware identifier) • UINT32 RecordIdx: The data record number • UINT32 DataSize: The data record size

Table 11-66 SDataRecordInfo - .NET (C#)

Syntax	<pre>struct SDataRecordInfo { UInt32 HardwareId; UInt32 RecordIdx; UInt32 DataSize; }</pre>
Member	• UInt32 ID: The ID of the hardware module • UInt32 RecordIdx: The data record number • UInt32 DataSize: The data record size

11.6.16 SDataRecord

Description

This structure contains read/write data record information and data records.

Table 11-67 SDataRecord - .NET (C#)

Syntax	<pre>struct SDataRecord { UInt32 HardwareId; byte[] Data }</pre>
Member	• SDataRecordInfo Info: The data record information, see SDataRecordInfo (Page 270) • byte[] Data: The array length

11.6.17 SConfiguredProcessEvents

Description

This structure contains information about the configured process events.

Table 11-68 SConfiguredProcessEvents - Native C++

Syntax	<pre>struct SConfiguredProcessEvents { UINT16 HardwareIdentifier; UINT16 Channel; EProcessEventType ProcessEventType; WCHAR Name[DPROCESS_EVENT_NAME_MAX_LENGTH]; };</pre>
Member	• UINT16 HardwareIdentifier: The HW identifier • UINT16 Channel: The channel of the I/O module which generates the process event. • EProcessEventType ProcessEventType: The type of the configured process event • WCHAR Name[DPROCESS_EVENT_NAME_MAX_LENGTH]: The name of the process event

Table 11-69 SConfiguredProcessEvents - .NET (C#)

Syntax	<pre>public struct SConfiguredProcessEvents { ushort HardwareIdentifier; ushort Channel; EProcessEventType ProcessEventType; string Name; }</pre>
Member	• ushort HardwareIdentifier: The HW identifier • ushort Channel: The channel of the I/O module which generates the process event. • EProcessEventType ProcessEventType: The type of the configured process event • String name: The name of the process event

11.6.18 SDiagExtChannelDescription

Description

This structure contains read/write data record information and data records.

Table 11-70 SDiagExtChannelDescription - Native C++

Syntax	<pre>struct SDiagExtChannelDescription { UINT16 ChannelNumber; UINT16 ErrorType; UINT16 ExtErrorType; EDiagSeverity Severity; EDiagProperty Direction; };</pre>
Member	• UINT16 ChannelNumber: If the interrupt relates to a specific channel of the IO device (e.g. short circuit), this parameter must contain the number of the faulty channel. If the interrupt was generated by a module or submodule, the number of the channel must be set to 0x8000. • UINT16 ErrorType: The parameter defines error types according to PROFINET standard, see "Error types" section. • EDiagSeverity Severity: The value of the severity for the diagnostics, see EDiagSeverity (Page 293). • EDiagProperty Direction: The value for the incoming/outgoing information, see EDiagProperty (Page 292). • UINT16 ExtErrorType: This parameter provides the option of defining more details for the diagnostic interrupt. This is helpful in combination with PDEV error types which are generated for CPU-internal modules. Should be 0 by default.

Table 11-71 SDiagExtChannelDescription - .NET (C#)

Syntax	<pre>struct SDiagExtChannelDescription { UInt16 ChannelNumber; UInt16 ErrorType; UInt16 ExtErrorType; EDiagSeverity Severity; EDiagProperty Direction; };</pre>
Member	• UInt16 ChannelNumber: If the interrupt relates to a specific channel of the IO device (e.g. short circuit), this parameter must contain the number of the faulty channel. If the interrupt was generated by a module or submodule, the number of the channel must be set to 0x8000. • UInt16 ErrorType: The parameter defines error types according to PROFINET standard, see "Error types" section. • EDiagSeverity Severity: The value of the severity for the diagnostics, see EDiagSeverity (Page 293). • EDiagProperty Direction: The value for the incoming/outgoing information, see EDiagProperty (Page 292). • UInt16 ExtErrorType: This parameter provides the option of defining more details for the diagnostic interrupt. This is helpful in combination with PDEV error types which are generated for CPU-internal modules. Should be 0 by default.

Error types

The following table contains important error types (`ErrorType`) according to PROFINET standard:

Table 11-72 Error types according to PROFINET standard

Value	Meaning
0x0000	Reserved / unknown error
0x0001	Short-circuit
0x0002	Undervoltage
0x0003	Overvoltage
0x0004	Overload
0x0005	Overtemperature
0x0006	Wire break
0x0007	High limit violated
0x0008:	Low limit violated
0x0009	Error

The following table contains error types `ExtChannelErrorType` for `ChannelErrorType` "Remote mismatch":

Table 11-73 `ExtChannelErrType` error types

Value	Meaning	Use
0x0000	Reserved	-
0x0001 to 0x7FFF	Manufacturer ID	Interrupt/diagnostics
0x8000	Peer name of station mismatch	Interrupt/diagnostics
0x8001	Peer name of port mismatch	Interrupt/diagnostics
0x8002	Peer RT_CLASS_3 mismatch	Interrupt/diagnostics
0x8003	Peer MAU Type mismatch	Interrupt/diagnostics

11.6.19 SAutodiscoverData

Description

This structure contains the IP address, the port, the Runtime version, and the name of the computer that has a Runtime Manager ready to make a remote connection.

Table 11-74 `SAutodiscoverData` - Native C++

Syntax	<pre>public struct SAutodiscoverData { UIP IP; UINT16 Port; DWORD RuntimeVersion; WCHAR ComputerName[MAX_COMPUTERNAME_LENGTH + 1]; };</pre>
--------	---

Table 11-75 SAutodiscoverData - .NET (C#)

Syntax	<pre>public struct SAutodiscoverData { public SIP IP; public ushort Port; public uint RuntimeVersion; public string ComputerName; }</pre>
--------	---

11.6.20 SMAC address

Description

Defines the structure of a MAC address.

Table 11-76 SMACAddress - Native C++

Syntax	<pre>#define DMAC_ADDRESS_LEN 6 struct SMACAddress { BYTE Octets[DMAC_ADDRESS_LEN]; };</pre>
--------	---

11.6.21 SNetInterfaceInfo

Description

Describes the structure of a network interface.

Table 11-77 SNetInterfaceInfo - Native C++

Syntax	<pre>struct SNetInterfaceInfo { DWORD interfaceIndex; UMACAddress MACAddress; WCHAR interfaceName[64]; WCHAR interfaceDescription[64]; };</pre>
--------	---

11.7 Enumerations

The following enumerations are available:

- [ERuntimeErrorCode \(Page 275\)](#)
- [EArea \(Page 278\)](#)
- [EOperatingState \(Page 278\)](#)
- [EOperatingMode \(Page 279\)](#)
- [ECPUType \(Page 280\)](#)
- [ECommunicationInterface \(Page 282\)](#)
- [EDisplayKey \(Page 282\)](#)
- [ENetworkMode \(Page 283\)](#)

- [EPLCInterface \(Page 283\)](#)
- [ELEDType \(Page 283\)](#)
- [ELEDMode \(Page 284\)](#)
- [EPrimitiveDataType \(Page 285\)](#)
- [EDataType \(Page 287\)](#)
- [ETagListDetails \(Page 289\)](#)
- [ERuntimeConfigChanged \(Page 290\)](#)
- [EPullOrPlugEventType \(Page 291\)](#)
- [EProcessEventTypes \(Page 291\)](#)
- [ESoftwareConfigChanged \(Page 292\)](#)
- [EDirection \(Page 292\)](#)
- [EDiagProperty \(Page 292\)](#)
- [EDiagSeverity \(Page 293\)](#)
- [ERackOrStationFaultType \(Page 293\)](#)
- [ECycleTimeMonitoringMode \(Page 293\)](#)
- [EAutodiscoverType \(Page 294\)](#)
- [ESecurityMode \(Page 294\)](#)

11.7.1 ERuntimeErrorCode

Description

This enumeration contains all error codes that are used by the Simulation Runtime API. Most API functions return one of these error codes. If the function is successful, the return value is always `SREC_OK` / `OK`. Errors are returned with negative values, warnings with positive values.

Table 11-78 ERuntimeErrorCode - Native C++

Syntax	<pre>enum ERuntimeErrorCode { SREC_OK = 0, SREC_INVALID_ERROR_CODE = -1, SREC_NOT_IMPLEMENTED = -2, SREC_INDEX_OUT_OF_RANGE = -3, SREC_DOES_NOT_EXIST = -4, SREC_ALREADY_EXISTS = -5, SREC_UNKNOWN_MESSAGE_TYPE = -6, SREC_INVALID_MESSAGE_ID = -7, SREC_WRONG_ARGUMENT = -8, SREC_WRONG_PIPE = -9, SREC_CONNECTION_ERROR = -10, SREC_TIMEOUT = -11, SREC_MESSAGE_CORRUPT = -12, SREC_WRONG_VERSION = -13, SREC_INSTANCE_NOT_RUNNING = -14, SREC_INTERFACE_REMOVED = -15, SREC_SHARED_MEMORY_NOT_INITIALIZED = -16, SREC_API_NOT_INITIALIZED = -17, SREC_WARNING_ALREADY_EXISTS = 18, SREC_NOT_SUPPORTED = -19, }</pre>
--------	---

11.7 Enumerations

	<pre> SREC_WARNING_INVALID_CALL = 20, SREC_ERROR_LOADING_DLL = -21, SREC_SIGNAL_NAME DOES NOT EXIST = -22, SREC_SIGNAL_TYPE_MISMATCH = -23, SREC_SIGNAL_CONFIGURATION_ERROR = -24, SREC_NO_SIGNAL_CONFIGURATION_LOADED = -25, SREC_CONFIGURED_CONNECTION_NOT_FOUND = -26, SREC_CONFIGURED_DEVICE_NOT_FOUND = -27, SREC_INVALID_CONFIGURATION = -28, SREC_TYPE_MISMATCH = -29, SREC_LICENSE_NOT_FOUND = -30, SREC_NO_LICENSE_AVAILABLE = -31, SREC_WRONG_COMMUNICATION_INTERFACE = -32, SREC_LIMIT_REACHED = -33, SREC_NO_STORAGE_PATH_SET = -34, SREC_STORAGE_PATH_ALREADY_IN_USE = -35, SREC_MESSAGE_INCOMPLETE = -36, SREC_ARCHIVE_STORAGE_NOT_CREATED = -37, SREC_RETRIEVE_STORAGE_FAILURE = -38, SREC_INVALID_OPERATING_STATE = -39, SREC_INVALID_ARCHIVE_PATH = -40, SREC_DELETE_EXISTING_STORAGE_FAILED = -41, SREC_CREATE_DIRECTORIES_FAILED = -42, SREC_NOT_ENOUGH_MEMORY = -43, SREC_WARNING_TRIAL_MODE_ACTIVE = 44, SREC_NOT_RUNNING = -45, SREC_NOT_EMPTY = -46, SREC_NOT_UP_TO_DATE = -47, SREC_COMMUNICATION_INTERFACE_NOT_AVAILABLE = -48, SREC_WARNING_NOT_COMPLETE = 49, SREC_VIRTUAL_SWITCH_MISCONFIGURED = -50, SREC_RUNTIME_NOT_AVAILABLE = -51, SREC_IS_EMPTY = -52, SREC_WRONG_MODULE_STATE = -53, SREC_WRONG_MODULE_TYPE = -54, SREC_NOT_SUPPORTED_BY_MODULE = -55, SREC_INTERNAL_ERROR = -56, SREC_STORAGE_TRANSFER_ERROR = -57, SREC_ANOTHER_VARIANT_OF_PLCSIM_RUNNING = -58, SREC_ACCESS_DENIED = -59, SREC_NOT_ALLOWED_DURING_DOWNLOAD = -60, SREC_AUTODISCOVER_ALREADY_RUNNING = -61, SREC_INVALID_STORAGE = -62, SREC_WARNING_UNSUPPORTED_PCAP_DRIVER = 63, // (Deprecated since V4) SREC_WARNING_RUNNING_ON_TIA_PORTAL_TEST_SUITE = 64, SREC_PCAP_DRIVER_NOT_RUNNING = -65, SREC_UNSUPPORTED_PCAP_DRIVER = -66, SREC_CONFIG_FILE_ERROR = -67, SREC_WARNING_PASSWORD_PROTECTION_ERROR = 68, SREC_BUFFER_OVERFLOW = -69, SREC_NOT_ALLOWED = -70, SREC_WARNING_NO_EXTERNAL_COMMUNICATION = 71, SREC_INSTANCE_ALREADY_RUNNING = -72, SREC_OUT_OF_MEMORY = -73, SREC_WARNING_UNSECURED_COMMUNICATION = -74 }; </pre>
--	--

Table 11-79 ERuntimeErrorCode - .NET (C#)

Syntax	<pre> enum ERuntimeErrorCode { OK = 0, InvalidErrorCode = -1, NotImplemented = -2, IndexOutOfRange = -3, DoesNotExist = -4, AlreadyExists = -5, UnknownMessageType = -6, InvalidMessageId = -7, WrongArgument = -8, } </pre>
--------	--

```

WrongPipe = -9,
ConnectionError = -10,
Timeout = -11,
MessageCorrupt = -12,
WrongVersion = -13,
InstanceNotRunning = -14,
InterfaceRemoved = -15,
SharedMemoryNotInitialized = -16,
ApiNotInitialized = -17,
WarningAlreadyExists = 18,
NotSupported = -19,
WarningInvalidCall = 20,
ErrorLoadingDll = -21,
SignalNameDoesNotExist = -22,
SignalTypeMismatch = -23,
SignalConfigurationError = -24,
NoSignalConfigurationLoaded = -25,
ConfiguredConnectionNotFound = -26,
ConfiguredDeviceNotFound = -27,
InvalidConfiguration = -28,
TypeMismatch = -29,
LicenseNotFound = -30,
NoLicenseAvailable = -31,
WrongCommunicationInterface = -32,
LimitReached = -33,
NoStartupPathSet = -34,
StartupPathAlreadyInUse = -35,
MessageIncomplete = -36,
ArchiveStorageNotCreated = -37,
RetrieveStorageFailure = -38,
InvalidOperatingState = -39,
InvalidArchivePath = -40,
DeleteExistingStorageFailed = -41,
CreateDirectoriesFailed = -42,
NotEnoughMemory = -43,
WarningTrialModeActive = 44,
NotRunning = -45,
NotEmpty = -46,
NotUpToDate = -47,
CommunicationInterfaceNotAvailable = -48,
WarningNotComplete = 49,
VirtualSwitchMisconfigured = -50,
RuntimeNotAvailable = -51,
IsEmpty = -52,
WrongModuleState = -53,
WrongModuleType = -54,
NotSupportedByModule = -55,
InternalError = -56,
StorageTransferError = -57,
AnotherVariantOfPlcsimRunning = -58,
AccessDenied = -59,
NotAllowedDuringDownload = -60,
AutodiscoverAlreadyRunning = -61,
InvalidStorage = -62,
WarningUnsupportedPcapDriver = 63, //(Deprecated since V4)
WarningRunningOnTiaPortalTestSuite = 64,
PCAPDriverNotRunning = -65,
UnsupportedPcapDriver = -66,
ConfigFileError = -67,
WarningPasswordForProtectionError = 68,
BufferOverflow = - 69,
NotAllowed = -70,
WarningNoExternalCommunication = 71
InstanceAlreadyRunning = -72
OutOfMemory = -73
WarningUnsecuredCommunication = -74
}

```

11.7.2 EArea

Description

This enumeration contains all PLC areas that contain the available PLC tags.

Table 11-80 EArea - Native C++

Syntax	<pre>enum EArea { SRA_INVALID_AREA = 0, SRA_INPUT = 1, SRA_MARKER = 2, SRA_OUTPUT = 3, SRA_COUNTER = 4, SRA_TIMER = 5, SRA_DATABLOCK = 6, SRA_ENUMERATION_SIZE = 7 };</pre>
--------	---

Table 11-81 EArea - .NET (C#)

Syntax	<pre>public enum EArea { InvalidArea = 0, Input = 1, Marker = 2, Output = 3, Counter = 4, Timer = 5, DataBlock = 6, }</pre>
--------	---

11.7.3 EOperatingState

Description

This enumeration contains the operating states of a simulated PLC instance.

Table 11-82 EOperatingState - Native C++

Syntax	<pre>enum EOperatingState { SROS_INVALID_OPERATING_STATE = 0, SROS_OFF = 1, SROS_BOOTING = 2, SROS_STOP = 3, SROS_STARTUP = 4, SROS_RUN = 5, SROS_FREEZE = 6, SROS_SHUTTING_DOWN = 7, SROS_HOLD = 8, SROS_ENUMERATION_SIZE = 9 };</pre>
--------	--

Table 11-83 EOperatingState - .NET (C#)

Syntax	<pre>enum EOperatingState { InvalidOperatingState = 0, Off = 1, Booting = 2, Stop = 3, Startup = 4, Run = 5, Freeze = 6, ShuttingDown = 7, Hold = 8 }</pre>
--------	---

11.7.4 EOperatingMode

Description

This enumeration contains the operating modes of a simulated PLC instance.

Table 11-84 EOperatingMode - Native C++

Syntax	<pre>enum EOperatingMode { SROM_DEFAULT = 0, SROM_SINGLE_STEP_C = 1, SROM_SINGLE_STEP_CT = 2, SROM_TIMESPAN_SYNCHNRONIZED_C = 3, SROM_SINGLE_STEP_P = 4, SROM_TIMESPAN_SYNCHNRONIZED_P = 5, SROM_SINGLE_STEP_CP = 6, SROM_SINGLE_STEP_CPT = 7, SROM_TIMESPAN_SYNCHNRONIZED_CP = 8, SROM_SINGLE_STEP_BUS = 9 };</pre>
--------	--

Table 11-85 EOperatingMode - .NET (C#)

Syntax	<pre>public enum EOperatingMode { Default = 0, SingleStep_C = 1, SingleStep_CT = 2, TimespanSynchronized_C = 3, SingleStep_P = 4, TimespanSynchronized_P = 5, SingleStep_CP = 6, SingleStep_CPT = 7, TimespanSynchronized_CP = 8, SingleStep_Bus = 9 }</pre>
--------	--

11.7.5 ECPUType

Description

This enumeration contains all CPU types that can be loaded in a simulated PLC instance.

Table 11-86 ECPUType - Native C++

Syntax	<pre>enum ECPUType { SRCT_1500_Unspecified = 0x000005DC, SRCT_1511 = 0x000005E7, SRCT_1513 = 0x000005E9, SRCT_1515 = 0x000005EB, SRCT_1516 = 0x000005EC, SRCT_1517 = 0x000005ED, SRCT_1518 = 0x000005EE, SRCT_1511C = 0x000405E7, SRCT_1512C = 0x000405E8, SRCT_1511F = 0x000105E7, SRCT_1513F = 0x000105E9, SRCT_1515F = 0x000105EB, SRCT_1516F = 0x000105EC, SRCT_1517F = 0x000105ED, SRCT_1518F = 0x000105EE, SRCT_1511T = 0x000805E7, SRCT_1515T = 0x000805EB, SRCT_1516T = 0x000805EC, SRCT_1517T = 0x000805ED, SRCT_1511TF = 0x000905E7, SRCT_1515TF = 0x000905EB, SRCT_1516TF = 0x000905EC, SRCT_1517TF = 0x000905ED, SRCT_1518ODK = 0x001005EE, SRCT_1518FODK = 0x001105EE, SRCT_1518MFP = 0x004005EE, SRCT_1518FMFP = 0x004105EE, SRCT_1518T = 0x000805EE, SRCT_1518TF = 0x000905EE, SRCT_1504DTF = 0x000905E0, SRCT_1507DTF = 0x000905E3, SRCT_ET200SP_Unspecified = 0x000205DC, SRCT_1510SP = 0x000205E6, SRCT_1512SP = 0x000205E8, SRCT_1510SPF = 0x000305E6, SRCT_1512SPF = 0x000305E8, SRCT_1514SP = 0x000205E7, SRCT_1514SPT = 0x000205E9, SRCT_1514SPF = 0x000305E7, SRCT_1514SPTF = 0x000305E9, SRCT_1514PA = 0x000305EB, SRCT_1500_RH_Unspecified = 0x008005DC, SRCT_1513R = 0x008005E9, SRCT_1515R = 0x008005EB, SRCT_1517H = 0x008005ED, SRCT_1518HF = 0x008105EE, SRCT_ET200PRO_Unspecified = 0x002005DC, SRCT_1513PRO = 0x002005E9, SRCT_1513PROF = 0x002105E9, SRCT_1516PRO = 0x002005EC, SRCT_1516PROF = 0x002105EC, SRCT_1500_SW_OC_Unspecified = 0x010005DC, SRCT_1507S = 0x010005E3, SRCT_1507SF = 0x010105E3, SRCT_1508S = 0x010005E4, SRCT_1508SF = 0x010105E4, SRCT_1508ST = 0x010805E4, SRCT_1508STF = 0x010905E4, SRCT_1505SP = 0x010005E1, </pre>
--------	--

	<pre> SRCT_1505SPF = 0x010105E1, SRCT_1505SPT = 0x010805E1, SRCT_1505SPTF = 0x010905E1 } </pre>
--	---

Table 11-87 ECPUType - .NET (C#)

Syntax	<pre> enum ECPUType { CPU1500_Unspecified = 0x000005DC, CPU1511_ = 0x000005E7, CPU1513 = 0x000005E9, CPU1515 = 0x000005EB, CPU1516 = 0x000005EC, CPU1517 = 0x000005ED, CPU1518 = 0x000005EE, CPU1511C = 0x000405E7, CPU1512C = 0x000405E8, CPU1511F = 0x000105E7, CPU1513F = 0x000105E9, CPU1515F = 0x000105EB, CPU1516F = 0x000105EC, CPU1517F = 0x000105ED, CPU1518F = 0x000105EE, CPU1511T = 0x000805E7, CPU1515T = 0x000805EB, CPU1516T = 0x000805EC, CPU1517T = 0x000805ED, CPU1511TF = 0x000905E7, CPU1515TF = 0x000905EB, CPU1516TF = 0x000905EC, CPU1517TF = 0x000905ED, CPU1518ODK = 0x001005EE, CPU1518FODK = 0x001105EE, CPU1518MFP = 0x004005EE, CPU1518FMFP = 0x004105EE, CPU1518T = 0x000805EE, CPU1518TF = 0x000905EE, CPU1504DTF = 0x000905E0, CPU1507DTF = 0x000905E3, CPUET200SP_Unspecified = 0x000205DC, CPU1510SP = 0x000205E6, CPU1512SP = 0x000205E8, CPU1510SPF = 0x000305E6, CPU1512SPF = 0x000305E8, CPU1514SP = 0x000205E7, CPU1514SPT = 0x000205E9, CPU1514SPF = 0x000305E7, CPU1514SPTF = 0x000305E9, SRCT_1514PA = 0x000305EB, CPU1500_RH_Unspecified = 0x008005DC, CPU1513R = 0x008005E9, CPU1515R = 0x008005EB, CPU1517H = 0x008005ED, CPU1518HF = 0x008105EE, CPUET200PRO_Unspecified = 0x002005DC, CPU1513PRO = 0x002005E9, CPU1513PROF = 0x002105E9, CPU1516PRO = 0x002005EC, CPU1516PROF = 0x002105EC, SRCT_1500_SW_OC_Unspecified = 0x010005DC, SRCT_1507S = 0x010005E3, SRCT_1507SF = 0x010105E3, SRCT_1508S = 0x010005E4, SRCT_1508SF = 0x010105E4, SRCT_1508ST = 0x010805E4, SRCT_1508STF = 0x010905E4, SRCT_1505SP = 0x010005E1, SRCT_1505SPF = 0x010105E1, SRCT_1505SPT = 0x010805E1, </pre>
--------	--

11.7 Enumerations

	<pre>SRCT_1505SPTF = 0x010905E1 }</pre>
--	---

11.7.6 ECommunicationInterface

Description

This enumeration contains the available communication interfaces of a simulated controller.

Table 11-88 ECommunicationInterface - Native C++

Syntax	<pre>enum ECommunicationInterface { SRCI_NONE = 0, SRCI_SOFTBUS = 1, SRCI_TCPIP = 2, SRCI_ENUMERATION_SIZE = 3 };</pre>
--------	---

Table 11-89 ECommunicationInterface - .NET (C#)

Syntax	<pre>enum ECommunicationInterface { None = 0, Softbus = 1, TCPIP = 2, }</pre>
--------	---

11.7.7 EDisplayKey

Description

Enum containing the IDs of possible keys to be pressed on the display of an instance.

Table 11-90 EDisplayKey - Native C++

Syntax	<pre>enum EDisplayKey { SRFK_UP = 0, SRFK_DOWN = 1, SRFK_LEFT = 2, SRFK_RIGHT = 3, SRFK_OK = 4, SRFK_ESC = 5, SRFK_LONGESC = 6 };</pre>
--------	---

Table 11-91 EDisplayKey - .NET C#

Syntax	<pre>enum EDisplayKey { SRFK_UP = 0, SRFK_DOWN = 1, SRFK_LEFT = 2, SRFK_RIGHT = 3, SRFK_OK = 4, SRFK_ESC = 5, SRFK_LONGESC = 6 };</pre>
--------	---

11.7.8 ENetworkMode

Description

This enumeration contains the communication modes of a simulated PLC instance.

Table 11-92 ENetworkMode - Native C++

Syntax	<pre>enum class ENetworkMode { STATE_UNKNOWN = -1, TCPIP_MULTIPLE_ADAPTER = 0, TCPIP_SINGLE_ADAPTER = 1, SOFTBUS = 2 };</pre>
--------	---

Table 11-93 ENetworkMode - .NET (C#)

Syntax	<pre>public enum class ENetworkMode { UnknownState TCIPIMultipleAdapter TCIPISingleAdapter Softbus };</pre>
--------	---

11.7.9 EPLCInterface

Description

This enumeration includes the available interfaces of a simulated PLC instance.

Table 11-94 EPLCInterface - Native C++

Syntax	<pre>enum class EPLCInterface { IE1 = 0, IE2, IE3 };</pre>
--------	--

11.7.10 ELEDType

Description

This enumeration contains the types of LEDs for a simulated PLC instance.

Table 11-95 ELEDType - Native C++

Syntax	<pre>enum ELEDType { SRLT_STOP = 0, SRLT_RUN = 1, SRLT_ERROR = 2, SRLT_MAINT = 3, SRLT_REDUND = 4, SRLT_FORCE = 5, SRLT_BUSF1 = 6, SRLT_BUSF2 = 7, SRLT_BUSF3 = 8, SRLT_BUSF4 = 9, SRLT_ENUMERATION_SIZE = 10 };</pre>
--------	--

11.7 Enumerations

	};
--	----

Table 11-96 ELEDType - .NET (C#)

Syntax	<pre>enum ELEDType { Stop = 0, Run = 1, Error = 2, Maint = 3, Redund = 4, Force = 5, Busf1 = 6, Busf2 = 7, Busf3 = 8, Busf4 = 9 }</pre>
--------	---

11.7.11 ELEDMode

Description

This enumeration contains the LEDs of a simulated PLC instance.

Table 11-97 ELEDMode - Native C++

Syntax	<pre>enum ELEDMode { SRLM_OFF = 0, SRLM_ON = 1, SRLM_FLASH_FAST = 2, SRLM_FLASH_SLOW = 3, SRLM_INVALID = 4 };</pre>
--------	---

Table 11-98 ELEDMode - .NET (C#)

Syntax	<pre>enum ELEDMode { Off = 0, On = 1, FlashFast = 2, FlashSlow = 3, Invalid = 4 }</pre>
--------	---

11.7.12 EPrimitiveDataType

Description

This list contains all the primitive data types that are used by the I/O access functions.

Table 11-99 EPrimitiveDataType - Native C++

Syntax	<pre>enum EPrimitiveDataType { SRPDT_UNSPECIFIC = 0, SRPDT_STRUCT = 1, SRPDT_BOOL = 2, SRPDT_INT8 = 3, SRPDT_INT16 = 4, SRPDT_INT32 = 5, SRPDT_INT64 = 6, SRPDT_UINT8 = 7, SRPDT_UINT16 = 8, SRPDT_UINT32 = 9, SRPDT_UINT64 = 10, SRPDT_FLOAT = 11, SRPDT_DOUBLE = 12, SRPDT_CHAR = 13, SRPDT_WCHAR = 14 };</pre>
--------	---

Table 11-100 EPrimitiveDataType - .NET (C#)

Syntax	<pre>enum EPrimitiveDataType { Unspecific = 0, Struct = 1, Bool = 2, Int8 = 3, Int16 = 4, Int32 = 5, Int64 = 6, UInt8 = 7, UInt16 = 8, UInt32 = 9, UInt64 = 10, Float = 11, Double = 12, Char = 13, WChar = 14 };</pre>
--------	---

Compatible primitive data types

The following tables shows the primitive data types of the user interface (API) and the data types of the S7-PLCSIM Advanced instance that are configured in the stored tag list. The compatible data types are marked with "X".

Table 11-101 Compatible primitive data types - Reading

API	S7-PLCSIM Advanced instance												
	Bool	INT				UINT				Float	Double	Char	WChar
		8	16	32	64	8	16	32	64				
Bool	X												
INT8		X											
INT16		X	X			X							
INT32		X	X	X		X	X						
INT64		X	X	X	X	X	X	X					
UINT8						X							
UINT16						X	X						
UINT32						X	X	X					
UINT64						X	X	X	X				
Float										X			
Double											X		
Char												X	
WChar													X

Table 11-102 Compatible primitive data types - Write

API	S7-PLCSIM Advanced instance												
	Bool	INT				UINT				Float	Double	Char	WChar
		8	16	32	64	8	16	32	64				
Bool	X												
INT8		X	X	X	X								
INT16			X	X	X								
INT32				X	X								
INT64					X								
UINT8			X	X	X	X	X	X	X				
UINT16				X	X		X	X	X				
UINT32					X			X	X				
UINT64									X				
Float										X			
Double											X		
Char												X	
WChar													X

11.7.13 EDataType

Description

This enumeration contains all the PLC data types (STEP 7).

Table 11-103 EDataType - Native C++

Syntax	<pre>enum EDataType { SRDT_UNKNOWN = 0, SRDT_BOOL = 1, SRDT_BYTE = 2, SRDT_CHAR = 3, SRDT_WORD = 4, SRDT_INT = 5, SRDT_DWORD = 6, SRDT_DINT = 7, SRDT_REAL = 8, SRDT_DATE = 9, SRDT_TIME_OF_DAY = 10, SRDT_TIME = 11, SRDT_S5TIME = 12, SRDT_DATE_AND_TIME = 14, SRDT_STRUCT = 17, SRDT_STRING = 19, SRDT_COUNTER = 28, SRDT_TIMER = 29, SRDT_IEC_Counter = 30, SRDT_IEC_Timer = 31, SRDT_LREAL = 48, SRDT_ULINT = 49, SRDT_LINT = 50, SRDT_LWORD = 51, SRDT_USINT = 52, SRDT_UINT = 53, SRDT_UDINT = 54, SRDT_SINT = 55, SRDT_WCHAR = 61, SRDT_WSTRING = 62, SRDT_LTIME = 64, SRDT_LTIME_OF_DAY = 65, SRDT_LDT = 66, SRDT_DTL = 67, SRDT_IEC_LTimer = 68, SRDT_IEC_SCounter = 69, SRDT_IEC_DCounter = 70, SRDT_IEC_LCounter = 71, SRDT_IEC_UCounter = 72, SRDT_IEC_USCounter = 73, SRDT_IEC_UDCounter = 74, SRDT_IEC_ULCounter = 75, SRDT_ERROR_STRUCT = 97, SRDT_NREF = 98, SRDT_CREF = 101, SRDT_AOM_IDENT = 128, SRDT_EVENT_ANY = 129, SRDT_EVENT_ATT = 130, SRDT_EVENT_HWINT = 131, SRDT_HW_ANY = 144, SRDT_HW_IOSYSTEM = 145, SRDT_HW_DPMASTER = 146, SRDT_HW_DEVICE = 147, SRDT_HW_DPSLAVE = 148, SRDT_HW_IO = 149, SRDT_HW_MODULE = 150, SRDT_HW_SUBMODULE = 151, SRDT_HW_HSC = 152, SRDT_HW_PWM = 153, }</pre>
--------	---

11.7 Enumerations

	<pre> SRDT_HW_PTO = 154, SRDT_HW_INTERFACE = 155, SRDT_HW_IEPORT = 156, SRDT_OB_ANY = 160, SRDT_OB_DELAY = 161, SRDT_OB_TOD = 162, SRDT_OB_CYCLIC = 163, SRDT_OB_ATT = 164, SRDT_CONN_ANY = 168, SRDT_CONN_PRG = 169, SRDT_CONN_OUC = 170, SRDT_CONN_R_ID = 171, SRDT_PORT = 173, SRDT_RTM = 174, SRDT_PIP = 175, SRDT_OB_PCYLE = 192, SRDT_OB_HWINT = 193, SRDT_OB_DIAG = 195, SRDT_OB_TIMEERROR = 196, SRDT_OB_STARTUP = 197, SRDT_DB_ANY = 208, SRDT_DB_WWW = 209, SRDT_DB_DYN = 210, SRDT_DB = 257 }; </pre>
--	--

Table 11-104 EDataType - .NET (C#)

Syntax	<pre> public enum EDataType { Unknown = 0, Bool = 1, Byte = 2, Char = 3, Word = 4, Int = 5, DWord = 6, DInt = 7, Real = 8, Date = 9, TimeOfDay = 10, Time = 11, S5Time = 12, DateAndTime = 14, Struct = 17, String = 19, Counter = 28, Timer = 29, IEC_Counter = 30, IEC_Timer = 31, LReal = 48, ULInt = 49, LInt = 50, LWord = 51, USInt = 52, UInt = 53, UDInt = 54, SInt = 55, WChar = 61, WString = 62, LTime = 64, LTimeOfDay = 65, LDT = 66, DTL = 67, IEC_LTimer = 68, IEC_SCounter = 69, IEC_DCounter = 70, IEC_LCounter = 71, IEC_UCounter = 72, IEC_USCounter = 73, } </pre>
--------	--


```

IEC_UDCounte = 74,
IEC_ULCounter = 75,
ErrorStruct = 97,
NREF = 98,
CREF = 101,
Aom_Ident = 128,
Event_Any = 129,
Event_Att = 130,
Event_HwInt = 131,
Hw_Any = 144,
Hw_IoSystem = 145,
Hw_DpMaster = 146,
Hw_Device = 147,
Hw_DpSlave = 148,
Hw_Io = 149,
Hw_Module = 150,
Hw_SubModule = 151,
Hw_Hsc = 152,
Hw_Pwm = 153,
Hw_Pto = 154,
Hw_Interface = 155,
Hw_IEPort = 156,
OB_Any = 160,
OB_Delay = 161,
OB_Tod = 162,
OB_Cyclic = 163,
OB_Att = 164,
Conn_Any = 168,
Conn_Prg = 169,
Conn_Ouc = 170,
Conn_R_ID = 171,
Port = 173,
Rtm = 174,
Pip = 175,
OB_PCycle = 192,
OB_HwInt = 193,
OB_Diag = 195,
OB_TimeError = 196,
OB_Startup = 197,
DB_Any = 208,
DB_WWW = 209,
DB_Dyn = 210,
DB = 257
}

```

11.7.14 ETagListDetails

Description

This list contains all PLC areas that can be used as a filter to update the tag table.

Table 11-105 ETagListDetails - Native C++

Syntax	<pre> enum ETagListDetails { SRTLD_NONE = 0, SRTLD_IO = 1, SRTLD_M = 2, SRTLD_IOM = 3, SRTLD_CT = 4, SRTLD_IOCT = 5, SRTLD_MCT = 6, SRTLD_IOMCT = 7, SRTLD_DB = 8, SRTLD_IODB = 9, SRTLD_MDB = 10, SRTLD_IOMDB = 11, </pre>
--------	---

11.7 Enumerations

	<pre> SRTLD_CTDB = 12, SRTLD_IOCTDB = 13, SRTLD_MCTDB = 14, SRTLD_IOMCTDB = 15 }; </pre>
--	--

Table 11-106 ETagListDetails - .NET (C#)

Syntax	<pre> enum ETagListDetails { None = 0, IO = 1, M = 2, IOM = 3, CT = 4, IOCT = 5, MCT = 6, IOMCT = 7, DB = 8, IODB = 9, MDB = 10, IOMDB = 11, CTDB = 12, IOCTDB = 13, MCTDB = 14, IOMCTDB = 15 } </pre>
--------	--

11.7.15 ERuntimeConfigChanged

Description

This list contains all possible causes of an OnConfigurationChanged event that the Runtime Manager sends.

Table 11-107 ERuntimeConfigChanged - Native C++

Syntax	<pre> enum ERuntimeConfigChanged { SRCC_INSTANCE_REGISTERED = 0, SRCC_INSTANCE_UNREGISTERED = 1, SRCC_CONNECTION_OPENED = 2, SRCC_CONNECTION_CLOSED = 3, SRCC_PORT_OPENED = 4, SRCC_PORT_CLOSED = 5, SRCC_NETWORK_MODE_CHANGED = 6, SRCC_STRICT_MOTION_CHANGED = 7 }; </pre>
--------	--

Table 11-108 ERuntimeConfigChanged - .NET (C#)

Syntax	<pre> enum ERuntimeConfigChanged { InstanceRegistered = 0, InstanceUnregistered = 1, ConnectionOpened = 2, ConnectionClosed = 3, PortOpened = 4, PortClosed = 5, NetworkModeChanged = 6, StrictMotionChanged = 7 } </pre>
--------	---

11.7.16 EPullOrPlugEventType

Description

This enumeration contains predefined types of pull/plug events for S7 modules.

Table 11-109 EPullOrPlugEventType - Native C++

Syntax	<pre>enum EPullOrPlugEventType { SR_PPE_UNDEFINED = 0, SR_PPE_PULL_EVENT = 1, SR_PPE_PLUG_EVENT = 2, SR_PPE_PLUG_EVENT_ERROR_REMAINS = 3, SR_PPE_PLUG_WRONG_MODULE_EVENT = 4 };</pre>
--------	---

Table 11-110 EPullOrPlugEventType - .NET (C#)

Syntax	<pre>enum EPullOrPlugEventType { Undefined = 0, Pull = 1, Plug = 2, PlugErrorRemains = 3, PlugWrongModule = 4 };</pre>
--------	--

11.7.17 EProcessEventType

Description

This enumeration contains predefined types of process events for S7 modules.

Table 11-111 EProcessEventType - Native C++

Syntax	<pre>enum EProcessEventType { SR_PET_UNDEFINED = 0, SR_PET_RISING_EDGE = 1, SR_PET_FALLING_EDGE = 2, SR_PET_LIMIT1_UNDERRUN = 3, SR_PET_LIMIT1_OVERRUN = 4, SR_PET_LIMIT2_UNDERRUN = 5, SR_PET_LIMIT2_OVERRUN = 6 };</pre>
--------	--

Table 11-112 EProcessEventType - .NET (C#)

Syntax	<pre>enum EProcessEventType { Undefined = 0, RisingEdge = 1, FallingEdge = 2, Limit_1_Underrun = 3, Limit_1_Overrun = 4, Limit_2_Underrun = 5, Limit_2_Overrun = 6 };</pre>
--------	---

11.7.18 ESoftwareConfigChanged

Description

This enumeration contains download case indicators for an OnSoftwareConfigurationChanged event.

Table 11-113 ESoftwareConfigChanged - Native C++

Syntax	<pre>enum ESoftwareConfigChanged { SRSCC_SOFTWARE_CHANGED_IN_STOP, SRSCC_SOFTWARE_CHANGED_IN_RUN };</pre>
--------	---

11.7.19 EDirection

Description

This enumeration contains properties of the diagnostic alarm.

Table 11-114 EDirection - Native C++

Syntax	<pre>enum EDirection { SRD_DIRECTION_INPUT = 0, SRD_DIRECTION_OUTPUT = 1 };</pre>
--------	---

Table 11-115 EDirection - .NET (C#)

Syntax	<pre>enum EDirection { Input = 0, Output = 1 }</pre>
--------	--

11.7.20 EDiagProperty

Description

This enumeration contains the incoming/outgoing information of the diagnostic alarm.

Table 11-116 EDiagProperty - Native C++

Syntax	<pre>enum EDiagProperty { SRP_DIAG_APPEAR = 1, SRP_DIAG_DISAPPEAR = 2 };</pre>
--------	--

Table 11-117 EDiagProperty - .NET (C#)

Syntax	<pre>enum EDiagProperty { Appear = 1, Disappear = 2 }</pre>
--------	---

11.7.21 EDiagSeverity

Description

This enumeration contains the severity of the diagnostic alarm (error, maintenance demanded, maintenance required).

Table 11-118 EDiagSeverity - Native C++

Syntax	<pre>enum EDiagSeverity { SRDS_SEVERITY_FAILURE = 0, SRDS_SEVERITY_MAINTENANCE_DEMANDED = 1, SRDS_SEVERITY_MAINTENANCE_REQUIRED = 2 };</pre>
--------	--

Table 11-119 EDiagSeverity - .NET (C#)

Syntax	<pre>enum EDiagSeverity { Failure = 0, MaintDemanded = 1, MaintRequired = 2 };</pre>
--------	--

11.7.22 ERackOrStationFaultType

Description

This enumeration contains the types of the RackOrStationFault event.

Table 11-120 ERackOrStationFaultType - Native C++

Syntax	<pre>enum ERackOrStationFaultType { SR_RS_FFAULT = 0, SR_RS_FRETURN = 1 };</pre>
--------	--

Table 11-121 ERackOrStationFaultType - .NET (C#)

Syntax	<pre>enum ERackOrStationFaultType { Fault = 0, Return = 1 };</pre>
--------	--

11.7.23 ECycleTimeMonitoringMode

Description

This enumeration contains the sources of the timer for the maximum cycle time monitoring.

Table 11-122 ECycleTimeMonitoringMode - Native C++

Syntax	<pre>enum ECycleTimeMonitoringMode { SRCTMM_DOWNLOADED = 0, SRCTMM_IGNORED = 1, SRCTMM_SPECIFIED = 2 };</pre>
--------	---

11.7 Enumerations

Table 11-123 ECycleTimeMonitoringMode - .NET (C#)

Syntax	<pre>enum ECycleTimeMonitoringMode { Downloaded = 0, Ignored = 1, Specified = 2 }</pre>
--------	---

11.7.24 EAutodiscoverType

Description

This enumeration is used in the Autodiscover Callback function.

Table 11-124 EAutodiscoverType - Native C++

Syntax	<pre>enum EAutodiscoverType { SRRSI_DISCOVER_STARTED = 0, SRRSI_DISCOVER_DATA = 1, SRRSI_DISCOVER_FINISHED = 2 };</pre>
--------	---

Table 11-125 EAutodiscoverType - .NET (C#)

Syntax	<pre>public enum EAutodiscoverType { AutodiscoverStarted = 0, AutodiscoverData = 1, AutodiscoverFinished = 2 }</pre>
--------	--

11.7.25 ESecurityMode

Description

This enumeration is used in the OpenPort API function to set the protection level of remote runtime communication.

Table 11-126 EAutodiscoverType - Native C++

Syntax	<pre>enum class ESecurityMode { UNSECURED = 0, ENCRYPTED_COMMUNICATION = 1, UNKNOWN = -1 };</pre>
--------	---

Table 11-127 EAutodiscoverType - .NET (C#)

Syntax	<pre>public enum class ESecurityMode { Unsecured, EncryptedCommunication, Unknown }</pre>
--------	---

Examples/applications

12.1 Example: Communication via TCP/IP in Single Adapter Network Mode (promiscuous mode)

In Single Adapter Network Mode (promiscuous mode), the network adapter reads all incoming telegrams. The network adapter also reads telegrams that are not intended for the network adapter. The network adapter then forwards the data to the operating system for processing.

However, on virtualization platforms or corporate networks, it is often not permitted to run the network interface of the PC in Single Adapter Network Mode (promiscuous mode) for security reasons. For an example on Multi-Adapter Mode (non-promiscuous mode), see the next topic [\(Page 296\)](#).

The following figure provides an overview of the possibilities and differences between the following communication options:

- Local TCP/IP communication
- Single Adapter Network Mode (promiscuous mode)
- Multi-Adapter Network Mode (non-promiscuous mode)

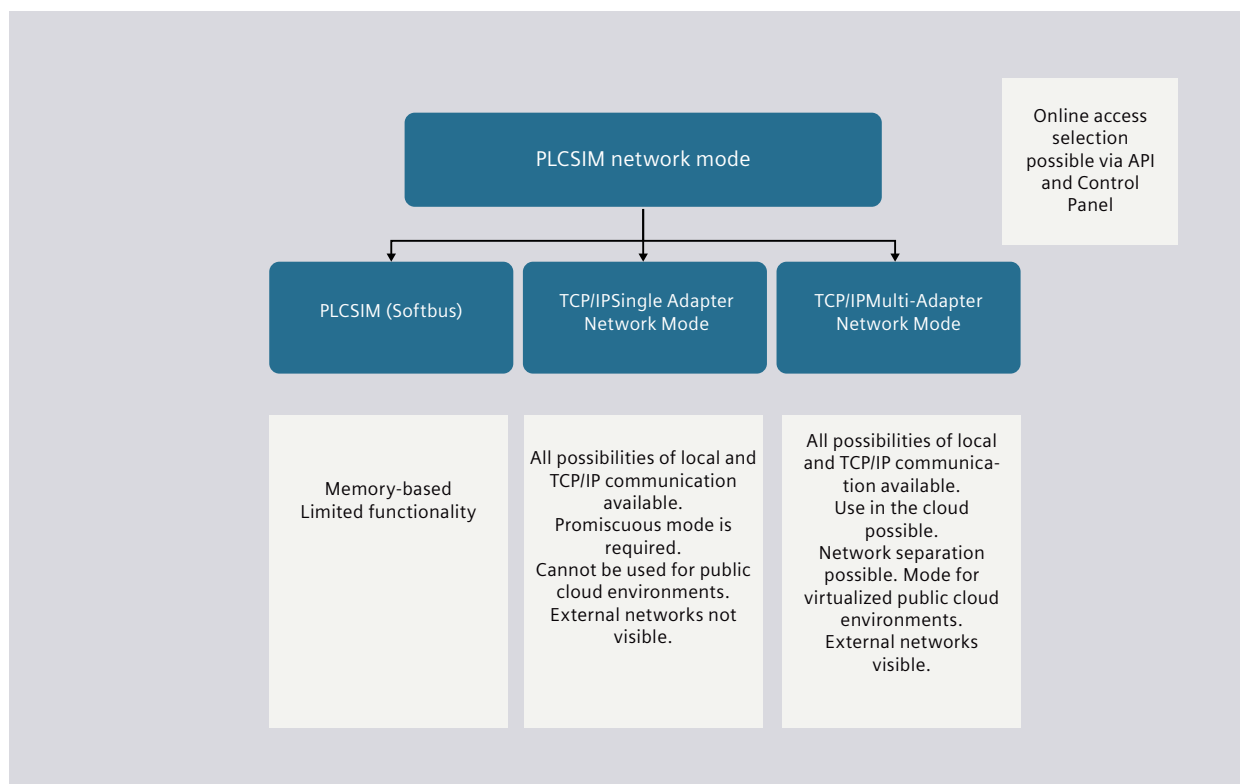


Figure 12-1 PLCSIM_networking

Example code

In this example, the physical interface is set to promiscuous mode and receives all packets; PCAP is filtering traffic. Each PLC interface has its own unique MAC address. Only two network adapters can be connected to the PLCSIM virtual switch.

```
#include "SimulationRuntimeApi.h"

ISimulationRuntimeManager* rtm = nullptr;
IInstance* instance0 = nullptr;
IInstance* instance1 = nullptr;
ERuntimeErrorCode result = SREC_INVALID_ERROR_CODE;

//initialize connection
InitializeApi (&rtm);

//set TCP/IP single adapter mode (softbus mode is default)
rtm->SetNetworkMode (ENetworkMode::TCP_IP_SINGLE_ADAPTER);

rtm->RegisterInstance (&instance0);
rtm->RegisterInstance (&instance1);
instance0->PowerOn (60000);
instance1->PowerOn (60000);

ShutdownAndFreeApi (rtm);
```

12.2 Example: Communication via TCP/IP in Multi-Adapter Network Mode (non-promiscuous mode)

In Multi-Adapter Network Mode (non-promiscuous mode), the evaluation of MAC addresses ensures that only data intended for the network adapter reaches the operating system.

Overview

The following figure provides an overview of the possibilities and differences between the following communication options:

- Local TCP/IP communication
- Single Adapter Network Mode (promiscuous mode)
- Multi-Adapter Network Mode (non-promiscuous mode)

12.2 Example: Communication via TCP/IP in Multi-Adapter Network Mode (non-promiscuous mode)

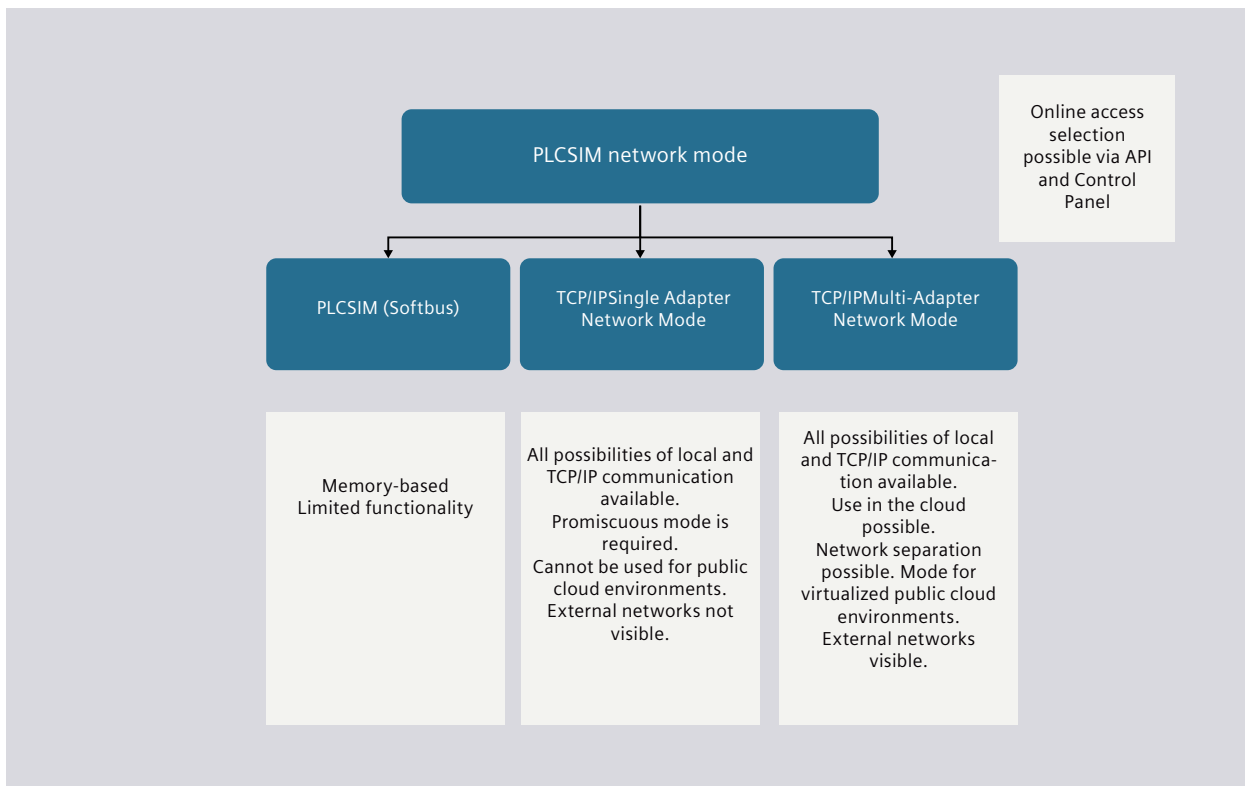


Figure 12-2 PLCSIM networking

Applications

The following applications describe how you can also communicate in Multi-Adapter Network Mode (non-promiscuous mode). You communicate in this mode by performing a one-to-one mapping of the MAC addresses between the CPU network interfaces and the network adapters of the PC.

The following applications assume that you are working in an environment where security restrictions apply and you have therefore switched to Multi-Adapter Network Mode (non-promiscuous mode).

Application 1:

You start multiple instances. Each CPU interface is assigned to the MAC address of the respective physical or virtual PC adapter.

12.2 Example: Communication via TCP/IP in Multi-Adapter Network Mode (non-promiscuous mode)

Precondition: Each interface of a running instance is assigned to its own physical or virtual PC interface (one-to-one mapping).

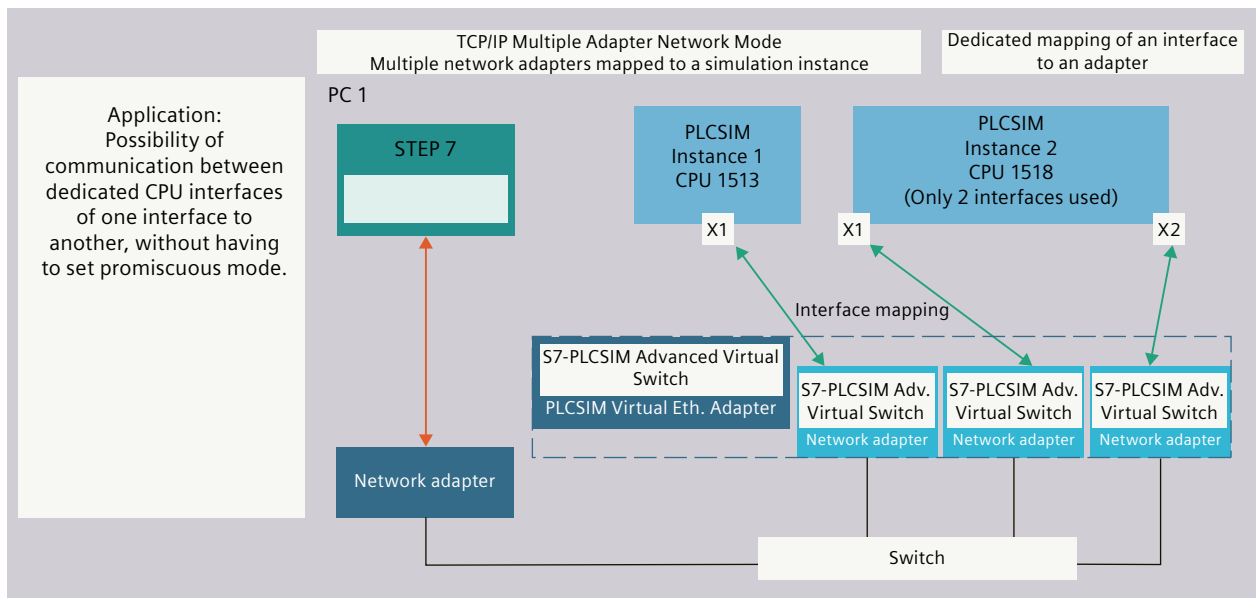


Figure 12-3 Application 1

Application 2:

The following application shows the behavior with multiple instances that have the same MAC addresses. Each CPU interface is assigned to the corresponding physical or virtual PC adapter.

12.2 Example: Communication via TCP/IP in Multi-Adapter Network Mode (non-promiscuous mode)

The scenario is only valid if only one instance is running at a time.

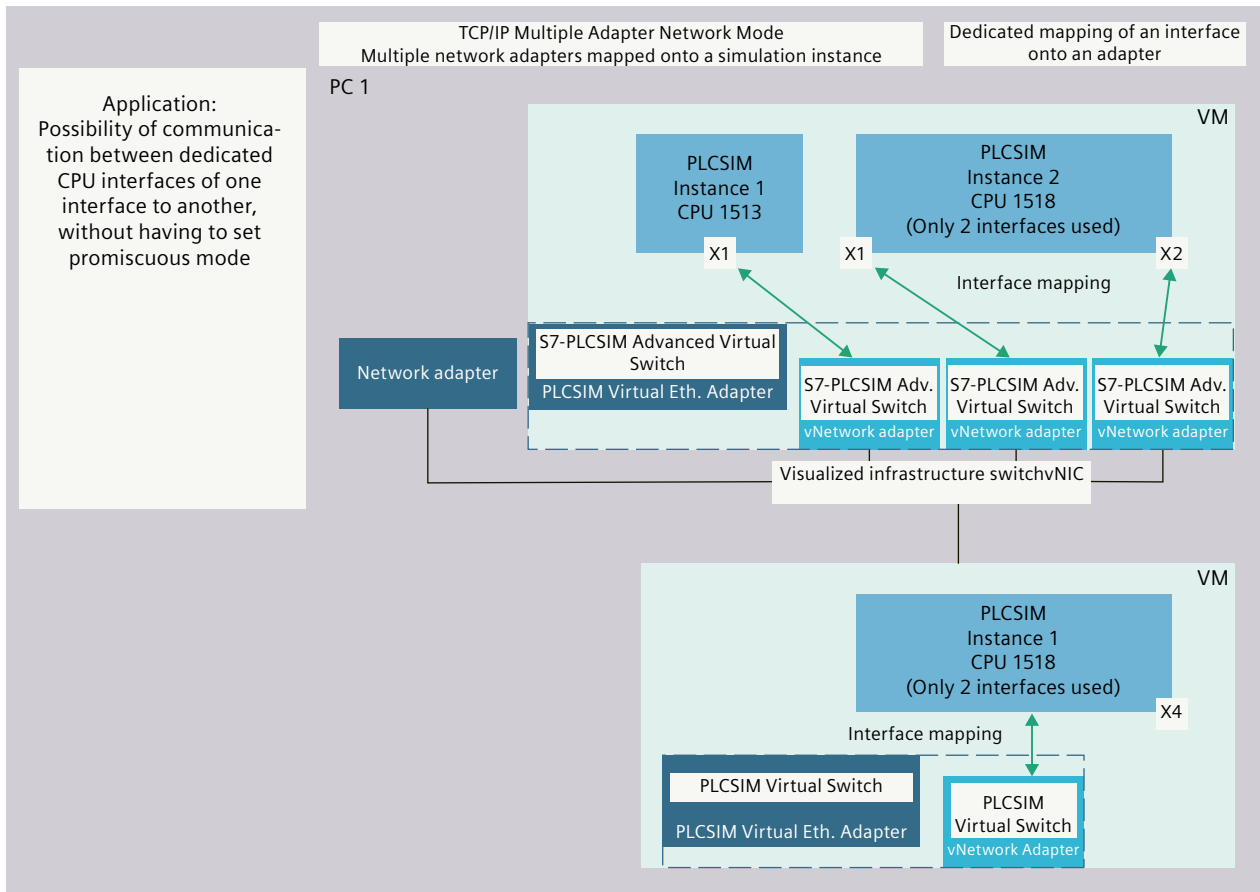


Figure 12-4 Application 2

Procedure

For external communication in Multi-Adapter Network Mode (non-promiscuous mode), the following steps are necessary depending on your application.

1. Use the following instruction to switch to Multi-Adapter Network Mode (non-promiscuous mode):
`SetNetworkMode(SRCM_VSWITCH_NON_PROMISCUOUS)`
2. To register a new instance, call `RegisterInstance()`.
3. To select which PC interface is assignable to which CPU interface by means of a list of PC interfaces, call `GetNetInterfaces()`.
4. To execute an interface mapping between CPU and PC interface, call
`SetNetInterfaceMapping(0, 23)`.
5. To check whether the configuration is correct, call `CheckNetInterfaceMapping()`.
6. To start the instance, call `PowerOn()`.

Before starting, the instance performs an internal check.

NOTE**Setting a storage path for a new instance**

If you use the `SetStoragePath()` function to set a storage path for a new instance, call this function before the MAC address is set. Otherwise, the MAC address is discarded.

This procedure is also valid for the stored CPU type and CPU serial number.

The `CleanupStoragePath()` function deletes not only the storage path but also the assignments of the network interfaces.

Examples in C++

The following examples show possible network configurations in C++.

For scenarios with more than one adapter, make sure to register the instance with a CPU that has the required number of interfaces. In the following examples we use three interfaces. Therefore we register the instance with a CPU with 3 PROFINET interfaces, such as the CPU 1518-4 PN/DP.

Example 1: Simplified settings

```
#include "SimulationRuntimeApi.h"

ISimulationRuntimeManager* rtm = nullptr;
IInstance* instance0 = nullptr;
IInstance* instance1 = nullptr;
ERuntimeErrorCode result = SREC_INVALID_ERROR_CODE;

//initialize connection
InitializeApi(&rtm);

//set NON-P mode
rtm->SetNetworkMode(ENetworkMode::TCPIP_MULTIPLE_ADAPTER);

rtm->RegisterInstance(ECPUType::SRCT_1518, &instance0);
instance0->SetNetInterfaceMapping(EPLCInterface::IE1, L"Ethernet");
instance0->SetNetInterfaceMapping(EPLCInterface::IE2, L"Ethernet 2");
instance0->SetNetInterfaceMapping(EPLCInterface::IE3, L"Ethernet 3");
instance0->PowerOn(60000);

rtm->RegisterInstance(ECPUType::SRCT_1518, &instance1);
instance1->SetNetInterfaceMapping(EPLCInterface::IE1, L"Ethernet 4");
instance1->SetNetInterfaceMapping(EPLCInterface::IE2, L"Ethernet 5");
```

12.2 Example: Communication via TCP/IP in Multi-Adapter Network Mode (non-promiscuous mode)

```
instance1->SetNetInterfaceMapping(EPLCInterface::IE3, L"Ethernet
6");
instance1->PowerOn(60000);
```

```
ShutdownAndFreeApi(rtm);
```

Example 2: Complete settings

```
#include <iostream>
#include "SimulationRuntimeApi.h"

ISimulationRuntimeManager* rtm = nullptr;
IInstance* instance0 = nullptr;
IInstance* instance1 = nullptr;
ERuntimeErrorCode result = SREC_INVALID_ERROR_CODE;

// initialize connection
InitializeApi(&rtm);

// User can get the current network mode using GetNetworkMode. The
// default mode is softbus.
ENetworkMode netMode;
rtm->GetNetworkMode(&netMode);

// If TCPIP_MULTIPLE_ADAPTER is set, the SetNetworkMode will not set
// it again.
rtm->SetNetworkMode(ENetworkMode::TCPIP_MULTIPLE_ADAPTER);

rtm->RegisterInstance(ECPUType::SRCT_1518, &instance0);
rtm->RegisterInstance(ECPUType::SRCT_1518, &instance1);

// gather network interfaces in computer
// get count of network interfaces
UINT count;
result = rtm->GetNetInterfaces(nullptr, &count);

// get network interfaces
// GetNetInterfaces will return adapters with disabled
// vSwitch and/or not connected as well
std::unique_ptr<SNetInterfaceInfo[]> netInterfaces =
std::make_unique<SNetInterfaceInfo[]>(count);
result = rtm->GetNetInterfaces(netInterfaces.get(), &count);

// map desired interface on instance interface
```

12.2 Example: Communication via TCP/IP in Multi-Adapter Network Mode (non-promiscuous mode)

```

// each instance interface must have unique or no mapping
// we can map interface by these three overloads
//
// interfaceIndex
result = instance0->SetNetInterfaceMapping(EPLCInterface::IE1,
netInterfaces.get()[0].interfaceIndex);
// friendly name
result = instance0->SetNetInterfaceMapping(EPLCInterface::IE2,
netInterfaces.get()[1].interfaceName);
// MAC address
result = instance0->SetNetInterfaceMapping(EPLCInterface::IE3,
&netInterfaces.get()[2].MACAddress);

// set mapping on second instance
result = instance1->SetNetInterfaceMapping(EPLCInterface::IE1,
&netInterfaces.get()[3].MACAddress);
result = instance1->SetNetInterfaceMapping(EPLCInterface::IE2,
&netInterfaces.get()[4].MACAddress);
result = instance1->SetNetInterfaceMapping(EPLCInterface::IE3,
&netInterfaces.get()[5].MACAddress);

// we can also check what is mapped on the interface
INT32 mappedInterfaceIndex = instance0-
>GetNetInterfaceMapping(EPLCInterface::IE1);
if (mappedInterfaceIndex == 0)
{
std::cout << "No mapping is set";
}

if (mappedInterfaceIndex == netInterfaces.get()[0].interfaceIndex)
{
std::cout << netInterfaces.get()[0].interfaceName << "is mapped on
EPLCInterface::IE1";
}

// set S7-PLCSIM Advanced Virtual Switch according to set mapping
// !! requires elevated (admin) mode !!
rtm->SetNetInterfaceBindings();

// check whether setting is valid - optional -
CheckNetInterfaceMapping() function is also called inside PowerOn()
result = instance0->CheckNetInterfaceMapping();
if (result != SREC_OK)
{
return result;
}

```

12.2 Example: Communication via TCP/IP in Multi-Adapter Network Mode (non-promiscuous mode)

```
instance0->PowerOn(60000);  
instance1->PowerOn(60000);  
  
ShutdownAndFreeApi(rtm);
```